

Curro Hillcrest Service tracker

Ruth-ann Rudolph

Grade 12

2022

Contents

Persistent storage 4

Accessing the text file (Logging in): 4

Updating the text file and tblStudents(Signing up): 6

Updating the textfile and tblAdmins (frontend): 9

Updating tblAdmins (frontend): 10

Updating tblStudents (backend): 12

Updating tblAdmins (backend): 12

Accessing tblStudents (frontend): 13

Updating tblStudents (frontend): 16

Accessing tblEvents (frontend): 18

Adding to tblEvents (frontend): 21

Adding to tblEvents (backend): 23

Updating tblEvents (frontend): 24

Accessing tblParticipation (frontend): 26

Adding to tblParticipation (frontend): 29

Adding to tblParticipation (backend): 29

Updating tblParticipation (frontend): 30

Updating tblParticipation (backend): 34

Defensive programming 34

1. Log-In 34
2. Sign-up 37
3. Home 40
4. Edit account details 40
5. Participation 43
6. Home Admin 43
7. Edit Admin account 44
8. New Admin account 46
9. Events Admin 48
10. Create new event 48
11. Edit Event 51
12. Participation Admin 53

13. Help 53
Externally sourced code 54
DBService 54
Play videos (help screen) 55
Data validation 56
Explanation of Critical Algorithms 58
Uploading events into the array 58
Adding event records into the array and database 59
Uploading participation records into the array 59
Adding participation records into the array and database 59
Uploading student records into the array 60
Advanced techniques 61
Complex multi-table database with complex joins 61
Any form of Artificial Intelligence – more than one method 62
Links to externally stored videos (YouTube) 64
Evaluation of Solution 65
Functional Testing 66
Test 1 66
Test 2 70
Data validation 75
Description table 75
Evidence of testing 76
Presence check 76
Standard 77
Abnormal 77
Length check 77
Standard 78
Extreme 78
Abnormal 79
Type check 79
Standard 80
Extreme 80

Abnormal 80
Range check 81
Standard 81
Extreme 82
Abnormal 82
Code: 82
UI Class: 82
AdminManager Class: 215
EventManager Class: 220
ParticipationManager Class: 223
StudentManager Class: 226
Admin Object Class: 233
Event Object Class: 235
Participation Object Class: 238
Student Object Class: 239
DBService Object Class: 243

Persistent storage

Accessing the text file (Logging in):

Allows an existing user to sign into their existing account by accessing the text file

```
private void btnLogInActionPerformed(java.awt.event.ActionEvent evt) {
    try{
        boolean correct; //creates a boolean to check that the username and password are correct

        try (Scanner scFile = new Scanner (new File ("UsernamesAndPasswords.txt")) //Creates a
scanner for the text file

        ) {

            String username; //creates a string to hold the username

            int current; //creates an integer to hold the ID value of the student or admin who loggs in

            String password; //creates a string to hold the password

            String title;//creates a string to hold the title admin or student
```

```
            correct = false; //sets the value of the boolean to false

            while(scFile.hasNext()) //Checks each line in the text file against the users entry

            {

                String line = scFile.nextLine(); //creates a string from the next line from the textfile

                try (Scanner scLine = new Scanner (line).useDelimiter("#")) { //creates a scanner to scan the
current line of the textfile

                    current = scLine.nextInt(); //sets current ID to the student ID from the textfile

                    username = scLine.next();//sets the username to the one from the textfile

                    password = scLine.next();//sets the password to the one from the texfile

                    title = scLine.next();//sets the title to the one from the textfile

                    //checks username and password against current line of the text file
                    if(password.equals(txfPassword.getText()) && username.equals(txfUsername.getText())
&& title.equalsIgnoreCase("Student") )
                    {
                       currentStu = current; //sets current student to the value of the ID from the textfile
                       correct = true; //sets the boolean value to true
                       if(currentStu != -1) //checks if the line from the text file was successfull in logging in
                       {
                          pnlLogIn.setVisible(false); //hides the log in pannel
                          tbdpStudent.setVisible(true);//shows the student tabbed pane
                          pnlHome.setVisible(true);//shows the student home page
                       }
                    }
                    else if (password.equals(txfPassword.getText()) &&
username.equals(txfUsername.getText()) && title.equalsIgnoreCase("Admin") ) // checks username and
password against current line of the text file
                    {
                       currentAdmin = current; //sets the value of current admin to the ID from the text file

                       correct = true; //sets the boolean value to true

                       tbdpAdmin.setVisible(true); //shows the admin tabbed pane

                       pnlLogIn.setVisible(false);//hides the log in pannel

                    }
                    scLine.close();
                }

            }
```

```
            scFile.close();
        }

      if (correct == false) //if the log in was not successful
      {
         //Displayes error if incorrect cradentials are entered
        JOptionPane. showMessageDialog(null, "Incorrect Username or password", "", JOptionPane.
ERROR_MESSAGE);
      }
      }
      catch(HeadlessException | FileNotFoundException e){ //catches any errors
         //displays an error message if the error is cought
         JOptionPane. showMessageDialog(null, "Error, pleae try again", "", JOptionPane.
ERROR_MESSAGE);
   }
   }
```

Updating the text file and tblStudents(Signing up):

Allows a user to create a new account by adding a line to the text file and adding a record to the database, to the tblStudents

```
   private void btnSignUpStuActionPerformed(java.awt.event.ActionEvent evt) {

      clearErrorLables(); //clears the error lables by calling the clear error lables method

      try{ try ( //Creates a new log in

         PrintWriter outFile = new PrintWriter(new FileWriter("UsernamesAndPasswords.txt",true)))
{//creates a file writer

         char gender = ' '; //creates a variable to hold the gender

         if(rbnNewStudentM.isSelected()) //checks if the male gender radio button is selected
         {
            gender = 'M';//sets the gender to M
         }
         else if (rbnNewStudentF.isSelected()) //checks if the female gender radio button is selected
         {
            gender = 'F';//sets the gender to F
         }
         else
         {
            lblNewStudentGenderError.setText("Please select a gender");//sets the error label if no gender
is selected
         }
         boolean password = false;//creates a boolean to check if the passwords entered match
         if (txfPassword2.getText().equals(txfConfirmPassword1.getText()))//Checks that both passwords
are the same
```

```
        {
            password = true;//sets boolean to true if passwords match
        }
        else
        {
            lblNewStudentConfPasswordError.setText("Passwords Do Not Match"); //gives an error is
passwords dont match
        }

        boolean validPortfolio = checkPortfolio();//presence check

        boolean validGrade = checkInt(txfNewStudentGrade, lblNewStudentGradeError,8,12);//type and
range check

        boolean ValidClass = checkInt(txfNewStudentClass, lblNewStudentClassError, 1,5);//type and
range check
        //presence, length and type check
        boolean validUserName = presenceAndLengthCheck(txfUsername1,
lblNewStudentUsernameError) && typeCheck(txfUsername1, lblNewStudentUsernameError, ".:,;'");
        //presence and length check
        boolean validPassword = presenceAndLengthCheck(txfPassword2,
lblNewStudentPasswordError);
        //presence, length and type check
        boolean validName = presenceAndLengthCheck(txfNewStudentName, lblNewStudentNameError)
&& typeCheck(txfNewStudentName, lblNewStudentNameError, " :,;'@#$%^&*()!+~") &&
SpaceCheck(txfNewStudentName, lblNewStudentNameError);
        //checks that all information is valid
        if((validUserName == true) && (validPassword == true) && (validName == true) && (gender != '
') && (validGrade == true) && (ValidClass == true) && (validPortfolio == true) && (password == true))
        {
            String name = txfNewStudentName.getText();//sets the name string to the one entered by the
user
            int grade = Integer.parseInt(txfNewStudentGrade.getText());//sets the grade integer to the one
entered by the user
            int classPoint =Integer.parseInt(txfNewStudentClass.getText());//sets the class integer to the
one entered by the user
            boolean catering = false;//creates a boolean to check if the user does catering
            boolean photography = false;//creates a boolean to check if the user does photography
            boolean firstAid = false;//creates a boolean to check if the user does first aid
            boolean av = false;//creates a boolean to check if the user does AV
            String lName = name.substring(name.lastIndexOf(" ")+1);//splits the last name from the first
name
            String fName = name.substring(0,name.lastIndexOf(" "));//splits the first name from the last
name

            if(cbxCatering.isSelected())//checks if the catering checkbox is selected
            {
                catering = true;//sets the catering boolean to true
            }
```

```
            if(cbxPotography.isSelected())//checks if the photography checkbox is selected
            {
               photography = true;//sets the photography boolean to true
            }
            if(cbxFirstAid.isSelected())//checks if the first aid checkbox is selected
            {
               firstAid = true;//sets the first aid boolean to true
            }
            if(cbxAV.isSelected()) //checks if the AV checkbox is selected
            {
               av = true; //sets the AV boolean to true
            }

            try {

               sm.add(fName,lName , grade, classPoint, gender, catering, photography, av, firstAid);//adds
a student

               String line = (sm.getLast())+ "#" + txfUsername1.getText() + "#" + txfPassword2.getText() +
"#Student"; //creates a new line in the text file

               currentStu = (sm.getLast());//sets current ctudent to the value of the next ID

               outFile.println(line);//adds the line to the textfile

            } catch (SQLException ex) {//catches any errors
               System.out.println("Could not create an account: Please try again");
            }
            try {
         if(sm.getCurrent(currentStu) != null)//checks that a new account has been created
         {
            pnlSignUp.setVisible(false);//hides the sign up page
            tbdpStudent.setVisible(true);//shows the student tabbed pane
         }
      } catch (SQLException ex) {//catches any erors
               System.out.println("Could not create an account: Please try again");//gives an error message
      }
         }  }
      }
       catch (IOException ex) { //cateches errors
      JOptionPane. showMessageDialog(null, "Please ensure all fields are filled out", "", JOptionPane.
ERROR_MESSAGE);

      }
   }
```

Updating the textfile and tblAdmins (frontend):

Allows an admin to create a new account by adding a line to the text file and adding a record to the database, to the tblAdmins

```
private void btnEditAdminEnter1ActionPerformed(java.awt.event.ActionEvent evt) {
        clearErrorLables(); //clears the error lables by calling the clear error lables method
        try{ try ( //Creates a new log in
            PrintWriter outFile = new PrintWriter(new FileWriter("UsernamesAndPasswords.txt",true)))
{//creates a file writer
            boolean password = false;//creates a boolean to check that the passwords match
            if (txfNewAdminPassword.getText().equals(txfNewAdminConfirmPassword.getText()))//Checks
that both passwords are the same
            {
                password = true;//sets the password boolean to true
            }
            else//if the passwords dont match
            {
                lblNewAdminConfirmPasswordError.setText("Passwords Do Not Match");//sets the lable to
display an error
            }
            boolean validPortfolio = checkPortfolioAdmin();//checks that the portfolio entries are valid
            //precence, length and type check
            boolean validUserName = presenceAndLengthCheck(txfNewAdminUsername,
lblNewAdminUsernameError) && typeCheck(txfNewAdminUsername, lblNewAdminUsernameError, ":,;'");
            //precence, length and type check
            boolean validPassword = presenceAndLengthCheck(txfNewAdminPassword,
lblNewAdminPasswordError);
            //precence, length and type check
            boolean validName = presenceAndLengthCheck(txfNewAdminName, lblNewAdminNameError)
&& typeCheck(txfNewAdminName, lblNewAdminNameError, " :,;'@#$%^&*()!+~") &&
SpaceCheck(txfNewAdminName, lblNewAdminNameError);
            //confirms that all the data is valid
            if((validUserName == true) && (validPassword == true) && (validName == true) &&
(validPortfolio == true) && (password == true))
            {
                String name = txfNewAdminName.getText();//sets the name to the value entered by the user
                boolean catering = false;//creates a boolean to check if the admin oversees catering
                boolean photography = false;//creates a boolean to check if the admin oversees photography
                boolean firstAid = false;//creates a boolean to check if the admin oversees first aid
                boolean av = false;//creates a boolean to check if the admin oversees AV
                String lName = name.substring(name.lastIndexOf(" ")+1);//puts the surname into a string
                String fName = name.substring(0,name.lastIndexOf(" "));//puts the first name into a string

                if(cbxNewAdminCatering.isSelected())//checks if the catering checkbox is selected
                {
                    catering = true;//sets the boolean to true
                }
                if(cbxNewAdminPhotography.isSelected())//checks if the photography checkbox is selected
```

```
            {
                photography = true;//sets the boolean to true
            }
            if(cbxNewAdminFirstAid.isSelected())//checks if the first aid checkbox is selected
            {
                firstAid = true;//sets the boolean to true
            }
            if(cbxNewAdminAv.isSelected())//checks if the AV checkbox is selected
            {
                av = true;//sets the boolean to true
            }

            try {

                am.add(fName,lName , catering, photography, av, firstAid);//adds a new admin
                //creates a line to be added to the text file
                String line = (am.getSize()-1) + "#" + txfNewAdminUsername.getText() + "#" +
txfNewAdminPassword.getText() + "#Admin";
                outFile.println(line);//adds the line to the textfile
                JOptionPane.showMessageDialog(null,"New Admin Created");//displays a message to
indicate success
                dlgNewAdmin.setVisible(false);//hides the dialog
            } catch (SQLException ex) {//catches any SQL errors
                Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
            }
        }  }
    }
     catch (IOException ex) {//catches any errors
        //displays an error
        JOptionPane. showMessageDialog(null, "Please ensure all fields are filled out", "", JOptionPane.
ERROR_MESSAGE);
    }
  }
```

Updating tblAdmins (frontend):

Allows the admin to edit their details and updates the database accordingly

```
private void btnEditAdminEnterActionPerformed(java.awt.event.ActionEvent evt) {
    clearErrorLables(); //clears the error lables by calling the clear error lables method
    boolean validPortfolio = checkPortfolioEditAdmin();
    boolean validName = presenceAndLengthCheck(txfEditAdminName, lblEditAdminNameError) &&
typeCheck(txfEditAdminName, lblEditAdminNameError, "-:,;'@#$%^&*()!+~ ");
    boolean validSurname = presenceAndLengthCheck(txfEditAdminSurname,
lblEditAdminSurnameError) && typeCheck(txfEditAdminSurname, lblEditAdminSurnameError, "
:,;'@#$%^&*()!+~");
    if((validName == true) && (validSurname == true) && (validPortfolio == true));
    {
        String name = txfEditAdminName.getText();
```

```
        String surname = txfEditAdminSurname.getText();
        boolean catering = false;
        boolean photography = false;
        boolean firstAid = false;
        boolean av = false;

        if(cbxCateringAdmin.isSelected())
        {
            catering = true;
        }
        if(cbxPhotographyAdmin.isSelected())
        {
            photography = true;
        }
        if(cbxFirstAidAdmin.isSelected())
        {
            firstAid = true;
        }
        if(cbxAVAdmin.isSelected())
        {
            av = true;
        }

        try {
            dbman.updateQuery("UPDATE tblAdmins SET AdminName = '" + name+ "',AdminSurname = '"
+ surname+ "', Catering = " + catering + ", Photography = " + photography + ", AV = " + av + ", [First Aid]
= " + firstAid + " WHERE AdminID = " + currentAdmin + ";");
            dlgEditAdmin.setVisible(false);
            tbdpAdmin.setVisible(true);
        } catch (SQLException ex) {
            System.out.println("Update Error, please try again");
        }
    }

    if(currentAdmin != -1)//checks that current admin has a value
    {
        try {
            txaAdminDetails.setText(am.getCurrent(currentAdmin));//calls a method to show the admin
details on the home page
        } catch (SQLException ex) {//catches any SQL errors
            Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
        }
    }

}
```

Updating tblStudents (backend):

Adds the student to the array and calls the method to insert the student into the database

```
public void add(String fn, String sn, int g, int c, char gen, boolean ca, boolean p, boolean a, boolean f)
throws SQLException//recives all the details of a student object except the auto generated StudentID
   {
      int x = size -1; //Sets x which will be used to allocate the place in the array where the new record will
be stored
      int num = (StudentArr[x].getStuID()) + 1; //Finds the last ID in the array and sets the value of num
one higher
      size++; //Increases size by 1
      StudentArr[size] = new Student(num,fn,sn,g,c,gen,ca,p,a,f);//adds new Student to the Array
      insertStudent(StudentArr[size]);//calls insert student method to add the student to the databse
   }
```

Inserts the student into the database

```
public void insertStudent(Student f) throws SQLException
   {
      String sql = "INSERT INTO tblStudents([firstName], [Surname], [Grade], [Class], [Gender],
[Catering], [Photography], [AV], [FirstAid]) VALUES ('"
         + f.getStuName() + "','" + f.getStuSurname() + "','" + f.getGrade() + "','" + f.getClassPoint() + "','"  +
f.getGender() + "','" + f.isCatering() + "','"
         + f.isPhotography() + "','" + f.isAv() + "','" + f.isFirstAid()+"');"; //SQL query to insert a student into
the database
      try
      {
         d.updateQuery(sql); //Executes the query
      }
      catch(SQLException e)
      {
         System.out.println("Insert error: Could not insert Student");
      }
   }
```

Updating tblAdmins (backend):

Adds the Admin to the array and calls the method to insert the admin into the database

```
public void add(String fn, String sn, boolean ca, boolean p, boolean a, boolean f) throws SQLException
//Recives all the information for a new admin except the auto generated Admin ID
   {
      int num = AdminArr[size-1].getID() + 1; //Finds the last ID in the array and sets the value of num one
higher
      Admin temp = new Admin(num,fn,sn,ca,p,a,f); //creates an admin admin object with all the new
details
      AdminArr[size] = new Admin(num,fn,sn,ca,p,a,f);//adds new Admin to the Array
```

```
        insertAdmin(AdminArr[size]); //calls insert admin method to add the admin to the databse
        size++; //Incriments size by 1
    }
```

Inserts the Admin into the database

```
public void insertAdmin(Admin f) throws SQLException
    {
        String sql = "INSERT INTO tblAdmins([AdminName], [AdminSurname], [Catering], [Photography], [AV], [First Aid]) VALUES ('"
        +  f.getAdminName() + "','" + f.getAdminSurname() + "','" + f.isCatering() + "','"
        + f.isPhotography() + "','" + f.isAv() + "','" + f.isFirstAid()+"');"; //SQL query to insert an admin into the database

        try
        {
            d.updateQuery(sql); //Executes the query
        }
        catch(SQLException e)
        {
            System.out.println("Insert error: Please try again");
        }
    }
```

Accessing tblStudents (frontend):

Displays all the students and their relevant details, including their accumulated hours of service for each portfolio

```
public void StudentsTBL(){

        try
        {

            DefaultTableModel dtm = (DefaultTableModel) tblStudents.getModel(); //Sets the default table model

            dtm.setRowCount(0); //sets the roow count to 0

            tblStudents.removeAll(); //Cleares the table

            model = (DefaultTableModel)tblStudents.getModel(); //Sets the default table model

            ResultSet rs = dbman.queryDatabase("SELECT * FROM tblStudents;"); //Selects all the students from the database

            while(rs.next()) //Creates a line in the JTable for every database entry
            {
```

```
            String stu = rs.getString("StudentID");//Sets the student ID to the ID value of the current record in the database

            int catering = 0; //Creates a variable to hold the catering hours total

            int photography = 0; //Creates a variable to hold the photography hours total

            int firstAid = 0; //Creates a variable to hold the first aid hours total

            int av = 0; //Creates a variable to hold the AV hours total

        ResultSet ch = dbman.queryDatabase("SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS CateringHours\n" +
                "FROM tblStudents, tblEvents, tblParticipation\n" +
                "WHERE tblParticipation.StudentID = tblStudents.StudentID AND tblEvents.EventID = tblParticipation.EventID AND tblEvents.Catering = True\n" +
                "GROUP BY tblStudents.StudentID;"); //Sums the total catering hours for each student
        ResultSet ph = dbman.queryDatabase("SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS PhotographyHours\n" +
                "FROM tblStudents, tblEvents, tblParticipation\n" +
                "WHERE tblParticipation.StudentID = tblStudents.StudentID AND tblEvents.EventID = tblParticipation.EventID AND tblEvents.Photography = True\n" +
                "GROUP BY tblStudents.StudentID;"); //Sums the total photography hours for each student
        ResultSet avh = dbman.queryDatabase("SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS AVHours\n" +
                "FROM tblStudents, tblEvents, tblParticipation\n" +
                "WHERE tblParticipation.StudentID = tblStudents.StudentID AND tblEvents.EventID = tblParticipation.EventID AND tblEvents.AV = True\n" +
                "GROUP BY tblStudents.StudentID;"); //Sums the total AV hours for each student
        ResultSet fh = dbman.queryDatabase("SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS FirstAidHours\n" +
                "FROM tblStudents, tblEvents, tblParticipation\n" +
                "WHERE tblParticipation.StudentID = tblStudents.StudentID AND tblEvents.EventID = tblParticipation.EventID AND tblEvents.FirstAid = True\n" +
                "GROUP BY tblStudents.StudentID;"); //Sums the total first aid hours for each student

            while (ch.next()) //Loops throught the catering hours results
            {
                //checks if the admin oversees catering and that the current student ID from the main result set matches the catering hours result set matches the catering hours result set
                if(am.getPortfolio("Catering", currentAdmin) == true &&
stu.equals(ch.getString("StudentID")))

                {
                    catering = ch.getInt("CateringHours");//assigns the current students total catering hours to the catering variable
                }
            }
```

```
            while (ph.next()) //Loops throught the photography hours results
            {
                //checks if the admin oversees catering and that the current student ID from the main result set matches the catering hours result set matches the photography hours result set
                if(am.getPortfolio("Photography", currentAdmin) == true && stu.equals(ph.getString("StudentID")))

                {
                    photography = ph.getInt("PhotographyHours");//assigns the current students total photography hours to the photography variable
                }
            }


            while (avh.next()) //Loops throught the AV hours results
            {
                //checks if the admin oversees catering and that the current student ID from the main result set matches the catering hours result set matches the AV hours result set
                if(am.getPortfolio("AV", currentAdmin) == true && stu.equals(avh.getString("StudentID")))

                {
                    av = avh.getInt("AVHours"); //assigns the current students total AV hours to the AV variable
                }
            }


            while (fh.next()) //Loops throught the first aid hours results
            {
                //checks if the admin oversees catering and that the current student ID from the main result set matches the catering hours result set matches the first aid hours result set
                if(am.getPortfolio("First Aid", currentAdmin) == true && stu.equals(fh.getString("StudentID")))
                {
                    firstAid = fh.getInt("FirstAidHours"); //assigns the current students total first aid hours to the first aid variable
                }
            }
            String grade = rs.getInt("Grade") + "." + rs.getInt("Class"); //Creates a string to contain the grade and class of the learner in a formatted manner
            String name = rs.getString("FirstName") + " " + rs.getString("Surname"); //Creates a string to contain the students full name
            String cater = Integer.toString(catering);//Converts the catering hours to a string
            String aV = Integer.toString(av); //converts the AV hours to a string
            String photo = Integer.toString(photography); //Converts the photography hours to a string
            String first =Integer.toString(firstAid); //Converts the first aid hours to a string
            if (am.getPortfolio("First Aid", currentAdmin) == false) //checks if the admin has permissions for first aid
            {
```

```
                first = "NA"; //if the admin does not have permissions for first aid, sets the first aid hours collum to 'NA'
            }
            if (am.getPortfolio("AV", currentAdmin) == false)//checks if the admin has permissions for AV
            {
                aV = "NA"; //if the admin does not have permissions for AV, sets the AV hours collum to 'NA'
            }
            if(am.getPortfolio("Catering", currentAdmin) == false) //checks if the admin has permissions for catering
            {
                cater = "NA"; //if the admin does not have permissions for catering, sets the catering hours collum to 'NA'
            }
            if (am.getPortfolio("Photography", currentAdmin) == false) //checks if the admin has permissions for photography
            {
                photo = "NA"; //if the admin does not have permissions for photography, sets the photography hours collum to 'NA'
            }
            //Creates a new object to be added to the table
             model.addRow(new Object[]{rs.getInt("StudentID"), name, grade, rs.getString("Gender"),cater, aV, photo,first });

            }
        model.addRow(new Object[]{}); //adds each object to its own row in the table

      }
      catch(SQLException e)

      {

         System.out.println("Could Not Load Students Table");//Displays an error message if an error occurs

      }

   }
```

Updating tblStudents (frontend):

Allows the student to edit any of their information and update the table accordingly

```
private void btnEditStuEnterActionPerformed(java.awt.event.ActionEvent evt) {
      clearErrorLables(); //clears the error lables by calling the clear error lables method
      char gender = ' ';//creates a variable to hold the gender
      if(rbnEditStuGenderM.isSelected())//checks if the radio button is selected
      {
         gender = 'M';//sets the gender character to M
      }
```

```
        else if (rbnEditStuGenderF.isSelected())//checks if the radio button is selected
        {
            gender = 'F';//sets the gender character to F
        }
        else//if neither radio buttons were selected
        {
            lblEditStuGenderError.setText("Please select a gender");//displays an error message
        }

        boolean validPortfolio = checkEditPortfolio();//checks that the portfolio selection is valid
        boolean validGrade = checkInt(txfEditStuGrade, lblEditStuGradeError,8,12);//Type,presence and
range check
        boolean validClass = checkInt(txfEditStuClass, lblEditStuClassError, 1,5);//Type,presence and range
check
        //precence, length and type check
        boolean validName = presenceAndLengthCheck(txfEditStuName, lblEditStuNameError) &&
typeCheck(txfEditStuName, lblEditStuNameError, " -:,;'@#$%^&*()!+~");
        //precence, length and type check
        boolean validSurname = presenceAndLengthCheck(txfEditStuSurname, lblEditStuSurnameError)
&& typeCheck(txfEditStuSurname, lblEditStuSurnameError, " -:,;'@#$%^&*()!+~");
        //confirms that all the data is valid
        if((validName == true) && (validSurname == true) && (gender != ' ') && (validGrade == true) &&
(validClass == true) && (validPortfolio == true))
        {
            String name = txfEditStuName.getText();//sets the name to the value entered by the user
            String surname = txfEditStuSurname.getText();//sets the surname to the value entered by the user
            int grade = Integer.parseInt(txfEditStuGrade.getText());//sets the grade to the value entered by the
user and converts it to an integer
            int classPoint =Integer.parseInt(txfEditStuClass.getText());//sets the class to the value entered by
the user and converts it to an integer

            boolean catering = false;//creates a boolean to hold catering
            boolean photography = false;//creates a boolean to hold photography
            boolean firstAid = false;//creates a boolean to hold first aid
            boolean av = false;//creates a boolean to hold AV

            if(cbxCatering1.isSelected())//checks if the catering checkbox is ticked
            {
                catering = true;//sets the catering boolean to true
            }
            if(cbxPhotography1.isSelected())//checks if the photography checkbox is ticked
            {
                photography = true;//sets the photography boolean to true
            }
            if(cbxFirstAid1.isSelected())//checks if the first aid checkbox is ticked
            {
                firstAid = true;//sets the first aid boolean to true
            }
            if(cbxAV1.isSelected())//checks if the AV checkbox is ticked
```

```
        {
            av = true;//sets the AV boolean to true
        }

        try {
            //updates the student record in the database
            dbman.updateQuery("UPDATE tblStudents SET firstName = '" + name+ "',Surname = '" +
surname+ "', Grade = " + grade+  ",Class = " + classPoint + ", Gender = '" + gender + "', Catering = " +
catering + ", Photography = " + photography + ", AV = " + av + ", FirstAid = " + firstAid + " WHERE
StudentID = " + currentStu + ";");
            dlgEditStudent.setVisible(false);//hides the dialog
            tbdpStudent.setVisible(true);//shows the student tabbed pane
        } catch (SQLException ex) {//catches any SQL errors
            Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
        }
    }
    else//if the data was not valid
    {
         JOptionPane. showMessageDialog(null, "UNABLE TO UPDATE!", "", JOptionPane.
ERROR_MESSAGE);//displays an error message
    }
    try {
        if(currentStu != -1)//checks that the current student has a value
        {
          txaStudent.setText(sm.getCurrent(currentStu));//calls a method to show the student details on
the home page
        }
    } catch (SQLException ex) {//catches SQL errors
        Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
    }
  }
```

Accessing tblEvents (frontend):

Displays the events that the admin has permissions for on the admin events page:

```
public void AdminEventsTBL(boolean c, boolean p, boolean a, boolean f){
    try
    {
        DefaultTableModel dtm = (DefaultTableModel) tblEventsAdmin.getModel(); //Sets the default table
moddel

        dtm.setRowCount(0); //Sets the row count to 0

        tblEventsAdmin.removeAll();//clears the table

        model = (DefaultTableModel)tblEventsAdmin.getModel(); //Sets the default table moddel
```

```
        ResultSet rs = dbman.queryDatabase("SELECT * FROM tblEvents"); //Selects data from the
databse

        while(rs.next()) //Creates a line in the JTable for every database entry
        {
            String portfolio = ""; //Creates a string to store the portfolio of the event
            if(rs.getBoolean("Catering") == true) //Checks if the event is for catering
            {
                portfolio = "Catering"; //Sets the portfolio string to catering
            }
            else if(rs.getBoolean("Photography") == true) //Checks if the event is for photography
            {
                portfolio = "Photography"; //Sets the portfolio string to photography
            }
            else if(rs.getBoolean("FirstAid") == true) //Checks if the event is for first aid
            {
                portfolio = "First Aid"; ////Sets the portfolio string to First aid
            }
            else if(rs.getBoolean("AV") == true) //Checks if the event is for AV
            {
                portfolio = "AV"; //Sets the portfolio string to AV
            }
            //Checks that the admin has permission to view the event
            if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true &&
portfolio.equalsIgnoreCase("Photography")) || (a == true && portfolio.equalsIgnoreCase("AV")) || (f ==
true && portfolio.equalsIgnoreCase("First Aid")))
            {
                model.addRow(new Object[]{rs.getInt("EventID"),rs.getString("EventName"),
                 rs.getInt("Hours"),rs.getDate("EventDate"), portfolio}); //Creates a new object to be added to
the table
            }
            }

        model.addRow(new Object[]{});//adds each object to its own row in the table

    }

    catch(SQLException e)

    {

        System.out.println("Could Not Load Events Table: Please restart the program"); //Displays an
error message if there is an error

    }

  }
```

Displays the events related to the portfolios the student participates in

```
public void StudentEventsTBL(boolean c, boolean p, boolean a, boolean f){
    try

    {

        DefaultTableModel dtm = (DefaultTableModel) tblEventStu.getModel(); //Sets the default table
model

        dtm.setRowCount(0);//sets the roow count to 0

        tblEventStu.removeAll();//clears the table

        model = (DefaultTableModel)tblEventStu.getModel(); //Sets the default table model

        ResultSet rs = dbman.queryDatabase("SELECT * FROM tblEvents"); //Selects data from the
databse



        while(rs.next()) //Creates a line in the JTable for every database entry

        {

            String portfolio = ""; //Creates a string to store the portfolio of the event

            if(rs.getBoolean("Catering") == true) //Checks if the event is for catering

            {

                portfolio = "Catering";//Sets the portfolio string to catering

            }

            else if(rs.getBoolean("Photography") == true) //Checks if the event is for photography

            {

                portfolio = "Photography"; //Sets the portfolio string to photography

            }

            else if(rs.getBoolean("FirstAid") == true)//Checks if the event is for first aid

            {

                portfolio = "First Aid"; //Sets the portfolio string to first aid

            }
```

```
            else if(rs.getBoolean("AV") == true)//Checks if the event is for AV

            {

                portfolio = "AV"; //Sets the portfolio string to AV

            }
            //Checks that the student has permission to view the event
            if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true &&
portfolio.equalsIgnoreCase("Photography")) || (a == true && portfolio.equalsIgnoreCase("AV")) || (f ==
true && portfolio.equalsIgnoreCase("First Aid")))
            {
                model.addRow(new Object[]{rs.getInt("EventID"),rs.getString("EventName"),
                 rs.getInt("Hours"),rs.getDate("EventDate"), portfolio}); //Creates a new object to be added to
the table

            }
            }

        model.addRow(new Object[]{});//adds each object to its own row in the table
      }

      catch(SQLException e)

      {
         System.out.println("Could Not Load Events Table");//Displays an error message if an error occurs

      }

   }
```

Adding to tblEvents (frontend):

Allows admins to add an event by entering all the information for the new event

```
private void btnNewEventEnterActionPerformed(java.awt.event.ActionEvent evt) {

      clearErrorLables(); //clears the error lables by calling the clear error lables method
      String name;
      //Presence, Length and Type check
      boolean validName = presenceAndLengthCheck(txfNewEventName, lblNewNameError) &&
typeCheck(txfNewEventName, lblNewNameError, " :,;'");
      //Precence check
      boolean validDate = CheckDate(lblNewDateError);

      boolean catering = false;//sets catering boolean to false
```

```
        boolean av = false;//sets AV boolean to false

        boolean photography = false;//sets photography boolean to false

        boolean firstAid = false;//sets first aid boolean to false

        boolean selected = false;//sets boolean selected to false, to indicate if any radio buttons have been
selected

        //Precence check
        if(rbnNewEventCatering.isSelected()) //checks if the catering radio button is selected
        {
            catering = true; //sets catering boolean to true

            selected = true; //sets selected boolean to true
        }

        else if (rbnNewEventAV.isSelected()) //checks if the AV radio button is selected
        {
            av = true; //sets AV boolean to true

            selected = true; //sets selected boolean to true
        }

        else if (rbnNewEventPhotography.isSelected()) //checks if the Photography radio button is selected
        {
            photography = true;//sets photography boolean to true

            selected = true; //sets selected boolean to true
        }

        else if (rbnNewEventFirstAid.isSelected()) //checks if the first aid radio button is selected
        {
            firstAid = true;//sets first aid boolean to true

            selected = true; //sets selected boolean to true
        }

        else
        {
            lblNewPortfolioError.setText("Select a portfolio"); //sets the error label to display an error
        }

        //Type check and range check
        String hours = checkHours(txfNewEventHours, lblNewHoursError); //type check

        double h = Double.parseDouble(hours); //converts hours from a string to a double
```

```
      if(validName == true &&  validDate == true && hours != null && selected == true) //checks that all the data is valid
      {
         try {

            name = txfNewEventName.getText(); //sets name variable to the text in the text feild

            LocalDate dateEvent = dpkDate.getDate();//Sets the date to the date entered

            em.add(name,h ,dateEvent, catering, photography, firstAid, av); //adds the new event

            dlgEvents.setVisible(false); //Hides the event dialog

            JOptionPane.showMessageDialog(null, "Event successfully added"); //Displays a message to indicate success
            //refreshes the table
            AdminEventsTBL((am.getPortfolio("Catering", currentAdmin)),(am.getPortfolio("Photography", currentAdmin)),(am.getPortfolio("av", currentAdmin)),(am.getPortfolio("First aid", currentAdmin)));

         } catch (SQLException ex) { //catches any sql errors

            System.out.println("SQL ERROR");//prints an error message

         }

      }

   }
```

Adding to tblEvents (backend):

Adds an event to the array and calls a method to insert the event into the database

```
public void add(String n, double h,LocalDate d, boolean c, boolean p, boolean f, boolean a) throws SQLException
   {
      int x = size -1; //Sets x which will be used to allocate the place in the array where the new record will be stored
      int num = EventArr[x].getEventNum() + 1; //Finds the last ID in the array and sets the value of num one higher
      EventArr[size] = new Event(num,n,h,d,c,p,f,a);//Adds a new event to the array
      insertEvent(EventArr[size]);//calls the insert event method
      size++; //Adds 1 to size
   }
```

Inserts the event into the database

```
public void insertEvent(Event f) throws SQLException
```

```
    {
        String sql = "INSERT INTO tblEvents([EventName], [Hours], [EventDate], [Catering], [Photography], [FirstAid], [AV]) VALUES ('"
            + f.getEventName() + "','" + f.getNumHours() + "','" + f.getDate() + "','" + f.isCatering() + "','"
            + f.isPhotography() + "','" + f.isFirstAid() + "','" + f.isAudioVisual()+"');"; //SQL query to insert an event into the database

        try
        {
            d.updateQuery(sql); //Executes the query
        }
        catch(SQLException e)
        {
            System.out.println("Insert error: could not insert event");
        }
    }
```

Updating tblEvents (frontend):

Allows the user to change any information about an event and updates the event accordingly

```
clearErrorLables(); //clears the error lables by calling the clear error lables method
        String name; //holds the event name
        //Presence, Length and Type check
        boolean validName = presenceAndLengthCheck(txfEditEventName, lblEditNameError) &&
typeCheck(txfEditEventName, lblEditNameError, " :,;'");
        //Precence check
        boolean validDate = CheckEditDate(lblEditDateError);
        boolean catering = false;//holds the boolean for if the event is for catering
        boolean av = false;//holds the boolean for if the event is for AV
        boolean photography = false;//holds the boolean for if the event is for photography
        boolean firstAid = false;//holds the boolean for if the event is for first aid
        boolean selected = false;//checks that a portfolio has been selected

        //Precence check
        if(rbnEditEventCatering.isSelected())//checks if the catering radio button is selected
        {
            catering = true;//sets the catering boolean to true

            selected = true;//sets the selected boolean to true
        }
        else if (rbnEditEventAV.isSelected())//checks if the AV radio button is selected
        {
            av = true;//sets the AV boolean to true

            selected = true;//sets the selected boolean to true
        }
```

```
        else if (rbnEditEventPhotography.isSelected())//checks if the Photography radio button is selected
        {
            photography = true;//sets the photography boolean to true

            selected = true;//sets the selected boolean to true
        }
        else if (rbnEditEventFirstAid.isSelected())//checks if the first aid radio button is selected
        {
            firstAid = true;//sets the first aid boolean to true

            selected = true;//sets the selected boolean to true
        }
        else//if none of the buttons are selected
        {
            lblEditPortfolioError.setText("Select a portfolio");//sets the lable to an error
        }

        //Type check and range check
        String hours = checkHours(txfEditEventHours, lblEditHoursError);

        double h = Double.parseDouble(hours);//converts hours to a double

        //checks that all the values are valid
        if(validName == true &&  validDate == true && hours != null && selected == true)
        {
            try {
                name = txfEditEventName.getText();//sets name to the name entered by the user

                LocalDate dateEvent = dpkEditDate.getDate();//gets the date from the date picker
                //creates the update event query
                String sql = ("UPDATE tblEvents SET EventName = '" + name+ "',Hours = " + hours+ ",
EventDate = #" + dateEvent + "#, Catering = " + catering + ", Photography = " + photography +  ", FirstAid
= " + firstAid + ", AV = " + av + " WHERE EventID = " + txfEditEventID.getText() + ";");

                dbman.updateQuery(sql);//executes the query

                dlgEditEvent.setVisible(false);//hides the dialog

                JOptionPane.showMessageDialog(null, "Event successfully Updated");//shows a message to
indicate a successfull entry

            } catch (SQLException ex) {//catches SQL errors
                System.out.println("SQL ERROR");//gives an error
            }
        }
        //reloads the events table for the admin
        AdminEventsTBL((am.getPortfolio("Catering", currentAdmin)),(am.getPortfolio("Photography",
currentAdmin)),(am.getPortfolio("av", currentAdmin)),(am.getPortfolio("First aid", currentAdmin)));
```

```
}
```

Accessing tblParticipation (frontend):

Allows the admin to view all participation records related to their portfolios

```
public void AdminParticipationTBL(boolean c, boolean p, boolean a, boolean f){ //Loads data from database into the Jtable
    try
    {
        DefaultTableModel dtm = (DefaultTableModel) tblParticipationAdmin.getModel(); //Sets the default table model
        dtm.setRowCount(0); //sets the roow count to 0
        tblParticipationAdmin.removeAll(); //Cleares the table
        model = (DefaultTableModel)tblParticipationAdmin.getModel(); //Sets the default table model
        //Selects all the participation records from the database
        ResultSet rs = dbman.queryDatabase("SELECT DISTINCT tblStudents.StudentID, tblStudents.firstName, tblStudents.Surname, tblEvents.EventID, tblEvents.EventName, tblEvents.hours, tblEvents.Catering, tblEvents.Photography, tblEvents.AV, tblEvents.FirstAid FROM tblEvents, tblStudents, tblParticipation Where tblParticipation.StudentID = tblStudents.StudentID AND tblParticipation.EventID = tblEvents.EventID");
        while(rs.next()) //Creates a line in the JTable for every database entry
        {
            String portfolio = ""; //Creates a string to store the portfolio of the event
            if(rs.getBoolean("Catering") == true) //Checks if the event of the participation record is for catering
            {
                portfolio = "Catering"; //Sets the portfolio string to catering
            }
            else if(rs.getBoolean("Photography") == true) //Checks if the event of the participation record is for photography
            {
                portfolio = "Photography"; //Sets the portfolio string to photography
            }
            else if(rs.getBoolean("FirstAid") == true) //Checks if the event of the participation record is for first aid
            {
                portfolio = "First Aid"; //Sets the portfolio string to first aid
            }
            else if(rs.getBoolean("AV") == true) //Checks if the event of the participation record is for AV
            {
                portfolio = "AV"; //Sets the portfolio string to AV
            }
            //Checks that the admin has permission to view the participation record
            if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true && portfolio.equalsIgnoreCase("Photography")) || (a == true && portfolio.equalsIgnoreCase("AV")) || (f == true && portfolio.equalsIgnoreCase("First Aid")))
```

```
            {
                String name = rs.getString("firstname") + " " + rs.getString("surname");//Creates a string to
hold the students full name
                model.addRow(new Object[]{rs.getInt("StudentID"),name, rs.getInt("EventID"),
rs.getString("EventName"),
                 rs.getInt("Hours"), portfolio}); //Creates a new object to be added to the table
            }

            }

        model.addRow(new Object[]{});//adds each object to its own row in the table

      }

      catch(SQLException e)

      {

          System.out.println("Could Not Execute Load Participation Table");//Displays an error message if
an error occurs

      }

   }
```

Allows a student to view all of their own participation records

```
public void ParticipationTBL(boolean c, boolean p, boolean a, boolean f){
      try
      {

          DefaultTableModel dtm = (DefaultTableModel) tblParticipation.getModel();//Sets the default table
model

          dtm.setRowCount(0); //Sets the row count to 0

          tblParticipation.removeAll(); //Cleares the table

          model = (DefaultTableModel)tblParticipation.getModel();//Sets the default table model

          //Selects the participation records for the student currently logged in
          ResultSet rs = dbman.queryDatabase("SELECT DISTINCT tblStudents.StudentID,
tblEvents.EventID, tblEvents.EventName, tblEvents.hours, tblEvents.Catering, tblEvents.Photography,
tblEvents.AV, tblEvents.FirstAid FROM tblEvents, tblStudents, tblParticipation Where
tblParticipation.StudentID = tblStudents.StudentID AND tblParticipation.EventID = tblEvents.EventID AND
tblParticipation.StudentID =" + currentStu);

          while(rs.next()) //Creates a line in the JTable for every database entry
```

```
        {
            String portfolio = ""; //Creates a string to store the portfolio of the event
            if(rs.getBoolean("Catering") == true)//Checks if the event of the participation record is for
catering
            {
               portfolio = "Catering"; //Sets the portfolio string to catering
            }
            else if(rs.getBoolean("Photography") == true) //Checks if the event of the participation record is
for photography
            {
               portfolio = "Photography"; //Sets the portfolio string to photography
            }
            else if(rs.getBoolean("FirstAid") == true)//Checks if the event of the participation record is for
first aid
            {
               portfolio = "First Aid"; //Sets the portfolio string to first aid
            }
            else if(rs.getBoolean("AV") == true) //Checks if the event of the participation record is for AV
            {
               portfolio = "AV"; //Sets the portfolio string to AV
            }
            //Checks that the student has permission to view the participation record
            if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true &&
portfolio.equalsIgnoreCase("Photography")) || (a == true && portfolio.equalsIgnoreCase("AV")) || (f ==
true && portfolio.equalsIgnoreCase("First Aid")))

            {
               model.addRow(new Object[]{rs.getInt("StudentID"), rs.getInt("EventID"),
rs.getString("EventName"),
                rs.getInt("Hours"), portfolio}); //Creates a new object to be added to the table
            }
            }

         model.addRow(new Object[]{}); //adds each object to its own row in the table

      }

      catch(SQLException e)

      {

         System.out.println("Could Not Load Participation Table"); //Displays an error message if an error
occurs

      }

   }
```

Adding to tblParticipation (frontend):

Allows a student to add a participation record

```
private void btnCreateParticipationActionPerformed(java.awt.event.ActionEvent evt) {
        try {
            int Event = Integer.parseInt(txfEventIDCreate.getText());//places the entered event ID into a variable
            pm.add(Event,currentStu);//adds a participation record with the current student ID and entered event ID
            JOptionPane.showMessageDialog(null,"Successfull entry");//displays a message to indicate success
            txfEventIDCreate.setText("e.g 5");//re-sets the text field
        } catch (SQLException ex) {//catches any SQL errors
            Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
        }
        //re-loads the participation table for the student
        ParticipationTBL((sm.getPortfolio("Catering", currentStu)),(sm.getPortfolio("Photography", currentStu)),(sm.getPortfolio("av", currentStu)),(sm.getPortfolio("First aid", currentStu)));
    }
```

Allows an admin to add a participation record

```
private void btnCreateParticipationAdminActionPerformed(java.awt.event.ActionEvent evt) {
        try {
            int Event = Integer.parseInt(txfCreatePartEventID.getText());//creates a variable to hold the event ID entered by the user
            pm.add(Event,Integer.parseInt(txfCreatePartStuID.getText()));//creates a variable to hold the student ID entered by the user
            JOptionPane.showMessageDialog(null,"Successfull entry");//displays a message to indicate success
        } catch (SQLException ex) {//catches any SQL errors
            Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
        }
        //re-loads the admin participation table
        AdminParticipationTBL((am.getPortfolio("Catering", currentAdmin)),(am.getPortfolio("Photography", currentAdmin)),(am.getPortfolio("av", currentAdmin)),(am.getPortfolio("First aid", currentAdmin)));
    }
```

Adding to tblParticipation (backend):

Adds a participation record to the array and calls a method to insert the record into the database

```
public void add(int studentID, int eventID) throws SQLException//Recives a Student ID and Event ID for a new participation record
    {
        PartArr[size] = new Participation(studentID, eventID);//Adds a new Participation to the array
```

```
        size++; //Adds 1 to size
        insertParticipation(eventID,studentID);//calls the insert participation mathod
    }
```

Inserts the participation record into the database

```
public void insertParticipation(int s,int e) throws SQLException
    {
        String sql = "INSERT INTO tblParticipation([EventID],[StudentID]) VALUES ('"
            + e + "','" + s+ "');"; //SQL query to insert a participation record into the database
        try
        {
            d.updateQuery(sql); //Executes the query
        }
        catch(SQLException p)
        {
            System.out.println("Insert error: Could not create participation record");
        }
    }
```

Updating tblParticipation (frontend):

Allows a student to delete a participation record

```
private void btnDeleteStuPartActionPerformed(java.awt.event.ActionEvent evt) {
        try //Searches for an entry by event ID and returns it to the table if found
        {
            DefaultTableModel dtm = (DefaultTableModel) tblParticipation.getModel(); //Sets the default table
model
            tblParticipation.removeAll();//Empties the Jtable
            dtm.setRowCount(0); //sets the row count to 0
            model = (DefaultTableModel)tblParticipation.getModel(); //Sets the default table model
            ResultSet rs = dbman.queryDatabase("SELECT DISTINCT tblStudents.StudentID,
tblEvents.EventID, tblEvents.EventName, tblEvents.hours, tblEvents.Catering, tblEvents.Photography,
tblEvents.AV, tblEvents.FirstAid FROM tblEvents, tblStudents, tblParticipation Where
tblParticipation.StudentID = tblStudents.StudentID AND tblParticipation.EventID = tblEvents.EventID AND
tblEvents.EventID = " + Integer.parseInt(txfEventIDDelete.getText()) + " AND tblStudents.StudentID = " +
currentStu); //Retrives entry(s) from the database
            while(rs.next())//loops through all the entrys returned
            {
                String portfolio = "";//creates a string to hold the portfolio
                if(rs.getBoolean("Catering") == true)//checks if the participation record is for catering
                {
                    portfolio = "Catering";//sets portfolio to catering
                }
                else if(rs.getBoolean("Photography") == true)//checks if the participation record is for
photography
                {
```

```
                portfolio = "Photography";//sets portfolio to photography
            }
            else if(rs.getBoolean("FirstAid") == true)//checks if the participation record is for first aid
            {
                portfolio = "First Aid";//sets portfolio to first aid
            }
            else if(rs.getBoolean("AV") == true)//checks if the participation record is for AV
            {
                portfolio = "AV"; //sets portfolio to AV
            }
            boolean c = sm.getPortfolio("Catering", currentStu);//checks if the current student does catering
            boolean p = sm.getPortfolio("Photography", currentStu);//checks if the current student does
photography
            boolean a = sm.getPortfolio("av", currentStu);//checks if the current student does AV
            boolean f = sm.getPortfolio("First aid", currentStu);//checks if the current student does first aid
            //checks that the student has permision to view the participation record
            if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true &&
portfolio.equalsIgnoreCase("Photography")) || (a == true && portfolio.equalsIgnoreCase("AV")) || (f ==
true && portfolio.equalsIgnoreCase("First Aid")))
            {
                int s = rs.getInt("StudentID");//places the Student ID into a variable
                int e = rs.getInt("EventID");//places the event ID into a variable
                String en = rs.getString("EventName");//places the event name into a variable
                int h = rs.getInt("Hours");//places the number of hours into a variable
                model.addRow(new Object[]{s,e,en,h, portfolio});//adds the row to an object

                int answer = JOptionPane.showConfirmDialog(btnDeleteStuPart,
                "Are you sure you want to delete this record?\n" + s + "-" + e + "-"+ en +"-"+h, "DELETE
PARTICIPATION RECORD",
                JOptionPane.YES_NO_OPTION);//displays a yes/no message to check if the user wants to
delete the record

                if (answer == JOptionPane.NO_OPTION) { //if the user selects no
                    JOptionPane.showMessageDialog(null,"Delete Canceled");//displays a cancelled
message
                } else if(answer == JOptionPane.YES_OPTION) {//if the user selects yes
                    pm.deleteEntry(Integer.parseInt(txfEventIDDelete.getText()),currentStu); //deletes the
record
                    //reloads the participation table for the student
                    ParticipationTBL((sm.getPortfolio("Catering", currentStu)),(sm.getPortfolio("Photography",
currentStu)),(sm.getPortfolio("av", currentStu)),(sm.getPortfolio("First aid", currentStu)));
                }
            }
        }
            txfEventIDDelete.setText("eg.5"); //Clears the text field
            model.addRow(new Object[]{}); //Adds row to the table
        }
        catch(SQLException e) //Catches any sql Errors
        {
```

```
        JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message
    }
    //reloads the participation table for the student
    ParticipationTBL((sm.getPortfolio("Catering", currentStu)),(sm.getPortfolio("Photography",
currentStu)),(sm.getPortfolio("av", currentStu)),(sm.getPortfolio("First aid", currentStu)));
  }
```

Allows an admin to delete a participation record

```
private void btnDeleteParticipationActionPerformed(java.awt.event.ActionEvent evt) {
    try //Searches for an entry by dive number and returns it to the table if found
    {
        DefaultTableModel dtm = (DefaultTableModel) tblParticipationAdmin.getModel(); //Sets the default
table model
        tblParticipationAdmin.removeAll();//Empties the Jtable
        dtm.setRowCount(0); //sets the row count to 0
        model = (DefaultTableModel)tblParticipationAdmin.getModel(); //Sets the default table model
        ResultSet rs = dbman.queryDatabase("SELECT DISTINCT tblStudents.StudentID,
tblStudents.firstName, tblStudents.surname, tblEvents.EventID, tblEvents.EventName, tblEvents.hours,
tblEvents.Catering, tblEvents.Photography, tblEvents.AV, tblEvents.FirstAid FROM tblEvents,
tblStudents, tblParticipation Where tblParticipation.StudentID = tblStudents.StudentID AND
tblParticipation.EventID = tblEvents.EventID AND tblEvents.EventID = " +
Integer.parseInt(txfDeleteEventID.getText()) + " AND tblStudents.StudentID = " +
txfDeleteStuID.getText()); //Retrives entry(s) from the database
        while(rs.next())//Loops through each result
        {
            String portfolio = "";//creates a string to hold the portfolios
            if(rs.getBoolean("Catering") == true)//checks if the participation record is for catering
            {
                portfolio = "Catering";//sets the string to catering
            }
            else if(rs.getBoolean("Photography") == true)//checks if the participation record is for
photography
            {
                portfolio = "Photography";//sets the string to photography
            }
            else if(rs.getBoolean("FirstAid") == true)//checks if the participation record is fornfirst aid
            {
                portfolio = "First Aid";//sets the string to first aid
            }
            else if(rs.getBoolean("AV") == true)//checks if the participation record is for AV
            {
                portfolio = "AV";//sets the string to AV
            }
            boolean c = am.getPortfolio("Catering", currentAdmin);//checks if the current admin oversees
catering
```

```
            boolean p = am.getPortfolio("Photography", currentAdmin);//checks if the current admin
oversees photography
            boolean a = am.getPortfolio("av", currentAdmin);//checks if the current admin oversees AV
            boolean f = am.getPortfolio("First aid", currentAdmin);//checks if the current admin oversees
first aid
            //checks that all the data entered is valid
            if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true &&
portfolio.equalsIgnoreCase("Photography")) || (a == true && portfolio.equalsIgnoreCase("AV")) || (f ==
true && portfolio.equalsIgnoreCase("First Aid")))
            {
                int s = rs.getInt("StudentID");//puts the student ID into a variable
                String name = rs.getString("firstname") + " " + rs.getString("surname");//puts the student full
name into a variable
                int e = rs.getInt("EventID");//puts the event ID into a variable
                String en = rs.getString("EventName");//puts the event name into a variable
                int h = rs.getInt("Hours");//puts the event hours into a variable
                model.addRow(new Object[]{s,name,e,en,h, portfolio});//creates an object for each row
                //displays a message to the user with a yes/no option
                int answer = JOptionPane.showConfirmDialog(btnDeleteParticipation,
                "Are you sure you want to delete this record?\n" + s + "-" + name + "-" + e + "-"+ en +"-"+h,
"DELETE PARTICIPATION RECORD",
                JOptionPane.YES_NO_OPTION);
                   if (answer == JOptionPane.NO_OPTION) {//if the user selects no
                   JOptionPane.showMessageDialog(null,"Delete Canceled");//displays a message to
explain that the delete has been canceled
                } else if(answer == JOptionPane.YES_OPTION) {

pm.deleteEntry(Integer.parseInt(txfDeleteEventID.getText()),Integer.parseInt(txfDeleteStuID.getText()));//
deletes event
                }
            }
        }
            txfDeleteEventID.setText("Enter Event ID"); //Clears the text field
            txfDeleteStuID.setText("Enter Student ID");//clears the text field
            model.addRow(new Object[]{}); //Adds row to the table
        }
        catch(SQLException e) //Catches any sql Errors
        {
            JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//displays an error message
        }
            //re-loads the admin participation table
            AdminParticipationTBL((am.getPortfolio("Catering",
currentAdmin)),(am.getPortfolio("Photography", currentAdmin)),(am.getPortfolio("av",
currentAdmin)),(am.getPortfolio("First aid", currentAdmin)));
    }
```

Updating tblParticipation (backend):

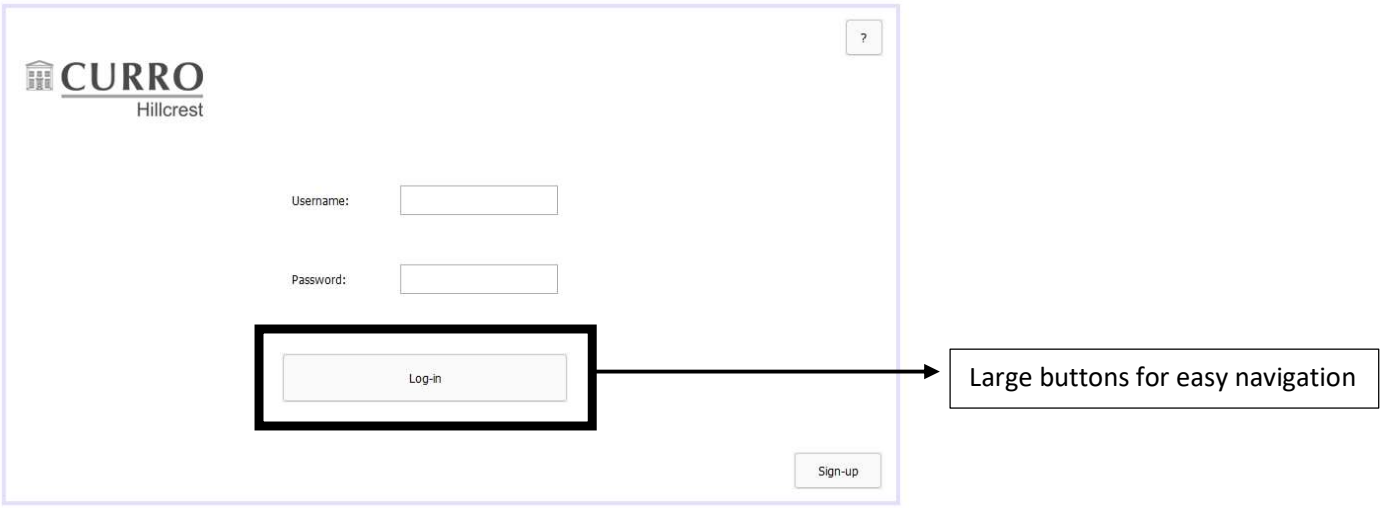
Deletes a participation record from the table

```
public String deleteEntry(int event, int student)
  {
     String sql = "DELETE * FROM tblParticipation WHERE EventID =" + event + " AND StudentID =" +
student; //Deletes all columbs where the row number is equal to what the user has entered
     try
     {
        d.updateQuery(sql); //Executes the query
        for (int i =0; i < size-1; i++) //Loops through the array
        {
           if(PartArr[i].getEventID() == event && PartArr[i].getStudentID() == student) //Checks if the
current array element in the loop matches the information given
           {
              shiftLeft(event-1); //Calls the shift left method to remove the event from the array
              return "You have successfully deleted the record from the database";
           }
        }
     }
     catch(SQLException e)
     {
        return "Error: Could not delete record";
     }
     return""; //Returns a string to indicate success or failure
  }
```

Defensive programming

1. Log-In

When a user clicks the log-in button, the username and password are checked against the text file to check that the account exists and that the username and passwords match. If the username and password do not match or the account does not exist, an error will appear.



Log-in code:

```
private void btnLogInActionPerformed(java.awt.event.ActionEvent evt) {
      try{
         boolean correct; //creates a boolean to check that the username and password are correct

         try (Scanner scFile = new Scanner (new File ("UsernamesAndPasswords.txt")) //Creates a
scanner for the text file

         ) {

            String username; //creates a string to hold the username

            int current; //creates an integer to hold the ID value of the student or admin who loggs in

            String password; //creates a string to hold the password

            String title;//creates a string to hold the title admin or student

            correct = false; //sets the value of the boolean to false

            while(scFile.hasNext()) //Checks each line in the text file against the users entry

            {

               String line = scFile.nextLine(); //creates a string from the next line from the textfile

               try (Scanner scLine = new Scanner (line).useDelimiter("#")) { //creates a scanner to scan the
current line of the textfile

                  current = scLine.nextInt(); //sets current ID to the student ID from the textfile

                  username = scLine.next();//sets the username to the one from the textfile
```

```
                password = scLine.next();//sets the password to the one from the texfile

                title = scLine.next();//sets the title to the one from the textfile

                //checks username and password against current line of the text file
                if(password.equals(txfPassword.getText()) && username.equals(txfUsername.getText())
&& title.equalsIgnoreCase("Student") )
                {
                   currentStu = current; //sets current student to the value of the ID from the textfile
                   correct = true; //sets the boolean value to true
                   if(currentStu != -1) //checks if the line from the text file was successfull in logging in
                   {
                      pnlLogIn.setVisible(false); //hides the log in pannel
                      tbdpStudent.setVisible(true);//shows the student tabbed pane
                      pnlHome.setVisible(true);//shows the student home page
                   }
                }
                else if (password.equals(txfPassword.getText()) &&
username.equals(txfUsername.getText()) && title.equalsIgnoreCase("Admin") ) // checks username and
password against current line of the text file
                {
                   currentAdmin = current; //sets the value of current admin to the ID from the text file

                   correct = true; //sets the boolean value to true

                   tbdpAdmin.setVisible(true); //shows the admin tabbed pane

                   pnlLogIn.setVisible(false);//hides the log in pannel

                }
                scLine.close();
             }

          }
          scFile.close();
       }

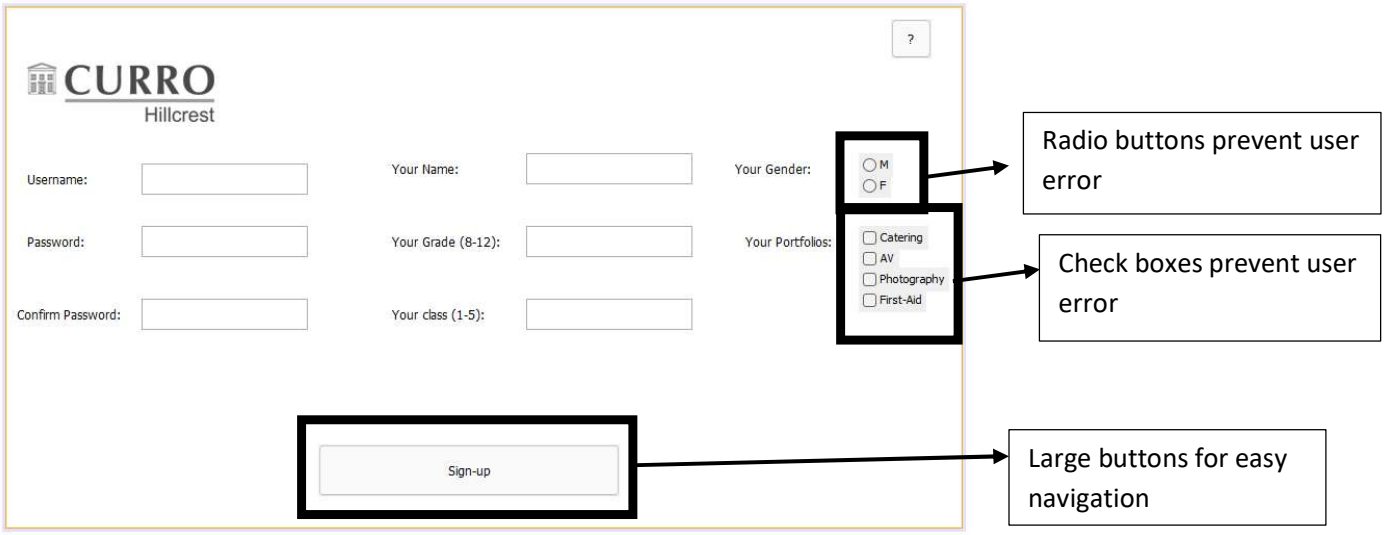
     if (correct == false) //if the log in was not successful
     {
        //Displayes error if incorrect cradentials are entered
       JOptionPane. showMessageDialog(null, "Incorrect Username or password", "", JOptionPane.
ERROR_MESSAGE);
     }
     }
     catch(HeadlessException | FileNotFoundException e){ //catches any errors
        //displays an error message if the error is cought
        JOptionPane. showMessageDialog(null, "Error, pleae try again", "", JOptionPane.
ERROR_MESSAGE);
```

```
}

}
```

2. Sign-up

When the user clicks a sign-up button, the information entered is validated and a new account is created in the text file and database if the data is valid. If the data entered is not valid, a specified error message will appear in a label next to the space where the invalid data has been entered.



Sign-up code:

```
private void btnSignUpStuActionPerformed(java.awt.event.ActionEvent evt) {

    clearErrorLables(); //clears the error lables by calling the clear error lables method

    try{ try ( //Creates a new log in

        PrintWriter outFile = new PrintWriter(new FileWriter("UsernamesAndPasswords.txt",true)))
{//creates a file writer

        char gender = ' '; //creates a variable to hold the gender

        if(rbnNewStudentM.isSelected()) //checks if the male gender radio button is selected
        {
            gender = 'M';//sets the gender to M
        }
        else if (rbnNewStudentF.isSelected()) //checks if the female gender radio button is selected
        {
            gender = 'F';//sets the gender to F
        }
```

```
        else
        {
            lblNewStudentGenderError.setText("Please select a gender");//sets the error label if no gender is selected
        }
        boolean password = false;//creates a boolean to check if the passwords entered match
        if (txfPassword2.getText().equals(txfConfirmPassword1.getText()))//Checks that both passwords are the same
        {
            password = true;//sets boolean to true if passwords match
        }
        else
        {
            lblNewStudentConfPasswordError.setText("Passwords Do Not Match"); //gives an error is passwords dont match
        }

        boolean validPortfolio = checkPortfolio();//presence check

        boolean validGrade = checkInt(txfNewStudentGrade, lblNewStudentGradeError,8,12);//type and range check

        boolean ValidClass = checkInt(txfNewStudentClass, lblNewStudentClassError, 1,5);//type and range check
        //presence, length and type check
        boolean validUserName = presenceAndLengthCheck(txfUsername1, lblNewStudentUsernameError) && typeCheck(txfUsername1, lblNewStudentUsernameError, ".:,;'");
        //presence and length check
        boolean validPassword = presenceAndLengthCheck(txfPassword2, lblNewStudentPasswordError);
        //presence, length and type check
        boolean validName = presenceAndLengthCheck(txfNewStudentName, lblNewStudentNameError) && typeCheck(txfNewStudentName, lblNewStudentNameError, " :,;'@#$%^&*()!+~") && SpaceCheck(txfNewStudentName, lblNewStudentNameError);
        //checks that all information is valid
        if((validUserName == true) && (validPassword == true) && (validName == true) && (gender != ' ') && (validGrade == true) && (ValidClass == true) && (validPortfolio == true) && (password == true))
        {
            String name = txfNewStudentName.getText();//sets the name string to the one entered by the user
            int grade = Integer.parseInt(txfNewStudentGrade.getText());//sets the grade integer to the one entered by the user
            int classPoint =Integer.parseInt(txfNewStudentClass.getText());//sets the class integer to the one entered by the user
            boolean catering = false;//creates a boolean to check if the user does catering
            boolean photography = false;//creates a boolean to check if the user does photography
            boolean firstAid = false;//creates a boolean to check if the user does first aid
            boolean av = false;//creates a boolean to check if the user does AV
```

```
            String lName = name.substring(name.lastIndexOf(" ")+1);//splits the last name from the first
name
            String fName = name.substring(0,name.lastIndexOf(" "));//splits the first name from the last
name

            if(cbxCatering.isSelected())//checks if the catering checkbox is selected
            {
               catering = true;//sets the catering boolean to true
            }
            if(cbxPotography.isSelected())//checks if the photography checkbox is selected
            {
               photography = true;//sets the photography boolean to true
            }
            if(cbxFirstAid.isSelected())//checks if the first aid checkbox is selected
            {
               firstAid = true;//sets the first aid boolean to true
            }
            if(cbxAV.isSelected()) //checks if the AV checkbox is selected
            {
               av = true; //sets the AV boolean to true
            }

            try {

               sm.add(fName,lName , grade, classPoint, gender, catering, photography, av, firstAid);//adds
a student

               String line = (sm.getLast())+ "#" + txfUsername1.getText() + "#" + txfPassword2.getText() +
"#Student"; //creates a new line in the text file

               currentStu = (sm.getLast());//sets current ctudent to the value of the next ID

               outFile.println(line);//adds the line to the textfile

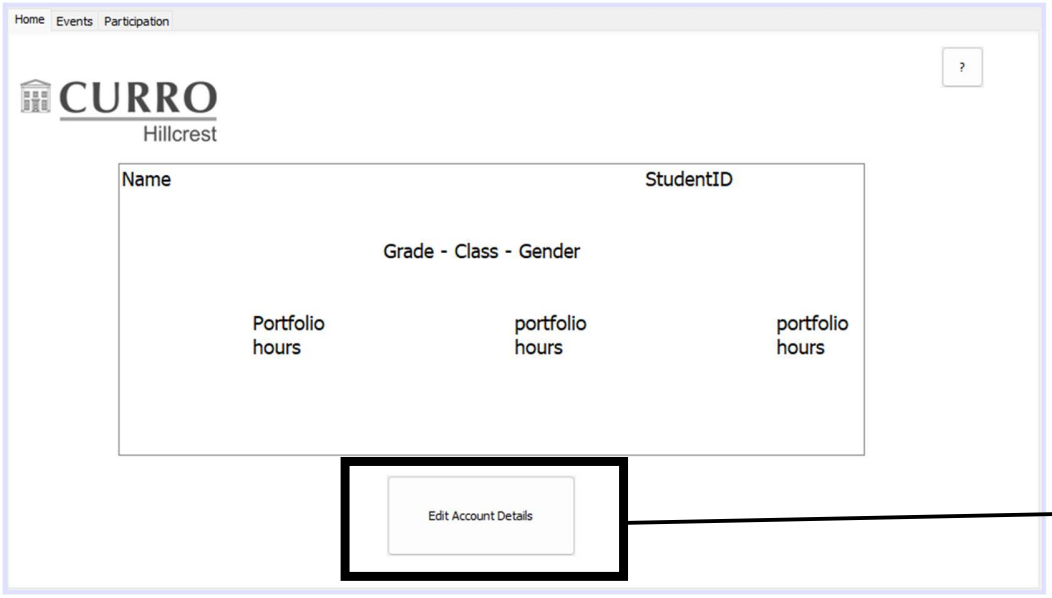
            } catch (SQLException ex) {//catches any errors
               System.out.println("Could not create an account: Please try again");
            }
            try {
         if(sm.getCurrent(currentStu) != null)//checks that a new account has been created
         {
            pnlSignUp.setVisible(false);//hides the sign up page
            tbdpStudent.setVisible(true);//shows the student tabbed pane
         }
      } catch (SQLException ex) {//catches any erors
               System.out.println("Could not create an account: Please try again");//gives an error message
      }
         }  }
      }
       catch (IOException ex) { //cateches errors
```

```
        JOptionPane. showMessageDialog(null, "Please ensure all fields are filled out", "", JOptionPane.
ERROR_MESSAGE);

    }
  }
```

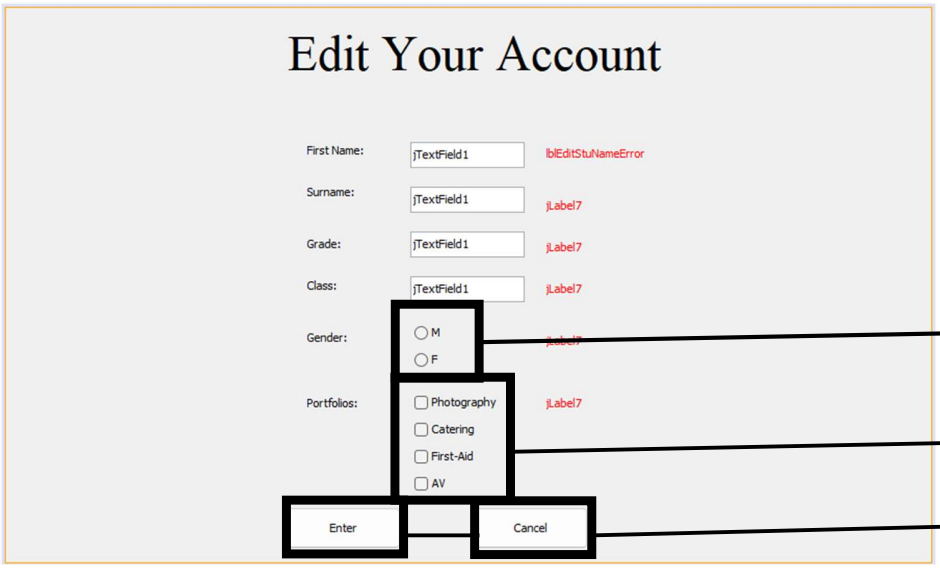
3. Home

The large action button allows the user to easily navigate to the edit account screen.



4. Edit account details

When the user clicks the enter button, the information entered is validated and the account updated in the database if the data is valid. If the data entered is not valid, a specified error message will appear in a label next to the space where the invalid data has been entered.



Edit account code:

```
private void btnEditStuEnterActionPerformed(java.awt.event.ActionEvent evt) {
        clearErrorLables(); //clears the error lables by calling the clear error lables method
        char gender = ' ';//creates a variable to hold the gender
        if(rbnEditStuGenderM.isSelected())//checks if the radio button is selected
        {
            gender = 'M';//sets the gender character to M
        }
        else if (rbnEditStuGenderF.isSelected())//checks if the radio button is selected
        {
            gender = 'F';//sets the gender character to F
        }
        else//if neither radio buttons were selected
        {
            lblEditStuGenderError.setText("Please select a gender");//displays an error message
        }

        boolean validPortfolio = checkEditPortfolio();//checks that the portfolio selection is valid
        boolean validGrade = checkInt(txfEditStuGrade, lblEditStuGradeError,8,12);//Type,presence and
range check
        boolean validClass = checkInt(txfEditStuClass, lblEditStuClassError, 1,5);//Type,presence and range
check
        //precence, length and type check
        boolean validName = presenceAndLengthCheck(txfEditStuName, lblEditStuNameError) &&
typeCheck(txfEditStuName, lblEditStuNameError, " -:,;'@#$%^&*()!+~");
        //precence, length and type check
        boolean validSurname = presenceAndLengthCheck(txfEditStuSurname, lblEditStuSurnameError)
&& typeCheck(txfEditStuSurname, lblEditStuSurnameError, " -:,;'@#$%^&*()!+~");
        //confirms that all the data is valid
        if((validName == true) && (validSurname == true) && (gender != ' ') && (validGrade == true) &&
(validClass == true) && (validPortfolio == true))
```

```
        {
            String name = txfEditStuName.getText();//sets the name to the value entered by the user
            String surname = txfEditStuSurname.getText();//sets the surname to the value entered by the user
            int grade = Integer.parseInt(txfEditStuGrade.getText());//sets the grade to the value entered by the user and converts it to an integer
            int classPoint =Integer.parseInt(txfEditStuClass.getText());//sets the class to the value entered by the user and converts it to an integer

            boolean catering = false;//creates a boolean to hold catering
            boolean photography = false;//creates a boolean to hold photography
            boolean firstAid = false;//creates a boolean to hold first aid
            boolean av = false;//creates a boolean to hold AV

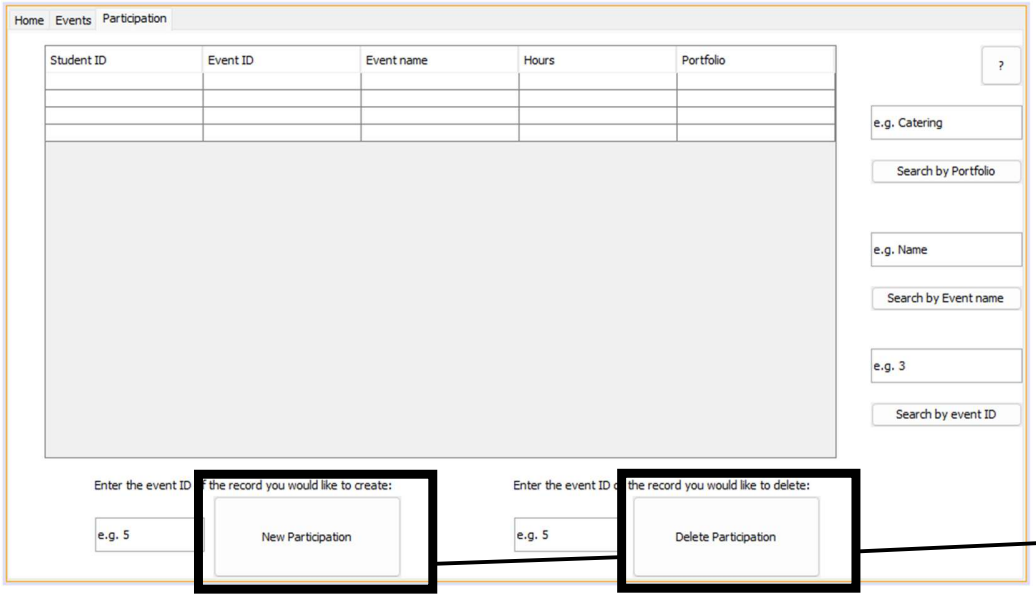
            if(cbxCatering1.isSelected())//checks if the catering checkbox is ticked
            {
                catering = true;//sets the catering boolean to true
            }
            if(cbxPhotography1.isSelected())//checks if the photography checkbox is ticked
            {
                photography = true;//sets the photography boolean to true
            }
            if(cbxFirstAid1.isSelected())//checks if the first aid checkbox is ticked
            {
                firstAid = true;//sets the first aid boolean to true
            }
            if(cbxAV1.isSelected())//checks if the AV checkbox is ticked
            {
                av = true;//sets the AV boolean to true
            }

            try {
                //updates the student record in the database
                dbman.updateQuery("UPDATE tblStudents SET firstName = '" + name+ "',Surname = '" + surname+ "', Grade = " + grade+  ",Class = " + classPoint + ", Gender = '" + gender + "', Catering = " + catering + ", Photography = " + photography + ", AV = " + av + ", FirstAid = " + firstAid + " WHERE StudentID = " + currentStu + ";");
                dlgEditStudent.setVisible(false);//hides the dialog
                tbdpStudent.setVisible(true);//shows the student tabbed pane
            } catch (SQLException ex) {//catches any SQL errors
                Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
            }
        }
        else//if the data was not valid
        {
             JOptionPane. showMessageDialog(null, "UNABLE TO UPDATE!", "", JOptionPane. ERROR_MESSAGE);//displays an error message
        }
        try {
            if(currentStu != -1)//checks that the current student has a value
```

```
            {
              txaStudent.setText(sm.getCurrent(currentStu));//calls a method to show the student details on the home page
            }
        } catch (SQLException ex) {//catches SQL errors
            Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
        }
    }
```

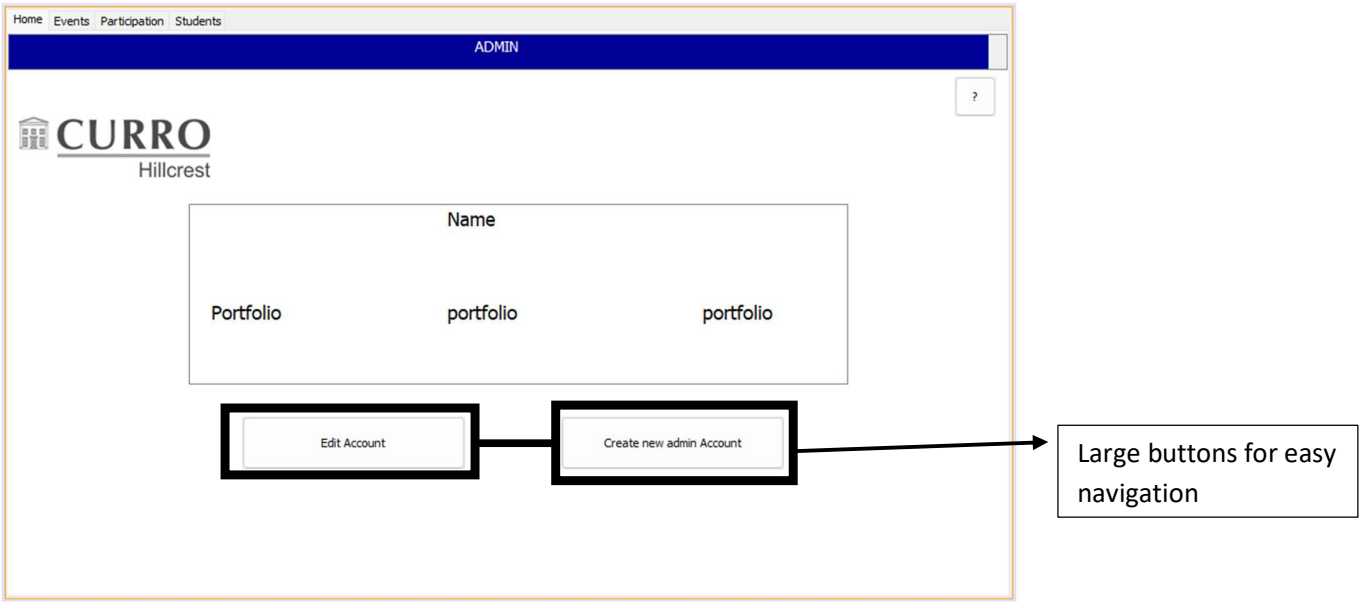
5. Participation

Large action buttons allow the user to easily perform tasks on this screen.



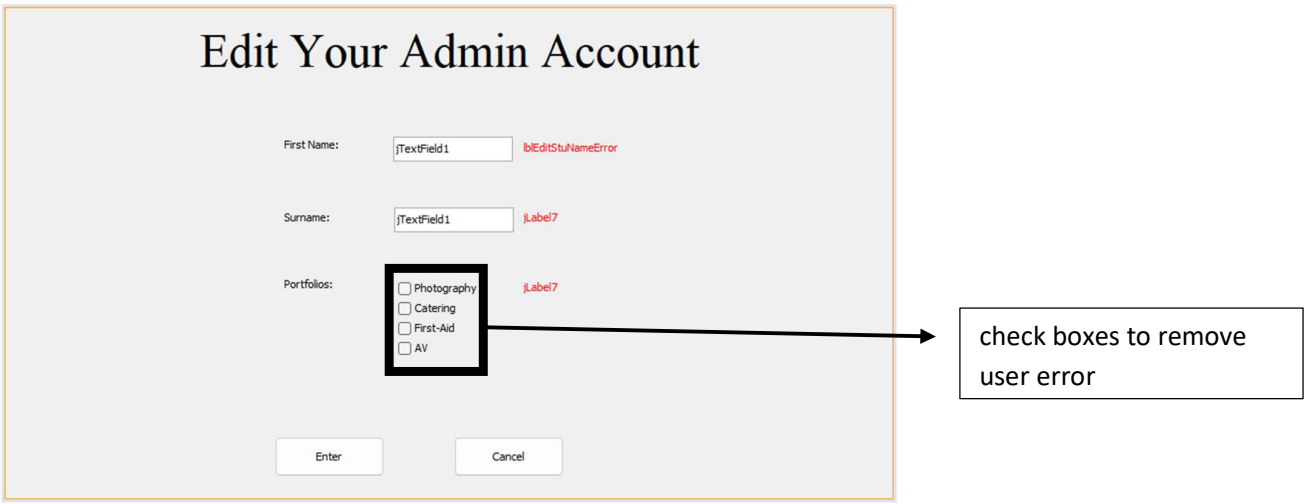
6. Home Admin

Large action buttons allow the user to easily navigate to other tasks from this screen.



7. Edit Admin account

When the user clicks the enter button, the information entered is validated and the account is updated in the database if the data is valid. If the data entered is not valid, a specified error message will appear in a label next to the space where the invalid data has been entered.



Edit Admin account code:

```
private void btnEditAdminEnterActionPerformed(java.awt.event.ActionEvent evt) {
    clearErrorLables(); //clears the error lables by calling the clear error lables method
    boolean validPortfolio = checkPortfolioEditAdmin();
```

```
        boolean validName = presenceAndLengthCheck(txfEditAdminName, lblEditAdminNameError) &&
typeCheck(txfEditAdminName, lblEditAdminNameError, "-:,;'@#$%^&*()!+~ ");
        boolean validSurname = presenceAndLengthCheck(txfEditAdminSurname,
lblEditAdminSurnameError) && typeCheck(txfEditAdminSurname, lblEditAdminSurnameError, "
:,;'@#$%^&*()!+~");
        if((validName == true) && (validSurname == true) && (validPortfolio == true));
        {
            String name = txfEditAdminName.getText();
            String surname = txfEditAdminSurname.getText();
            boolean catering = false;
            boolean photography = false;
            boolean firstAid = false;
            boolean av = false;

            if(cbxCateringAdmin.isSelected())
            {
                catering = true;
            }
            if(cbxPhotographyAdmin.isSelected())
            {
                photography = true;
            }
            if(cbxFirstAidAdmin.isSelected())
            {
                firstAid = true;
            }
            if(cbxAVAdmin.isSelected())
            {
                av = true;
            }

            try {
                dbman.updateQuery("UPDATE tblAdmins SET AdminName = '" + name+ "',AdminSurname = '"
+ surname+ "', Catering = " + catering + ", Photography = " + photography + ", AV = " + av + ", [First Aid]
= " + firstAid + " WHERE AdminID = " + currentAdmin + ";");
                dlgEditAdmin.setVisible(false);
                tbdpAdmin.setVisible(true);
            } catch (SQLException ex) {
                System.out.println("Update Error, please try again");
            }
        }

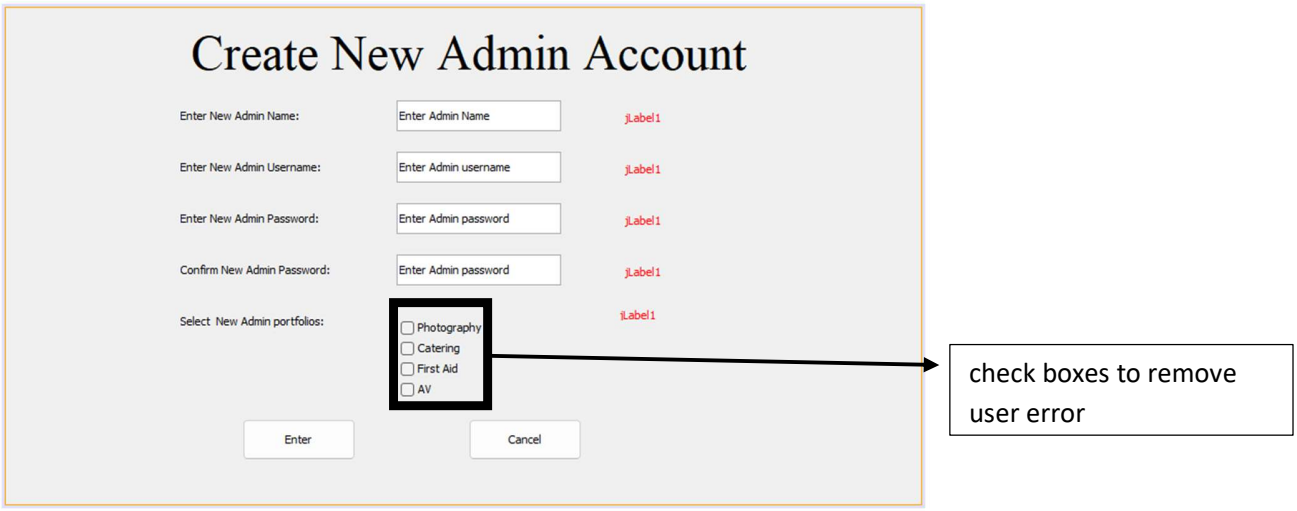
        if(currentAdmin != -1)//checks that current admin has a value
        {
            try {
                txaAdminDetails.setText(am.getCurrent(currentAdmin));//calls a method to show the admin
details on the home page
            } catch (SQLException ex) {//catches any SQL errors
                Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
```

```
        }
    }

}
```

8. New Admin account

When the user clicks a enter button, the information entered is validated and a new account is created in the text file and database if the data is valid. If the data entered is not valid, a specified error message will appear in a label next to the space where the invalid data has been entered.



New admin account code:

```
private void btnEditAdminEnter1ActionPerformed(java.awt.event.ActionEvent evt) {
        clearErrorLables(); //clears the error lables by calling the clear error lables method
        try{ try ( //Creates a new log in
            PrintWriter outFile = new PrintWriter(new FileWriter("UsernamesAndPasswords.txt",true)))
{//creates a file writer
            boolean password = false;//creates a boolean to check that the passwords match
            if (txfNewAdminPassword.getText().equals(txfNewAdminConfirmPassword.getText()))//Checks
that both passwords are the same
            {
                password = true;//sets the password boolean to true
            }
            else//if the passwords dont match
            {
                lblNewAdminConfirmPasswordError.setText("Passwords Do Not Match");//sets the lable to
display an error
            }
            boolean validPortfolio = checkPortfolioAdmin();//checks that the portfolio entries are valid
            //precence, length and type check
```

```
        boolean validUserName = presenceAndLengthCheck(txfNewAdminUsername,
lblNewAdminUsernameError) && typeCheck(txfNewAdminUsername, lblNewAdminUsernameError, ":,;'");
        //precence, length and type check
        boolean validPassword = presenceAndLengthCheck(txfNewAdminPassword,
lblNewAdminPasswordError);
        //precence, length and type check
        boolean validName = presenceAndLengthCheck(txfNewAdminName, lblNewAdminNameError)
&& typeCheck(txfNewAdminName, lblNewAdminNameError, " :,;'@#$%^&*()!+~") &&
SpaceCheck(txfNewAdminName, lblNewAdminNameError);
        //confirms that all the data is valid
        if((validUserName == true) && (validPassword == true) && (validName == true) &&
(validPortfolio == true) && (password == true))
        {
            String name = txfNewAdminName.getText();//sets the name to the value entered by the user
            boolean catering = false;//creates a boolean to check if the admin oversees catering
            boolean photography = false;//creates a boolean to check if the admin oversees photography
            boolean firstAid = false;//creates a boolean to check if the admin oversees first aid
            boolean av = false;//creates a boolean to check if the admin oversees AV
            String lName = name.substring(name.lastIndexOf(" ")+1);//puts the surname into a string
            String fName = name.substring(0,name.lastIndexOf(" "));//puts the first name into a string

            if(cbxNewAdminCatering.isSelected())//checks if the catering checkbox is selected
            {
                catering = true;//sets the boolean to true
            }
            if(cbxNewAdminPhotography.isSelected())//checks if the photography checkbox is selected
            {
                photography = true;//sets the boolean to true
            }
            if(cbxNewAdminFirstAid.isSelected())//checks if the first aid checkbox is selected
            {
                firstAid = true;//sets the boolean to true
            }
            if(cbxNewAdminAv.isSelected())//checks if the AV checkbox is selected
            {
                av = true;//sets the boolean to true
            }

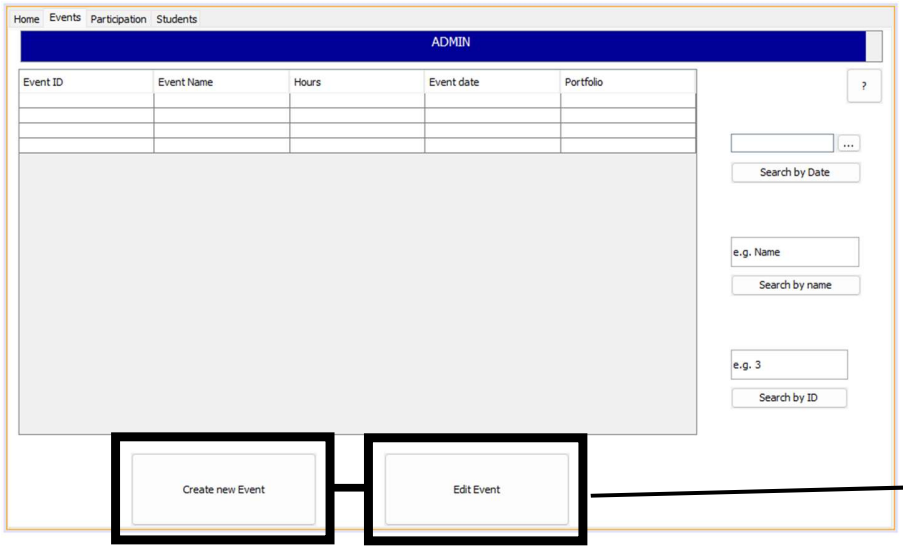
            try {

                am.add(fName,lName , catering, photography, av, firstAid);//adds a new admin
                //creates a line to be added to the text file
                String line = (am.getSize()-1) + "#" + txfNewAdminUsername.getText() + "#" +
txfNewAdminPassword.getText() + "#Admin";
                outFile.println(line);//adds the line to the textfile
                JOptionPane.showMessageDialog(null,"New Admin Created");//displays a message to
indicate success
                dlgNewAdmin.setVisible(false);//hides the dialog
            } catch (SQLException ex) {//catches any SQL errors
```

```
                Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
            }
        } }
    }
     catch (IOException ex) {//catches any errors
        //displays an error
        JOptionPane. showMessageDialog(null, "Please ensure all fields are filled out", "", JOptionPane.
ERROR_MESSAGE);
    }
  }
```

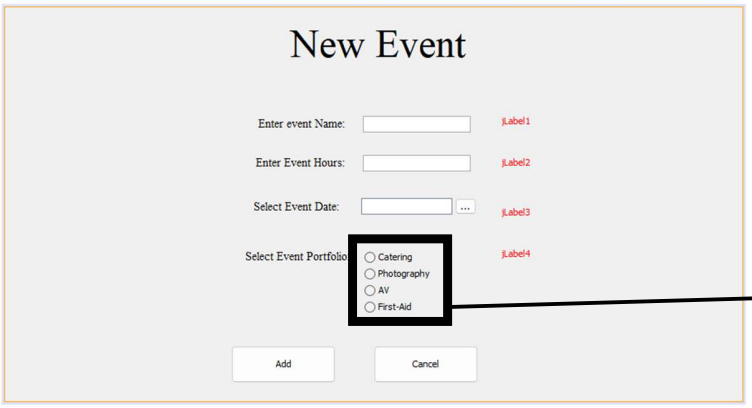
9. Events Admin

Large action buttons allow the user to easily navigate to other tasks from this screen.



10. Create new event

When the user clicks a enter button, the information entered is validated and a new event is created in the database if the data is valid. If the data entered is not valid, a specified error message will appear in a label next to the space where the invalid data has been entered.



Radio buttons remove user error

New event code:

```
private void btnNewEventEnterActionPerformed(java.awt.event.ActionEvent evt) {

        clearErrorLables(); //clears the error lables by calling the clear error lables method
        String name;
        //Presence, Length and Type check
        boolean validName = presenceAndLengthCheck(txfNewEventName, lblNewNameError) &&
typeCheck(txfNewEventName, lblNewNameError, " :,;'"");
        //Precence check
        boolean validDate = CheckDate(lblNewDateError);

        boolean catering = false;//sets catering boolean to false

        boolean av = false;//sets AV boolean to false

        boolean photography = false;//sets photography boolean to false

        boolean firstAid = false;//sets first aid boolean to false

        boolean selected = false;//sets boolean selected to false, to indicate if any radio buttons have been
selected

        //Precence check
        if(rbnNewEventCatering.isSelected()) //checks if the catering radio button is selected
        {
            catering = true; //sets catering boolean to true

            selected = true; //sets selected boolean to true
        }

        else if (rbnNewEventAV.isSelected()) //checks if the AV radio button is selected
        {
            av = true; //sets AV boolean to true

            selected = true; //sets selected boolean to true
```

```
        }

        else if (rbnNewEventPhotography.isSelected()) //checks if the Photography radio button is selected
        {
            photography = true;//sets photography boolean to true

            selected = true; //sets selected boolean to true
        }

        else if (rbnNewEventFirstAid.isSelected()) //checks if the first aid radio button is selected
        {
            firstAid = true;//sets first aid boolean to true

            selected = true; //sets selected boolean to true
        }

        else
        {
            lblNewPortfolioError.setText("Select a portfolio"); //sets the error label to display an error
        }

        //Type check and range check
        String hours = checkHours(txfNewEventHours, lblNewHoursError); //type check

        double h = Double.parseDouble(hours); //converts hours from a string to a double

        if(validName == true &&  validDate == true && hours != null && selected == true) //checks that all
the data is valid
        {
            try {

                name = txfNewEventName.getText(); //sets name variable to the text in the text feild

                LocalDate dateEvent = dpkDate.getDate();//Sets the date to the date entered

                em.add(name,h ,dateEvent, catering, photography, firstAid, av); //adds the new event

                dlgEvents.setVisible(false); //Hides the event dialog

                JOptionPane.showMessageDialog(null, "Event successfully added"); //Displays a message to
indicate success
                //refreshes the table
                AdminEventsTBL((am.getPortfolio("Catering", currentAdmin)),(am.getPortfolio("Photography",
currentAdmin)),(am.getPortfolio("av", currentAdmin)),(am.getPortfolio("First aid", currentAdmin)));

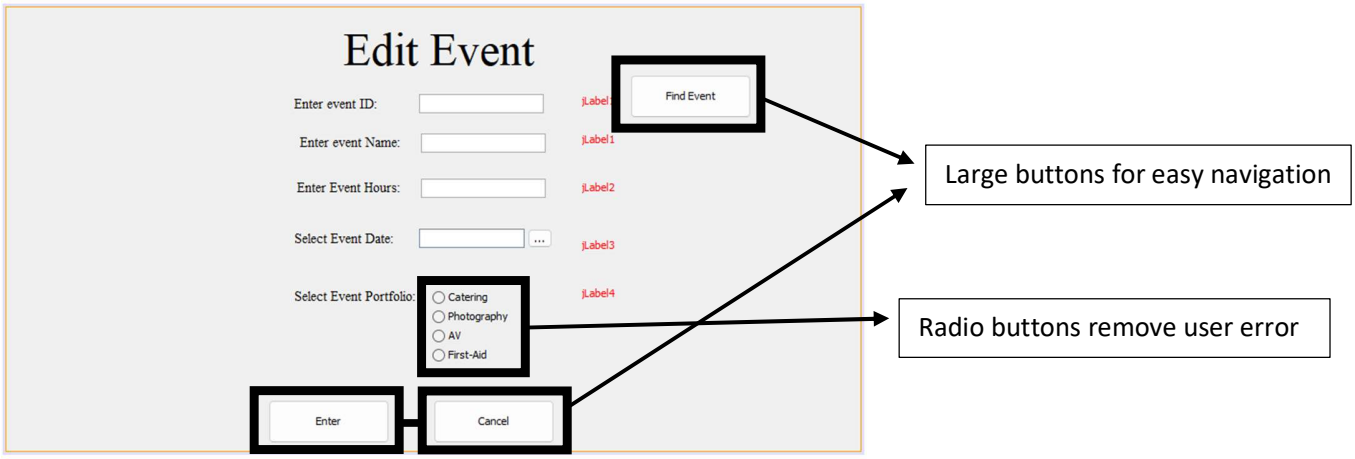
            } catch (SQLException ex) { //catches any sql errors

                System.out.println("SQL ERROR");//prints an error message
```

```
        }
    }
}
```

11. Edit Event

When the user clicks the enter button, the information entered is validated and the event is updated in the database if the data is valid. If the data entered is not valid, a specified error message will appear in a label next to the space where the invalid data has been entered.



Edit event code:

```
private void btnEditEventEnterActionPerformed(java.awt.event.ActionEvent evt) {
        clearErrorLables(); //clears the error lables by calling the clear error lables method
        String name; //holds the event name
        //Presence, Length and Type check
        boolean validName = presenceAndLengthCheck(txfEditEventName, lblEditNameError) &&
typeCheck(txfEditEventName, lblEditNameError, " :,;'");
        //Precence check
        boolean validDate = CheckEditDate(lblEditDateError);
        boolean catering = false;//holds the boolean for if the event is for catering
        boolean av = false;//holds the boolean for if the event is for AV
        boolean photography = false;//holds the boolean for if the event is for photography
        boolean firstAid = false;//holds the boolean for if the event is for first aid
        boolean selected = false;//checks that a portfolio has been selected

        //Precence check
        if(rbnEditEventCatering.isSelected())//checks if the catering radio button is selected
        {
            catering = true;//sets the catering boolean to true
```

```
            selected = true;//sets the selected boolean to true
        }
        else if (rbnEditEventAV.isSelected())//checks if the AV radio button is selected
        {
            av = true;//sets the AV boolean to true

            selected = true;//sets the selected boolean to true
        }
        else if (rbnEditEventPhotography.isSelected())//checks if the Photography radio button is selected
        {
            photography = true;//sets the photography boolean to true

            selected = true;//sets the selected boolean to true
        }
        else if (rbnEditEventFirstAid.isSelected())//checks if the first aid radio button is selected
        {
            firstAid = true;//sets the first aid boolean to true

            selected = true;//sets the selected boolean to true
        }
        else//if none of the buttons are selected
        {
            lblEditPortfolioError.setText("Select a portfolio");//sets the lable to an error
        }

        //Type check and range check
        String hours = checkHours(txfEditEventHours, lblEditHoursError);

        double h = Double.parseDouble(hours);//converts hours to a double

        //checks that all the values are valid
        if(validName == true &&  validDate == true && hours != null && selected == true)
        {
            try {
                name = txfEditEventName.getText();//sets name to the name entered by the user

                LocalDate dateEvent = dpkEditDate.getDate();//gets the date from the date picker
                //creates the update event query
                String sql = ("UPDATE tblEvents SET EventName = '" + name+ "',Hours = " + hours+ ",
EventDate = #" + dateEvent + "#, Catering = " + catering + ", Photography = " + photography +  ", FirstAid
= " + firstAid + ", AV = " + av + " WHERE EventID = " + txfEditEventID.getText() + ";");

                dbman.updateQuery(sql);//executes the query

                dlgEditEvent.setVisible(false);//hides the dialog

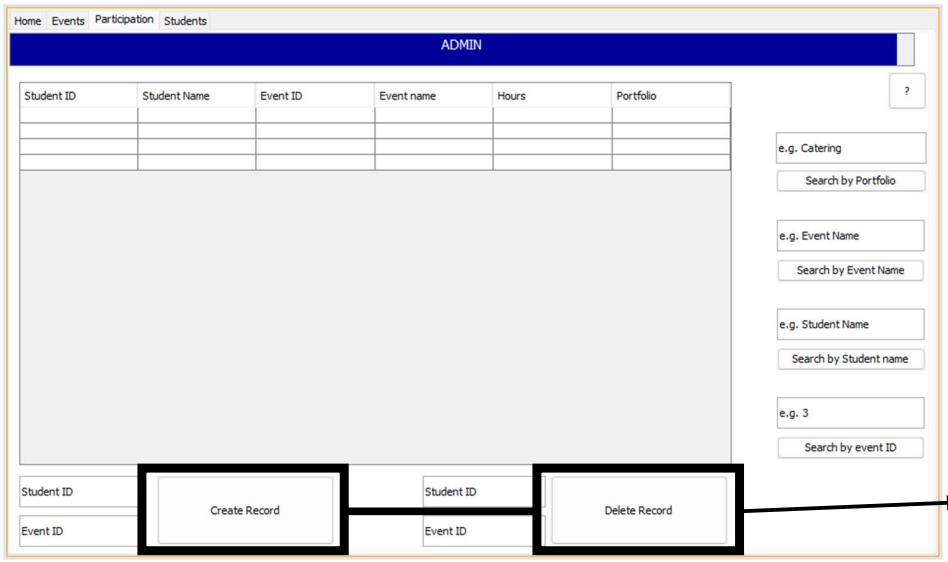
                JOptionPane.showMessageDialog(null, "Event successfully Updated");//shows a message to
indicate a successfull entry
```

```
        } catch (SQLException ex) {//catches SQL errors
            System.out.println("SQL ERROR");//gives an error
        }
    }
    //reloads the events table for the admin
    AdminEventsTBL((am.getPortfolio("Catering", currentAdmin)),(am.getPortfolio("Photography",
currentAdmin)),(am.getPortfolio("av", currentAdmin)),(am.getPortfolio("First aid", currentAdmin)));

}
```

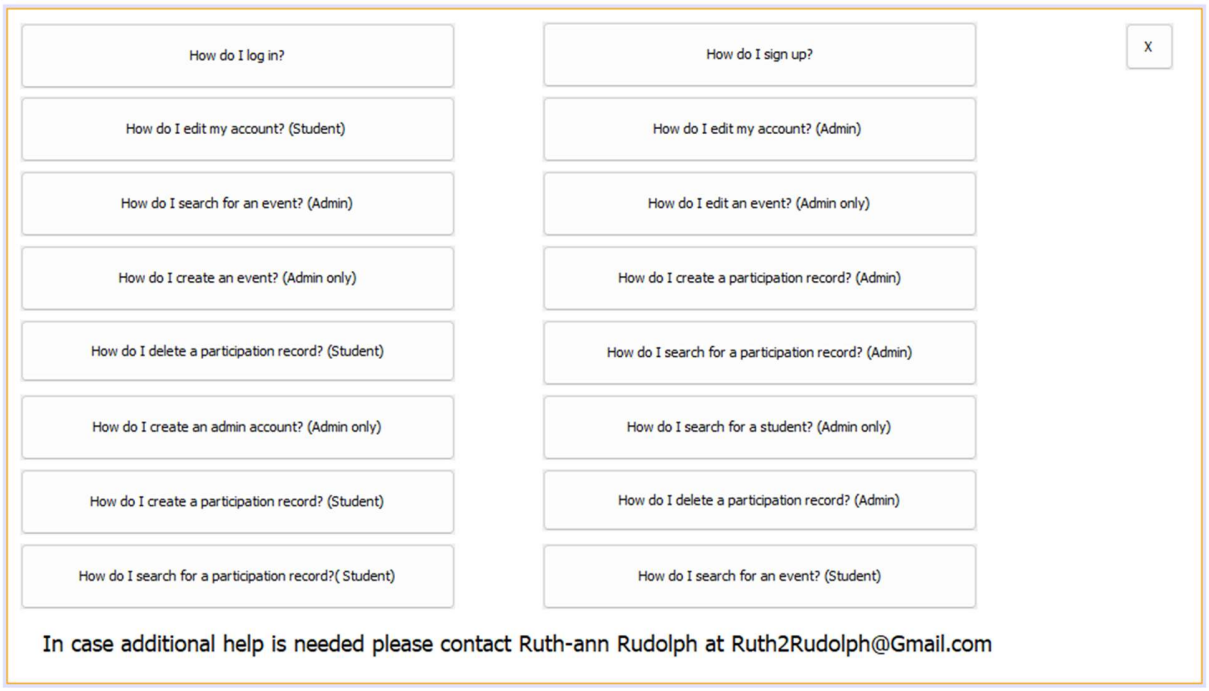
12. Participation Admin

Large action buttons allow the user to easily perform tasks on this screen.



13. Help

All help buttons are large for easy reading and navigation, and the help screen consists only of buttons to limit user input.



Externally sourced code

DBService

Petri Mostert

```
/*
 * To change this license header, choose License Headers in Project Properties.
 * To change this template file, choose Tools | Templates
 * and open the template in the editor.
 */

package Objects;
//Code adapted from Petri Mostert, previous teacher
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

/**
 * Establishes a connection to the database
 * @author ruth2
 */
```

```
public class DBService {
    private String url = "jdbc:ucanaccess://2022PAT.accdb";
    private Connection conn = null;

    /**
     * Establishes a connection to the database or throws an error
     */
    public DBService()
    {
        try
        {
            conn = DriverManager.getConnection(url); // Tries to establish a connection using the connection
string
        }
        catch(SQLException e) // Throws SQLException if connection cannot be established.
        {

        }
    }

    /**
     * Sends and receives SELECT queries between DB and Java
     * @param sql the SQL query as a String
     * @return the result of the SQL query
     * @throws SQLException
     */
    public ResultSet queryDatabase(String sql) throws SQLException    {
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet set = stmt.executeQuery(); // Executes the query received from paramater
        return set;
    }

    /**
     *Updates the DB using queries in Java
     * @param sql the SQl query as a String
     * @throws SQLException
     */
    public void updateQuery(String sql) throws SQLException
    {
        PreparedStatement stmt = conn.prepareStatement(sql);
        stmt.execute(); // Executes the query received from paramater
    }
}
```

Play videos (help screen)

Source: https://www.youtube.com/watch?v=Nm-e9aWPH8U

```
final URI url;
```

```
        try {
            url = new URI("https://youtu.be/7diFPRxfVFs");//creates a link to the youtube video
            play(url);//Attempts to open the browser and play the video
        } catch (URISyntaxException ex) {//catches any errors
            Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
        }


private static void play(URI url) //Code Adapted from https://www.youtube.com/watch?v=Nm-e9aWPH8U
    {
        if(Desktop.isDesktopSupported())//checks if the action can be performed on the pc
        {
            try{
                Desktop.getDesktop().browse(url);//opens the b,video in the browser
            }
            catch(IOException e)//catches any errors
                {
                    System.out.println("Could not play video");//gives an error to indicate that the video could
not be played
                }
        }
    }
```

Data validation

adapted from Exploring IT: Java programming, Grade 12 textbook

```
private boolean typeCheck(javax.swing.JTextField txf, javax.swing.JLabel lbl, String validStr)
    {
        if(txf.getText().isEmpty())//checks if the text field is empty
        {
            lbl.setText("!Required");//gives an error in the label
            return false;//returns false if invalid
        }
        String txfText = txf.getText();//puts the text field value into a string
        for (int i = 0; i < txfText.length(); i++)//loops through the string untill the end
        {
            if(!Character.isLetterOrDigit(txfText.charAt(i))&& !(validStr.indexOf(txfText.charAt(i))>=0))//checks
that each character is of the correct type
            {
                lbl.setText("Invalid character(s)" + txfText.charAt(i));//gives an invalid character error
                return false;//returns false if invalid
            }
        }
        return true;//returns true if all is valid
    }

    private boolean presenceAndLengthCheck(javax.swing.JTextField txf, javax.swing.JLabel lbl)
    {
```

```
        if ((txf.getText()).length() <= 1)//checks that the length is longer than 1 character
        {
            lbl.setText("Enter at least two characters");//gives an error to indicate that it is not long enough
            return false;//returns false to indicate invalid data
        }
        else
        {
            return true;//returns true to indicate valid data
        }
    }

    private String checkHours(javax.swing.JTextField txf, javax.swing.JLabel lbl)
    {
        String temp = "";//creates a string to hold the number of hours
        String txfText = txf.getText();//puts the value entered by the user into a string
        if(txf.getText().equals(""))//checks if the string from the user is empty
        {
            lbl.setText("Enter a number");//sets the label to an error
        } else//if the string is not empty
        {
            boolean test = true;//creates a boolean to check if each character is valid
            int i = 0;//creates a variable to count how many characters have been checked
            while (test == true && i < txfText.length())//loops untill the end of the string is reached or a
character is not valid
            {
                if (Character.isDigit(txfText.charAt(i)) || (txfText.charAt(i)) == '.')//checks if the character is a
digit or a full stop
                {
                    temp += txfText.charAt(i);//adds the character to the string
                    i++;//adds 1 to i
                } else //if the character is not a digit
                {
                    test = false;//sets test to false, to end the loop
                    lbl.setText("Hours number must only contain digits or a full stop");//sets the lable to an error
                    return null;//returns null to indicate that the data is not valid
                }
            }
            String hours = temp;//makes a new string to contain temp
            double num;//creates a double to hold the number of hours
            int lastDigitPos = hours.length() - 1;//creates an integer to hold the length of the string
            int checkDigit = Character.getNumericValue(hours.charAt(lastDigitPos));//checks the last
character of the string
            num = Double.parseDouble(hours);//converts the string to a double
            if(num < 0)//checks that the number is bigger than 0
            {
                lblNewHoursError.setText("Hours must be more than zero");//sets the lable to an error
                return null;//returns null to indicate that the data is not valid
            }
            else if (num > 12)//checks that the number is less than 12
```

```
        {
            lblNewHoursError.setText("Hours must be less than 12");//sets the lable to an error
            return null;//returns null to indicate that the data is not valid
        }
        else
        {
            return hours;//returns the number of hours if all data is valid
        }
    }
    return null;//returns null to indicate that the data is not valid
}
```

Explanation of Critical Algorithms

Uploading events into the array

This algorithm is essential for the easy access and display of the events for the user.

```
sql ← "SELECT * FROM tblEvents;"
s ← query Database (sql)
WHILE s has next
    n ← s.getInt("EventID")
    name ← s.getString("EventName")
    h ← s.getDouble("Hours")
    date ← s.getDate("EventDate")
    localDate ← date.toLocalDate()
    catering ← s.getBoolean("Catering")
    photography ← s.getBoolean("Photography")
    firstAid ← s.getBoolean("FirstAid")
    AV ← s.getBoolean("AV")
    EventArr [size] ← new Event (n, name, h, localDate, catering, photography, firstAid, AV)
    size++
END WHILE
```

Adding event records into the array and database

This algorithm is essential for the addition of new events to the database and array.

```
x ← size -1

num ← EventArr[x] get Event Num + 1

EventArr [size] ← new Event(num, n, h, d, c, p, f, a)

Insert Event (EventArr[size])

size++


sql ← "INSERT INTO tblEvents([EventName], [Hours], [EventDate], [Catering], [Photography], [FirstAid], [AV]) VALUES ('"  + f.getEventName() + "','" + f.getNumHours() + "','" + f.getDate() + "','" + f.isCatering() + "','" + f.isPhotography() + "','" + f.isFirstAid() + "','" + f.isAudioVisual()+"');"

update Query (sql)
```

Uploading participation records into the array

This algorithm is essential for the easy access and display of the participation records for the user.

```
    sql ← "SELECT * FROM tblParticipation;"

    s ← query Database (sql)

    WHILE s has next

      sn ← s.getInt("StudentID")

      n ← s.getInt("EventID")

      PartArr [size] ← new Participation (sn, n)

      size++

    END WHILE
```

Adding participation records into the array and database

This algorithm is essential for the addition of new participation records to the database and array.

```
PartArr [size] ← new Participation (studentID, eventID)
```

```
size++
insert Participation (eventID, studentID);

sql ← "INSERT INTO tblParticipation([EventID],[StudentID]) VALUES ('"+ e + "','" + s+ "');"
Update Query (sql)
```

Uploading student records into the array

This algorithm is essential for the easy access and display of the student records for the user.

```
    sql ← "SELECT * FROM tblStudents;"
    s ← d.queryDatabase(sql)
    WHILE s has next
      n ← s.getInt("StudentID")
      name ← s.getString("FirstName")
      surName ← s.getString("Surname")
      grade ← s.getInt("Grade")
      classPoint ← s.getInt("Class")
      g ← (s.getString("Gender")) char At(0)
      catering ← s.getBoolean("Catering")
      photography ← s.getBoolean("Photography")
      AV ← s.getBoolean("AV")
      firstAid ← s.getBoolean("FirstAid")
      StudentArr [size] ← new Student(n, name ,surName, grade, classPoint, g, catering,
photography, AV, firstAid)
      size++
    END WHILE
```

Advanced techniques

Complex multi-table database with complex joins

```
public void ParticipationTBL(boolean c, boolean p, boolean a, boolean f){
        try
        {

            DefaultTableModel dtm = (DefaultTableModel)
tblParticipation.getModel();//Sets the default table model

            dtm.setRowCount(0); //Sets the row count to 0

            tblParticipation.removeAll(); //Cleares the table

            model = (DefaultTableModel)tblParticipation.getModel();//Sets the
default table model

            //Selects the participation records for the student currently
logged in
            ResultSet rs = dbman.queryDatabase("SELECT DISTINCT
tblStudents.StudentID, tblEvents.EventID, tblEvents.EventName,
tblEvents.hours, tblEvents.Catering, tblEvents.Photography, tblEvents.AV,
tblEvents.FirstAid FROM tblEvents, tblStudents, tblParticipation Where
tblParticipation.StudentID = tblStudents.StudentID AND
tblParticipation.EventID = tblEvents.EventID AND tblParticipation.StudentID
=" + currentStu);

            while(rs.next()) //Creates a line in the JTable for every
database entry
            {
                String portfolio = ""; //Creates a string to store the
portfolio of the event
                if(rs.getBoolean("Catering") == true)//Checks if the event of
the participation record is for catering
                {
                    portfolio = "Catering"; //Sets the portfolio string to
catering
                }
                else if(rs.getBoolean("Photography") == true) //Checks if the
event of the participation record is for photography
                {
                    portfolio = "Photography"; //Sets the portfolio string to
photography
                }
                else if(rs.getBoolean("FirstAid") == true)//Checks if the
event of the participation record is for first aid
                {
                    portfolio = "First Aid"; //Sets the portfolio string to
first aid
```

```
                }
                else if(rs.getBoolean("AV") == true) //Checks if the event of
the participation record is for AV
                {
                    portfolio = "AV"; //Sets the portfolio string to AV
                }
                //Checks that the student has permission to view the
participation record
                if((c == true && portfolio.equalsIgnoreCase("Catering")) ||
(p == true && portfolio.equalsIgnoreCase("Photography")) || (a == true &&
portfolio.equalsIgnoreCase("AV")) || (f == true &&
portfolio.equalsIgnoreCase("First Aid")))

                {
                    model.addRow(new Object[]{rs.getInt("StudentID"),
rs.getInt("EventID"), rs.getString("EventName"),
                     rs.getInt("Hours"), portfolio}); //Creates a new object
to be added to the table
                }
                }

            model.addRow(new Object[]{}); //adds each object to its own row in
the table

        }

        catch(SQLException e)

        {

            System.out.println("Could Not Load Participation Table");
//Displays an error message if an error occurs

        }

    }
```

Any form of Artificial Intelligence – more than one method

```
if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true &&
portfolio.equalsIgnoreCase("Photography")) || (a == true &&
portfolio.equalsIgnoreCase("AV")) || (f == true &&
portfolio.equalsIgnoreCase("First Aid")))

                {
                    model.addRow(new Object[]{rs.getInt("StudentID"),
rs.getInt("EventID"), rs.getString("EventName"),
                     rs.getInt("Hours"), portfolio}); //Creates a new object
to be added to the table
```

```
                }


 pnlHelp.setVisible(false); //hides the help pannel
        switch (help) {
            case 1 -> pnlLogIn.setVisible(true);//if the help variable value
is 1, takes the user to the log in pannel
            case 2 -> pnlSignUp.setVisible(true);//if the help variable value
is 2, takes the user to the sign up pannel
            case 3 -> {
                tbdpAdmin.setVisible(true); //if the help variable value is
3, takes the user to the Admin tabbed pane, on the home page
                pnlHomeAdmin.setVisible(true);
            }
            case 4 -> {
                tbdpAdmin.setVisible(true); //if the help variable value is
4, takes the user to the Admin tabbed pane, on the events pannel
                pnlEvents1.setVisible(true);
            }
            case 5 -> {
                tbdpAdmin.setVisible(true); //if the help variable value is
5, takes the user to the Admin tabbed pane, on the participation pannel
                pnlParticipation1.setVisible(true);
            }
            case 6 -> {
                tbdpAdmin.setVisible(true); //if the help variable value is
6, takes the user to the Admin tabbed pane, on the students pannel
                pnlStudent.setVisible(true);
            }
            case 7 -> {
                tbdpStudent.setVisible(true); //if the help variable value is
7, takes the user to the Students tabbed pane, on the home pannel
                pnlHome.setVisible(true);
            }
            case 8 -> {
                tbdpStudent.setVisible(true); //if the help variable value is
8, takes the user to the Students tabbed pane, on the Events pannel
                pnlEvents.setVisible(true);
            }
            case 9 -> {
                tbdpStudent.setVisible(true); //if the help variable value is
9, takes the user to the Students tabbed pane, on the participation pannel
                pnlParticipation.setVisible(true);
            }
            default -> {
            }
        }
```

```
if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true &&
portfolio.equalsIgnoreCase("Photography")) || (a == true &&
portfolio.equalsIgnoreCase("AV")) || (f == true &&
portfolio.equalsIgnoreCase("First Aid")))
                {
                    model.addRow(new
Object[]{rs.getInt("EventID"),rs.getString("EventName"),
                     rs.getInt("Hours"),rs.getDate("EventDate"), portfolio});
//Creates a new object to be added to the table
                    count++; //adds 1 to count
                }


if((AdminArr[j].getID()) == i)//checks each Admin ID aginst i
            {
                i = i-1;
            if(c.equalsIgnoreCase("Catering") &&
AdminArr[i].isCatering())//checks if the string entered is catering and if
the current Admin oversees catering
            {
                test = true;
            }
            else if (c.equalsIgnoreCase("Photography") &&
AdminArr[i].isPhotography())//checks if the string entered is photography and
if the current Admin oversees photography
            {
                test = true;
            }
            else if (c.equalsIgnoreCase("av") && AdminArr[i].isAv())//checks
if the string entered is AV and if the current Admin oversees AV
            {
                test = true;
            }
            else if (c.equalsIgnoreCase("First Aid") &&
AdminArr[i].isFirstAid())//checks if the string entered is First Aid and if
the current Admin oversees First Aid
            {
                test = true;
            }
            }
```

Links to externally stored videos (YouTube)

```
final URI url;
        try {
            url = new URI("https://youtu.be/8ffuBUkUIUA");//creates a link to
the youtube video
            play(url);//Attempts to open the browser and play the video
        } catch (URISyntaxException ex) {//catches any errors
```

```
            Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null,
ex);
        }


private static void play(URI url)
    {
        if(Desktop.isDesktopSupported())//checks if the action can be
performed on the pc
        {
            try{
                Desktop.getDesktop().browse(url);//opens the b,video in the
browser
            }
            catch(IOException e)//catches any errors
                    {
                        System.out.println("Could not play video");//gives an
error to indicate that the video could not be played
                    }
        }
    }
```

Evaluation of Solution

My project entirely matches the idea that I had, and laid out, in the beginning of the development stage with a few minor adjustments including permissions and minor design changes. The users will not be allowed to delete any events as it was realised that this may cause unnecessary loss of data.

I was able to meet all my goals laid out in the design and specification documents. Including my overall goal of creating an interactive system which will save time and energy by limiting the admin work done by teachers at Curro Hillcrest to calculate and track the service hours completed by the students.

I feel that I was able to manage my time carefully and spread out my workload well over the time allocated for the project, which allowed me to deliver a high-quality project.

Areas where I would like to improve include:

- Creating a data validation check for new users to ensure that the username does not already exist in the database.
- Creating a function which will require an admin user to “Sign-off” on participation records logged by the students to ensure that all data is secure and that no cheating or falsifying of records takes place.

- I would like to expand the program to include outreach hours and peer tutoring hours completed.
- I would like to develop an application which will allow the system to be used at Curro Hillcrest

Functional Testing

Test 1

Tester Name: Gerard Rudolph **Date: 21/07/2022**

Functions	Achieved? Yes/No
Log In Screen Exists	Yes
Allow users to log in using a username and password to access an admin or learner account	Yes
Allow users to press a button that will direct them to the sign-up page where they can sign up for a learner account	Yes
Allow users to select the help button	Yes
Sign Up Screen Exists	Yes
Allow learners to register/sign up by entering a username and by entering the same password twice	Yes
Allow learners to enter their full name	Yes
Allow learners to enter their grade	1.Yes but
Allow learners to enter their class	Yes
Allow learners to select a gender	Yes
Allow learners to select portfolio options	Yes
Allow learners to select a button to create their account provided all the above information has been entered	Yes
Allow users to select the help button	Yes
Student Home Page Exists	Yes
Allow learners to view their details (name, surname, username, grade, class, gender and portfolios with their accumulated hours in each portfolio)	Yes
Allow learners to select an “Edit account” button	Yes
Allow learners to navigate to other parts of the program through tabs	Yes
Allow users to select the help button	Yes
Edit Account (Student) Screen Exists	Yes
Allows users to edit their existing details, in separate fields (Name, surname, grade, class, portfolios and gender)	Yes
Allow users to select an “Enter” button to confirm the changes	Yes

Allow users to select a “Cancel” button to exit the edit screen and return to the home screen	Yes
Events Screen (Student) Exists	Yes
Allow learners to view existing events related to their portfolio	Yes
Allow learners to search for events by date which will return event ID, event name, event date, portfolio and how long the event took (Hours)	Yes
Allow learners to search for events by name which will return event ID, event name, event date, portfolio and how long the event took (Hours)	Yes
Allow learners to search for events by ID which will return event ID, event name, event date, portfolio and how long the event took (Hours)	Yes
Allow learners to navigate to other parts of the program through tabs	Yes
Allow users to select the help button	Yes
Participation Screen (Student) Exists	Yes
Allow learners to log their participation in events	Yes
Allow learners to view records of their participation in events	Yes
Allow learners to delete record of their participation in events	Yes
Allow learners to search for events that they have participated in, by portfolio which will return the event ID, the event name, Student ID, hours, and portfolio.	Whole text only
Allow learners to search for events that they have participated in, by event name which will return the event ID, the event name, Student ID, hours, and portfolio.	Yes
Allow learners to search for events that they have participated in, by event ID which will return the event ID, the event name, Student ID, hours, and portfolio.	Yes
Allow learners to navigate to other parts of the program through tabs	Yes
Allow users to select the help button	Yes
Admin Home Page Exists	Yes
Allow admins to view their details (Name, surname and portfolios)	Yes
Allow admins to select an “Edit account” button	Yes
Allow admins to select a “Create new admin account’ button	Yes
Allow admins to navigate to other parts of the program through tabs	Yes
Allow users to select the help button	Yes
Edit Admin Page Exists	Yes
Will allow admins to edit their existing details in separate fields (Name, surname, and portfolios)	Yes

Allow admins to select an “Enter” button to confirm any changes they have made	Yes
Allow admins to select a “Cancel” button to exit the screen and discard any changes	Yes
Create New Admin Page Exists	Yes
Allow an existing admin to enter details for a new admin (Full name, username, password, confirmation of password, Portfolios)	Yes
Allow admin to select an “Enter” button to create the new admin account	Yes
Allow the admin to select a “cancel” button to exit the screen and discard any information they have entered.	Yes
Participation (Admin) Page Exists	Yes
Allow admins to view all participation records related to their portfolios	Yes
Allow admins to search for participation records by Event ID which will return the event ID, the event name, the student ID, student name, the relevant portfolio and the number of hours for which the event counts.	Yes
Allow admins to search for participation records by event name which will return the event ID, the event name, the student ID, student name, the relevant portfolio and the number of hours for which the event counts.	Yes
Allow admins to search for participation records by student name which will return the event ID, the event name, the student ID, student name, the relevant portfolio and the number of hours for which the event counts.	Yes
Allow admins to search for participation records by portfolio which will return the event ID, the event name, the student ID, student name, the relevant portfolio and the number of hours for which the event counts.	Yes
Allow admins to delete participation records	Yes
Allow admins to create participation records	Yes
Allow admins to navigate to other parts of the program through tabs	Yes
Allow users to select the help button	Yes
Events (Admin) Page Exists	Yes
Allow admins to view existing events related to their portfolio	Yes
Allow admins to select a “create new event” button	Yes
Allow admins to select an “edit existing event” button	Yes

Allow admins to search for events, by date which will return event ID, event name, event date, portfolio and how long the event took	Yes
Allow admins to search for events, by name which will return event ID, event name, event date, portfolio and how long the event took	Yes
Allow admins to search for events, by event ID which will return event ID, event name, event date, portfolio and how long the event took	Yes
Allow admins to navigate to other parts of the program through tabs	Yes
Allow users to select the help button	Yes
Edit Event (Admin) Page Exists	Yes
Will initially present the admin with a single space to enter an event ID	Yes
Will initially present the admin with a “Find event” button to search for an event matching the ID they have entered.	Yes
Will allow the admin to select an “Enter” button to confirm the changes they have made	Yes
Will allow the admin to select a “cancel” button to exit the screen and discard any changes they have made.	Yes
After an event matching an entered ID has been found, the rest of the screen will be displayed with fields containing the current values assigned to that event	Yes
Will allow the admin to change the existing values assigned to an event.	Yes
Create New Event (Admin) Page Exists	Yes
Will allow the admin to enter all the details of an event (name, hours, date, and portfolio)	Yes
Will allow the admin to select an “Enter” button to create the event provided all the fields are complete	Nothing happened
Will allow the admin to select a “cancel” button to exit the screen and discard any details they have entered	Yes
Students Page Exists	Yes
Allow admins to search for learners by name which will return student ID, student name, grade, class, gender (M/F), catering hours, photography hours, AV hours and first aid hours	No (double space?)
Allow admins to search for learners by grade which will return student ID, student name, grade, class, gender (M/F), catering hours, photography hours, AV hours and first aid hours	4.Partial

Allow admins to search for learners by Student ID which will return student ID, student name, grade, class, gender (M/F), catering hours, photography hours, AV hours and first aid hours	Only double digits
Allow admins to navigate to other parts of the program through tabs	Yes
Allow users to select the help button	Yes
Help Page Exists	Yes
Allow users to select a button that relates to their problem which will direct them to a video that explains how to solve the problem or perform the task	Yes
Allow users to contact someone if none of the solutions provided could assist them	No

Please write any general comments below:

…well I like it!

I'd prefer a partial string search – rather than having to know the whole name I'm searching for and needing to spell it exactly the way it was entered.
1. Yes but – could enter invalid options without receiving a warning

~~2.Yes… but~~ – nope – this works

~~3.Kinda –~~ nope – this works

4.Partial – returned some (but not all) results if I type only digit(s) before decimal
If I type the decimal – no results, even with correct grade selected, e.g 10.5

Test 2

Functional Testing

Tester Name: Kieran Cullum **Date:** **27/07/2022**

Functions	Achieved? Yes/No
Log In Screen Exists	yes
Allow users to log in using a username and password to access an admin or learner account	yes
Allow users to press a button that will direct them to the sign-up page where they can sign up for a learner account	yes
Allow users to select the help button	yes

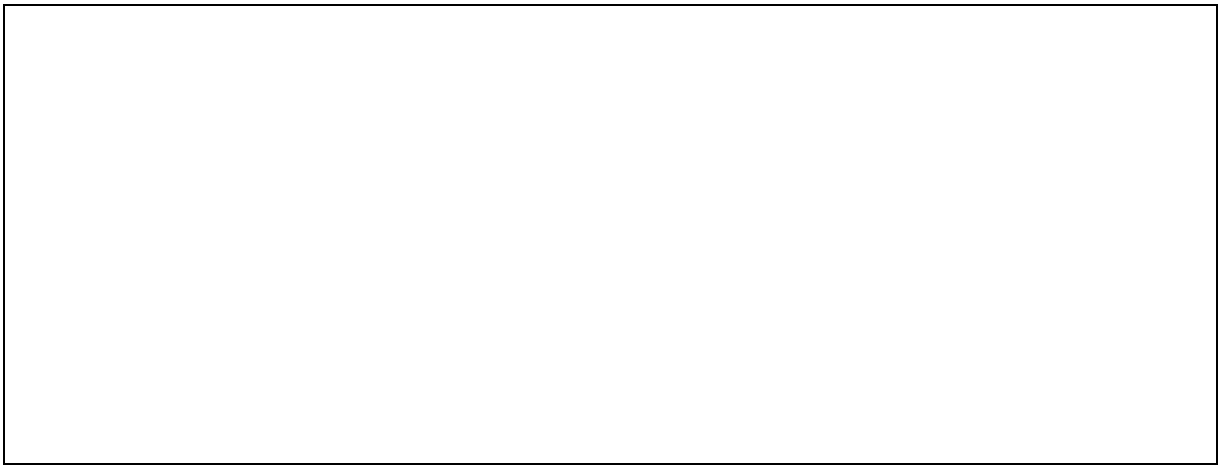
Sign Up Screen Exists	yes
Allow learners to register/sign up by entering a username and by entering the same password twice	yes
Allow learners to enter their full name	yes
Allow learners to enter their grade	yes
Allow learners to enter their class	yes
Allow learners to select a gender	yes
Allow learners to select portfolio options	yes
Allow learners to select a button to create their account provided all the above information has been entered	yes
Allow users to select the help button	yes
Student Home Page Exists	yes
Allow learners to view their details (name, surname, username, grade, class, gender and portfolios with their accumulated hours in each portfolio)	yes
Allow learners to select an “Edit account” button	yes
Allow learners to navigate to other parts of the program through tabs	yes
Allow users to select the help button	yes
Edit Account (Student) Screen Exists	yes
Allows users to edit their existing details, in separate fields (Name, surname, grade, class, portfolios and gender)	yes
Allow users to select an “Enter” button to confirm the changes	yes
Allow users to select a “Cancel” button to exit the edit screen and return to the home screen	yes
Events Screen (Student) Exists	yes
Allow learners to view existing events related to their portfolio	yes
Allow learners to search for events by date which will return event ID, event name, event date, portfolio and how long the event took (Hours)	yes
Allow learners to search for events by name which will return event ID, event name, event date, portfolio and how long the event took (Hours)	yes
Allow learners to search for events by ID which will return event ID, event name, event date, portfolio and how long the event took (Hours)	yes
Allow learners to navigate to other parts of the program through tabs	yes
Allow users to select the help button	yes
Participation Screen (Student) Exists	yes
Allow learners to log their participation in events	yes

Allow learners to view records of their participation in events	yes
Allow learners to delete record of their participation in events	yes
Allow learners to search for events that they have participated in, by portfolio which will return the event ID, the event name, Student ID, hours, and portfolio.	yes
Allow learners to search for events that they have participated in, by event name which will return the event ID, the event name, Student ID, hours, and portfolio.	yes
Allow learners to search for events that they have participated in, by event ID which will return the event ID, the event name, Student ID, hours, and portfolio.	yes
Allow learners to navigate to other parts of the program through tabs	yes
Allow users to select the help button	yes
Admin Home Page Exists	yes
Allow admins to view their details (Name, surname and portfolios)	yes
Allow admins to select an “Edit account” button	yes
Allow admins to select a “Create new admin account’ button	yes
Allow admins to navigate to other parts of the program through tabs	yes
Allow admins to select the help button	yes
Edit Admin Page Exists	yes
Will allow admins to edit their existing details in separate fields (Name, surname, and portfolios)	yes
Allow admins to select an “Enter” button to confirm any changes they have made	yes
Allow admins to select a “Cancel” button to exit the screen and discard any changes	yes
Create New Admin Page Exists	yes
Allow an existing admin to enter details for a new admin (Full name, username, password, confirmation of password, Portfolios)	yes
Allow admin to select an “Enter” button to create the new admin account	yes
Allow the admin to select a “cancel” button to exit the screen and discard any information they have entered.	yes
Participation (Admin) Page Exists	yes
Allow admins to view all participation records related to their portfolios	yes
Allow admins to search for participation records by Event ID which will return the event ID, the event name, the student ID, student	yes

name, the relevant portfolio and the number of hours for which the event counts.	
Allow admins to search for participation records by event name which will return the event ID, the event name, the student ID, student name, the relevant portfolio and the number of hours for which the event counts.	yes
Allow admins to search for participation records by student name which will return the event ID, the event name, the student ID, student name, the relevant portfolio and the number of hours for which the event counts.	yes
Allow admins to search for participation records by portfolio which will return the event ID, the event name, the student ID, student name, the relevant portfolio and the number of hours for which the event counts.	yes
Allow admins to delete participation records	yes
Allow admins to create participation records	yes
Allow admins to navigate to other parts of the program through tabs	yes
Allow admins to select the help button	yes
Events (Admin) Page Exists	yes
Allow admins to view existing events related to their portfolio	yes
Allow admins to select a “create new event” button	yes
Allow admins to select an “edit existing event” button	yes
Allow admins to search for events, by date which will return event ID, event name, event date, portfolio and how long the event took	yes
Allow admins to search for events, by name which will return event ID, event name, event date, portfolio and how long the event took	yes
Allow admins to search for events, by event ID which will return event ID, event name, event date, portfolio and how long the event took	yes
Allow admins to navigate to other parts of the program through tabs	yes
Allow users to select the help button	yes
Edit Event (Admin) Page Exists	yes
Will initially present the admin with a single space to enter an event ID	yes
Will initially present the admin with a “Find event” button to search for an event matching the ID they have entered.	yes
Will allow the admin to select an “Enter” button to confirm the changes they have made	yes

Will allow the admin to select a “cancel” button to exit the screen and discard any changes they have made.	yes
After an event matching an entered ID has been found, the rest of the screen will be displayed with fields containing the current values assigned to that event	yes
Will allow the admin to change the existing values assigned to an event.	yes
Create New Event (Admin) Page Exists	yes
Will allow the admin to enter all the details of an event (name, hours, date, and portfolio)	yes
Will allow the admin to select an “Enter” button to create the event provided all the fields are complete	yes
Will allow the admin to select a “cancel” button to exit the screen and discard any details they have entered	yes
Students Page Exists	yes
Allow admins to search for learners by name which will return student ID, student name, grade, class, gender (M/F), catering hours, photography hours, AV hours and first aid hours	yes
Allow admins to search for learners by grade which will return student ID, student name, grade, class, gender (M/F), catering hours, photography hours, AV hours and first aid hours	yes
Allow admins to search for learners by Student ID which will return student ID, student name, grade, class, gender (M/F), catering hours, photography hours, AV hours and first aid hours	yes
Allow admins to search for learners by student ID which will return student ID, student name, grade, class, gender (M/F), catering hours, photography hours, AV hours and first aid hours	yes
Allow admins to navigate to other parts of the program through tabs	yes
Allow users to select the help button	yes
Help Page Exists	yes
Allow users to select a button that relates to their problem which will direct them to a video that explains how to solve the problem or perform the task	yes
Allow users to contact someone if none of the solutions provided could assist them	yes

Please write any general comments below:



Data validation

Description table

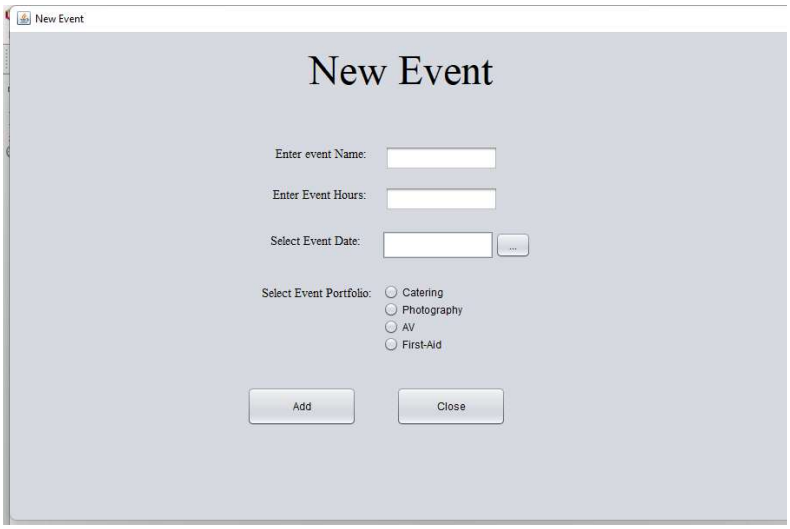
Name and type of field	Input component	Reason for input component	Data validation	Reason for data validation
String - name	Text field	User can enter any name for the event	- Presence - Length	- Name must be present - Name must be longer than 1 digit
Double – hours	Text field	User can enter any number of hours for the event, with any decimals they would like	- Presence - type - Range	- Hours must have a value - all characters must be digits or a comma - hours must be larger than 0 and less then 12
LocalDate - dateEvent	Date picker	Limits the range of dates and ensures	- Presence - range	- Date must be present

		dates are in the correct format		- date must not be more than a year in advance
Boolean – catering /photography /firstAid /av	4 Radio buttons	Allows the user to choose any one of the 4 portfolios to set as “true” and the rest will be set to false	- Presence	- A portfolio must be selected

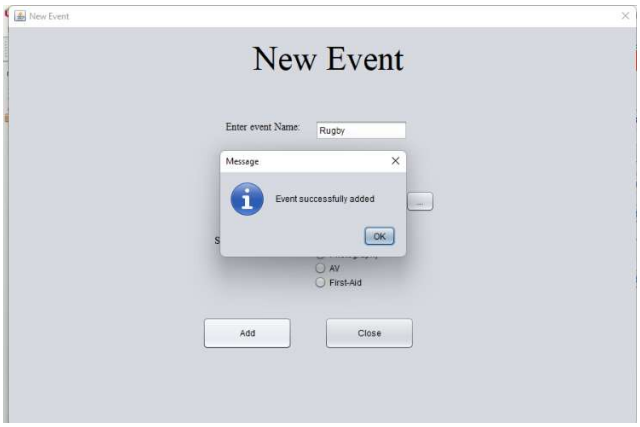
Evidence of testing

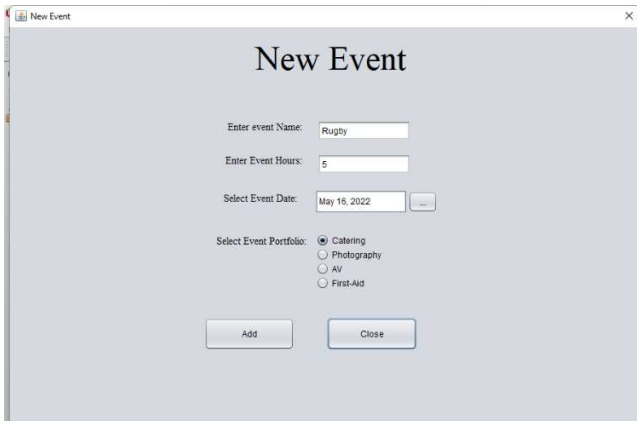
Presence check

Field to be tested	Standard data	Extreme data	Abnormal data
Name	Rugby Match	NA	No data
Hours	5	NA	No data
dateEvent	05/16/2022	NA	No data
portfolios	Select one portfolio	NA	No data

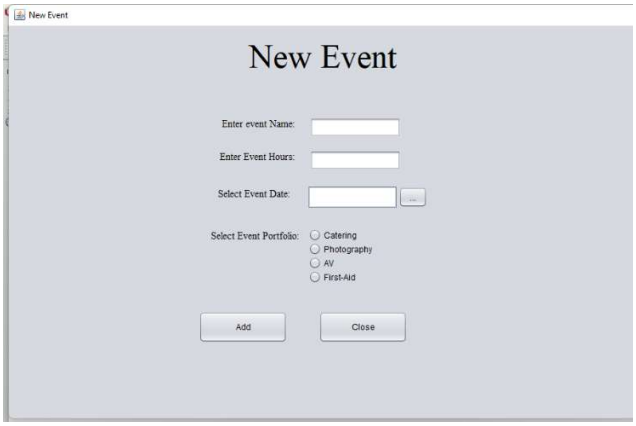


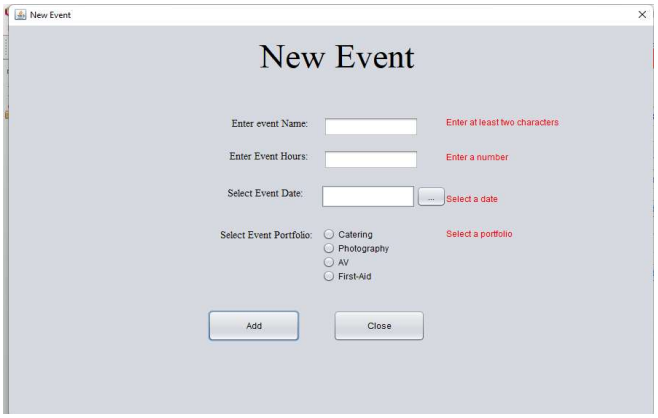
Standard





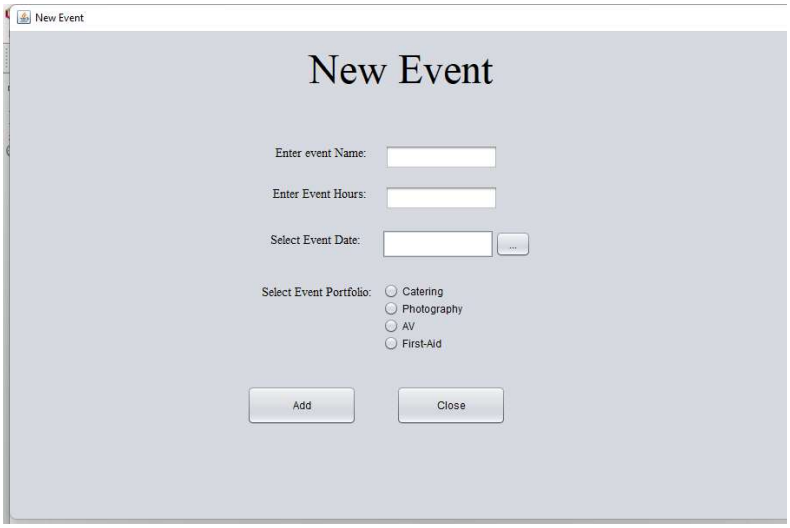
Abnormal



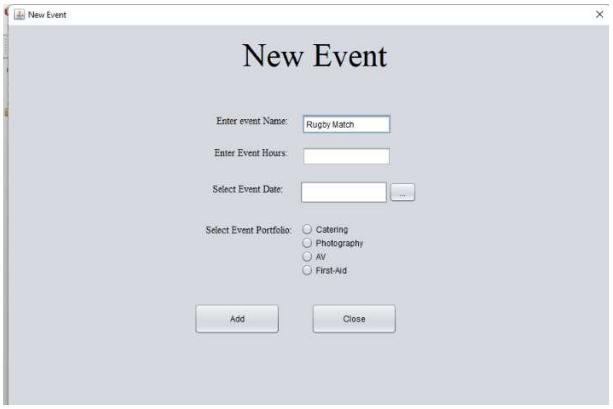


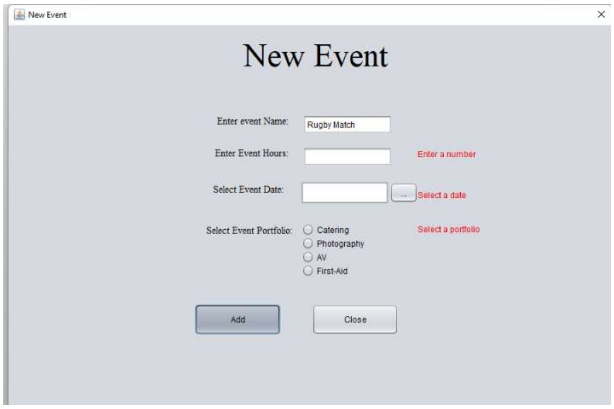
Length check

Field to be tested	Standard data	Extreme data	Abnormal data
Name	Rugby Match	RB	Q

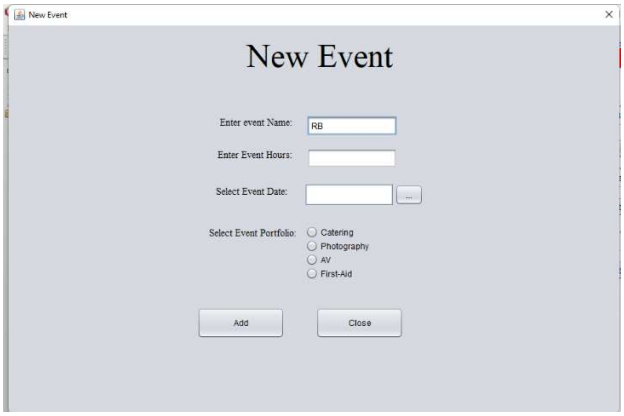


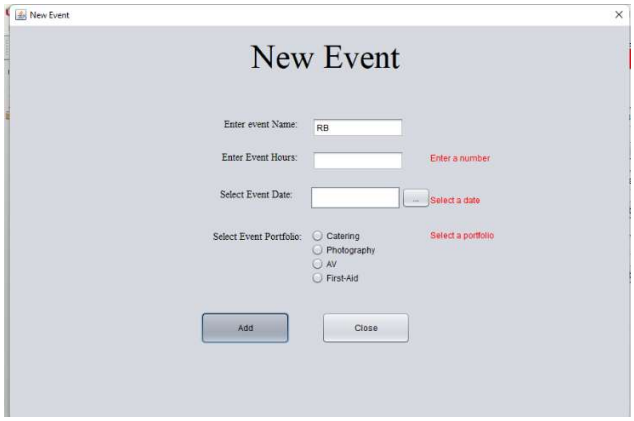
Standard



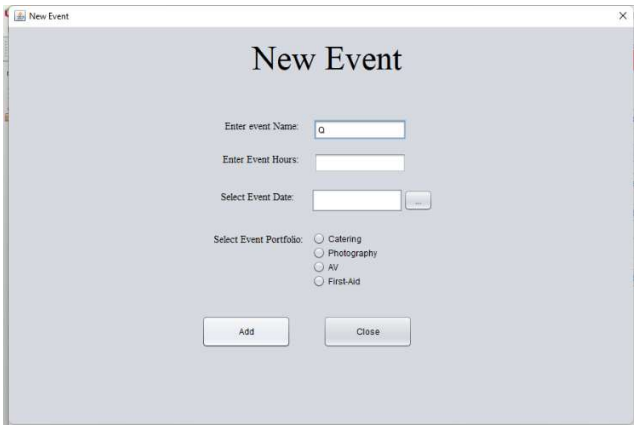


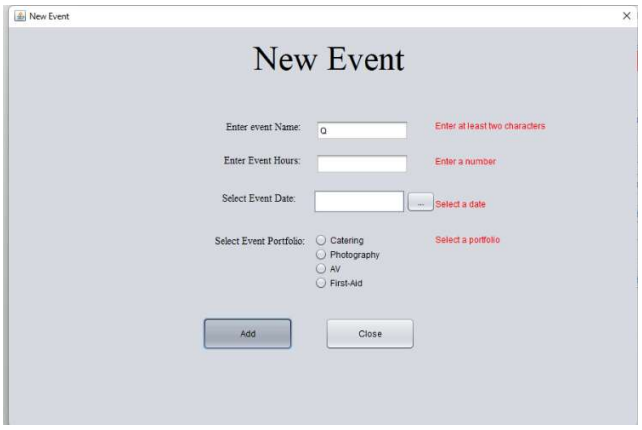
Extreme





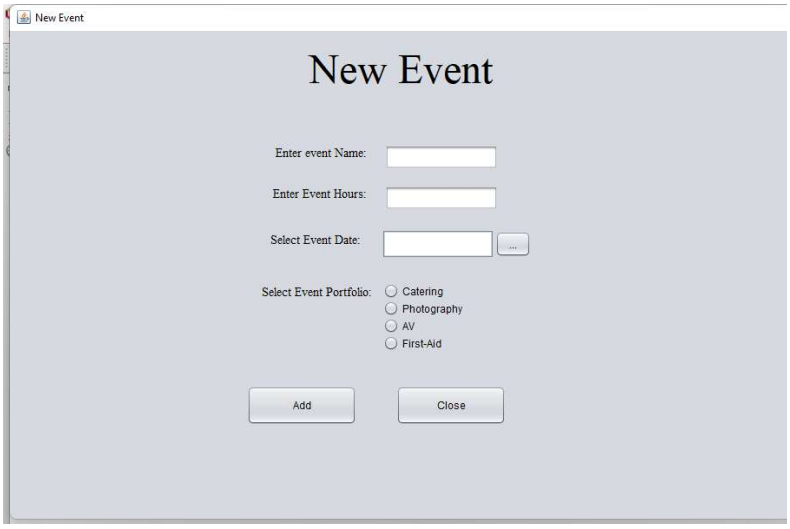
Abnormal



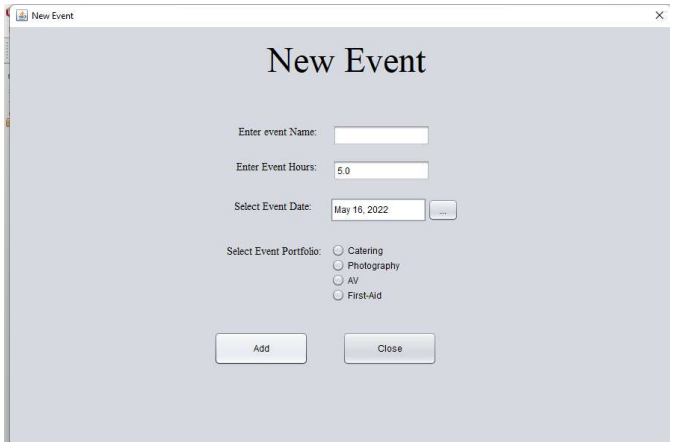


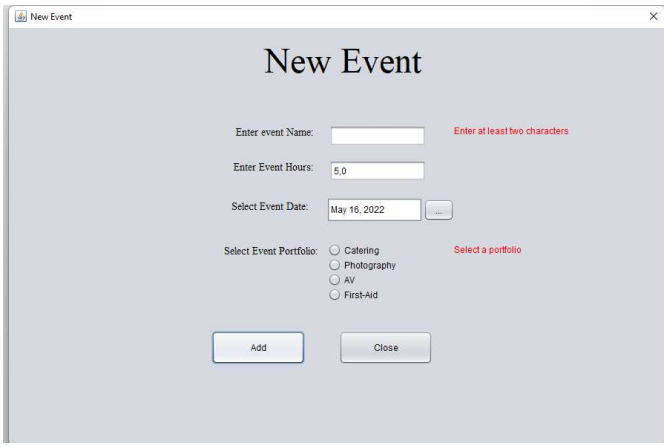
Type check

Field to be tested	Standard data	Extreme data	Abnormal data
Hours	5,0	11,99999	Ten
dateEvent	05/16/2022	05/16/2023	Not a Date

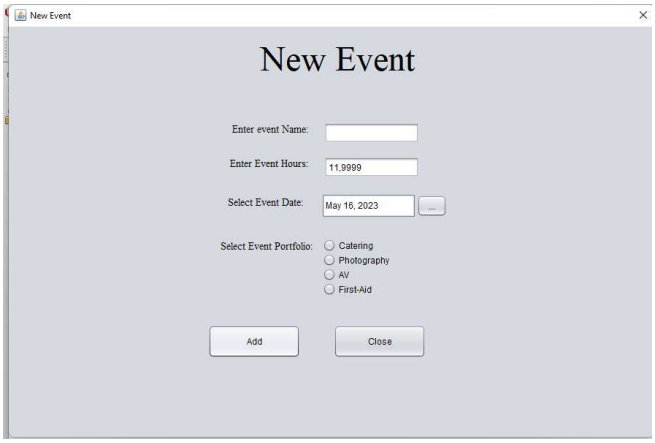


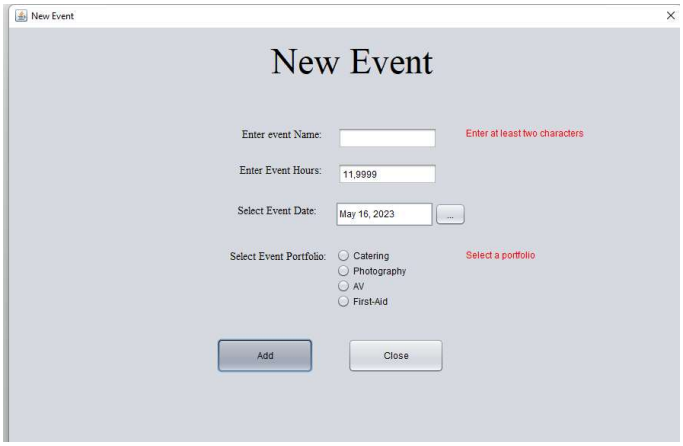
Standard



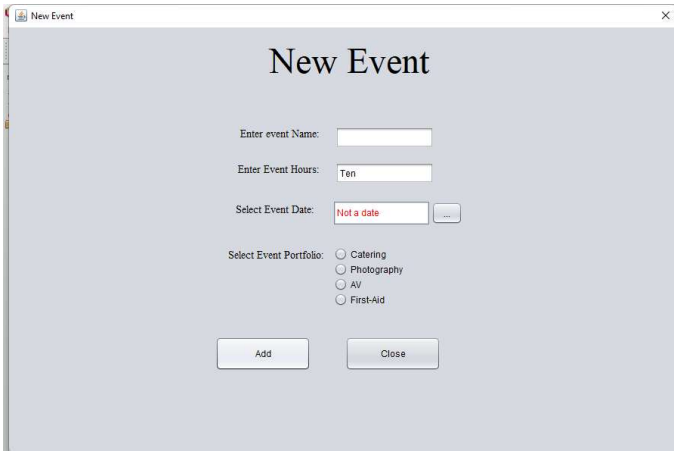


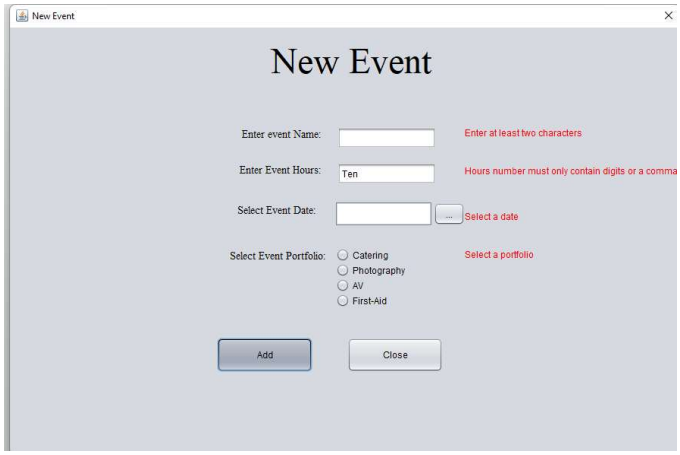
Extreme





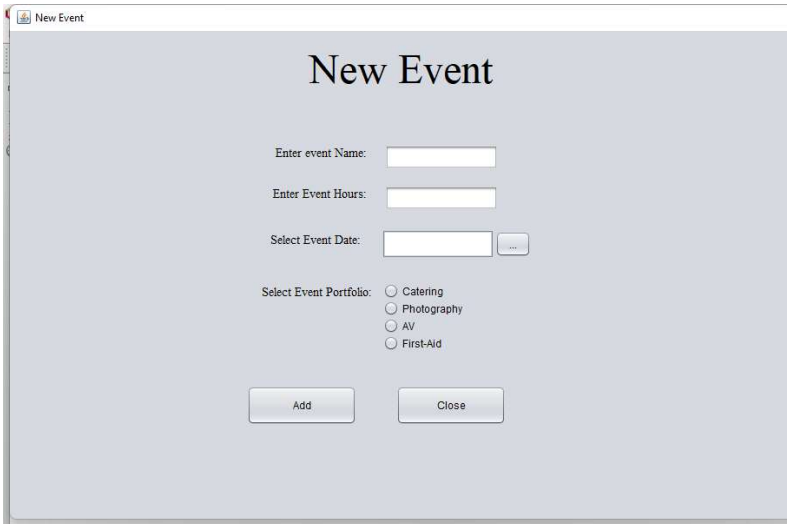
Abnormal



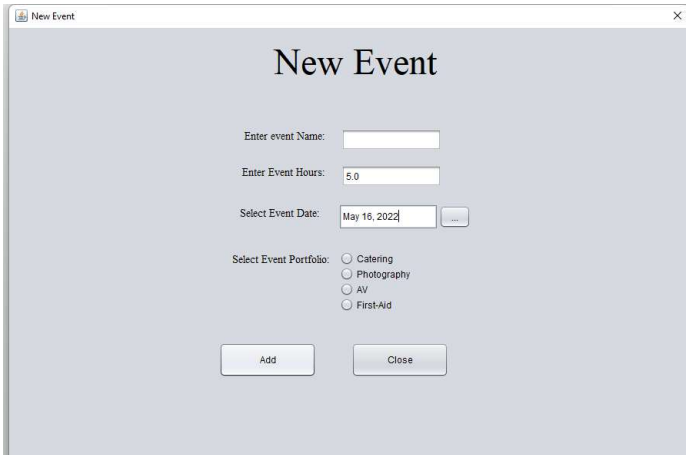


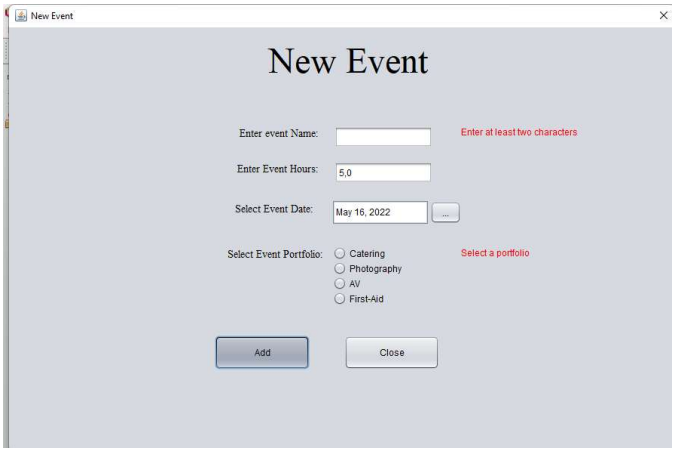
Range check

Field to be tested	Standard data	Extreme data	Abnormal data
Hours	5.0	11.99999	12
dateEvent	05/16/2022	05/16/2023	05/16/2024

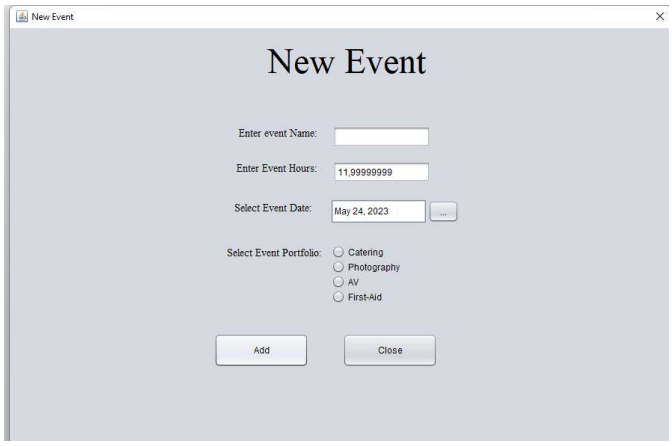


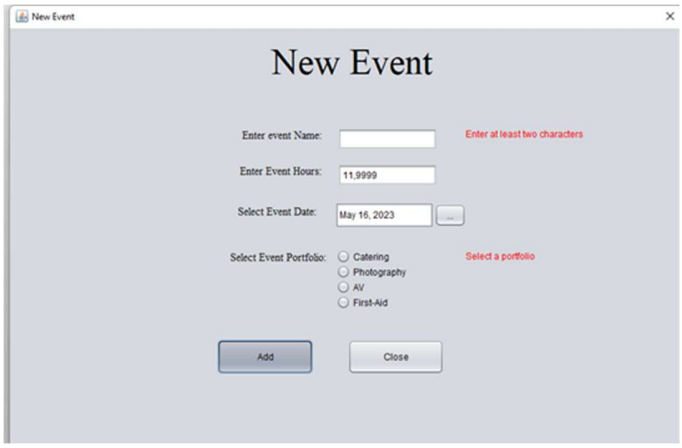
Standard



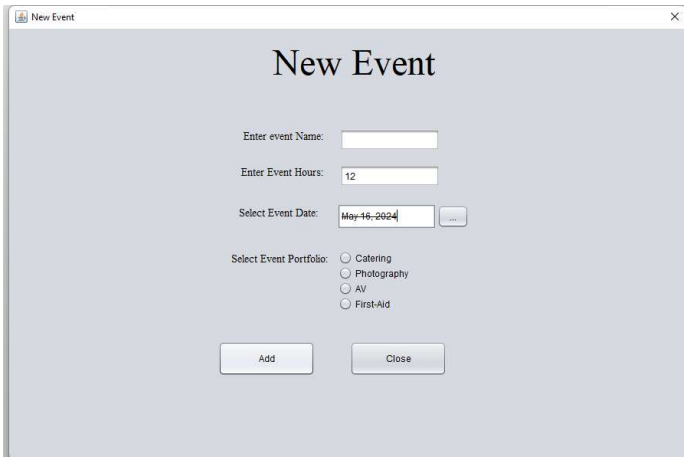


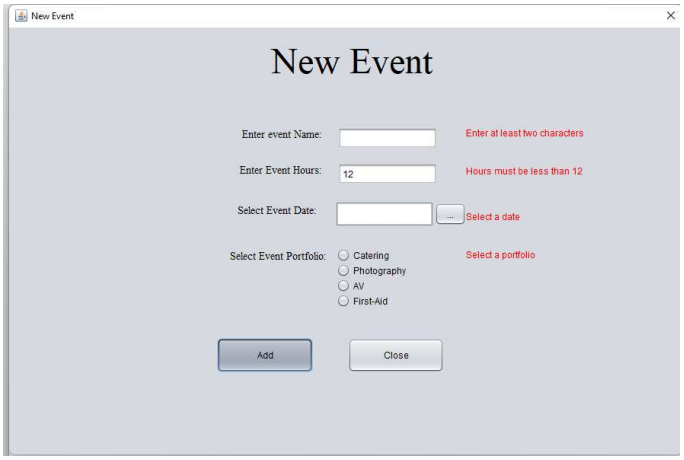
Extreme





Abnormal





Code:

UI Class:

```
package Application;


import Mannager.AdminManager;

import Mannager.EventManager;

import Mannager.ParticipationManager;

import Mannager.StudentManager;

import Objects.DBService;
```

```
import com.github.lgooddatepicker.components.DatePickerSettings;
import java.awt.Desktop;

import java.awt.HeadlessException;

import java.io.File;

import java.io.FileNotFoundException;

import java.io.FileWriter;

import java.io.IOException;

import java.io.PrintWriter;
import java.net.URI;
import java.net.URISyntaxException;

import java.sql.ResultSet;

import java.sql.SQLException;

import java.time.LocalDate;

import java.util.Scanner;

import java.util.logging.Level;

import java.util.logging.Logger;

import javax.swing.JOptionPane;

import javax.swing.table.DefaultTableModel;



/**

 *

 * @author ruth2

 */



public class frmMain extends javax.swing.JFrame {

    EventManager em = new EventManager(); //Instantiates Event manager
```

```
    AdminManager am = new AdminManager();//Instamtiates Admin manager

    StudentManager sm = new StudentManager();//Instantiates Student manager

    ParticipationManager pm = new ParticipationManager();//Instantiates Participation manager

    int currentAdmin = -1; //Sets current student to -1

    int lastAdmin = -1;//Sets last admin to -1

    int currentStu = -1;//Sets current student to -1

    int lastStu = -1;//Sets last student to -1

    int help = -1;//Sets help to -1

    private final DBService dbman = new DBService();//Creates a connection to the databse

    private DefaultTableModel model = null;//Sets the default table modle to null

    /**
     *
     * @throws SQLException
     */
    public frmMain() throws SQLException {

        initComponents();

        //Adapted from Exploring IT: Java programming, Grade 12 text book

        DatePickerSettings settings = dpkDate.getSettings(); //Sets the settings of the create event date
picker

        DatePickerSettings settings2 = dpkSearch.getSettings(); //Sets the settings of the search event date
picker (Admin)

        DatePickerSettings settings3 = dpkSearchDate.getSettings();//Sets the settings of the search event
date picker (students)

        LocalDate now = LocalDate.now().plusYears(1);//Sets the date limit to 1 year from the current date

        LocalDate startDate = LocalDate.now().minusYears(1);//Sets the begining limit to 1 year before the
current date

        LocalDate now2 = LocalDate.now().plusYears(2); //Sets the date limit to 2 year from the current date

        LocalDate startDate2 = LocalDate.now().minusYears(2);//Sets the begining limit to 2 year before the
current date
```

```
        settings.setDateRangeLimits(startDate,now); //Sets the limits to the date picker

        settings2.setDateRangeLimits(startDate2,now2); //Sets the limits to the date picker

        settings3.setDateRangeLimits(startDate2,now2); //Sets the limits to the date picker

        pnlHome.setVisible(false); //Hides the home pannel

        tbdpStudent.setVisible(false); //Hides the student tabbed pane

    }

    /**
     * Loads data from database into the JTable
     * @param c Boolean indicating admin permissions for catering
     * @param p Boolean indicating admin permissions for photography
     * @param a Boolean indicating admin permissions for AV
     * @param f Boolean indicating admin permissions for first aid
     */
    public void AdminEventsTBL(boolean c, boolean p, boolean a, boolean f){
        try
        {
            DefaultTableModel dtm = (DefaultTableModel) tblEventsAdmin.getModel(); //Sets the default table
moddel

            dtm.setRowCount(0); //Sets the row count to 0

            tblEventsAdmin.removeAll();//clears the table

            model = (DefaultTableModel)tblEventsAdmin.getModel(); //Sets the default table moddel

            ResultSet rs = dbman.queryDatabase("SELECT * FROM tblEvents"); //Selects data from the
databse

            while(rs.next()) //Creates a line in the JTable for every database entry
            {
                String portfolio = ""; //Creates a string to store the portfolio of the event
                if(rs.getBoolean("Catering") == true) //Checks if the event is for catering
                {
                    portfolio = "Catering"; //Sets the portfolio string to catering
                }
                else if(rs.getBoolean("Photography") == true) //Checks if the event is for photography
                {
                    portfolio = "Photography"; //Sets the portfolio string to photography
                }
                else if(rs.getBoolean("FirstAid") == true) //Checks if the event is for first aid
                {
                    portfolio = "First Aid"; ////Sets the portfolio string to First aid
```

```
            }
            else if(rs.getBoolean("AV") == true) //Checks if the event is for AV
            {
               portfolio = "AV"; //Sets the portfolio string to AV
            }
            //Checks that the admin has permission to view the event
            if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true &&
portfolio.equalsIgnoreCase("Photography")) || (a == true && portfolio.equalsIgnoreCase("AV")) || (f ==
true && portfolio.equalsIgnoreCase("First Aid")))
            {
               model.addRow(new Object[]{rs.getInt("EventID"),rs.getString("EventName"),
                rs.getInt("Hours"),rs.getDate("EventDate"), portfolio}); //Creates a new object to be added to
the table
            }
            }

         model.addRow(new Object[]{});//adds each object to its own row in the table

      }

      catch(SQLException e)

      {

         System.out.println("Could Not Load Events Table: Please restart the program"); //Displays an
error message if there is an error

      }

   }

   /**
    * Loads data from database into the JTable
    * @param c Boolean checks if the student does catering
    * @param p Boolean checks if the student does photography
    * @param a Boolean checks if the student does AV
    * @param f Boolean checks if the student does first aid
    */
   public void StudentEventsTBL(boolean c, boolean p, boolean a, boolean f){
      try

      {

         DefaultTableModel dtm = (DefaultTableModel) tblEventStu.getModel(); //Sets the default table
model

         dtm.setRowCount(0);//sets the roow count to 0

         tblEventStu.removeAll();//clears the table
```

```
        model = (DefaultTableModel)tblEventStu.getModel(); //Sets the default table model

        ResultSet rs = dbman.queryDatabase("SELECT * FROM tblEvents"); //Selects data from the
databse



        while(rs.next()) //Creates a line in the JTable for every database entry

        {

            String portfolio = ""; //Creates a string to store the portfolio of the event

            if(rs.getBoolean("Catering") == true) //Checks if the event is for catering

            {

                portfolio = "Catering";//Sets the portfolio string to catering

            }

            else if(rs.getBoolean("Photography") == true) //Checks if the event is for photography

            {

                portfolio = "Photography"; //Sets the portfolio string to photography

            }

            else if(rs.getBoolean("FirstAid") == true)//Checks if the event is for first aid

            {

                portfolio = "First Aid"; //Sets the portfolio string to first aid

            }

            else if(rs.getBoolean("AV") == true)//Checks if the event is for AV

            {

                portfolio = "AV"; //Sets the portfolio string to AV

            }
            //Checks that the student has permission to view the event
            if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true &&
portfolio.equalsIgnoreCase("Photography")) || (a == true && portfolio.equalsIgnoreCase("AV")) || (f ==
true && portfolio.equalsIgnoreCase("First Aid")))
```

```
            {
                model.addRow(new Object[]{rs.getInt("EventID"),rs.getString("EventName"),
                 rs.getInt("Hours"),rs.getDate("EventDate"), portfolio}); //Creates a new object to be added to
the table

            }
            }

          model.addRow(new Object[]{});//adds each object to its own row in the table
        }

        catch(SQLException e)

        {
            System.out.println("Could Not Load Events Table");//Displays an error message if an error occurs

        }

    }

    /**
     * Loads data from database into the JTable
     * @param c Boolean indicating if the student does catering
     * @param p Boolean indicating if the student does photography
     * @param a Boolean indicating if the student does AV
     * @param f Boolean indicating if the student does first aid
     */
    public void ParticipationTBL(boolean c, boolean p, boolean a, boolean f){
        try
        {

            DefaultTableModel dtm = (DefaultTableModel) tblParticipation.getModel();//Sets the default table
model

            dtm.setRowCount(0); //Sets the row count to 0

            tblParticipation.removeAll(); //Cleares the table

            model = (DefaultTableModel)tblParticipation.getModel();//Sets the default table model

            //Selects the participation records for the student currently logged in
            ResultSet rs = dbman.queryDatabase("SELECT DISTINCT tblStudents.StudentID,
tblEvents.EventID, tblEvents.EventName, tblEvents.hours, tblEvents.Catering, tblEvents.Photography,
tblEvents.AV, tblEvents.FirstAid FROM tblEvents, tblStudents, tblParticipation Where
tblParticipation.StudentID = tblStudents.StudentID AND tblParticipation.EventID = tblEvents.EventID AND
tblParticipation.StudentID =" + currentStu);

            while(rs.next()) //Creates a line in the JTable for every database entry
            {
```

```
            String portfolio = ""; //Creates a string to store the portfolio of the event
            if(rs.getBoolean("Catering") == true)//Checks if the event of the participation record is for
catering
            {
               portfolio = "Catering"; //Sets the portfolio string to catering
            }
            else if(rs.getBoolean("Photography") == true) //Checks if the event of the participation record is
for photography
            {
               portfolio = "Photography"; //Sets the portfolio string to photography
            }
            else if(rs.getBoolean("FirstAid") == true)//Checks if the event of the participation record is for
first aid
            {
               portfolio = "First Aid"; //Sets the portfolio string to first aid
            }
            else if(rs.getBoolean("AV") == true) //Checks if the event of the participation record is for AV
            {
               portfolio = "AV"; //Sets the portfolio string to AV
            }
            //Checks that the student has permission to view the participation record
            if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true &&
portfolio.equalsIgnoreCase("Photography")) || (a == true && portfolio.equalsIgnoreCase("AV")) || (f ==
true && portfolio.equalsIgnoreCase("First Aid")))

            {
               model.addRow(new Object[]{rs.getInt("StudentID"), rs.getInt("EventID"),
rs.getString("EventName"),
                rs.getInt("Hours"), portfolio}); //Creates a new object to be added to the table
            }
            }

         model.addRow(new Object[]{}); //adds each object to its own row in the table

      }

      catch(SQLException e)

      {

         System.out.println("Could Not Load Participation Table"); //Displays an error message if an error
occurs

      }

   }

   /**
    * Loads data from database into the JTable
```

```
     * @param c Boolean indicating admin permissions for catering
     * @param p Boolean indicating admin permissions for photography
     * @param a Boolean indicating admin permissions for AV
     * @param f Boolean indicating admin permissions for first aid
     */
    public void AdminParticipationTBL(boolean c, boolean p, boolean a, boolean f){ //Loads data from database into the Jtable
        try
        {
            DefaultTableModel dtm = (DefaultTableModel) tblParticipationAdmin.getModel(); //Sets the default table model
            dtm.setRowCount(0); //sets the roow count to 0
            tblParticipationAdmin.removeAll(); //Cleares the table
            model = (DefaultTableModel)tblParticipationAdmin.getModel(); //Sets the default table model
            //Selects all the participation records from the database
            ResultSet rs = dbman.queryDatabase("SELECT DISTINCT tblStudents.StudentID, tblStudents.firstName, tblStudents.Surname, tblEvents.EventID, tblEvents.EventName, tblEvents.hours, tblEvents.Catering, tblEvents.Photography, tblEvents.AV, tblEvents.FirstAid FROM tblEvents, tblStudents, tblParticipation Where tblParticipation.StudentID = tblStudents.StudentID AND tblParticipation.EventID = tblEvents.EventID");
            while(rs.next()) //Creates a line in the JTable for every database entry
            {
                String portfolio = ""; //Creates a string to store the portfolio of the event
                if(rs.getBoolean("Catering") == true) //Checks if the event of the participation record is for catering
                {
                    portfolio = "Catering"; //Sets the portfolio string to catering
                }
                else if(rs.getBoolean("Photography") == true) //Checks if the event of the participation record is for photography
                {
                    portfolio = "Photography"; //Sets the portfolio string to photography
                }
                else if(rs.getBoolean("FirstAid") == true) //Checks if the event of the participation record is for first aid
                {
                    portfolio = "First Aid"; //Sets the portfolio string to first aid
                }
                else if(rs.getBoolean("AV") == true) //Checks if the event of the participation record is for AV
                {
                    portfolio = "AV"; //Sets the portfolio string to AV
                }
                //Checks that the admin has permission to view the participation record
                if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true && portfolio.equalsIgnoreCase("Photography")) || (a == true && portfolio.equalsIgnoreCase("AV")) || (f == true && portfolio.equalsIgnoreCase("First Aid")))

                {
```

```
                String name = rs.getString("firstname") + " " + rs.getString("surname");//Creates a string to
hold the students full name
                model.addRow(new Object[]{rs.getInt("StudentID"),name, rs.getInt("EventID"),
rs.getString("EventName"),
                 rs.getInt("Hours"), portfolio}); //Creates a new object to be added to the table
            }

            }

        model.addRow(new Object[]{});//adds each object to its own row in the table

    }

    catch(SQLException e)

    {

        System.out.println("Could Not Execute Load Participation Table");//Displays an error message if
an error occurs

    }

  }

  /**
   * Loads data from database into the JTable
   */
  public void StudentsTBL(){

    try
    {

        DefaultTableModel dtm = (DefaultTableModel) tblStudents.getModel(); //Sets the default table
model

        dtm.setRowCount(0); //sets the roow count to 0

        tblStudents.removeAll(); //Cleares the table

        model = (DefaultTableModel)tblStudents.getModel(); //Sets the default table model

        ResultSet rs = dbman.queryDatabase("SELECT * FROM tblStudents;"); //Selects all the students
from the database

        while(rs.next()) //Creates a line in the JTable for every database entry
        {
            String stu = rs.getString("StudentID");//Sets the student ID to the ID value of the current record
in the database
```

```
            int catering = 0; //Creates a variable to hold the catering hours total

            int photography = 0; //Creates a variable to hold the photography hours total

            int firstAid = 0; //Creates a variable to hold the first aid hours total

            int av = 0; //Creates a variable to hold the AV hours total

        ResultSet ch = dbman.queryDatabase("SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS
CateringHours\n" +
                "FROM tblStudents, tblEvents, tblParticipation\n" +
                "WHERE tblParticipation.StudentID = tblStudents.StudentID AND tblEvents.EventID =
tblParticipation.EventID AND tblEvents.Catering = True\n" +
                "GROUP BY tblStudents.StudentID;"); //Sums the total catering hours for each student
        ResultSet ph = dbman.queryDatabase("SELECT tblStudents.StudentID, SUM(tblEvents.hours)
AS PhotographyHours\n" +
                "FROM tblStudents, tblEvents, tblParticipation\n" +
                "WHERE tblParticipation.StudentID = tblStudents.StudentID AND tblEvents.EventID =
tblParticipation.EventID AND tblEvents.Photography = True\n" +
                "GROUP BY tblStudents.StudentID;"); //Sums the total photography hours for each student
        ResultSet avh = dbman.queryDatabase("SELECT tblStudents.StudentID, SUM(tblEvents.hours)
AS AVHours\n" +
                "FROM tblStudents, tblEvents, tblParticipation\n" +
                "WHERE tblParticipation.StudentID = tblStudents.StudentID AND tblEvents.EventID =
tblParticipation.EventID AND tblEvents.AV = True\n" +
                "GROUP BY tblStudents.StudentID;"); //Sums the total AV hours for each student
        ResultSet fh = dbman.queryDatabase("SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS
FirstAidHours\n" +
                "FROM tblStudents, tblEvents, tblParticipation\n" +
                "WHERE tblParticipation.StudentID = tblStudents.StudentID AND tblEvents.EventID =
tblParticipation.EventID AND tblEvents.FirstAid = True\n" +
                "GROUP BY tblStudents.StudentID;"); //Sums the total first aid hours for each student

            while (ch.next()) //Loops throught the catering hours results
            {
                //checks if the admin oversees catering and that the current student ID from the main result
set matches the catering hours result set matches the catering hours result set
                if(am.getPortfolio("Catering", currentAdmin) == true &&
stu.equals(ch.getString("StudentID")))

                {
                    catering = ch.getInt("CateringHours");//assigns the current students total catering hours to
the catering variable
                }
            }


            while (ph.next()) //Loops throught the photography hours results
            {
```

```
            //checks if the admin oversees catering and that the current student ID from the main result set matches the catering hours result set matches the photography hours result set
            if(am.getPortfolio("Photography", currentAdmin) == true && stu.equals(ph.getString("StudentID")))

            {
              photography = ph.getInt("PhotographyHours");//assigns the current students total photography hours to the photography variable
            }
          }


          while (avh.next()) //Loops throught the AV hours results
          {
            //checks if the admin oversees catering and that the current student ID from the main result set matches the catering hours result set matches the AV hours result set
            if(am.getPortfolio("AV", currentAdmin) == true && stu.equals(avh.getString("StudentID")))

            {
              av = avh.getInt("AVHours"); //assigns the current students total AV hours to the AV variable
            }
          }


          while (fh.next()) //Loops throught the first aid hours results
          {
            //checks if the admin oversees catering and that the current student ID from the main result set matches the catering hours result set matches the first aid hours result set
            if(am.getPortfolio("First Aid", currentAdmin) == true && stu.equals(fh.getString("StudentID")))
            {
              firstAid = fh.getInt("FirstAidHours"); //assigns the current students total first aid hours to the first aid variable
            }
          }
          String grade = rs.getInt("Grade") + "." + rs.getInt("Class"); //Creates a string to contain the grade and class of the learner in a formatted manner
          String name = rs.getString("FirstName") + " " + rs.getString("Surname"); //Creates a string to contain the students full name
          String cater = Integer.toString(catering);//Converts the catering hours to a string
          String aV = Integer.toString(av); //converts the AV hours to a string
          String photo = Integer.toString(photography); //Converts the photography hours to a string
          String first =Integer.toString(firstAid); //Converts the first aid hours to a string
          if (am.getPortfolio("First Aid", currentAdmin) == false) //checks if the admin has permissions for first aid
          {
            first = "NA"; //if the admin does not have permissions for first aid, sets the first aid hours collum to 'NA'
          }
```

```
            if (am.getPortfolio("AV", currentAdmin) == false)//checks if the admin has permissions for AV
            {
                aV = "NA"; //if the admin does not have permissions for AV, sets the AV hours collum to 'NA'
            }
            if(am.getPortfolio("Catering", currentAdmin) == false) //checks if the admin has permissions for
catering
            {
                cater = "NA"; //if the admin does not have permissions for catering, sets the catering hours
collum to 'NA'
            }
            if (am.getPortfolio("Photography", currentAdmin) == false) //checks if the admin has
permissions for photography
            {
                photo = "NA"; //if the admin does not have permissions for photography, sets the
photography hours collum to 'NA'
            }
            //Creates a new object to be added to the table
             model.addRow(new Object[]{rs.getInt("StudentID"), name, grade, rs.getString("Gender"),cater,
aV, photo,first });

            }
          model.addRow(new Object[]{}); //adds each object to its own row in the table

        }
        catch(SQLException e)

        {

            System.out.println("Could Not Load Students Table");//Displays an error message if an error
occurs

        }

    }

    /**

     * This method is called from within the constructor to initialize the form.

     * WARNING: Do NOT modify this code. The content of this method is always

     * regenerated by the Form Editor.

     */

    @SuppressWarnings("unchecked")

    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {
```

```
btgChangeAdmin = new javax.swing.ButtonGroup();
bngNewAdmin = new javax.swing.ButtonGroup();
bngEditStuGender = new javax.swing.ButtonGroup();
bngNewEvent = new javax.swing.ButtonGroup();
bngEditEvent = new javax.swing.ButtonGroup();
bngNewStudentGender = new javax.swing.ButtonGroup();
dlgEditStudent = new javax.swing.JDialog();
cbxCatering1 = new javax.swing.JCheckBox();
cbxAV1 = new javax.swing.JCheckBox();
cbxFirstAid1 = new javax.swing.JCheckBox();
cbxPhotography1 = new javax.swing.JCheckBox();
lblEditStudent = new javax.swing.JLabel();
txfEditStuClass = new javax.swing.JTextField();
rbnEditStuGenderM = new javax.swing.JRadioButton();
rbnEditStuGenderF = new javax.swing.JRadioButton();
txfEditStuSurname = new javax.swing.JTextField();
txfEditStuGrade = new javax.swing.JTextField();
txfEditStuName = new javax.swing.JTextField();
lblEditStuPortfolios = new javax.swing.JLabel();
lblEditStuName = new javax.swing.JLabel();
lblEditStuSurname = new javax.swing.JLabel();
lblEditStuGrade = new javax.swing.JLabel();
lblEditStuClass = new javax.swing.JLabel();
lblEditStuGender = new javax.swing.JLabel();
lblEditStuSurnameError = new javax.swing.JLabel();
lblEditStuGradeError = new javax.swing.JLabel();
lblEditStuClassError = new javax.swing.JLabel();
lblEditStuGenderError = new javax.swing.JLabel();
lblEditStuPortfoliosError = new javax.swing.JLabel();
lblEditStuNameError = new javax.swing.JLabel();
btnEditStuCancel = new javax.swing.JButton();
btnEditStuEnter = new javax.swing.JButton();
dlgEvents = new javax.swing.JDialog();
txfNewEventName = new javax.swing.JTextField();
rbnNewEventCatering = new javax.swing.JRadioButton();
rbnNewEventAV = new javax.swing.JRadioButton();
rbnNewEventFirstAid = new javax.swing.JRadioButton();
rbnNewEventPhotography = new javax.swing.JRadioButton();
txfNewEventHours = new javax.swing.JTextField();
lblNewEvent = new javax.swing.JLabel();
lblNewEventName = new javax.swing.JLabel();
lblNewEventHours = new javax.swing.JLabel();
lblNewEventDate = new javax.swing.JLabel();
lblNewEventPortfolio = new javax.swing.JLabel();
dpkDate = new com.github.lgooddatepicker.components.DatePicker();
btnNewEventEnter = new javax.swing.JButton();
btnNewEventClose = new javax.swing.JButton();
lblNewNameError = new javax.swing.JLabel();
```

```
lblNewHoursError = new javax.swing.JLabel();
lblNewDateError = new javax.swing.JLabel();
lblNewPortfolioError = new javax.swing.JLabel();
dlgEditEvent = new javax.swing.JDialog();
txfEditEventID = new javax.swing.JTextField();
rbnEditEventCatering = new javax.swing.JRadioButton();
rbnEditEventAV = new javax.swing.JRadioButton();
rbnEditEventFirstAid = new javax.swing.JRadioButton();
rbnEditEventPhotography = new javax.swing.JRadioButton();
txfEditEventHours = new javax.swing.JTextField();
lblEditEvent = new javax.swing.JLabel();
lblEditEventID = new javax.swing.JLabel();
lblEditEventHours = new javax.swing.JLabel();
lblEditEventDate = new javax.swing.JLabel();
lblEditEventPortfolio = new javax.swing.JLabel();
dpkEditDate = new com.github.lgooddatepicker.components.DatePicker();
btnEditEventSearch = new javax.swing.JButton();
btnEditEventClose = new javax.swing.JButton();
lblEditIDError = new javax.swing.JLabel();
lblEditHoursError = new javax.swing.JLabel();
lblEditDateError = new javax.swing.JLabel();
lblEditPortfolioError = new javax.swing.JLabel();
lblEditEventName = new javax.swing.JLabel();
txfEditEventName = new javax.swing.JTextField();
lblEditNameError = new javax.swing.JLabel();
btnEditEventEnter = new javax.swing.JButton();
dlgEditAdmin = new javax.swing.JDialog();
cbxCateringAdmin = new javax.swing.JCheckBox();
cbxAVAdmin = new javax.swing.JCheckBox();
cbxFirstAidAdmin = new javax.swing.JCheckBox();
cbxPhotographyAdmin = new javax.swing.JCheckBox();
lblEditAdmin = new javax.swing.JLabel();
txfEditAdminSurname = new javax.swing.JTextField();
txfEditAdminName = new javax.swing.JTextField();
lblEditAdminPortfolios = new javax.swing.JLabel();
lblEditAdminName = new javax.swing.JLabel();
lblEditAdminSurname = new javax.swing.JLabel();
lblEditAdminSurnameError = new javax.swing.JLabel();
lblEditAdminPortfoliosError = new javax.swing.JLabel();
lblEditAdminNameError = new javax.swing.JLabel();
btnEditAdminCancel = new javax.swing.JButton();
btnEditAdminEnter = new javax.swing.JButton();
dlgNewAdmin = new javax.swing.JDialog();
txfNewAdminName = new javax.swing.JTextField();
txfNewAdminUsername = new javax.swing.JTextField();
txfNewAdminPassword = new javax.swing.JTextField();
lblNewAdminPortfolios = new javax.swing.JLabel();
cbxNewAdminPhotography = new javax.swing.JCheckBox();
cbxNewAdminAv = new javax.swing.JCheckBox();
```

```
cbxNewAdminFirstAid = new javax.swing.JCheckBox();
cbxNewAdminCatering = new javax.swing.JCheckBox();
lblNewAdminUsernameError = new javax.swing.JLabel();
lblNewAdminPasswordError = new javax.swing.JLabel();
lblNewAdminConfirmPasswordError = new javax.swing.JLabel();
lblNewAdminPortfoliosError = new javax.swing.JLabel();
lblNewAdminNameError = new javax.swing.JLabel();
btnEditAdminCancel1 = new javax.swing.JButton();
btnEditAdminEnter1 = new javax.swing.JButton();
lblNewAdminAccount = new javax.swing.JLabel();
txfNewAdminConfirmPassword = new javax.swing.JTextField();
lblNewAdminPassword = new javax.swing.JLabel();
lblNewAdminName = new javax.swing.JLabel();
lblNewAdminConfirmPassword = new javax.swing.JLabel();
lblNewAdminUsername = new javax.swing.JLabel();
pnlLogIn = new javax.swing.JPanel();
btnSignUp = new javax.swing.JButton();
btnHelp10 = new javax.swing.JButton();
lblPassword1 = new javax.swing.JLabel();
lblUsername1 = new javax.swing.JLabel();
txfPassword = new javax.swing.JTextField();
txfUsername = new javax.swing.JTextField();
btnLogIn = new javax.swing.JButton();
lblLogo1 = new javax.swing.JLabel();
pnlSignUp = new javax.swing.JPanel();
btnHelp11 = new javax.swing.JButton();
lblNewStudentPortfolios = new javax.swing.JLabel();
lblUsername = new javax.swing.JLabel();
txfNewStudentGrade = new javax.swing.JTextField();
txfUsername1 = new javax.swing.JTextField();
btnSignUpStu = new javax.swing.JButton();
lblLogo = new javax.swing.JLabel();
lblPassword2 = new javax.swing.JLabel();
txfPassword2 = new javax.swing.JTextField();
lblNewStudentGrade = new javax.swing.JLabel();
lblNewStudentClass = new javax.swing.JLabel();
lblCPassword4 = new javax.swing.JLabel();
txfConfirmPassword1 = new javax.swing.JTextField();
txfNewStudentClass = new javax.swing.JTextField();
txfNewStudentName = new javax.swing.JTextField();
rbnNewStudentF = new javax.swing.JRadioButton();
rbnNewStudentM = new javax.swing.JRadioButton();
lblNewStudentName = new javax.swing.JLabel();
lblNewStudentGender = new javax.swing.JLabel();
cbxAV = new javax.swing.JCheckBox();
cbxPotography = new javax.swing.JCheckBox();
cbxFirstAid = new javax.swing.JCheckBox();
cbxCatering = new javax.swing.JCheckBox();
lblNewStudentPasswordError = new javax.swing.JLabel();
```

```
lblNewStudentConfPasswordError = new javax.swing.JLabel();
lblNewStudentNameError = new javax.swing.JLabel();
lblNewStudentGradeError = new javax.swing.JLabel();
lblNewStudentClassError = new javax.swing.JLabel();
lblNewStudentGenderError = new javax.swing.JLabel();
lblNewStudentPortfolioError = new javax.swing.JLabel();
lblNewStudentUsernameError = new javax.swing.JLabel();
tbdpAdmin = new javax.swing.JTabbedPane();
pnlHomeAdmin = new javax.swing.JPanel();
btnHelp4 = new javax.swing.JButton();
lblLogoHome1 = new javax.swing.JLabel();
jScrollPane4 = new javax.swing.JScrollPane();
txaAdminDetails = new javax.swing.JTextArea();
jScrollPane5 = new javax.swing.JScrollPane();
txaAdmin = new javax.swing.JTextArea();
btnNewAdmin = new javax.swing.JButton();
btnConfirmPortfolio = new javax.swing.JButton();
pnlEvents1 = new javax.swing.JPanel();
btnHelp5 = new javax.swing.JButton();
jScrollPane6 = new javax.swing.JScrollPane();
tblEventsAdmin = new javax.swing.JTable();
btnSearchDate = new javax.swing.JButton();
btnSearchName = new javax.swing.JButton();
btnSearchID = new javax.swing.JButton();
txfSearchID = new javax.swing.JTextField();
txfSearchName = new javax.swing.JTextField();
jScrollPane7 = new javax.swing.JScrollPane();
txaAdmin1 = new javax.swing.JTextArea();
btnNewEvent = new javax.swing.JButton();
btnEditEvent = new javax.swing.JButton();
dpkSearch = new com.github.lgooddatepicker.components.DatePicker();
pnlParticipation1 = new javax.swing.JPanel();
btnHelp13 = new javax.swing.JButton();
jScrollPane8 = new javax.swing.JScrollPane();
tblParticipationAdmin = new javax.swing.JTable();
btnSearchPortfolioPart = new javax.swing.JButton();
btnSearchStudentNamePart = new javax.swing.JButton();
tbnSearchEventIDPart = new javax.swing.JButton();
txfSearchPortfolioPart = new javax.swing.JTextField();
txfSearchEventIDPart = new javax.swing.JTextField();
txfSearchStudentNamePart = new javax.swing.JTextField();
jScrollPane9 = new javax.swing.JScrollPane();
txaAdmin2 = new javax.swing.JTextArea();
txfSearchEventNamePart = new javax.swing.JTextField();
btnSearchEventNamePart = new javax.swing.JButton();
txfDeleteStuID = new javax.swing.JTextField();
txfDeleteEventID = new javax.swing.JTextField();
btnDeleteParticipation = new javax.swing.JButton();
txfCreatePartStuID = new javax.swing.JTextField();
```

```
txfCreatePartEventID = new javax.swing.JTextField();
btnCreateParticipationAdmin = new javax.swing.JButton();
pnlStudent = new javax.swing.JPanel();
btnHelp14 = new javax.swing.JButton();
jScrollPane10 = new javax.swing.JScrollPane();
tblStudents = new javax.swing.JTable();
btnSearchStuID = new javax.swing.JButton();
btnSearchStuName = new javax.swing.JButton();
btnSearchStuGrade = new javax.swing.JButton();
txfSearchStuID = new javax.swing.JTextField();
txfSearchStuGrade = new javax.swing.JTextField();
txfSearchStuName = new javax.swing.JTextField();
jScrollPane11 = new javax.swing.JScrollPane();
txaAdmin3 = new javax.swing.JTextArea();
tbdpStudent = new javax.swing.JTabbedPane();
pnlHome = new javax.swing.JPanel();
btnHelp2 = new javax.swing.JButton();
jScrollPane1 = new javax.swing.JScrollPane();
txaStudent = new javax.swing.JTextArea();
lblLogoHome = new javax.swing.JLabel();
btnChangeStudent = new javax.swing.JButton();
pnlEvents = new javax.swing.JPanel();
btnHelp3 = new javax.swing.JButton();
jScrollPane2 = new javax.swing.JScrollPane();
tblEventStu = new javax.swing.JTable();
btnSearchEventDate = new javax.swing.JButton();
tbnSearchEventName = new javax.swing.JButton();
btnSearchEventID = new javax.swing.JButton();
tfxSearchEventID = new javax.swing.JTextField();
txfSearchEventName = new javax.swing.JTextField();
dpkSearchDate = new com.github.lgooddatepicker.components.DatePicker();
pnlParticipation = new javax.swing.JPanel();
btnHelp12 = new javax.swing.JButton();
jScrollPane3 = new javax.swing.JScrollPane();
tblParticipation = new javax.swing.JTable();
btnSearchPartPortfolio = new javax.swing.JButton();
btnSearchPartName = new javax.swing.JButton();
btnSearchPartID = new javax.swing.JButton();
txfSearchPartPortfolio = new javax.swing.JTextField();
txfSearchPartID = new javax.swing.JTextField();
txfSearchPartName = new javax.swing.JTextField();
btnDeleteStuPart = new javax.swing.JButton();
btnCreateParticipation = new javax.swing.JButton();
txfEventIDDelete = new javax.swing.JTextField();
lblCreateParticipation = new javax.swing.JLabel();
txfEventIDCreate = new javax.swing.JTextField();
lblDeleteParticipation = new javax.swing.JLabel();
pnlHelp = new javax.swing.JPanel();
btnExitHelp = new javax.swing.JButton();
```

```
        btnHowLogIn = new javax.swing.JButton();
        btnHowSignUp = new javax.swing.JButton();
        btnHowNewAdmin = new javax.swing.JButton();
        btnHowEditMyAccountStudent = new javax.swing.JButton();
        btnHowSearchEventStu = new javax.swing.JButton();
        btnHowCreateEvent = new javax.swing.JButton();
        btnHowDeleteParticipationAdmin = new javax.swing.JButton();
        btnHowSearchStudent = new javax.swing.JButton();
        btnHowEditEvent = new javax.swing.JButton();
        btnHowSearchParticipationStu = new javax.swing.JButton();
        btnHowMakeParticipationStudent = new javax.swing.JButton();
        btnHowEditAccountAdmin = new javax.swing.JButton();
        btnHowMakeParticipation = new javax.swing.JButton();
        btnHowDeleteParticipationStudent = new javax.swing.JButton();
        lblHelpContact = new javax.swing.JLabel();
        btnHowSearchParticipationAdmin = new javax.swing.JButton();
        btnHowSearchEventAdmin = new javax.swing.JButton();

        dlgEditStudent.getContentPane().setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());

        cbxCatering1.setText("Catering");
        dlgEditStudent.getContentPane().add(cbxCatering1, new
org.netbeans.lib.awtextra.AbsoluteConstraints(389, 394, -1, -1));

        cbxAV1.setText("AV");
        dlgEditStudent.getContentPane().add(cbxAV1, new
org.netbeans.lib.awtextra.AbsoluteConstraints(389, 446, -1, -1));

        cbxFirstAid1.setText("First-Aid");
        cbxFirstAid1.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                cbxFirstAid1ActionPerformed(evt);
            }
        });
        dlgEditStudent.getContentPane().add(cbxFirstAid1, new
org.netbeans.lib.awtextra.AbsoluteConstraints(389, 420, -1, -1));

        cbxPhotography1.setText("Photography");
        dlgEditStudent.getContentPane().add(cbxPhotography1, new
org.netbeans.lib.awtextra.AbsoluteConstraints(389, 368, -1, -1));

        lblEditStudent.setFont(new java.awt.Font("Serif", 0, 48)); // NOI18N
        lblEditStudent.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
        lblEditStudent.setText("Edit Your Account");
        dlgEditStudent.getContentPane().add(lblEditStudent, new
org.netbeans.lib.awtextra.AbsoluteConstraints(10, 11, 880, -1));

        txfEditStuClass.setText("jTextField1");
```

```
        dlgEditStudent.getContentPane().add(txfEditStuClass, new
org.netbeans.lib.awtextra.AbsoluteConstraints(389, 258, 110, 25));

        bngEditStuGender.add(rbnEditStuGenderM);
        rbnEditStuGenderM.setText("M");
        dlgEditStudent.getContentPane().add(rbnEditStuGenderM, new
org.netbeans.lib.awtextra.AbsoluteConstraints(389, 301, -1, -1));

        bngEditStuGender.add(rbnEditStuGenderF);
        rbnEditStuGenderF.setText("F");
        dlgEditStudent.getContentPane().add(rbnEditStuGenderF, new
org.netbeans.lib.awtextra.AbsoluteConstraints(389, 327, -1, -1));

        txfEditStuSurname.setText("jTextField1");
        dlgEditStudent.getContentPane().add(txfEditStuSurname, new
org.netbeans.lib.awtextra.AbsoluteConstraints(389, 172, 110, 25));

        txfEditStuGrade.setText("jTextField1");
        dlgEditStudent.getContentPane().add(txfEditStuGrade, new
org.netbeans.lib.awtextra.AbsoluteConstraints(389, 215, 110, 25));

        txfEditStuName.setText("jTextField1");
        dlgEditStudent.getContentPane().add(txfEditStuName, new
org.netbeans.lib.awtextra.AbsoluteConstraints(389, 129, 110, 25));

        lblEditStuPortfolios.setText("Portfolios:");
        dlgEditStudent.getContentPane().add(lblEditStuPortfolios, new
org.netbeans.lib.awtextra.AbsoluteConstraints(290, 370, 70, 20));

        lblEditStuName.setText("First Name:");
        dlgEditStudent.getContentPane().add(lblEditStuName, new
org.netbeans.lib.awtextra.AbsoluteConstraints(290, 130, 80, -1));

        lblEditStuSurname.setText("Surname:");
        dlgEditStudent.getContentPane().add(lblEditStuSurname, new
org.netbeans.lib.awtextra.AbsoluteConstraints(290, 170, 70, -1));

        lblEditStuGrade.setText("Grade:");
        dlgEditStudent.getContentPane().add(lblEditStuGrade, new
org.netbeans.lib.awtextra.AbsoluteConstraints(290, 220, 60, -1));

        lblEditStuClass.setText("Class:");
        dlgEditStudent.getContentPane().add(lblEditStuClass, new
org.netbeans.lib.awtextra.AbsoluteConstraints(290, 260, 50, -1));

        lblEditStuGender.setText("Gender:");
        dlgEditStudent.getContentPane().add(lblEditStuGender, new
org.netbeans.lib.awtextra.AbsoluteConstraints(290, 310, 60, -1));
```

```
        lblEditStuSurnameError.setForeground(new java.awt.Color(255, 0, 0));
        lblEditStuSurnameError.setText("jLabel7");
        dlgEditStudent.getContentPane().add(lblEditStuSurnameError, new
org.netbeans.lib.awtextra.AbsoluteConstraints(520, 180, 280, 20));

        lblEditStuGradeError.setForeground(new java.awt.Color(255, 0, 0));
        lblEditStuGradeError.setText("jLabel7");
        dlgEditStudent.getContentPane().add(lblEditStuGradeError, new
org.netbeans.lib.awtextra.AbsoluteConstraints(520, 220, 320, 20));

        lblEditStuClassError.setForeground(new java.awt.Color(255, 0, 0));
        lblEditStuClassError.setText("jLabel7");
        dlgEditStudent.getContentPane().add(lblEditStuClassError, new
org.netbeans.lib.awtextra.AbsoluteConstraints(520, 260, 290, 20));

        lblEditStuGenderError.setForeground(new java.awt.Color(255, 0, 0));
        lblEditStuGenderError.setText("jLabel7");
        dlgEditStudent.getContentPane().add(lblEditStuGenderError, new
org.netbeans.lib.awtextra.AbsoluteConstraints(520, 310, 310, 20));

        lblEditStuPortfoliosError.setText("jLabel7");
        lblEditStuPortfoliosError.setForeground(new java.awt.Color(255, 0, 0));
        dlgEditStudent.getContentPane().add(lblEditStuPortfoliosError, new
org.netbeans.lib.awtextra.AbsoluteConstraints(520, 370, 280, 20));

        lblEditStuNameError.setForeground(new java.awt.Color(255, 0, 0));
        lblEditStuNameError.setText("lblEditStuNameError");
        dlgEditStudent.getContentPane().add(lblEditStuNameError, new
org.netbeans.lib.awtextra.AbsoluteConstraints(520, 130, 250, 20));

        btnEditStuCancel.setText("Cancel");
        btnEditStuCancel.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnEditStuCancelActionPerformed(evt);
            }
        });
        dlgEditStudent.getContentPane().add(btnEditStuCancel, new
org.netbeans.lib.awtextra.AbsoluteConstraints(450, 480, 110, 40));

        btnEditStuEnter.setText("Enter");
        btnEditStuEnter.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnEditStuEnterActionPerformed(evt);
            }
        });
        dlgEditStudent.getContentPane().add(btnEditStuEnter, new
org.netbeans.lib.awtextra.AbsoluteConstraints(270, 480, 110, 40));

        dlgEvents.setTitle("New Event");
```

```
        dlgEvents.setEnabled(false);
        dlgEvents.addComponentListener(new java.awt.event.ComponentAdapter() {
            public void componentShown(java.awt.event.ComponentEvent evt) {
                dlgEventsComponentShown(evt);
            }
        });
        dlgEvents.getContentPane().setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());
        dlgEvents.getContentPane().add(txfNewEventName, new
org.netbeans.lib.awtextra.AbsoluteConstraints(432, 131, 130, -1));

        bngNewEvent.add(rbnNewEventCatering);
        rbnNewEventCatering.setText("Catering");
        dlgEvents.getContentPane().add(rbnNewEventCatering, new
org.netbeans.lib.awtextra.AbsoluteConstraints(430, 290, -1, -1));

        bngNewEvent.add(rbnNewEventAV);
        rbnNewEventAV.setText("AV");
        dlgEvents.getContentPane().add(rbnNewEventAV, new
org.netbeans.lib.awtextra.AbsoluteConstraints(430, 330, -1, -1));

        bngNewEvent.add(rbnNewEventFirstAid);
        rbnNewEventFirstAid.setText("First-Aid");
        dlgEvents.getContentPane().add(rbnNewEventFirstAid, new
org.netbeans.lib.awtextra.AbsoluteConstraints(430, 350, -1, -1));

        bngNewEvent.add(rbnNewEventPhotography);
        rbnNewEventPhotography.setText("Photography");
        dlgEvents.getContentPane().add(rbnNewEventPhotography, new
org.netbeans.lib.awtextra.AbsoluteConstraints(430, 310, -1, -1));
        dlgEvents.getContentPane().add(txfNewEventHours, new
org.netbeans.lib.awtextra.AbsoluteConstraints(432, 178, 130, -1));

        lblNewEvent.setFont(new java.awt.Font("Serif", 0, 48)); // NOI18N
        lblNewEvent.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
        lblNewEvent.setText("New Event");
        dlgEvents.getContentPane().add(lblNewEvent, new
org.netbeans.lib.awtextra.AbsoluteConstraints(10, 11, 880, -1));

        lblNewEventName.setFont(new java.awt.Font("Serif", 0, 14)); // NOI18N
        lblNewEventName.setText("Enter event Name: ");
        dlgEvents.getContentPane().add(lblNewEventName, new
org.netbeans.lib.awtextra.AbsoluteConstraints(306, 130, -1, -1));

        lblNewEventHours.setFont(new java.awt.Font("Serif", 0, 14)); // NOI18N
        lblNewEventHours.setText("Enter Event Hours: ");
        dlgEvents.getContentPane().add(lblNewEventHours, new
org.netbeans.lib.awtextra.AbsoluteConstraints(303, 177, -1, -1));

        lblNewEventDate.setText("Select Event Date: ");
```

```
        lblNewEventDate.setFont(new java.awt.Font("Serif", 0, 14)); // NOI18N
        dlgEvents.getContentPane().add(lblNewEventDate, new
org.netbeans.lib.awtextra.AbsoluteConstraints(300, 230, -1, -1));

        lblNewEventPortfolio.setFont(new java.awt.Font("Serif", 0, 14)); // NOI18N
        lblNewEventPortfolio.setText("Select Event Portfolio:");
        dlgEvents.getContentPane().add(lblNewEventPortfolio, new
org.netbeans.lib.awtextra.AbsoluteConstraints(290, 290, -1, -1));
        dlgEvents.getContentPane().add(dpkDate, new org.netbeans.lib.awtextra.AbsoluteConstraints(430,
230, -1, -1));

        btnNewEventEnter.setText("Add");
        btnNewEventEnter.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnNewEventEnterActionPerformed(evt);
            }
        });
        dlgEvents.getContentPane().add(btnNewEventEnter, new
org.netbeans.lib.awtextra.AbsoluteConstraints(273, 408, 126, 45));

        btnNewEventClose.setText("Cancel");
        btnNewEventClose.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnNewEventCloseActionPerformed(evt);
            }
        });
        dlgEvents.getContentPane().add(btnNewEventClose, new
org.netbeans.lib.awtextra.AbsoluteConstraints(445, 408, 126, 45));

        lblNewNameError.setText("jLabel1");
        lblNewNameError.setForeground(new java.awt.Color(255, 0, 0));
        dlgEvents.getContentPane().add(lblNewNameError, new
org.netbeans.lib.awtextra.AbsoluteConstraints(600, 130, -1, -1));

        lblNewHoursError.setText("jLabel2");
        lblNewHoursError.setForeground(new java.awt.Color(255, 0, 0));
        dlgEvents.getContentPane().add(lblNewHoursError, new
org.netbeans.lib.awtextra.AbsoluteConstraints(600, 180, -1, -1));

        lblNewDateError.setText("jLabel3");
        lblNewDateError.setForeground(new java.awt.Color(255, 0, 0));
        dlgEvents.getContentPane().add(lblNewDateError, new
org.netbeans.lib.awtextra.AbsoluteConstraints(600, 240, -1, -1));

        lblNewPortfolioError.setText("jLabel4");
        lblNewPortfolioError.setForeground(new java.awt.Color(255, 0, 0));
        dlgEvents.getContentPane().add(lblNewPortfolioError, new
org.netbeans.lib.awtextra.AbsoluteConstraints(600, 290, -1, -1));
```

```
        dlgEditEvent.setTitle("New Event");
        dlgEditEvent.setEnabled(false);
        dlgEditEvent.addComponentListener(new java.awt.event.ComponentAdapter() {
            public void componentShown(java.awt.event.ComponentEvent evt) {
                dlgEditEventComponentShown(evt);
            }
        });
        dlgEditEvent.getContentPane().setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());

        txfEditEventID.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                txfEditEventIDActionPerformed(evt);
            }
        });
        dlgEditEvent.getContentPane().add(txfEditEventID, new
org.netbeans.lib.awtextra.AbsoluteConstraints(430, 90, 130, -1));

        bngEditEvent.add(rbnEditEventCatering);
        rbnEditEventCatering.setText("Catering");
        dlgEditEvent.getContentPane().add(rbnEditEventCatering, new
org.netbeans.lib.awtextra.AbsoluteConstraints(440, 290, -1, -1));

        bngEditEvent.add(rbnEditEventAV);
        rbnEditEventAV.setText("AV");
        dlgEditEvent.getContentPane().add(rbnEditEventAV, new
org.netbeans.lib.awtextra.AbsoluteConstraints(440, 330, -1, -1));

        bngEditEvent.add(rbnEditEventFirstAid);
        rbnEditEventFirstAid.setText("First-Aid");
        dlgEditEvent.getContentPane().add(rbnEditEventFirstAid, new
org.netbeans.lib.awtextra.AbsoluteConstraints(440, 350, -1, -1));

        bngEditEvent.add(rbnEditEventPhotography);
        rbnEditEventPhotography.setText("Photography");
        dlgEditEvent.getContentPane().add(rbnEditEventPhotography, new
org.netbeans.lib.awtextra.AbsoluteConstraints(440, 310, -1, -1));
        dlgEditEvent.getContentPane().add(txfEditEventHours, new
org.netbeans.lib.awtextra.AbsoluteConstraints(432, 178, 130, -1));

        lblEditEvent.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
        lblEditEvent.setText("Edit Event");
        lblEditEvent.setFont(new java.awt.Font("Serif", 0, 48)); // NOI18N
        dlgEditEvent.getContentPane().add(lblEditEvent, new
org.netbeans.lib.awtextra.AbsoluteConstraints(10, 11, 880, -1));

        lblEditEventID.setFont(new java.awt.Font("Serif", 0, 14)); // NOI18N
        lblEditEventID.setText("Enter event ID: ");
        dlgEditEvent.getContentPane().add(lblEditEventID, new
org.netbeans.lib.awtextra.AbsoluteConstraints(300, 90, -1, -1));
```

```
        lblEditEventHours.setFont(new java.awt.Font("Serif", 0, 14)); // NOI18N
        lblEditEventHours.setText("Enter Event Hours: ");
        dlgEditEvent.getContentPane().add(lblEditEventHours, new
org.netbeans.lib.awtextra.AbsoluteConstraints(303, 177, -1, -1));

        lblEditEventDate.setFont(new java.awt.Font("Serif", 0, 14)); // NOI18N
        lblEditEventDate.setText("Select Event Date: ");
        dlgEditEvent.getContentPane().add(lblEditEventDate, new
org.netbeans.lib.awtextra.AbsoluteConstraints(300, 230, -1, -1));

        lblEditEventPortfolio.setFont(new java.awt.Font("Serif", 0, 14)); // NOI18N
        lblEditEventPortfolio.setText("Select Event Portfolio:");
        dlgEditEvent.getContentPane().add(lblEditEventPortfolio, new
org.netbeans.lib.awtextra.AbsoluteConstraints(300, 290, -1, -1));
        dlgEditEvent.getContentPane().add(dpkEditDate, new
org.netbeans.lib.awtextra.AbsoluteConstraints(430, 230, -1, -1));

        btnEditEventSearch.setText("Find Event");
        btnEditEventSearch.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnEditEventSearchActionPerformed(evt);
            }
        });
        dlgEditEvent.getContentPane().add(btnEditEventSearch, new
org.netbeans.lib.awtextra.AbsoluteConstraints(650, 70, 126, 45));

        btnEditEventClose.setText("Cancel");
        btnEditEventClose.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnEditEventCloseActionPerformed(evt);
            }
        });
        dlgEditEvent.getContentPane().add(btnEditEventClose, new
org.netbeans.lib.awtextra.AbsoluteConstraints(445, 408, 126, 45));

        lblEditIDError.setForeground(new java.awt.Color(255, 0, 0));
        lblEditIDError.setText("jLabel1");
        dlgEditEvent.getContentPane().add(lblEditIDError, new
org.netbeans.lib.awtextra.AbsoluteConstraints(600, 90, -1, -1));

        lblEditHoursError.setForeground(new java.awt.Color(255, 0, 0));
        lblEditHoursError.setText("jLabel2");
        dlgEditEvent.getContentPane().add(lblEditHoursError, new
org.netbeans.lib.awtextra.AbsoluteConstraints(600, 180, -1, -1));

        lblEditDateError.setForeground(new java.awt.Color(255, 0, 0));
        lblEditDateError.setText("jLabel3");
```

```
        dlgEditEvent.getContentPane().add(lblEditDateError, new
org.netbeans.lib.awtextra.AbsoluteConstraints(600, 240, -1, -1));

        lblEditPortfolioError.setForeground(new java.awt.Color(255, 0, 0));
        lblEditPortfolioError.setText("jLabel4");
        dlgEditEvent.getContentPane().add(lblEditPortfolioError, new
org.netbeans.lib.awtextra.AbsoluteConstraints(600, 290, -1, -1));

        lblEditEventName.setFont(new java.awt.Font("Serif", 0, 14)); // NOI18N
        lblEditEventName.setText("Enter event Name: ");
        dlgEditEvent.getContentPane().add(lblEditEventName, new
org.netbeans.lib.awtextra.AbsoluteConstraints(306, 130, -1, -1));
        dlgEditEvent.getContentPane().add(txfEditEventName, new
org.netbeans.lib.awtextra.AbsoluteConstraints(432, 131, 130, -1));

        lblEditNameError.setForeground(new java.awt.Color(255, 0, 0));
        lblEditNameError.setText("jLabel1");
        dlgEditEvent.getContentPane().add(lblEditNameError, new
org.netbeans.lib.awtextra.AbsoluteConstraints(600, 130, -1, -1));

        btnEditEventEnter.setText("Enter");
        btnEditEventEnter.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnEditEventEnterActionPerformed(evt);
            }
        });
        dlgEditEvent.getContentPane().add(btnEditEventEnter, new
org.netbeans.lib.awtextra.AbsoluteConstraints(273, 408, 126, 45));

        dlgEditAdmin.getContentPane().setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());

        cbxCateringAdmin.setText("Catering");
        dlgEditAdmin.getContentPane().add(cbxCateringAdmin, new
org.netbeans.lib.awtextra.AbsoluteConstraints(390, 290, -1, -1));

        cbxAVAdmin.setText("AV");
        dlgEditAdmin.getContentPane().add(cbxAVAdmin, new
org.netbeans.lib.awtextra.AbsoluteConstraints(390, 330, -1, -1));

        cbxFirstAidAdmin.setText("First-Aid");
        cbxFirstAidAdmin.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                cbxFirstAidAdminActionPerformed(evt);
            }
        });
        dlgEditAdmin.getContentPane().add(cbxFirstAidAdmin, new
org.netbeans.lib.awtextra.AbsoluteConstraints(390, 310, -1, -1));

        cbxPhotographyAdmin.setText("Photography");
```

```
        dlgEditAdmin.getContentPane().add(cbxPhotographyAdmin, new
org.netbeans.lib.awtextra.AbsoluteConstraints(390, 270, -1, -1));

        lblEditAdmin.setFont(new java.awt.Font("Serif", 0, 48)); // NOI18N
        lblEditAdmin.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
        lblEditAdmin.setText("Edit Your Admin Account");
        dlgEditAdmin.getContentPane().add(lblEditAdmin, new
org.netbeans.lib.awtextra.AbsoluteConstraints(0, 10, 890, -1));

        txfEditAdminSurname.setText("jTextField1");
        dlgEditAdmin.getContentPane().add(txfEditAdminSurname, new
org.netbeans.lib.awtextra.AbsoluteConstraints(390, 200, 120, 25));

        txfEditAdminName.setText("jTextField1");
        dlgEditAdmin.getContentPane().add(txfEditAdminName, new
org.netbeans.lib.awtextra.AbsoluteConstraints(389, 129, 120, 25));

        lblEditAdminPortfolios.setText("Portfolios:");
        dlgEditAdmin.getContentPane().add(lblEditAdminPortfolios, new
org.netbeans.lib.awtextra.AbsoluteConstraints(280, 270, 80, -1));

        lblEditAdminName.setText("First Name:");
        dlgEditAdmin.getContentPane().add(lblEditAdminName, new
org.netbeans.lib.awtextra.AbsoluteConstraints(280, 130, 90, -1));

        lblEditAdminSurname.setText("Surname:");
        dlgEditAdmin.getContentPane().add(lblEditAdminSurname, new
org.netbeans.lib.awtextra.AbsoluteConstraints(280, 200, 80, 20));

        lblEditAdminSurnameError.setForeground(new java.awt.Color(255, 0, 0));
        lblEditAdminSurnameError.setText("jLabel7");
        dlgEditAdmin.getContentPane().add(lblEditAdminSurnameError, new
org.netbeans.lib.awtextra.AbsoluteConstraints(520, 200, 110, 20));

        lblEditAdminPortfoliosError.setForeground(new java.awt.Color(255, 0, 0));
        lblEditAdminPortfoliosError.setText("jLabel7");
        dlgEditAdmin.getContentPane().add(lblEditAdminPortfoliosError, new
org.netbeans.lib.awtextra.AbsoluteConstraints(520, 270, 110, 20));

        lblEditAdminNameError.setForeground(new java.awt.Color(255, 0, 0));
        lblEditAdminNameError.setText("lblEditStuNameError");
        dlgEditAdmin.getContentPane().add(lblEditAdminNameError, new
org.netbeans.lib.awtextra.AbsoluteConstraints(520, 130, 110, 20));

        btnEditAdminCancel.setText("Cancel");
        btnEditAdminCancel.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnEditAdminCancelActionPerformed(evt);
            }
```

```
        });
        dlgEditAdmin.getContentPane().add(btnEditAdminCancel, new
org.netbeans.lib.awtextra.AbsoluteConstraints(450, 430, 110, 40));

        btnEditAdminEnter.setText("Enter");
        btnEditAdminEnter.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnEditAdminEnterActionPerformed(evt);
            }
        });
        dlgEditAdmin.getContentPane().add(btnEditAdminEnter, new
org.netbeans.lib.awtextra.AbsoluteConstraints(270, 430, 110, 40));

        dlgNewAdmin.getContentPane().setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());

        txfNewAdminName.setText("Enter Admin Name");
        txfNewAdminName.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                txfNewAdminNameActionPerformed(evt);
            }
        });
        dlgNewAdmin.getContentPane().add(txfNewAdminName, new
org.netbeans.lib.awtextra.AbsoluteConstraints(380, 90, 160, 30));

        txfNewAdminUsername.setText("Enter Admin username");
        txfNewAdminUsername.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                txfNewAdminUsernameActionPerformed(evt);
            }
        });
        dlgNewAdmin.getContentPane().add(txfNewAdminUsername, new
org.netbeans.lib.awtextra.AbsoluteConstraints(380, 140, 160, 30));

        txfNewAdminPassword.setText("Enter Admin password");
        txfNewAdminPassword.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                txfNewAdminPasswordActionPerformed(evt);
            }
        });
        dlgNewAdmin.getContentPane().add(txfNewAdminPassword, new
org.netbeans.lib.awtextra.AbsoluteConstraints(380, 190, 160, 30));

        lblNewAdminPortfolios.setText("Select  New Admin portfolios:");
        dlgNewAdmin.getContentPane().add(lblNewAdminPortfolios, new
org.netbeans.lib.awtextra.AbsoluteConstraints(170, 290, 170, 30));

        cbxNewAdminPhotography.setText("Photography");
        dlgNewAdmin.getContentPane().add(cbxNewAdminPhotography, new
org.netbeans.lib.awtextra.AbsoluteConstraints(380, 300, -1, -1));
```

```
        cbxNewAdminAv.setText("AV");
        dlgNewAdmin.getContentPane().add(cbxNewAdminAv, new
org.netbeans.lib.awtextra.AbsoluteConstraints(380, 360, -1, -1));

        cbxNewAdminFirstAid.setText("First Aid");
        dlgNewAdmin.getContentPane().add(cbxNewAdminFirstAid, new
org.netbeans.lib.awtextra.AbsoluteConstraints(380, 340, -1, -1));

        cbxNewAdminCatering.setText("Catering");
        dlgNewAdmin.getContentPane().add(cbxNewAdminCatering, new
org.netbeans.lib.awtextra.AbsoluteConstraints(380, 320, -1, -1));

        lblNewAdminUsernameError.setForeground(new java.awt.Color(255, 0, 0));
        lblNewAdminUsernameError.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
        lblNewAdminUsernameError.setText("jLabel1");
        dlgNewAdmin.getContentPane().add(lblNewAdminUsernameError, new
org.netbeans.lib.awtextra.AbsoluteConstraints(540, 150, 160, -1));

        lblNewAdminPasswordError.setForeground(new java.awt.Color(255, 0, 0));
        lblNewAdminPasswordError.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
        lblNewAdminPasswordError.setText("jLabel1");
        dlgNewAdmin.getContentPane().add(lblNewAdminPasswordError, new
org.netbeans.lib.awtextra.AbsoluteConstraints(540, 200, 160, -1));

        lblNewAdminConfirmPasswordError.setForeground(new java.awt.Color(255, 0, 0));
        lblNewAdminConfirmPasswordError.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
        lblNewAdminConfirmPasswordError.setText("jLabel1");
        dlgNewAdmin.getContentPane().add(lblNewAdminConfirmPasswordError, new
org.netbeans.lib.awtextra.AbsoluteConstraints(540, 250, 160, -1));

        lblNewAdminPortfoliosError.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
        lblNewAdminPortfoliosError.setText("jLabel1");
        lblNewAdminPortfoliosError.setForeground(new java.awt.Color(255, 0, 0));
        dlgNewAdmin.getContentPane().add(lblNewAdminPortfoliosError, new
org.netbeans.lib.awtextra.AbsoluteConstraints(580, 294, 70, 10));

        lblNewAdminNameError.setForeground(new java.awt.Color(255, 0, 0));
        lblNewAdminNameError.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
        lblNewAdminNameError.setText("jLabel1");
        dlgNewAdmin.getContentPane().add(lblNewAdminNameError, new
org.netbeans.lib.awtextra.AbsoluteConstraints(540, 100, 160, -1));

        btnEditAdminCancel1.setText("Cancel");
        btnEditAdminCancel1.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnEditAdminCancel1ActionPerformed(evt);
            }
        });
```

```
        dlgNewAdmin.getContentPane().add(btnEditAdminCancel1, new
org.netbeans.lib.awtextra.AbsoluteConstraints(450, 400, 110, 40));

        btnEditAdminEnter1.setText("Enter");
        btnEditAdminEnter1.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnEditAdminEnter1ActionPerformed(evt);
            }
        });
        dlgNewAdmin.getContentPane().add(btnEditAdminEnter1, new
org.netbeans.lib.awtextra.AbsoluteConstraints(230, 400, 110, 40));

        lblNewAdminAccount.setFont(new java.awt.Font("Serif", 0, 48)); // NOI18N
        lblNewAdminAccount.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
        lblNewAdminAccount.setText("Create New Admin Account");
        dlgNewAdmin.getContentPane().add(lblNewAdminAccount, new
org.netbeans.lib.awtextra.AbsoluteConstraints(10, 11, 880, -1));

        txfNewAdminConfirmPassword.setText("Enter Admin password");
        txfNewAdminConfirmPassword.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                txfNewAdminConfirmPasswordActionPerformed(evt);
            }
        });
        dlgNewAdmin.getContentPane().add(txfNewAdminConfirmPassword, new
org.netbeans.lib.awtextra.AbsoluteConstraints(380, 240, 160, 30));

        lblNewAdminPassword.setText("Enter New Admin Password:");
        dlgNewAdmin.getContentPane().add(lblNewAdminPassword, new
org.netbeans.lib.awtextra.AbsoluteConstraints(170, 190, 180, 30));

        lblNewAdminName.setText("Enter New Admin Name:");
        dlgNewAdmin.getContentPane().add(lblNewAdminName, new
org.netbeans.lib.awtextra.AbsoluteConstraints(170, 90, 150, 30));

        lblNewAdminConfirmPassword.setText("Confirm New Admin Password:");
        dlgNewAdmin.getContentPane().add(lblNewAdminConfirmPassword, new
org.netbeans.lib.awtextra.AbsoluteConstraints(170, 240, 180, 30));

        lblNewAdminUsername.setText("Enter New Admin Username:");
        dlgNewAdmin.getContentPane().add(lblNewAdminUsername, new
org.netbeans.lib.awtextra.AbsoluteConstraints(170, 140, 170, 30));

        setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
        setMinimumSize(new java.awt.Dimension(900, 550));
        getContentPane().setLayout(new java.awt.CardLayout());

        pnlLogIn.setBackground(new java.awt.Color(255, 255, 255));
        pnlLogIn.setMaximumSize(new java.awt.Dimension(900, 550));
```

```
pnlLogIn.setMinimumSize(new java.awt.Dimension(890, 490));
pnlLogIn.setPreferredSize(new java.awt.Dimension(900, 550));
pnlLogIn.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());

btnSignUp.setText("Sign-up");
btnSignUp.setFont(new java.awt.Font("Tahoma", 0, 12)); // NOI18N
btnSignUp.setPreferredSize(new java.awt.Dimension(30, 30));
btnSignUp.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        btnSignUpActionPerformed(evt);
    }
});
pnlLogIn.add(btnSignUp, new org.netbeans.lib.awtextra.AbsoluteConstraints(799, 450, 90, 38));

btnHelp10.setText("?");
btnHelp10.setFont(new java.awt.Font("Tahoma", 0, 12)); // NOI18N
btnHelp10.setPreferredSize(new java.awt.Dimension(30, 30));
btnHelp10.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        btnHelp10ActionPerformed(evt);
    }
});
pnlLogIn.add(btnHelp10, new org.netbeans.lib.awtextra.AbsoluteConstraints(851, 11, 39, 38));

lblPassword1.setFont(new java.awt.Font("Tahoma", 0, 12)); // NOI18N
lblPassword1.setText("Password:");
pnlLogIn.add(lblPassword1, new org.netbeans.lib.awtextra.AbsoluteConstraints(290, 260, 60, 30));

lblUsername1.setFont(new java.awt.Font("Tahoma", 0, 12)); // NOI18N
lblUsername1.setText("Username:");
pnlLogIn.add(lblUsername1, new org.netbeans.lib.awtextra.AbsoluteConstraints(290, 180, 60, 30));

txfPassword.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        txfPasswordActionPerformed(evt);
    }
});
pnlLogIn.add(txfPassword, new org.netbeans.lib.awtextra.AbsoluteConstraints(400, 260, 160, 30));

txfUsername.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        txfUsernameActionPerformed(evt);
    }
});
pnlLogIn.add(txfUsername, new org.netbeans.lib.awtextra.AbsoluteConstraints(400, 180, 160, 30));

btnLogIn.setText("Log-in");
btnLogIn.setFont(new java.awt.Font("Tahoma", 0, 12)); // NOI18N
btnLogIn.addActionListener(new java.awt.event.ActionListener() {
```

```
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnLogInActionPerformed(evt);
            }
        });
        pnlLogIn.add(btnLogIn, new org.netbeans.lib.awtextra.AbsoluteConstraints(280, 350, 290, 50));

        lblLogo1.setIcon(new
javax.swing.ImageIcon("C:\\Users\\ruth2\\OneDrive\\Documents\\NetBeansProjects\\PAT\\curro_logo.png
")); // NOI18N
        lblLogo1.setText("jLabel2");
        pnlLogIn.add(lblLogo1, new org.netbeans.lib.awtextra.AbsoluteConstraints(10, -20, 230, 170));

        getContentPane().add(pnlLogIn, "card2");

        pnlSignUp.setBackground(new java.awt.Color(255, 255, 255));
        pnlSignUp.setMinimumSize(new java.awt.Dimension(890, 490));
        pnlSignUp.setPreferredSize(new java.awt.Dimension(200, 500));
        pnlSignUp.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());

        btnHelp11.setText("?");
        btnHelp11.setFont(new java.awt.Font("Tahoma", 0, 12)); // NOI18N
        btnHelp11.setPreferredSize(new java.awt.Dimension(30, 30));
        btnHelp11.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnHelp11ActionPerformed(evt);
            }
        });
        pnlSignUp.add(btnHelp11, new org.netbeans.lib.awtextra.AbsoluteConstraints(851, 11, 39, 38));

        lblNewStudentPortfolios.setText("Your Portfolios:");
        lblNewStudentPortfolios.setFont(new java.awt.Font("Tahoma", 0, 12)); // NOI18N
        pnlSignUp.add(lblNewStudentPortfolios, new org.netbeans.lib.awtextra.AbsoluteConstraints(830,
130, 100, 30));

        lblUsername.setFont(new java.awt.Font("Tahoma", 0, 12)); // NOI18N
        lblUsername.setText("Username:");
        pnlSignUp.add(lblUsername, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 150, 60, 30));

        txfNewStudentGrade.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                txfNewStudentGradeActionPerformed(evt);
            }
        });
        pnlSignUp.add(txfNewStudentGrade, new org.netbeans.lib.awtextra.AbsoluteConstraints(540, 200,
160, 30));

        txfUsername1.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                txfUsername1ActionPerformed(evt);
```

```
        }
    });
    pnlSignUp.add(txfUsername1, new org.netbeans.lib.awtextra.AbsoluteConstraints(130, 150, 160,
30));

    btnSignUpStu.setText("Sign-up");
    btnSignUpStu.setFont(new java.awt.Font("Tahoma", 0, 12)); // NOI18N
    btnSignUpStu.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            btnSignUpStuActionPerformed(evt);
        }
    });
    pnlSignUp.add(btnSignUpStu, new org.netbeans.lib.awtextra.AbsoluteConstraints(300, 450, 290,
50));

    lblLogo.setIcon(new
javax.swing.ImageIcon("C:\\Users\\ruth2\\OneDrive\\Documents\\NetBeansProjects\\PAT\\curro_logo.png
")); // NOI18N
    lblLogo.setText("jLabel2");
    pnlSignUp.add(lblLogo, new org.netbeans.lib.awtextra.AbsoluteConstraints(10, -20, 230, 170));

    lblPassword2.setFont(new java.awt.Font("Tahoma", 0, 12)); // NOI18N
    lblPassword2.setText("Password:");
    pnlSignUp.add(lblPassword2, new org.netbeans.lib.awtextra.AbsoluteConstraints(60, 210, 60, 30));

    txfPassword2.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            txfPassword2ActionPerformed(evt);
        }
    });
    pnlSignUp.add(txfPassword2, new org.netbeans.lib.awtextra.AbsoluteConstraints(130, 210, 160,
30));

    lblNewStudentGrade.setText("Your Grade (8-12):");
    lblNewStudentGrade.setFont(new java.awt.Font("Tahoma", 0, 12)); // NOI18N
    pnlSignUp.add(lblNewStudentGrade, new org.netbeans.lib.awtextra.AbsoluteConstraints(420, 200,
120, 30));

    lblNewStudentClass.setText("Your class (1-5):");
    lblNewStudentClass.setFont(new java.awt.Font("Tahoma", 0, 12)); // NOI18N
    pnlSignUp.add(lblNewStudentClass, new org.netbeans.lib.awtextra.AbsoluteConstraints(440, 260,
100, 30));

    lblCPassword4.setText("Confirm Password:");
    lblCPassword4.setFont(new java.awt.Font("Tahoma", 0, 12)); // NOI18N
    pnlSignUp.add(lblCPassword4, new org.netbeans.lib.awtextra.AbsoluteConstraints(20, 280, 100,
30));

    txfConfirmPassword1.addActionListener(new java.awt.event.ActionListener() {
```

```
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            txfConfirmPassword1ActionPerformed(evt);
        }
    });
    pnlSignUp.add(txfConfirmPassword1, new org.netbeans.lib.awtextra.AbsoluteConstraints(130, 280,
160, 30));

    txfNewStudentClass.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            txfNewStudentClassActionPerformed(evt);
        }
    });
    pnlSignUp.add(txfNewStudentClass, new org.netbeans.lib.awtextra.AbsoluteConstraints(540, 260,
160, 30));

    txfNewStudentName.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            txfNewStudentNameActionPerformed(evt);
        }
    });
    pnlSignUp.add(txfNewStudentName, new org.netbeans.lib.awtextra.AbsoluteConstraints(540, 140,
160, 30));

    bngNewStudentGender.add(rbnNewStudentF);
    rbnNewStudentF.setText("F");
    pnlSignUp.add(rbnNewStudentF, new org.netbeans.lib.awtextra.AbsoluteConstraints(550, 340, -1, -
1));

    bngNewStudentGender.add(rbnNewStudentM);
    rbnNewStudentM.setText("M");
    pnlSignUp.add(rbnNewStudentM, new org.netbeans.lib.awtextra.AbsoluteConstraints(550, 320, -1, -
1));

    lblNewStudentName.setText("Your Full Name:");
    lblNewStudentName.setFont(new java.awt.Font("Tahoma", 0, 12)); // NOI18N
    pnlSignUp.add(lblNewStudentName, new org.netbeans.lib.awtextra.AbsoluteConstraints(420, 140,
110, 30));

    lblNewStudentGender.setText("Your Gender:");
    lblNewStudentGender.setFont(new java.awt.Font("Tahoma", 0, 12)); // NOI18N
    pnlSignUp.add(lblNewStudentGender, new org.netbeans.lib.awtextra.AbsoluteConstraints(450, 310,
100, 30));

    cbxAV.setText("AV");
    pnlSignUp.add(cbxAV, new org.netbeans.lib.awtextra.AbsoluteConstraints(840, 260, -1, -1));

    cbxPotography.setText("Photography");
    pnlSignUp.add(cbxPotography, new org.netbeans.lib.awtextra.AbsoluteConstraints(840, 200, -1, -
1));
```

```
        cbxFirstAid.setText("First-Aid");
        pnlSignUp.add(cbxFirstAid, new org.netbeans.lib.awtextra.AbsoluteConstraints(840, 230, -1, -1));

        cbxCatering.setText("Catering");
        cbxCatering.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                cbxCateringActionPerformed(evt);
            }
        });
        pnlSignUp.add(cbxCatering, new org.netbeans.lib.awtextra.AbsoluteConstraints(840, 170, -1, -1));

        lblNewStudentPasswordError.setForeground(new java.awt.Color(255, 0, 0));
        lblNewStudentPasswordError.setText("jLabel1");
        pnlSignUp.add(lblNewStudentPasswordError, new
org.netbeans.lib.awtextra.AbsoluteConstraints(300, 220, -1, -1));

        lblNewStudentConfPasswordError.setForeground(new java.awt.Color(255, 0, 0));
        lblNewStudentConfPasswordError.setText("jLabel1");
        pnlSignUp.add(lblNewStudentConfPasswordError, new
org.netbeans.lib.awtextra.AbsoluteConstraints(300, 290, -1, -1));

        lblNewStudentNameError.setForeground(new java.awt.Color(255, 0, 0));
        lblNewStudentNameError.setText("jLabel1");
        pnlSignUp.add(lblNewStudentNameError, new org.netbeans.lib.awtextra.AbsoluteConstraints(720,
150, -1, -1));

        lblNewStudentGradeError.setForeground(new java.awt.Color(255, 0, 0));
        lblNewStudentGradeError.setText("jLabel1");
        pnlSignUp.add(lblNewStudentGradeError, new org.netbeans.lib.awtextra.AbsoluteConstraints(720,
210, -1, -1));

        lblNewStudentClassError.setText("jLabel1");
        lblNewStudentClassError.setForeground(new java.awt.Color(255, 0, 0));
        pnlSignUp.add(lblNewStudentClassError, new org.netbeans.lib.awtextra.AbsoluteConstraints(720,
270, -1, -1));

        lblNewStudentGenderError.setForeground(new java.awt.Color(255, 0, 0));
        lblNewStudentGenderError.setText("jLabel1");
        pnlSignUp.add(lblNewStudentGenderError, new org.netbeans.lib.awtextra.AbsoluteConstraints(610,
330, -1, -1));

        lblNewStudentPortfolioError.setForeground(new java.awt.Color(255, 0, 0));
        lblNewStudentPortfolioError.setText("jLabel1");
        pnlSignUp.add(lblNewStudentPortfolioError, new
org.netbeans.lib.awtextra.AbsoluteConstraints(820, 310, -1, -1));

        lblNewStudentUsernameError.setForeground(new java.awt.Color(255, 0, 0));
        lblNewStudentUsernameError.setText("jLabel1");
```

```
        pnlSignUp.add(lblNewStudentUsernameError, new
org.netbeans.lib.awtextra.AbsoluteConstraints(300, 160, -1, -1));

        getContentPane().add(pnlSignUp, "card2");

        tbdpAdmin.addComponentListener(new java.awt.event.ComponentAdapter() {
            public void componentShown(java.awt.event.ComponentEvent evt) {
                tbdpAdminComponentShown(evt);
            }
        });

        pnlHomeAdmin.setBackground(new java.awt.Color(255, 255, 255));
        pnlHomeAdmin.addComponentListener(new java.awt.event.ComponentAdapter() {
            public void componentShown(java.awt.event.ComponentEvent evt) {
                pnlHomeAdminComponentShown(evt);
            }
        });
        pnlHomeAdmin.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());

        btnHelp4.setText("?");
        btnHelp4.setPreferredSize(new java.awt.Dimension(30, 30));
        btnHelp4.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnHelp4ActionPerformed(evt);
            }
        });
        pnlHomeAdmin.add(btnHelp4, new org.netbeans.lib.awtextra.AbsoluteConstraints(890, 40, 39, 38));

        lblLogoHome1.setIcon(new
javax.swing.ImageIcon(getClass().getResource("/Application/curro_logo.png"))); // NOI18N
        pnlHomeAdmin.add(lblLogoHome1, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 40, -1,
100));

        jScrollPane4.setFont(new java.awt.Font("Tahoma", 0, 12)); // NOI18N

        txaAdminDetails.setColumns(20);
        txaAdminDetails.setFont(new java.awt.Font("Tahoma", 0, 18)); // NOI18N
        txaAdminDetails.setRows(5);
        txaAdminDetails.setText("\t\tName\n\n\n\n    Portfolio \t\tportfolio \t\tportfolio");
        jScrollPane4.setViewportView(txaAdminDetails);

        pnlHomeAdmin.add(jScrollPane4, new org.netbeans.lib.awtextra.AbsoluteConstraints(170, 160,
620, 170));

        txaAdmin.setEditable(false);
        txaAdmin.setBackground(new java.awt.Color(0, 0, 153));
        txaAdmin.setColumns(20);
        txaAdmin.setFont(new java.awt.Font("Tahoma", 0, 14)); // NOI18N
        txaAdmin.setForeground(new java.awt.Color(255, 255, 255));
```

```
        txaAdmin.setRows(5);
        txaAdmin.setText("\t\t\t\t\t            ADMIN");
        txaAdmin.setCaretColor(new java.awt.Color(255, 255, 255));
        txaAdmin.setMaximumSize(new java.awt.Dimension(500, 21));
        txaAdmin.setVerifyInputWhenFocusTarget(false);
        jScrollPane5.setViewportView(txaAdmin);
        txaAdmin.getAccessibleContext().setAccessibleDescription("");

        pnlHomeAdmin.add(jScrollPane5, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 0, 940,
34));

        btnNewAdmin.setText("Create new admin Account");
        btnNewAdmin.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnNewAdminActionPerformed(evt);
            }
        });
        pnlHomeAdmin.add(btnNewAdmin, new org.netbeans.lib.awtextra.AbsoluteConstraints(520, 360,
210, 50));

        btnConfirmPortfolio.setText("Edit Account");
        btnConfirmPortfolio.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnConfirmPortfolioActionPerformed(evt);
            }
        });
        pnlHomeAdmin.add(btnConfirmPortfolio, new org.netbeans.lib.awtextra.AbsoluteConstraints(220,
360, 210, 50));

        tbdpAdmin.addTab("Home", pnlHomeAdmin);

        pnlEvents1.setBackground(new java.awt.Color(255, 255, 255));
        pnlEvents1.addComponentListener(new java.awt.event.ComponentAdapter() {
            public void componentShown(java.awt.event.ComponentEvent evt) {
                pnlEvents1ComponentShown(evt);
            }
        });
        pnlEvents1.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());

        btnHelp5.setText("?");
        btnHelp5.setPreferredSize(new java.awt.Dimension(30, 30));
        btnHelp5.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnHelp5ActionPerformed(evt);
            }
        });
        pnlEvents1.add(btnHelp5, new org.netbeans.lib.awtextra.AbsoluteConstraints(893, 40, 39, 38));

        tblEventsAdmin.setModel(new javax.swing.table.DefaultTableModel(
```

```
            new Object [][] {
                {null, null, null, null, null},
                {null, null, null, null, null},
                {null, null, null, null, null},
                {null, null, null, null, null}
            },
            new String [] {
                "Event ID", "Event Name", "Hours", "Event date", "Portfolio"
            }
        ));
        jScrollPane6.setViewportView(tblEventsAdmin);

        pnlEvents1.add(jScrollPane6, new org.netbeans.lib.awtextra.AbsoluteConstraints(10, 41, 724,
390));

        btnSearchDate.setText("Search by Date");
        btnSearchDate.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnSearchDateActionPerformed(evt);
            }
        });
        pnlEvents1.add(btnSearchDate, new org.netbeans.lib.awtextra.AbsoluteConstraints(770, 140, 139, -
1));

        btnSearchName.setText("Search by name");
        btnSearchName.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnSearchNameActionPerformed(evt);
            }
        });
        pnlEvents1.add(btnSearchName, new org.netbeans.lib.awtextra.AbsoluteConstraints(770, 260, 139,
-1));

        btnSearchID.setText("Search by ID");
        btnSearchID.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnSearchIDActionPerformed(evt);
            }
        });
        pnlEvents1.add(btnSearchID, new org.netbeans.lib.awtextra.AbsoluteConstraints(770, 380, 125, -
1));

        txfSearchID.setText("e.g. 3");
        txfSearchID.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                txfSearchIDActionPerformed(evt);
            }
        });
```

```
        pnlEvents1.add(txfSearchID, new org.netbeans.lib.awtextra.AbsoluteConstraints(770, 340, 125,
32));

        txfSearchName.setText("e.g. Name");
        txfSearchName.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                txfSearchNameActionPerformed(evt);
            }
        });
        pnlEvents1.add(txfSearchName, new org.netbeans.lib.awtextra.AbsoluteConstraints(770, 220, 137,
32));

        txaAdmin1.setColumns(20);
        txaAdmin1.setFont(new java.awt.Font("Tahoma", 0, 14)); // NOI18N
        txaAdmin1.setRows(5);
        txaAdmin1.setText("\t\t\t\t\t            ADMIN");
        txaAdmin1.setBackground(new java.awt.Color(0, 0, 153));
        txaAdmin1.setForeground(new java.awt.Color(255, 255, 255));
        jScrollPane7.setViewportView(txaAdmin1);

        pnlEvents1.add(jScrollPane7, new org.netbeans.lib.awtextra.AbsoluteConstraints(12, 0, 920, 34));

        btnNewEvent.setText("Create new Event");
        btnNewEvent.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnNewEventActionPerformed(evt);
            }
        });
        pnlEvents1.add(btnNewEvent, new org.netbeans.lib.awtextra.AbsoluteConstraints(130, 450, 198,
78));

        btnEditEvent.setText("Edit Event");
        btnEditEvent.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnEditEventActionPerformed(evt);
            }
        });
        pnlEvents1.add(btnEditEvent, new org.netbeans.lib.awtextra.AbsoluteConstraints(400, 450, 198,
78));
        pnlEvents1.add(dpkSearch, new org.netbeans.lib.awtextra.AbsoluteConstraints(770, 110, -1, -1));

        tbdpAdmin.addTab("Events", pnlEvents1);

        pnlParticipation1.setBackground(new java.awt.Color(255, 255, 255));
        pnlParticipation1.addComponentListener(new java.awt.event.ComponentAdapter() {
            public void componentShown(java.awt.event.ComponentEvent evt) {
                pnlParticipation1ComponentShown(evt);
            }
        });
```

```
        pnlParticipation1.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());

        btnHelp13.setText("?");
        btnHelp13.setPreferredSize(new java.awt.Dimension(30, 30));
        btnHelp13.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnHelp13ActionPerformed(evt);
            }
        });
        pnlParticipation1.add(btnHelp13, new org.netbeans.lib.awtextra.AbsoluteConstraints(893, 40, 39,
38));

        tblParticipationAdmin.setModel(new javax.swing.table.DefaultTableModel(
            new Object [][] {
                {null, null, null, null, null, null},
                {null, null, null, null, null, null},
                {null, null, null, null, null, null},
                {null, null, null, null, null, null}
            },
            new String [] {
                "Student ID", "Student Name", "Event ID", "Event name", "Hours", "Portfolio"
            }
        ));
        jScrollPane8.setViewportView(tblParticipationAdmin);

        pnlParticipation1.add(jScrollPane8, new org.netbeans.lib.awtextra.AbsoluteConstraints(10, 50, 724,
389));

        btnSearchPortfolioPart.setText("Search by Portfolio");
        btnSearchPortfolioPart.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnSearchPortfolioPartActionPerformed(evt);
            }
        });
        pnlParticipation1.add(btnSearchPortfolioPart, new
org.netbeans.lib.awtextra.AbsoluteConstraints(779, 139, 153, -1));

        btnSearchStudentNamePart.setText("Search by Student name");
        btnSearchStudentNamePart.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnSearchStudentNamePartActionPerformed(evt);
            }
        });
        pnlParticipation1.add(btnSearchStudentNamePart, new
org.netbeans.lib.awtextra.AbsoluteConstraints(780, 320, -1, -1));

        tbnSearchEventIDPart.setText("Search by event ID");
        tbnSearchEventIDPart.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
```

```
            tbnSearchEventIDPartActionPerformed(evt);
        }
    });
    pnlParticipation1.add(tbnSearchEventIDPart, new
org.netbeans.lib.awtextra.AbsoluteConstraints(780, 410, 150, -1));

    txfSearchPortfolioPart.setText("e.g. Catering");
    txfSearchPortfolioPart.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            txfSearchPortfolioPartActionPerformed(evt);
        }
    });
    pnlParticipation1.add(txfSearchPortfolioPart, new
org.netbeans.lib.awtextra.AbsoluteConstraints(779, 101, 153, 32));

    txfSearchEventIDPart.setText("e.g. 3");
    txfSearchEventIDPart.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            txfSearchEventIDPartActionPerformed(evt);
        }
    });
    pnlParticipation1.add(txfSearchEventIDPart, new
org.netbeans.lib.awtextra.AbsoluteConstraints(780, 370, 150, 32));

    txfSearchStudentNamePart.setText("e.g. Student Name");
    txfSearchStudentNamePart.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            txfSearchStudentNamePartActionPerformed(evt);
        }
    });
    pnlParticipation1.add(txfSearchStudentNamePart, new
org.netbeans.lib.awtextra.AbsoluteConstraints(780, 280, 150, 32));

    txaAdmin2.setColumns(20);
    txaAdmin2.setFont(new java.awt.Font("Tahoma", 0, 14)); // NOI18N
    txaAdmin2.setRows(5);
    txaAdmin2.setText("\t\t\t\t\t            ADMIN");
    txaAdmin2.setBackground(new java.awt.Color(0, 0, 153));
    txaAdmin2.setForeground(new java.awt.Color(255, 255, 255));
    jScrollPane9.setViewportView(txaAdmin2);

    pnlParticipation1.add(jScrollPane9, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 0, 920,
34));

    txfSearchEventNamePart.setText("e.g. Event Name");
    txfSearchEventNamePart.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            txfSearchEventNamePartActionPerformed(evt);
        }
```

```
        });
        pnlParticipation1.add(txfSearchEventNamePart, new
org.netbeans.lib.awtextra.AbsoluteConstraints(780, 190, 150, 32));

        btnSearchEventNamePart.setText("Search by Event Name");
        btnSearchEventNamePart.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnSearchEventNamePartActionPerformed(evt);
            }
        });
        pnlParticipation1.add(btnSearchEventNamePart, new
org.netbeans.lib.awtextra.AbsoluteConstraints(780, 230, 150, -1));

        txfDeleteStuID.setText("Student ID");
        txfDeleteStuID.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                txfDeleteStuIDActionPerformed(evt);
            }
        });
        pnlParticipation1.add(txfDeleteStuID, new org.netbeans.lib.awtextra.AbsoluteConstraints(420, 450,
125, 32));

        txfDeleteEventID.setText("Event ID");
        txfDeleteEventID.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                txfDeleteEventIDActionPerformed(evt);
            }
        });
        pnlParticipation1.add(txfDeleteEventID, new org.netbeans.lib.awtextra.AbsoluteConstraints(420,
490, 125, 32));

        btnDeleteParticipation.setText("Delete Record");
        btnDeleteParticipation.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnDeleteParticipationActionPerformed(evt);
            }
        });
        pnlParticipation1.add(btnDeleteParticipation, new
org.netbeans.lib.awtextra.AbsoluteConstraints(550, 450, 180, 70));

        txfCreatePartStuID.setText("Student ID");
        txfCreatePartStuID.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                txfCreatePartStuIDActionPerformed(evt);
            }
        });
        pnlParticipation1.add(txfCreatePartStuID, new org.netbeans.lib.awtextra.AbsoluteConstraints(10,
450, 125, 32));
```

```
        txfCreatePartEventID.setText("Event ID");
        txfCreatePartEventID.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                txfCreatePartEventIDActionPerformed(evt);
            }
        });
        pnlParticipation1.add(txfCreatePartEventID, new org.netbeans.lib.awtextra.AbsoluteConstraints(10,
490, 125, 32));

        btnCreateParticipationAdmin.setText("Create Record");
        btnCreateParticipationAdmin.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnCreateParticipationAdminActionPerformed(evt);
            }
        });
        pnlParticipation1.add(btnCreateParticipationAdmin, new
org.netbeans.lib.awtextra.AbsoluteConstraints(150, 450, 180, 70));

        tbdpAdmin.addTab("Participation", pnlParticipation1);

        pnlStudent.setBackground(new java.awt.Color(255, 255, 255));
        pnlStudent.addComponentListener(new java.awt.event.ComponentAdapter() {
            public void componentShown(java.awt.event.ComponentEvent evt) {
                pnlStudentComponentShown(evt);
            }
        });
        pnlStudent.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());

        btnHelp14.setText("?");
        btnHelp14.setPreferredSize(new java.awt.Dimension(30, 30));
        btnHelp14.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnHelp14ActionPerformed(evt);
            }
        });
        pnlStudent.add(btnHelp14, new org.netbeans.lib.awtextra.AbsoluteConstraints(893, 40, 39, 38));

        tblStudents.setModel(new javax.swing.table.DefaultTableModel(
            new Object [][] {
                {null, null, null, null, null, null, null, null},
                {null, null, null, null, null, null, null, null},
                {null, null, null, null, null, null, null, null},
                {null, null, null, null, null, null, null, null}
            },
            new String [] {
                "Student ID", "Student Name", "Grade", "Gender", "Catering hours", "AV hours", "Photography
hours", "FirstAid Hours"
            }
        ));
```

```
        jScrollPane10.setViewportView(tblStudents);

        pnlStudent.add(jScrollPane10, new org.netbeans.lib.awtextra.AbsoluteConstraints(10, 44, 765,
420));

        btnSearchStuID.setText("Search by Student ID");
        btnSearchStuID.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnSearchStuIDActionPerformed(evt);
            }
        });
        pnlStudent.add(btnSearchStuID, new org.netbeans.lib.awtextra.AbsoluteConstraints(790, 130, -1, -
1));

        btnSearchStuName.setText("Search by Student name");
        btnSearchStuName.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnSearchStuNameActionPerformed(evt);
            }
        });
        pnlStudent.add(btnSearchStuName, new org.netbeans.lib.awtextra.AbsoluteConstraints(790, 260, -
1, -1));

        btnSearchStuGrade.setText("Search by grade");
        btnSearchStuGrade.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnSearchStuGradeActionPerformed(evt);
            }
        });
        pnlStudent.add(btnSearchStuGrade, new org.netbeans.lib.awtextra.AbsoluteConstraints(790, 400, -
1, -1));

        txfSearchStuID.setText("e.g. 4");
        txfSearchStuID.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                txfSearchStuIDActionPerformed(evt);
            }
        });
        pnlStudent.add(txfSearchStuID, new org.netbeans.lib.awtextra.AbsoluteConstraints(790, 90, 135,
32));

        txfSearchStuGrade.setText("e.g. 11");
        txfSearchStuGrade.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                txfSearchStuGradeActionPerformed(evt);
            }
        });
        pnlStudent.add(txfSearchStuGrade, new org.netbeans.lib.awtextra.AbsoluteConstraints(790, 360,
125, 32));
```

```
        txfSearchStuName.setText("e.g. Student Name");
        txfSearchStuName.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                txfSearchStuNameActionPerformed(evt);
            }
        });
        pnlStudent.add(txfSearchStuName, new org.netbeans.lib.awtextra.AbsoluteConstraints(790, 220,
130, 32));

        txaAdmin3.setColumns(20);
        txaAdmin3.setFont(new java.awt.Font("Tahoma", 0, 14)); // NOI18N
        txaAdmin3.setRows(5);
        txaAdmin3.setText("\t\t\t\t\t              ADMIN");
        txaAdmin3.setBackground(new java.awt.Color(0, 0, 153));
        txaAdmin3.setForeground(new java.awt.Color(255, 255, 255));
        jScrollPane11.setViewportView(txaAdmin3);

        pnlStudent.add(jScrollPane11, new org.netbeans.lib.awtextra.AbsoluteConstraints(0, 0, 920, 34));

        tbdpAdmin.addTab("Students", pnlStudent);

        getContentPane().add(tbdpAdmin, "card10");

        tbdpStudent.setBackground(new java.awt.Color(255, 255, 255));
        tbdpStudent.addComponentListener(new java.awt.event.ComponentAdapter() {
            public void componentResized(java.awt.event.ComponentEvent evt) {
                tbdpStudentComponentResized(evt);
            }
            public void componentShown(java.awt.event.ComponentEvent evt) {
                tbdpStudentComponentShown(evt);
            }
        });

        pnlHome.setBackground(new java.awt.Color(255, 255, 255));
        pnlHome.addComponentListener(new java.awt.event.ComponentAdapter() {
            public void componentShown(java.awt.event.ComponentEvent evt) {
                pnlHomeComponentShown(evt);
            }
        });

        btnHelp2.setText("?");
        btnHelp2.setPreferredSize(new java.awt.Dimension(30, 30));
        btnHelp2.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnHelp2ActionPerformed(evt);
            }
        });
```

```
        jScrollPane1.setFont(new java.awt.Font("Tahoma", 0, 12)); // NOI18N

        txaStudent.setColumns(20);
        txaStudent.setFont(new java.awt.Font("Tahoma", 0, 18)); // NOI18N
        txaStudent.setRows(5);
        txaStudent.setText("Name\t\t\t\tStudentID\n\n\n\t\tGrade - Class - Gender\n\n\n\tPortfolio \t\tportfolio
\t\tportfolio\n\thours\t\thours\t\thours");
        jScrollPane1.setViewportView(txaStudent);

        lblLogoHome.setIcon(new
javax.swing.ImageIcon(getClass().getResource("/Application/curro_logo.png"))); // NOI18N
        lblLogoHome.addComponentListener(new java.awt.event.ComponentAdapter() {
            public void componentShown(java.awt.event.ComponentEvent evt) {
                lblLogoHomeComponentShown(evt);
            }
        });

        btnChangeStudent.setText("Edit Account Details");
        btnChangeStudent.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnChangeStudentActionPerformed(evt);
            }
        });

        javax.swing.GroupLayout pnlHomeLayout = new javax.swing.GroupLayout(pnlHome);
        pnlHome.setLayout(pnlHomeLayout);
        pnlHomeLayout.setHorizontalGroup(
            pnlHomeLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(pnlHomeLayout.createSequentialGroup()
                .addComponent(lblLogoHome, javax.swing.GroupLayout.PREFERRED_SIZE, 230,
javax.swing.GroupLayout.PREFERRED_SIZE)
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
                .addComponent(btnHelp2, javax.swing.GroupLayout.PREFERRED_SIZE, 39,
javax.swing.GroupLayout.PREFERRED_SIZE)
                .addGap(50, 50, 50))
            .addGroup(pnlHomeLayout.createSequentialGroup()

.addGroup(pnlHomeLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                    .addGroup(pnlHomeLayout.createSequentialGroup()
                        .addGap(100, 100, 100)
                        .addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE, 684,
javax.swing.GroupLayout.PREFERRED_SIZE))
                    .addGroup(pnlHomeLayout.createSequentialGroup()
                        .addGap(346, 346, 346)
                        .addComponent(btnChangeStudent, javax.swing.GroupLayout.PREFERRED_SIZE, 173,
javax.swing.GroupLayout.PREFERRED_SIZE)))
                .addContainerGap(159, Short.MAX_VALUE))
        );
```

```
        pnlHomeLayout.setVerticalGroup(
            pnlHomeLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(pnlHomeLayout.createSequentialGroup()

.addGroup(pnlHomeLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                .addGroup(pnlHomeLayout.createSequentialGroup()
                    .addContainerGap()
                    .addComponent(btnHelp2, javax.swing.GroupLayout.PREFERRED_SIZE, 38,
javax.swing.GroupLayout.PREFERRED_SIZE))
                .addComponent(lblLogoHome, javax.swing.GroupLayout.PREFERRED_SIZE, 107,
javax.swing.GroupLayout.PREFERRED_SIZE))
            .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
            .addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE, 268,
javax.swing.GroupLayout.PREFERRED_SIZE)
            .addGap(18, 18, 18)
            .addComponent(btnChangeStudent, javax.swing.GroupLayout.PREFERRED_SIZE, 74,
javax.swing.GroupLayout.PREFERRED_SIZE)
            .addContainerGap(28, Short.MAX_VALUE))
        );

        tbdpStudent.addTab("Home", pnlHome);

        pnlEvents.setBackground(new java.awt.Color(255, 255, 255));
        pnlEvents.addComponentListener(new java.awt.event.ComponentAdapter() {
            public void componentShown(java.awt.event.ComponentEvent evt) {
                pnlEventsComponentShown(evt);
            }
        });

        btnHelp3.setText("?");
        btnHelp3.setPreferredSize(new java.awt.Dimension(30, 30));
        btnHelp3.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnHelp3ActionPerformed(evt);
            }
        });

        tblEventStu.setModel(new javax.swing.table.DefaultTableModel(
            new Object [][] {
                {null, null, null, null, null},
                {null, null, null, null, null},
                {null, null, null, null, null},
                {null, null, null, null, null}
            },
            new String [] {
                "Event ID", "Event Name", "Hours", "Event date", "Portfolio"
            }
        ));
        jScrollPane2.setViewportView(tblEventStu);
```

```
        btnSearchEventDate.setText("Search by Date");
        btnSearchEventDate.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnSearchEventDateActionPerformed(evt);
            }
        });

        tbnSearchEventName.setText("Search by name");
        tbnSearchEventName.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                tbnSearchEventNameActionPerformed(evt);
            }
        });

        btnSearchEventID.setText("Search by ID");
        btnSearchEventID.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnSearchEventIDActionPerformed(evt);
            }
        });

        tfxSearchEventID.setText("e.g. 3");
        tfxSearchEventID.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                tfxSearchEventIDActionPerformed(evt);
            }
        });

        txfSearchEventName.setText("e.g. Name");
        txfSearchEventName.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                txfSearchEventNameActionPerformed(evt);
            }
        });

        javax.swing.GroupLayout pnlEventsLayout = new javax.swing.GroupLayout(pnlEvents);
        pnlEvents.setLayout(pnlEventsLayout);
        pnlEventsLayout.setHorizontalGroup(
            pnlEventsLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(pnlEventsLayout.createSequentialGroup()
                .addGap(24, 24, 24)
                .addComponent(jScrollPane2, javax.swing.GroupLayout.PREFERRED_SIZE, 724,
javax.swing.GroupLayout.PREFERRED_SIZE)
                .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 29,
Short.MAX_VALUE)

.addGroup(pnlEventsLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
```

```
.addGroup(pnlEventsLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING)
                    .addComponent(dpkSearchDate, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
                    .addComponent(btnHelp3, javax.swing.GroupLayout.PREFERRED_SIZE, 39,
javax.swing.GroupLayout.PREFERRED_SIZE))

.addGroup(pnlEventsLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING)
                    .addComponent(txfSearchEventName, javax.swing.GroupLayout.PREFERRED_SIZE,
107, javax.swing.GroupLayout.PREFERRED_SIZE)
                    .addComponent(btnSearchEventDate, javax.swing.GroupLayout.PREFERRED_SIZE,
109, javax.swing.GroupLayout.PREFERRED_SIZE)
                    .addComponent(tbnSearchEventName)
                    .addComponent(tfxSearchEventID, javax.swing.GroupLayout.PREFERRED_SIZE, 109,
javax.swing.GroupLayout.PREFERRED_SIZE)
                    .addComponent(btnSearchEventID, javax.swing.GroupLayout.PREFERRED_SIZE, 109,
javax.swing.GroupLayout.PREFERRED_SIZE)))
                .addGap(27, 27, 27))
        );
        pnlEventsLayout.setVerticalGroup(
            pnlEventsLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGroup(pnlEventsLayout.createSequentialGroup()
                .addGap(22, 22, 22)

.addGroup(pnlEventsLayout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
                    .addComponent(jScrollPane2, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
                    .addGroup(pnlEventsLayout.createSequentialGroup()
                        .addComponent(btnHelp3, javax.swing.GroupLayout.PREFERRED_SIZE, 38,
javax.swing.GroupLayout.PREFERRED_SIZE)
                        .addGap(38, 38, 38)
                        .addComponent(dpkSearchDate, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
                        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
                        .addComponent(btnSearchEventDate)
                        .addGap(62, 62, 62)
                        .addComponent(txfSearchEventName, javax.swing.GroupLayout.PREFERRED_SIZE, 32,
javax.swing.GroupLayout.PREFERRED_SIZE)
                        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
                        .addComponent(tbnSearchEventName)
                        .addGap(75, 75, 75)
                        .addComponent(tfxSearchEventID, javax.swing.GroupLayout.PREFERRED_SIZE, 32,
javax.swing.GroupLayout.PREFERRED_SIZE)
                        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
                        .addComponent(btnSearchEventID)))
                .addContainerGap(57, Short.MAX_VALUE))
        );

        tbdpStudent.addTab("Events", pnlEvents);
```

```
        pnlParticipation.setBackground(new java.awt.Color(255, 255, 255));
        pnlParticipation.addComponentListener(new java.awt.event.ComponentAdapter() {
            public void componentShown(java.awt.event.ComponentEvent evt) {
                pnlParticipationComponentShown(evt);
            }
        });
        pnlParticipation.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());

        btnHelp12.setText("?");
        btnHelp12.setPreferredSize(new java.awt.Dimension(30, 30));
        btnHelp12.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnHelp12ActionPerformed(evt);
            }
        });
        pnlParticipation.add(btnHelp12, new org.netbeans.lib.awtextra.AbsoluteConstraints(904, 11, 39,
38));

        tblParticipation.setModel(new javax.swing.table.DefaultTableModel(
            new Object [][] {
                {null, null, null, null, null},
                {null, null, null, null, null},
                {null, null, null, null, null},
                {null, null, null, null, null}
            },
            new String [] {
                "Student ID", "Event ID", "Event name", "Hours", "Portfolio"
            }
        ));
        jScrollPane3.setViewportView(tblParticipation);

        pnlParticipation.add(jScrollPane3, new org.netbeans.lib.awtextra.AbsoluteConstraints(33, 11, 737,
384));

        btnSearchPartPortfolio.setText("Search by Portfolio");
        btnSearchPartPortfolio.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnSearchPartPortfolioActionPerformed(evt);
            }
        });
        pnlParticipation.add(btnSearchPartPortfolio, new
org.netbeans.lib.awtextra.AbsoluteConstraints(802, 117, 141, -1));

        btnSearchPartName.setText("Search by Event name");
        btnSearchPartName.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnSearchPartNameActionPerformed(evt);
            }
```

```
        });
        pnlParticipation.add(btnSearchPartName, new org.netbeans.lib.awtextra.AbsoluteConstraints(802,
235, -1, -1));

        btnSearchPartID.setText("Search by event ID");
        btnSearchPartID.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnSearchPartIDActionPerformed(evt);
            }
        });
        pnlParticipation.add(btnSearchPartID, new org.netbeans.lib.awtextra.AbsoluteConstraints(802, 343,
141, -1));

        txfSearchPartPortfolio.setText("e.g. Catering");
        txfSearchPartPortfolio.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                txfSearchPartPortfolioActionPerformed(evt);
            }
        });
        pnlParticipation.add(txfSearchPartPortfolio, new org.netbeans.lib.awtextra.AbsoluteConstraints(802,
67, 141, 32));

        txfSearchPartID.setText("e.g. 3");
        txfSearchPartID.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                txfSearchPartIDActionPerformed(evt);
            }
        });
        pnlParticipation.add(txfSearchPartID, new org.netbeans.lib.awtextra.AbsoluteConstraints(802, 293,
141, 32));

        txfSearchPartName.setText("e.g. Name");
        txfSearchPartName.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                txfSearchPartNameActionPerformed(evt);
            }
        });
        pnlParticipation.add(txfSearchPartName, new org.netbeans.lib.awtextra.AbsoluteConstraints(802,
185, 141, 32));

        btnDeleteStuPart.setText("Delete Participation");
        btnDeleteStuPart.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnDeleteStuPartActionPerformed(evt);
            }
        });
        pnlParticipation.add(btnDeleteStuPart, new org.netbeans.lib.awtextra.AbsoluteConstraints(580, 430,
175, 76));
```

```
        btnCreateParticipation.setText("New Participation");
        btnCreateParticipation.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnCreateParticipationActionPerformed(evt);
            }
        });
        pnlParticipation.add(btnCreateParticipation, new org.netbeans.lib.awtextra.AbsoluteConstraints(190,
430, 175, 76));

        txfEventIDDelete.setText("e.g. 5");
        txfEventIDDelete.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                txfEventIDDeleteActionPerformed(evt);
            }
        });
        pnlParticipation.add(txfEventIDDelete, new org.netbeans.lib.awtextra.AbsoluteConstraints(470, 450,
102, 32));

        lblCreateParticipation.setText("Enter the event ID of the record you would like to create:");
        pnlParticipation.add(lblCreateParticipation, new org.netbeans.lib.awtextra.AbsoluteConstraints(80,
410, -1, 20));

        txfEventIDCreate.setText("e.g. 5");
        txfEventIDCreate.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                txfEventIDCreateActionPerformed(evt);
            }
        });
        pnlParticipation.add(txfEventIDCreate, new org.netbeans.lib.awtextra.AbsoluteConstraints(80, 450,
102, 32));

        lblDeleteParticipation.setText("Enter the event ID of the record you would like to delete:");
        pnlParticipation.add(lblDeleteParticipation, new org.netbeans.lib.awtextra.AbsoluteConstraints(470,
410, -1, 20));

        tbdpStudent.addTab("Participation", pnlParticipation);

        getContentPane().add(tbdpStudent, "card12");

        pnlHelp.setBackground(new java.awt.Color(255, 255, 255));
        pnlHelp.setLayout(new org.netbeans.lib.awtextra.AbsoluteLayout());

        btnExitHelp.setText("X");
        btnExitHelp.setPreferredSize(new java.awt.Dimension(30, 30));
        btnExitHelp.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnExitHelpActionPerformed(evt);
            }
        });
```

```
        pnlHelp.add(btnExitHelp, new org.netbeans.lib.awtextra.AbsoluteConstraints(899, 11, 39, 38));

        btnHowLogIn.setText("How do I log in?");
        btnHowLogIn.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnHowLogInActionPerformed(evt);
            }
        });
        pnlHelp.add(btnHowLogIn, new org.netbeans.lib.awtextra.AbsoluteConstraints(10, 11, 350, 53));

        btnHowSignUp.setText("How do I sign up?");
        btnHowSignUp.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnHowSignUpActionPerformed(evt);
            }
        });
        pnlHelp.add(btnHowSignUp, new org.netbeans.lib.awtextra.AbsoluteConstraints(430, 10, 350, 53));

        btnHowNewAdmin.setText("How do I create an admin account? (Admin only)");
        btnHowNewAdmin.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnHowNewAdminActionPerformed(evt);
            }
        });
        pnlHelp.add(btnHowNewAdmin, new org.netbeans.lib.awtextra.AbsoluteConstraints(10, 310, 350,
53));

        btnHowEditMyAccountStudent.setText("How do I edit my account? (Student) ");
        btnHowEditMyAccountStudent.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnHowEditMyAccountStudentActionPerformed(evt);
            }
        });
        pnlHelp.add(btnHowEditMyAccountStudent, new org.netbeans.lib.awtextra.AbsoluteConstraints(10,
70, 350, 53));

        btnHowSearchEventStu.setText("How do I search for an event? (Student)");
        btnHowSearchEventStu.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnHowSearchEventStuActionPerformed(evt);
            }
        });
        pnlHelp.add(btnHowSearchEventStu, new org.netbeans.lib.awtextra.AbsoluteConstraints(430, 430,
350, 53));

        btnHowCreateEvent.setText("How do I create an event? (Admin only)");
        btnHowCreateEvent.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnHowCreateEventActionPerformed(evt);
```

```
        }
    });
    pnlHelp.add(btnHowCreateEvent, new org.netbeans.lib.awtextra.AbsoluteConstraints(10, 190, 350,
53));

    btnHowDeleteParticipationAdmin.setText("How do I delete a participation record? (Admin)");
    btnHowDeleteParticipationAdmin.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            btnHowDeleteParticipationAdminActionPerformed(evt);
        }
    });
    pnlHelp.add(btnHowDeleteParticipationAdmin, new
org.netbeans.lib.awtextra.AbsoluteConstraints(430, 370, 350, 50));

    btnHowSearchStudent.setText("How do I search for a student? (Admin only)");
    btnHowSearchStudent.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            btnHowSearchStudentActionPerformed(evt);
        }
    });
    pnlHelp.add(btnHowSearchStudent, new org.netbeans.lib.awtextra.AbsoluteConstraints(430, 310,
350, 53));

    btnHowEditEvent.setText("How do I edit an event? (Admin only)");
    btnHowEditEvent.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            btnHowEditEventActionPerformed(evt);
        }
    });
    pnlHelp.add(btnHowEditEvent, new org.netbeans.lib.awtextra.AbsoluteConstraints(430, 130, 350,
53));

    btnHowSearchParticipationStu.setText("How do I search for a participation record?( Student)");
    btnHowSearchParticipationStu.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            btnHowSearchParticipationStuActionPerformed(evt);
        }
    });
    pnlHelp.add(btnHowSearchParticipationStu, new org.netbeans.lib.awtextra.AbsoluteConstraints(10,
430, 350, 53));

    btnHowMakeParticipationStudent.setText("How do I create a participation record? (Student)");
    btnHowMakeParticipationStudent.addActionListener(new java.awt.event.ActionListener() {
        public void actionPerformed(java.awt.event.ActionEvent evt) {
            btnHowMakeParticipationStudentActionPerformed(evt);
        }
    });
    pnlHelp.add(btnHowMakeParticipationStudent, new
org.netbeans.lib.awtextra.AbsoluteConstraints(10, 370, 350, 53));
```

```
        btnHowEditAccountAdmin.setText("How do I edit my account? (Admin) ");
        btnHowEditAccountAdmin.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnHowEditAccountAdminActionPerformed(evt);
            }
        });
        pnlHelp.add(btnHowEditAccountAdmin, new org.netbeans.lib.awtextra.AbsoluteConstraints(430, 70,
350, 53));

        btnHowMakeParticipation.setText("How do I create a participation record? (Admin)");
        btnHowMakeParticipation.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnHowMakeParticipationActionPerformed(evt);
            }
        });
        pnlHelp.add(btnHowMakeParticipation, new org.netbeans.lib.awtextra.AbsoluteConstraints(430,
190, 350, 53));

        btnHowDeleteParticipationStudent.setText("How do I delete a participation record? (Student)");
        btnHowDeleteParticipationStudent.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnHowDeleteParticipationStudentActionPerformed(evt);
            }
        });
        pnlHelp.add(btnHowDeleteParticipationStudent, new
org.netbeans.lib.awtextra.AbsoluteConstraints(10, 250, 350, 50));

        lblHelpContact.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
        lblHelpContact.setText("In case additional help is needed please contact Ruth-ann Rudolph at
Ruth2Rudolph@Gmail.com");
        lblHelpContact.setFont(new java.awt.Font("Tahoma", 0, 18)); // NOI18N
        pnlHelp.add(lblHelpContact, new org.netbeans.lib.awtextra.AbsoluteConstraints(10, 490, 800, 40));

        btnHowSearchParticipationAdmin.setText("How do I search for a participation record? (Admin)");
        btnHowSearchParticipationAdmin.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnHowSearchParticipationAdminActionPerformed(evt);
            }
        });
        pnlHelp.add(btnHowSearchParticipationAdmin, new
org.netbeans.lib.awtextra.AbsoluteConstraints(430, 250, 350, 53));

        btnHowSearchEventAdmin.setText("How do I search for an event? (Admin)");
        btnHowSearchEventAdmin.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                btnHowSearchEventAdminActionPerformed(evt);
            }
        });
```

```
        pnlHelp.add(btnHowSearchEventAdmin, new org.netbeans.lib.awtextra.AbsoluteConstraints(10,
130, 350, 53));

        getContentPane().add(pnlHelp, "card11");

        pack();
    }// </editor-fold>

    private void btnSignUpActionPerformed(java.awt.event.ActionEvent evt) {

        pnlSignUp.setVisible(true); //takes the user to the sign up page

        pnlLogIn.setVisible(false); //hides the log in page

        clearErrorLables(); //clears the error lables by calling the clear error lables method

    }

    private void btnHelp3ActionPerformed(java.awt.event.ActionEvent evt) {

        pnlHelp.setVisible(true); //Sets the help pannel to visible

        pnlEvents.setVisible(false); //Hides the students events pannel

        tbdpStudent.setVisible(false); //Hides the students tabbed pane

        tbdpAdmin.setVisible(false); //Hides the admins tabbed pane

        help = 8; //sets the help variable to

    }

    private void btnExitHelpActionPerformed(java.awt.event.ActionEvent evt) {

        pnlHelp.setVisible(false); //hides the help pannel
        switch (help) {
            case 1 -> pnlLogIn.setVisible(true);//if the help variable value is 1, takes the user to the log in
pannel
            case 2 -> pnlSignUp.setVisible(true);//if the help variable value is 2, takes the user to the sign up
pannel
            case 3 -> {
                tbdpAdmin.setVisible(true); //if the help variable value is 3, takes the user to the Admin tabbed
pane, on the home page
                pnlHomeAdmin.setVisible(true);
            }
            case 4 -> {
                tbdpAdmin.setVisible(true); //if the help variable value is 4, takes the user to the Admin tabbed
pane, on the events pannel
                pnlEvents1.setVisible(true);
```

```
            }
            case 5 -> {
                tbdpAdmin.setVisible(true); //if the help variable value is 5, takes the user to the Admin tabbed pane, on the participation pannel
                pnlParticipation1.setVisible(true);
            }
            case 6 -> {
                tbdpAdmin.setVisible(true); //if the help variable value is 6, takes the user to the Admin tabbed pane, on the students pannel
                pnlStudent.setVisible(true);
            }
            case 7 -> {
                tbdpStudent.setVisible(true); //if the help variable value is 7, takes the user to the Students tabbed pane, on the home pannel
                pnlHome.setVisible(true);
            }
            case 8 -> {
                tbdpStudent.setVisible(true); //if the help variable value is 8, takes the user to the Students tabbed pane, on the Events pannel
                pnlEvents.setVisible(true);
            }
            case 9 -> {
                tbdpStudent.setVisible(true); //if the help variable value is 9, takes the user to the Students tabbed pane, on the participation pannel
                pnlParticipation.setVisible(true);
            }
            default -> {
            }
        }

    }

    private void btnHelp10ActionPerformed(java.awt.event.ActionEvent evt) {

        pnlHelp.setVisible(true); //Sets the help pannel to visible

        pnlLogIn.setVisible(false); //Hides the log in pannel

        tbdpStudent.setVisible(false); //Hides the student tabbed pane

        tbdpAdmin.setVisible(false); //Hides the admin tabbed pane

        help = 1; //Sets help value to 1

    }

    private void txfPasswordActionPerformed(java.awt.event.ActionEvent evt) {

        // TODO add your handling code here:
```

```
    }

    private void txfUsernameActionPerformed(java.awt.event.ActionEvent evt) {

        // TODO add your handling code here:

    }

    private void btnHelp11ActionPerformed(java.awt.event.ActionEvent evt) {

        pnlHelp.setVisible(true); //Sets the help pannel to visible

        pnlSignUp.setVisible(false); //Hides the sign up pannel

        tbdpStudent.setVisible(false); //Hides the student tabbed pane

        tbdpAdmin.setVisible(false); //Hides the admin tabbed pane

        help = 2;//sets the help value to 2

    }

    private void txfNewStudentGradeActionPerformed(java.awt.event.ActionEvent evt) {

        // TODO add your handling code here:

    }

    private void txfUsername1ActionPerformed(java.awt.event.ActionEvent evt) {

        // TODO add your handling code here:

    }

    private void txfPassword2ActionPerformed(java.awt.event.ActionEvent evt) {

        // TODO add your handling code here:

    }

    private void btnSearchEventDateActionPerformed(java.awt.event.ActionEvent evt) {
        try //Searches for an entry by Event date and returns it to the table if found
        {
            DefaultTableModel dtm = (DefaultTableModel) tblEventStu.getModel(); //Sets the default table
model
            tblEventStu.removeAll(); //Empties the Jtable
            dtm.setRowCount(0); //sets the row count to 0
            model = (DefaultTableModel)tblEventStu.getModel(); //Sets the default table model
```

```
        LocalDate date = dpkSearchDate.getDate();//gets the date entered by the user
        ResultSet rs = dbman.queryDatabase("SELECT * FROM tblEvents WHERE EventDate =#" + date
+"#"); //Retrives entry(s) from the database
        int count = 0; //sets a counter to 0
        while(rs.next()) //checks if the result set has a next result and moves to it if it does
        {
            String portfolio = ""; //Creates a string to store the portfolio of the event

            if(rs.getBoolean("Catering") == true) //Checks if the event is for catering
            {

                portfolio = "Catering";//Sets the portfolio string to catering

            }
            else if(rs.getBoolean("Photography") == true) //Checks if the event is for photography
            {

                portfolio = "Photography"; //Sets the portfolio string to photography

            }
            else if(rs.getBoolean("FirstAid") == true)//Checks if the event is for first aid
            {

                portfolio = "First Aid"; //Sets the portfolio string to first aid

            }
            else if(rs.getBoolean("AV") == true)//Checks if the event is for AV
            {

                portfolio = "AV"; //Sets the portfolio string to AV

            }
            boolean c = sm.getPortfolio("Catering", currentStu); //checks if the student does catering
            boolean p = sm.getPortfolio("Photography", currentStu); //checks if the student does
photography
            boolean a = sm.getPortfolio("av", currentStu); //checks if the student does AV
            boolean f = sm.getPortfolio("First aid", currentStu); //checks if the student does first aid
            //Checks that the student has permission to view the event
            if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true &&
portfolio.equalsIgnoreCase("Photography")) || (a == true && portfolio.equalsIgnoreCase("AV")) || (f ==
true && portfolio.equalsIgnoreCase("First Aid")))
            {
                model.addRow(new Object[]{rs.getInt("EventID"),rs.getString("EventName"),
                 rs.getInt("Hours"),rs.getDate("EventDate"), portfolio}); //Creates a new object to be added to
the table
                count++; //adds 1 to count
            }
      }
        if(count == 0) //checks if count is till equal to 0
```

```
        {
            JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message to indicate that the event could not be found
        }
        dpkSearchDate.setText(""); //Clears the text field
        model.addRow(new Object[]{}); //Adds row to the table
    }
    catch(SQLException e) //Catches any sql Errors
    {
        JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message to indicate that the event could not be found
    }

  }

  private void tfxSearchEventIDActionPerformed(java.awt.event.ActionEvent evt) {

    // TODO add your handling code here:

  }

  private void txfSearchEventNameActionPerformed(java.awt.event.ActionEvent evt) {

    // TODO add your handling code here:

  }

  private void btnHelp12ActionPerformed(java.awt.event.ActionEvent evt) {

    pnlHelp.setVisible(true); //Sets the help pannel to visible

    pnlParticipation.setVisible(false); //Hides the participation pannel

    tbdpStudent.setVisible(false); //Hides the Student Tabbed pane

    tbdpAdmin.setVisible(false); //Hides the Admin tabbed pane

    help = 9; //Sets help value to 9

  }

  private void btnSearchPartPortfolioActionPerformed(java.awt.event.ActionEvent evt) {
    try //Searches for an entry by portfolio and returns it to the table if found
    {
        DefaultTableModel dtm = (DefaultTableModel) tblParticipation.getModel(); //Sets the default table
model
        tblParticipation.removeAll();//Empties the Jtable
        dtm.setRowCount(0); //sets the row count to 0
        model = (DefaultTableModel)tblParticipation.getModel(); //Sets the default table model
```

```
        ResultSet rs = dbman.queryDatabase("SELECT DISTINCT tblStudents.StudentID,
tblEvents.EventID, tblEvents.EventName, tblEvents.hours, tblEvents.Catering, tblEvents.Photography,
tblEvents.AV, tblEvents.FirstAid FROM tblEvents, tblStudents, tblParticipation Where
tblParticipation.StudentID = tblStudents.StudentID AND tblParticipation.EventID = tblEvents.EventID AND
tblStudents.StudentID = " + currentStu); //Retrives entry(s) from the database
        int count = 0; //sets a counter to 0
        while(rs.next()) //checks if the result set has a next result and moves to it if it does
        {
            String portfolio = "";//Creates a string to store the portfolio of the event
            //checks if the record is for catering and if the student does catering
            if(rs.getBoolean("Catering") == true &&
(txfSearchPartPortfolio.getText().equalsIgnoreCase("Catering") ||
txfSearchPartPortfolio.getText().charAt(0) == 'c' || txfSearchPartPortfolio.getText().charAt(0) == 'C'))
            {

                portfolio = "Catering"; //sets the portfolio string to catering

            }
            //checks if the record is for photography and if the student does photography
            else if(rs.getBoolean("Photography") == true &&
(txfSearchPartPortfolio.getText().equalsIgnoreCase("Photography") ||
txfSearchPartPortfolio.getText().charAt(0) == 'p' || txfSearchPartPortfolio.getText().charAt(0) == 'P'))
            {

                portfolio = "Photography";//set the portfolio string to photography

            }
            //checks if the record is for first aid and if the student does first aid
            else if(rs.getBoolean("FirstAid") == true &&
(txfSearchPartPortfolio.getText().equalsIgnoreCase("First Aid")||
txfSearchPartPortfolio.getText().charAt(0) == 'F' || txfSearchPartPortfolio.getText().charAt(0) == 'f'))
            {

                portfolio = "First Aid"; //set the portfolio string to first aid

            }
            //checks if the record is for AV and if the student does AV
            else if(rs.getBoolean("AV") == true &&
(txfSearchPartPortfolio.getText().equalsIgnoreCase("AV") || txfSearchPartPortfolio.getText().charAt(0) ==
'a' || txfSearchPartPortfolio.getText().charAt(0) == 'A'))
            {

                portfolio = "AV"; //sets the portfolio string to AV

            }
            boolean c = sm.getPortfolio("Catering", currentStu); //checks if the current student does
catering
            boolean p = sm.getPortfolio("Photography", currentStu); //checks if the current student does
photography
```

```
            boolean a = sm.getPortfolio("av", currentStu); //checks if the current student does AV
            boolean f = sm.getPortfolio("First aid", currentStu); //checks if the current student does first aid
            //Checks that the student has permission to view the participation record
            if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true &&
portfolio.equalsIgnoreCase("Photography")) || (a == true && portfolio.equalsIgnoreCase("AV")) || (f ==
true && portfolio.equalsIgnoreCase("First Aid")))
            {
                model.addRow(new
Object[]{rs.getInt("StudentID"),rs.getInt("EventID"),rs.getString("EventName"),
                    rs.getInt("Hours"), portfolio});//Creates a new object to be added to the table
                count++; //adds 1 to count
            }
        }
            if(count == 0) //checks if count is till equal to 0
            {
                JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message to indicate that the event could not be found
            }
            txfSearchPartPortfolio.setText(""); //Clears the text field
            model.addRow(new Object[]{}); //Adds row to the table
        }
        catch(SQLException e) //Catches any sql Errors
        {
            JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message to indicate that the event could not be found
        }
    }

    private void txfSearchPartPortfolioActionPerformed(java.awt.event.ActionEvent evt) {

        // TODO add your handling code here:

    }

    private void txfSearchPartIDActionPerformed(java.awt.event.ActionEvent evt) {

        // TODO add your handling code here:

    }

    private void txfSearchPartNameActionPerformed(java.awt.event.ActionEvent evt) {

        // TODO add your handling code here:

    }

    private void btnHelp4ActionPerformed(java.awt.event.ActionEvent evt) {

        pnlHelp.setVisible(true); //Sets the help pannel to visible
```

```
        pnlHomeAdmin.setVisible(false);//Hides the admin home pannel
        tbdpStudent.setVisible(false); //Hides the Student Tabbed pane
        tbdpAdmin.setVisible(false); //Hides the Admin tabbed pane
        help = 3; //Sets the help value to 3
    }
    private void txfNewAdminNameActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
    }
    private void txfNewAdminUsernameActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
    }
    private void txfNewAdminPasswordActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
    }
    private void btnHelp5ActionPerformed(java.awt.event.ActionEvent evt) {
        pnlHelp.setVisible(true); //Sets the help pannel to visible
        pnlEvents1.setVisible(false);//Hides the Events pannel
        tbdpStudent.setVisible(false); //Hides the Student Tabbed pane
        tbdpAdmin.setVisible(false); //Hides the Admin tabbed pane
        help = 4; //Sets the help value to 4
    }
    private void btnSearchDateActionPerformed(java.awt.event.ActionEvent evt) {
        try //Searches for an entry by event date and returns it to the table if found
        {
            DefaultTableModel dtm = (DefaultTableModel) tblEventsAdmin.getModel(); //Sets the default table
model
            tblEventsAdmin.removeAll();//Empties the Jtable
```

```
        dtm.setRowCount(0); //sets the row count to 0
        model = (DefaultTableModel)tblEventsAdmin.getModel(); //Sets the default table model
        LocalDate date = dpkSearch.getDate();//gets the date entered by the user
        ResultSet rs = dbman.queryDatabase("SELECT * FROM tblEvents WHERE EventDate =#" + date
+"#"); //Retrives entry(s) from the database
        int count = 0; //sets a counter to 0
        while(rs.next()) //checks if the result set has a next result and moves to it if it does
        {
            String portfolio = ""; //Creates a string to store the portfolio of the event
            if(rs.getBoolean("Catering") == true) //Checks if the event is for catering
            {
                portfolio = "Catering"; //Sets the portfolio string to catering
            }
            else if(rs.getBoolean("Photography") == true) //Checks if the event is for photography
            {
                portfolio = "Photography"; //Sets the portfolio string to photography
            }
            else if(rs.getBoolean("FirstAid") == true) //Checks if the event is for first aid
            {
                portfolio = "First Aid"; ////Sets the portfolio string to First aid
            }
            else if(rs.getBoolean("AV") == true) //Checks if the event is for AV
            {
                portfolio = "AV"; //Sets the portfolio string to AV
            }
            boolean c = am.getPortfolio("Catering", currentAdmin); //checks if the admin oversees catering
            boolean p = am.getPortfolio("Photography", currentAdmin); //checks if the admin oversees
photography
            boolean a = am.getPortfolio("av", currentAdmin); //checks if the admin oversees AV
            boolean f = am.getPortfolio("First aid", currentAdmin);//checks if the admin oversees first aid
            //Checks that the admin has permission to view the event
            if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true &&
portfolio.equalsIgnoreCase("Photography")) || (a == true && portfolio.equalsIgnoreCase("AV")) || (f ==
true && portfolio.equalsIgnoreCase("First Aid")))
            {
                count ++;//adds 1 to count
                model.addRow(new Object[]{rs.getInt("EventID"),rs.getString("EventName"),
                rs.getInt("Hours"),rs.getDate("EventDate"), portfolio}); //Creates a new object to be added to
the table
            }
        }
        if(count == 0) //checks if count is till equal to 0
        {
            JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message to indicate that the event could not be found
        }
        dpkSearch.setText(""); //Clears the text field
        model.addRow(new Object[]{}); //Adds row to the table
    }
```

```
        catch(SQLException e) //Catches any sql Errors
        {
            JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE); //Displays an error message to indicate that the event could not be found
        }

    }

    private void txfSearchIDActionPerformed(java.awt.event.ActionEvent evt) {

        // TODO add your handling code here:

    }

    private void txfSearchNameActionPerformed(java.awt.event.ActionEvent evt) {

        // TODO add your handling code here:

    }

    private void btnHelp13ActionPerformed(java.awt.event.ActionEvent evt) {

        pnlHelp.setVisible(true); //Sets the help pannel to visible

        pnlParticipation1.setVisible(false);//Hides the admin participation pannel

        tbdpStudent.setVisible(false); //Hides the Student Tabbed pane

        tbdpAdmin.setVisible(false); //Hides the Admin tabbed pane

        help = 5;//Sets the help value to 5

    }

    private void btnSearchPortfolioPartActionPerformed(java.awt.event.ActionEvent evt) {
        try //Searches for an entry by portfolio and returns it to the table if found
        {
            DefaultTableModel dtm = (DefaultTableModel) tblParticipationAdmin.getModel(); //Sets the default
table model
            tblParticipationAdmin.removeAll();//Empties the Jtable
            dtm.setRowCount(0); //sets the row count to 0
            model = (DefaultTableModel)tblParticipationAdmin.getModel(); //Sets the default table model
            ResultSet rs = dbman.queryDatabase("SELECT DISTINCT tblStudents.StudentID,
tblStudents.firstName, tblStudents.Surname, tblEvents.EventID, tblEvents.EventName, tblEvents.hours,
tblEvents.Catering, tblEvents.Photography, tblEvents.AV, tblEvents.FirstAid FROM tblEvents,
tblStudents, tblParticipation Where tblParticipation.StudentID = tblStudents.StudentID AND
tblParticipation.EventID = tblEvents.EventID  "); //Retrives entry(s) from the database
            int count = 0; //sets a counter to 0
            while(rs.next()) //checks if the result set has a next result and moves to it if it does
```

```
        {
            String portfolio = "";//Creates a string to store the portfolio of the event
            //checks if the record is for catering and if the admin oversees catering
            if(rs.getBoolean("Catering") == true &&
txfSearchPortfolioPart.getText().equalsIgnoreCase("Catering"))
            {
                portfolio = "Catering"; //Sets the string to catering
            }
            //checks if the record is for photography and if the admin oversees photography
            if(rs.getBoolean("Photography") == true &&
txfSearchPortfolioPart.getText().equalsIgnoreCase("Photography"))
            {
                portfolio = "Photography"; //Sets the string to photography
            }
            //checks if the record is for first aid and if the admin oversees first aid
            if(rs.getBoolean("FirstAid") == true &&
txfSearchPortfolioPart.getText().equalsIgnoreCase("First Aid"))
            {
                portfolio = "First Aid";//Sets the string to first aid
            }
            //checks if the record is for AV and if the admin oversees AV
            if(rs.getBoolean("AV") == true && txfSearchPortfolioPart.getText().equalsIgnoreCase("AV"))
            {
                portfolio = "AV"; //Sets the string to AV
            }
            boolean c = am.getPortfolio("Catering", currentAdmin);//checks if the admin oversees catering
            boolean p = am.getPortfolio("Photography", currentAdmin); //checks if the admin oversees
photography
            boolean a = am.getPortfolio("av", currentAdmin); //checks if the admin oversees AV
            boolean f = am.getPortfolio("First aid", currentAdmin); //checks if the admin oversees first aid

            if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true &&
portfolio.equalsIgnoreCase("Photography")) || (a == true && portfolio.equalsIgnoreCase("AV")) || (f ==
true && portfolio.equalsIgnoreCase("First Aid")))
            {
                String name = rs.getString("firstName") + " " + rs.getString("surname");//gets the student's full
name
                model.addRow(new
Object[]{rs.getInt("StudentID"),name,rs.getInt("EventID"),rs.getString("EventName"),
                 rs.getInt("Hours"), portfolio});//Creates a new object to be added to the table
                count++;//adds 1 to count
            }
        }
        if(count == 0) //checks if count is till equal to 0
        {
            JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message to indicate that the event could not be found
        }
        txfSearchPortfolioPart.setText(""); //Clears the text field
```

```
            model.addRow(new Object[]{}); //Adds row to the table
        }
        catch(SQLException e) //Catches any sql Errors
        {
            JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message to indicate that the event could not be found
        }

    }

    private void txfSearchPortfolioPartActionPerformed(java.awt.event.ActionEvent evt) {

        // TODO add your handling code here:

    }

    private void txfSearchEventIDPartActionPerformed(java.awt.event.ActionEvent evt) {

        // TODO add your handling code here:

    }

    private void txfSearchStudentNamePartActionPerformed(java.awt.event.ActionEvent evt) {

        // TODO add your handling code here:

    }

    private void txfSearchEventNamePartActionPerformed(java.awt.event.ActionEvent evt) {

        // TODO add your handling code here:

    }

    private void btnSearchEventNamePartActionPerformed(java.awt.event.ActionEvent evt) {
        try //Searches for an entry by Event name and returns it to the table if found
        {
            DefaultTableModel dtm = (DefaultTableModel) tblParticipationAdmin.getModel(); //Sets the default
table model
            tblParticipationAdmin.removeAll();//Empties the Jtable
            dtm.setRowCount(0); //sets the row count to 0
            model = (DefaultTableModel)tblParticipationAdmin.getModel(); //Sets the default table model
            ResultSet rs = dbman.queryDatabase("SELECT DISTINCT tblStudents.StudentID,
tblStudents.firstName, tblStudents.Surname, tblEvents.EventID, tblEvents.EventName, tblEvents.hours,
tblEvents.Catering, tblEvents.Photography, tblEvents.AV, tblEvents.FirstAid FROM tblEvents,
tblStudents, tblParticipation Where tblParticipation.StudentID = tblStudents.StudentID AND
tblParticipation.EventID = tblEvents.EventID AND EventName LIKE '*" +
txfSearchEventNamePart.getText() + "*'"); //Retrives entry(s) from the database
            int count = 0; //sets a counter to 0
```

```
            while(rs.next()) //checks if the result set has a next result and moves to it if it does
            {
                String portfolio = "";//Creates a string to store the portfolio of the event
                if(rs.getBoolean("Catering") == true) //Checks if the event of the participation record is for
catering
                {
                    portfolio = "Catering"; //Sets the portfolio string to catering
                }
                else if(rs.getBoolean("Photography") == true) //Checks if the event of the participation record is
for photography
                {
                    portfolio = "Photography"; //Sets the portfolio string to photography
                }
                else if(rs.getBoolean("FirstAid") == true) //Checks if the event of the participation record is for
first aid
                {
                    portfolio = "First Aid"; //Sets the portfolio string to first aid
                }
                else if(rs.getBoolean("AV") == true) //Checks if the event of the participation record is for AV
                {
                    portfolio = "AV"; //Sets the portfolio string to AV
                }
                boolean c = am.getPortfolio("Catering", currentAdmin);//checks if the admin oversees catering
                boolean p = am.getPortfolio("Photography", currentAdmin); //checks if the admin oversees
photography
                boolean a = am.getPortfolio("av", currentAdmin); //checks if the admin oversees AV
                boolean f = am.getPortfolio("First aid", currentAdmin); //checks if the admin oversees first aid

                if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true &&
portfolio.equalsIgnoreCase("Photography")) || (a == true && portfolio.equalsIgnoreCase("AV")) || (f ==
true && portfolio.equalsIgnoreCase("First Aid")))
                {
                    String name = rs.getString("firstName") + " " + rs.getString("surname");//gets the student's full
name
                    model.addRow(new
Object[]{rs.getInt("StudentID"),name,rs.getInt("EventID"),rs.getString("EventName"),
                     rs.getInt("Hours"), portfolio});//Creates a new object to be added to the table
                    count++;//adds 1 to count
                }
            }
            if(count == 0) //checks if count is till equal to 0
            {
                JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message to indicate that the event could not be found
            }
            txfSearchEventNamePart.setText(""); //Clears the text field
            model.addRow(new Object[]{}); //Adds row to the table
        }
        catch(SQLException e) //Catches any sql Errors
```

```
        {
            JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message to indicate that the event could not be found
        }
    }

    private void txfDeleteStuIDActionPerformed(java.awt.event.ActionEvent evt) {

        // TODO add your handling code here:

    }

    private void txfDeleteEventIDActionPerformed(java.awt.event.ActionEvent evt) {

        // TODO add your handling code here:

    }

    private void btnHelp14ActionPerformed(java.awt.event.ActionEvent evt) {

        pnlHelp.setVisible(true); //Sets the help pannel to visible

        pnlStudent.setVisible(false);//Hides the student pannel

        tbdpStudent.setVisible(false); //Hides the Student Tabbed pane

        tbdpAdmin.setVisible(false); //Hides the Admin tabbed pane

        help = 6; //sets the help value to 6

    }

    private void btnSearchStuIDActionPerformed(java.awt.event.ActionEvent evt) {
        try //Searches for an entry by Student ID and returns it to the table if found
        {
            DefaultTableModel dtm = (DefaultTableModel) tblStudents.getModel(); //Sets the default table
model
            tblStudents.removeAll(); //Empties the Jtable
            dtm.setRowCount(0); //sets the row count to 0
            model = (DefaultTableModel)tblStudents.getModel(); //Sets the default table model
            ResultSet rs = dbman.queryDatabase("SELECT * FROM tblStudents WHERE StudentID =" +
Integer.parseInt(txfSearchStuID.getText()) ); //Retrives entry(s) from the database
            int count = 0; //sets a counter to 0
            while(rs.next()) //Creates a line in the JTable for every database entry
            {
                String stu = rs.getString("StudentID");//Sets the student ID to the ID value of the current record
in the database

                int catering = 0; //Creates a variable to hold the catering hours total
```

```
            int photography = 0; //Creates a variable to hold the photography hours total

            int firstAid = 0; //Creates a variable to hold the first aid hours total

            int av = 0; //Creates a variable to hold the AV hours total

        ResultSet ch = dbman.queryDatabase("SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS
CateringHours\n" +
                "FROM tblStudents, tblEvents, tblParticipation\n" +
                "WHERE tblParticipation.StudentID = tblStudents.StudentID AND tblEvents.EventID =
tblParticipation.EventID AND tblEvents.Catering = True\n" +
                "GROUP BY tblStudents.StudentID;"); //Sums the total catering hours for each student
        ResultSet ph = dbman.queryDatabase("SELECT tblStudents.StudentID, SUM(tblEvents.hours)
AS PhotographyHours\n" +
                "FROM tblStudents, tblEvents, tblParticipation\n" +
                "WHERE tblParticipation.StudentID = tblStudents.StudentID AND tblEvents.EventID =
tblParticipation.EventID AND tblEvents.Photography = True\n" +
                "GROUP BY tblStudents.StudentID;"); //Sums the total photography hours for each student
        ResultSet avh = dbman.queryDatabase("SELECT tblStudents.StudentID, SUM(tblEvents.hours)
AS AVHours\n" +
                "FROM tblStudents, tblEvents, tblParticipation\n" +
                "WHERE tblParticipation.StudentID = tblStudents.StudentID AND tblEvents.EventID =
tblParticipation.EventID AND tblEvents.AV = True\n" +
                "GROUP BY tblStudents.StudentID;"); //Sums the total AV hours for each student
        ResultSet fh = dbman.queryDatabase("SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS
FirstAidHours\n" +
                "FROM tblStudents, tblEvents, tblParticipation\n" +
                "WHERE tblParticipation.StudentID = tblStudents.StudentID AND tblEvents.EventID =
tblParticipation.EventID AND tblEvents.FirstAid = True\n" +
                "GROUP BY tblStudents.StudentID;"); //Sums the total first aid hours for each student

            while (ch.next()) //Loops throught the catering hours results
            {
                //checks if the admin oversees catering and that the current student ID from the main result
set matches the catering hours result set matches the catering hours result set
                if(am.getPortfolio("Catering", currentAdmin) == true &&
stu.equals(ch.getString("StudentID")))

                {
                    catering = ch.getInt("CateringHours");//assigns the current students total catering hours to
the catering variable
                }
            }


            while (ph.next()) //Loops throught the photography hours results
            {
```

```
            //checks if the admin oversees catering and that the current student ID from the main result set matches the catering hours result set matches the photography hours result set
            if(am.getPortfolio("Photography", currentAdmin) == true && stu.equals(ph.getString("StudentID")))

            {
              photography = ph.getInt("PhotographyHours");//assigns the current students total photography hours to the photography variable
            }
          }


          while (avh.next()) //Loops throught the AV hours results
          {
            //checks if the admin oversees catering and that the current student ID from the main result set matches the catering hours result set matches the AV hours result set
            if(am.getPortfolio("AV", currentAdmin) == true && stu.equals(avh.getString("StudentID")))

            {
              av = avh.getInt("AVHours"); //assigns the current students total AV hours to the AV variable
            }
          }


          while (fh.next()) //Loops throught the first aid hours results
          {
            //checks if the admin oversees catering and that the current student ID from the main result set matches the catering hours result set matches the first aid hours result set
            if(am.getPortfolio("First Aid", currentAdmin) == true && stu.equals(fh.getString("StudentID")))
            {
              firstAid = fh.getInt("FirstAidHours"); //assigns the current students total first aid hours to the first aid variable
            }
          }
          String grade = rs.getInt("Grade") + "." + rs.getInt("Class"); //Creates a string to contain the grade and class of the learner in a formatted manner
          String name = rs.getString("FirstName") + " " + rs.getString("Surname"); //Creates a string to contain the students full name
          String cater = Integer.toString(catering);//Converts the catering hours to a string
          String aV = Integer.toString(av); //converts the AV hours to a string
          String photo = Integer.toString(photography); //Converts the photography hours to a string
          String first =Integer.toString(firstAid); //Converts the first aid hours to a string
          if (am.getPortfolio("First Aid", currentAdmin) == false) //checks if the admin has permissions for first aid
          {
            first = "NA"; //if the admin does not have permissions for first aid, sets the first aid hours collum to 'NA'
          }
```

```
            if (am.getPortfolio("AV", currentAdmin) == false)//checks if the admin has permissions for AV
            {
                aV = "NA"; //if the admin does not have permissions for AV, sets the AV hours collum to 'NA'
            }
            if(am.getPortfolio("Catering", currentAdmin) == false) //checks if the admin has permissions for
catering
            {
                cater = "NA"; //if the admin does not have permissions for catering, sets the catering hours
collum to 'NA'
            }
            if (am.getPortfolio("Photography", currentAdmin) == false) //checks if the admin has
permissions for photography
            {
                photo = "NA"; //if the admin does not have permissions for photography, sets the
photography hours collum to 'NA'
            }
            //Creates a new object to be added to the table
             model.addRow(new Object[]{rs.getInt("StudentID"), name, grade, rs.getString("Gender"),cater,
aV, photo,first });
             count++;
            }
         model.addRow(new Object[]{}); //adds each object to its own row in the table
          if(count == 0) //checks if count is till equal to 0
          {
             JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message to indicate that the event could not be found
          }
      }

      catch(SQLException e)
      {
          JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message to indicate that the event could not be found
      }

          txfSearchStuID.setText(""); //Clears the text field
          model.addRow(new Object[]{}); //Adds row to the table
   }

   private void txfSearchStuIDActionPerformed(java.awt.event.ActionEvent evt) {

      // TODO add your handling code here:

   }

   private void txfSearchStuGradeActionPerformed(java.awt.event.ActionEvent evt) {

      // TODO add your handling code here:
```

```
}

private void txfSearchStuNameActionPerformed(java.awt.event.ActionEvent evt) {

    // TODO add your handling code here:

}

private void txfConfirmPassword1ActionPerformed(java.awt.event.ActionEvent evt) {

    // TODO add your handling code here:

}

private void txfNewStudentClassActionPerformed(java.awt.event.ActionEvent evt) {

    // TODO add your handling code here:

}

private void txfNewStudentNameActionPerformed(java.awt.event.ActionEvent evt) {

    // TODO add your handling code here:

}

private void btnHelp2ActionPerformed(java.awt.event.ActionEvent evt) {

    pnlHelp.setVisible(true); //Sets the help pannel to visible

    pnlHome.setVisible(false);//Hides the student home pannel

    tbdpStudent.setVisible(false); //Hides the Student Tabbed pane

    tbdpAdmin.setVisible(false); //Hides the Admin tabbed pane

    help = 7; //Sets the help value to 7

}

private void btnChangeStudentActionPerformed(java.awt.event.ActionEvent evt) {
    clearErrorLables(); //clears the error lables by calling the clear error lables method

    dlgEditStudent.setEnabled(true);//enables the edit student account dialog

    dlgEditStudent.setSize(900,600); //sets the sie of the dialog

    dlgEditStudent.setLocationRelativeTo(txaStudent); //Sets the position of the dialog
```

```
        dlgEditStudent.setVisible(true);//Opens the edit account dialog

        txfEditStuName.setText("");//clears the edit student name text field
        txfEditStuSurname.setText("");//clears the edit student surname text field
        txfEditStuGrade.setText("");//clears the edit student grade text field
        txfEditStuClass.setText("");//clears the edit student class text field

        try
        {
            ResultSet rs = dbman.queryDatabase("SELECT * FROM tblStudents WHERE StudentID =" +
currentStu);//selects the current students details

            while(rs.next()) //goes to the next result
            {
               if(rs.getBoolean("Catering") == true) //checks if the student does catering
               {
                  cbxCatering1.setSelected(true);//sets the check box to selected
               }
               if(rs.getBoolean("Photography") == true) //checks if the student does photography
               {
                  cbxPhotography1.setSelected(true);//sets the check box to selected
               }
               if(rs.getBoolean("FirstAid") == true) //checks if the student does first aid
               {
                  cbxFirstAid1.setSelected(true);//sets the check box to selected
               }
               if(rs.getBoolean("AV") == true) //checks if the student does AV
               {
                  cbxAV1.setSelected(true);//sets the check box to selected
               }
               txfEditStuName.setText(rs.getString("firstname"));//fills in the student first name
               txfEditStuSurname.setText(rs.getString("surname"));//fills in the students surname
               txfEditStuGrade.setText(rs.getString("Grade"));//fills in the students grade
               txfEditStuClass.setText(rs.getString("Class"));//fills in the students class
               if(rs.getString("Gender").equalsIgnoreCase("F"))//checks if the student is female
               {
                  rbnEditStuGenderF.setSelected(true);//selects the female radio button
               }
               else if(rs.getString("Gender").equalsIgnoreCase("M"))//checks if the student is male
               {
                  rbnEditStuGenderM.setSelected(true);//selects the male radio button
               }
               else//checks if neither of the genders are filled in
               {
                  rbnEditStuGenderM.setSelected(true);//sets the male radio button to selected
                  lblEditStuGenderError.setText("!!!");//gives an error in the lable
               }
               }
        }
```

```
    catch(SQLException e)
    {
        System.out.println("Could Not Edit Student"); //prints out an error in case of an SQL exception
    }

}

private void cbxCateringActionPerformed(java.awt.event.ActionEvent evt) {

    // TODO add your handling code here:

}

private void btnNewEventActionPerformed(java.awt.event.ActionEvent evt) {

    clearErrorLables(); //clears the error lables by calling the clear error lables method

    dlgEvents.setEnabled(true);

    dlgEvents.setSize(900,600);

    dlgEvents.setLocationRelativeTo(tblEventsAdmin);

    dlgEvents.setVisible(true);

    rbnNewEventCatering.setVisible(true);

    rbnNewEventPhotography.setVisible(true);

    rbnNewEventAV.setVisible(true);

    rbnNewEventFirstAid.setVisible(true);

    if(am.getPortfolio("Catering", currentAdmin) == false)
    {
        rbnNewEventCatering.setVisible(false);
    }

    if (am.getPortfolio("Photography", currentAdmin) == false)
    {
        rbnNewEventPhotography.setVisible(false);
    }

    if (am.getPortfolio("AV", currentAdmin)== false)
    {
        rbnNewEventAV.setVisible(false);
    }

    if (am.getPortfolio("First Aid", currentAdmin) == false)
```

```
        {
            rbnNewEventFirstAid.setVisible(false);
        }
    }

    private void btnNewEventCloseActionPerformed(java.awt.event.ActionEvent evt) {

        dlgEvents.setVisible(false);

    }

    private void btnNewEventEnterActionPerformed(java.awt.event.ActionEvent evt) {

        clearErrorLables(); //clears the error lables by calling the clear error lables method
        String name;
        //Presence, Length and Type check
        boolean validName = presenceAndLengthCheck(txfNewEventName, lblNewNameError) &&
typeCheck(txfNewEventName, lblNewNameError, " :,;'");
        //Precence check
        boolean validDate = CheckDate(lblNewDateError);

        boolean catering = false;//sets catering boolean to false

        boolean av = false;//sets AV boolean to false

        boolean photography = false;//sets photography boolean to false

        boolean firstAid = false;//sets first aid boolean to false

        boolean selected = false;//sets boolean selected to false, to indicate if any radio buttons have been
selected

        //Precence check
        if(rbnNewEventCatering.isSelected()) //checks if the catering radio button is selected
        {
            catering = true; //sets catering boolean to true

            selected = true; //sets selected boolean to true
        }

        else if (rbnNewEventAV.isSelected()) //checks if the AV radio button is selected
        {
            av = true; //sets AV boolean to true

            selected = true; //sets selected boolean to true
        }

        else if (rbnNewEventPhotography.isSelected()) //checks if the Photography radio button is selected
        {
```

```
            photography = true;//sets photography boolean to true

            selected = true; //sets selected boolean to true
        }

        else if (rbnNewEventFirstAid.isSelected()) //checks if the first aid radio button is selected
        {
            firstAid = true;//sets first aid boolean to true

            selected = true; //sets selected boolean to true
        }

        else
        {
            lblNewPortfolioError.setText("Select a portfolio"); //sets the error label to display an error
        }

        //Type check and range check
        String hours = checkHours(txfNewEventHours, lblNewHoursError); //type check

        double h = Double.parseDouble(hours); //converts hours from a string to a double

        if(validName == true &&  validDate == true && hours != null && selected == true) //checks that all
the data is valid
        {
            try {

                name = txfNewEventName.getText(); //sets name variable to the text in the text feild

                LocalDate dateEvent = dpkDate.getDate();//Sets the date to the date entered

                em.add(name,h ,dateEvent, catering, photography, firstAid, av); //adds the new event

                dlgEvents.setVisible(false); //Hides the event dialog

                JOptionPane.showMessageDialog(null, "Event successfully added"); //Displays a message to
indicate success
                //refreshes the table
                AdminEventsTBL((am.getPortfolio("Catering", currentAdmin)),(am.getPortfolio("Photography",
currentAdmin)),(am.getPortfolio("av", currentAdmin)),(am.getPortfolio("First aid", currentAdmin)));

            } catch (SQLException ex) { //catches any sql errors

                System.out.println("SQL ERROR");//prints an error message

            }

        }
```

```
    }

    private void dlgEventsComponentShown(java.awt.event.ComponentEvent evt) {

        clearErrorLables(); //clears the error lables by calling the clear error lables method

    }

    private void pnlEvents1ComponentShown(java.awt.event.ComponentEvent evt) {
        if(currentAdmin != -1) //checks that there is an admin logged in
        {
            //Calls the admin events table method
            AdminEventsTBL((am.getPortfolio("Catering", currentAdmin)),(am.getPortfolio("Photography",
currentAdmin)),(am.getPortfolio("av", currentAdmin)),(am.getPortfolio("First aid", currentAdmin)));
        }
    }

    private void btnLogInActionPerformed(java.awt.event.ActionEvent evt) {
        try{
            boolean correct; //creates a boolean to check that the username and password are correct

            try (Scanner scFile = new Scanner (new File ("UsernamesAndPasswords.txt")) //Creates a
scanner for the text file

            ) {

                String username; //creates a string to hold the username

                int current; //creates an integer to hold the ID value of the student or admin who loggs in

                String password; //creates a string to hold the password

                String title;//creates a string to hold the title admin or student

                correct = false; //sets the value of the boolean to false

                while(scFile.hasNext()) //Checks each line in the text file against the users entry

                {

                    String line = scFile.nextLine(); //creates a string from the next line from the textfile

                    try (Scanner scLine = new Scanner (line).useDelimiter("#")) { //creates a scanner to scan the
current line of the textfile

                        current = scLine.nextInt(); //sets current ID to the student ID from the textfile

                        username = scLine.next();//sets the username to the one from the textfile
```

```
                password = scLine.next();//sets the password to the one from the texfile

                title = scLine.next();//sets the title to the one from the textfile

                //checks username and password against current line of the text file
                if(password.equals(txfPassword.getText()) && username.equals(txfUsername.getText())
&& title.equalsIgnoreCase("Student") )
                {
                   currentStu = current; //sets current student to the value of the ID from the textfile
                   correct = true; //sets the boolean value to true
                   if(currentStu != -1) //checks if the line from the text file was successfull in logging in
                   {
                      pnlLogIn.setVisible(false); //hides the log in pannel
                      tbdpStudent.setVisible(true);//shows the student tabbed pane
                      pnlHome.setVisible(true);//shows the student home page
                   }
                }
                else if (password.equals(txfPassword.getText()) &&
username.equals(txfUsername.getText()) && title.equalsIgnoreCase("Admin") ) // checks username and
password against current line of the text file
                {
                   currentAdmin = current; //sets the value of current admin to the ID from the text file

                   correct = true; //sets the boolean value to true

                   tbdpAdmin.setVisible(true); //shows the admin tabbed pane

                   pnlLogIn.setVisible(false);//hides the log in pannel

                }
                scLine.close();
             }

          }
          scFile.close();
       }

     if (correct == false) //if the log in was not successful
     {
        //Displayes error if incorrect cradentials are entered
       JOptionPane. showMessageDialog(null, "Incorrect Username or password", "", JOptionPane.
ERROR_MESSAGE);
     }
     }
     catch(HeadlessException | FileNotFoundException e){ //catches any errors
        //displays an error message if the error is cought
        JOptionPane. showMessageDialog(null, "Error, pleae try again", "", JOptionPane.
ERROR_MESSAGE);
  }
```

```
    }

    private void btnSignUpStuActionPerformed(java.awt.event.ActionEvent evt) {

        clearErrorLables(); //clears the error lables by calling the clear error lables method

        try{ try ( //Creates a new log in

            PrintWriter outFile = new PrintWriter(new FileWriter("UsernamesAndPasswords.txt",true)))
{//creates a file writer

            char gender = ' '; //creates a variable to hold the gender

            if(rbnNewStudentM.isSelected()) //checks if the male gender radio button is selected
            {
                gender = 'M';//sets the gender to M
            }
            else if (rbnNewStudentF.isSelected()) //checks if the female gender radio button is selected
            {
                gender = 'F';//sets the gender to F
            }
            else
            {
                lblNewStudentGenderError.setText("Please select a gender");//sets the error label if no gender
is selected
            }
            boolean password = false;//creates a boolean to check if the passwords entered match
            if (txfPassword2.getText().equals(txfConfirmPassword1.getText()))//Checks that both passwords
are the same
            {
                password = true;//sets boolean to true if passwords match
            }
            else
            {
                lblNewStudentConfPasswordError.setText("Passwords Do Not Match"); //gives an error is
passwords dont match
            }

            boolean validPortfolio = checkPortfolio();//presence check

            boolean validGrade = checkInt(txfNewStudentGrade, lblNewStudentGradeError,8,12);//type and
range check

            boolean ValidClass = checkInt(txfNewStudentClass, lblNewStudentClassError, 1,5);//type and
range check
            //presence, length and type check
            boolean validUserName = presenceAndLengthCheck(txfUsername1,
lblNewStudentUsernameError) && typeCheck(txfUsername1, lblNewStudentUsernameError, ".:,;'");
```

```
        //presence and length check
        boolean validPassword = presenceAndLengthCheck(txfPassword2,
lblNewStudentPasswordError);
        //presence, length and type check
        boolean validName = presenceAndLengthCheck(txfNewStudentName, lblNewStudentNameError)
&& typeCheck(txfNewStudentName, lblNewStudentNameError, " :,;'@#$%^&*()!+~") &&
SpaceCheck(txfNewStudentName, lblNewStudentNameError);
        //checks that all information is valid
        if((validUserName == true) && (validPassword == true) && (validName == true) && (gender != '
') && (validGrade == true) && (ValidClass == true) && (validPortfolio == true) && (password == true))
        {
            String name = txfNewStudentName.getText();//sets the name string to the one entered by the
user
            int grade = Integer.parseInt(txfNewStudentGrade.getText());//sets the grade integer to the one
entered by the user
            int classPoint =Integer.parseInt(txfNewStudentClass.getText());//sets the class integer to the
one entered by the user
            boolean catering = false;//creates a boolean to check if the user does catering
            boolean photography = false;//creates a boolean to check if the user does photography
            boolean firstAid = false;//creates a boolean to check if the user does first aid
            boolean av = false;//creates a boolean to check if the user does AV
            String lName = name.substring(name.lastIndexOf(" ")+1);//splits the last name from the first
name
            String fName = name.substring(0,name.lastIndexOf(" "));//splits the first name from the last
name

            if(cbxCatering.isSelected())//checks if the catering checkbox is selected
            {
               catering = true;//sets the catering boolean to true
            }
            if(cbxPotography.isSelected())//checks if the photography checkbox is selected
            {
               photography = true;//sets the photography boolean to true
            }
            if(cbxFirstAid.isSelected())//checks if the first aid checkbox is selected
            {
               firstAid = true;//sets the first aid boolean to true
            }
            if(cbxAV.isSelected()) //checks if the AV checkbox is selected
            {
               av = true; //sets the AV boolean to true
            }

            try {

               sm.add(fName,lName , grade, classPoint, gender, catering, photography, av, firstAid);//adds
a student
```

```
                String line = (sm.getLast())+ "#" + txfUsername1.getText() + "#" + txfPassword2.getText() +
"#Student"; //creates a new line in the text file

                currentStu = (sm.getLast());//sets current ctudent to the value of the next ID

                outFile.println(line);//adds the line to the textfile

            } catch (SQLException ex) {//catches any errors
                System.out.println("Could not create an account: Please try again");
            }
            try {
          if(sm.getCurrent(currentStu) != null)//checks that a new account has been created
          {
              pnlSignUp.setVisible(false);//hides the sign up page
              tbdpStudent.setVisible(true);//shows the student tabbed pane
          }
      } catch (SQLException ex) {//catches any erors
                System.out.println("Could not create an account: Please try again");//gives an error message
      }
         }  }
      }
       catch (IOException ex) { //cateches errors
      JOptionPane. showMessageDialog(null, "Please ensure all fields are filled out", "", JOptionPane.
ERROR_MESSAGE);

      }
   }

   private void tbdpAdminComponentShown(java.awt.event.ComponentEvent evt) {
      if(currentAdmin != -1)//checks that current admin has a value
      {
          try {
             txaAdminDetails.setText(am.getCurrent(currentAdmin));//calls a method to show the admin
details on the home page
          } catch (SQLException ex) {//catches any SQL errors
              Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
          }
      }
   }

   private void tbdpStudentComponentShown(java.awt.event.ComponentEvent evt) {
      try {
          if(currentStu != -1)//checks that the current student has a value
          {
            txaStudent.setText(sm.getCurrent(currentStu));//calls a method to show the student details on
the home page
          }
      } catch (SQLException ex) {//catches SQL errors
          Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
```

```
        }
    }

    private void pnlEventsComponentShown(java.awt.event.ComponentEvent evt) {
        //Uses the method to load the Events table for the student
        StudentEventsTBL((sm.getPortfolio("Catering", currentStu)),(sm.getPortfolio("Photography",
currentStu)),(sm.getPortfolio("av", currentStu)),(sm.getPortfolio("First aid", currentStu)));
    }

    private void pnlStudentComponentShown(java.awt.event.ComponentEvent evt) {
         if(currentAdmin != -1) //checks that there is a valid admin is logged in
        {
            StudentsTBL();//Loads the student table for the admin
        }

    }

    private void btnEditEventActionPerformed(java.awt.event.ActionEvent evt) {

        clearErrorLables(); //clears the error lables by calling the clear error lables method

        dlgEditEvent.setEnabled(true);//enables the dialog

        dlgEditEvent.setSize(900,600);//sets the dialog size

        dlgEditEvent.setLocationRelativeTo(tblEventsAdmin);//sets the position of the dialog

        dlgEditEvent.setVisible(true);//shows the dialog

        txfEditEventName.setVisible(false);//hides the text field

        txfEditEventHours.setVisible(false);//hides the text field

        dpkEditDate.setVisible(false);//hides the date picker

        rbnEditEventCatering.setVisible(false);//hides the radio button

        rbnEditEventFirstAid.setVisible(false);//hides the radio button

        rbnEditEventPhotography.setVisible(false);//hides the radio button

        rbnEditEventAV.setVisible(false);//hides the radio button

        btnEditEventEnter.setVisible(false);//hides the button

        lblEditEventName.setVisible(false);//hides the lable

        lblEditEventHours.setVisible(false);//hides the lable
```

```
        lblEditEventDate.setVisible(false);//hides the lable

        lblEditEventPortfolio.setVisible(false);//hides the lable

        btnEditEventSearch.setVisible(true);//hides the button

    }

    private void txfNewEventNameActionPerformed(java.awt.event.ActionEvent evt) {

        // TODO add your handling code here:

    }

    private void txfEditEventIDActionPerformed(java.awt.event.ActionEvent evt) {

        // TODO add your handling code here:

    }

    private void btnEditEventSearchActionPerformed(java.awt.event.ActionEvent evt) {
        if(!"".equals(txfEditEventID.getText()))//checks that the user has entered an event ID
        {
            int Event = Integer.parseInt(txfEditEventID.getText()); //converts the entered value to an integer

            txfEditEventName.setVisible(true);//shows the text field

            txfEditEventHours.setVisible(true);//shows the text field

            dpkEditDate.setVisible(true);//shows the date picker

            rbnEditEventCatering.setVisible(true);//shows the radio button

            rbnEditEventPhotography.setVisible(true);//shows the radio button

            rbnEditEventAV.setVisible(true);//shows the radio button

            rbnEditEventFirstAid.setVisible(true);//shows the radio button

            if(am.getPortfolio("Catering", currentAdmin) == false)//checks if the admin has permissions for
catering
            {
                rbnEditEventCatering.setVisible(false);//hides the radio button if the admin does not have
permission to create an event for that portfolio
            }

            if (am.getPortfolio("Photography", currentAdmin) == false) //checks if the admin has permissions
for photography
            {
```

```
            rbnEditEventPhotography.setVisible(false);//hides the radio button if the admin does not have permission to create an event for that portfolio
        }

        if (am.getPortfolio("AV", currentAdmin)== false)//checks if the admin has permissions for AV
        {
            rbnEditEventAV.setVisible(false);//hides the radio button if the admin does not have permission to create an event for that portfolio
        }

        if (am.getPortfolio("First Aid", currentAdmin) == false)//checks if the admin has permissions for first aid
        {
            rbnEditEventFirstAid.setVisible(false);//hides the radio button if the admin does not have permission to create an event for that portfolio
        }

        btnEditEventEnter.setVisible(true);//shows the button

        lblEditEventName.setVisible(true);//shows the lable

        lblEditEventHours.setVisible(true);//shows the lable

        lblEditEventDate.setVisible(true);//shows the lable

        lblEditEventPortfolio.setVisible(true);//shows the lable

        try //Searches for an entry by dive number and returns it to the table if found
    {
        ResultSet rs = dbman.queryDatabase("SELECT * FROM tblEvents WHERE EventID =" +txfEditEventID.getText()); //Retrives entry(s) from the database

        while(rs.next())//loops through all the matching entries from the databse
        {
            String portfolio = "";//sets portfolio string to nothing
            if(rs.getBoolean("Catering") == true) //checks if the event is listed for catering
            {

                rbnEditEventCatering.setSelected(true);//selects the catering radio button
                portfolio = "Catering";//sets the portfolio string to catering
            }
            else if(rs.getBoolean("Photography") == true)//checks if the event is listed for photography
            {
                rbnEditEventPhotography.setSelected(true);//selects the photography radio button
                portfolio = "Photography";//sets the portfolio string to photography
            }
            else if(rs.getBoolean("FirstAid") == true)//checks if the event is listed for first aid
            {
                rbnEditEventFirstAid.setSelected(true);//selects the first aid radio button
```

```
                portfolio = "First Aid";//sets the portfolio string to first aid
            }
            else if(rs.getBoolean("AV") == true)//checks if the event is listed for AV
            {
                portfolio = "AV";//sets the portfolio string to AV
                rbnEditEventAV.setSelected(true);//selects the AV radio button
            }
            boolean c = am.getPortfolio("Catering", currentAdmin);//checkss if the admin oversees catering
            boolean p = am.getPortfolio("Photography", currentAdmin);//checkss if the admin oversees
photography
            boolean a = am.getPortfolio("av", currentAdmin);//checkss if the admin oversees AV
            boolean f = am.getPortfolio("First aid", currentAdmin);//checkss if the admin oversees first aid
            //checks if the admin has permission to view the event
            if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true &&
portfolio.equalsIgnoreCase("Photography")) || (a == true && portfolio.equalsIgnoreCase("AV")) || (f ==
true && portfolio.equalsIgnoreCase("FirstAid")))

            {
                txfEditEventID.setText(rs.getString("EventID"));//sets the text field to the event ID
                txfEditEventName.setText(rs.getString("EventName"));//sets the txt field to the event name
                txfEditEventHours.setText(rs.getString("Hours"));//sets the text field to the event hours
                LocalDate Date = rs.getDate("EventDate").toLocalDate();//gets the date from the databse
                dpkEditDate.setDate(Date);//sets the date picker to that date
            }
        }
            btnEditEventSearch.setVisible(false);//hides the search button
        }
        catch(SQLException e) //Catches any sql Errors
        {
            JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//gives an error if the event was not found
        }
        }
        else //if no ID has been entered
        {
            lblEditEventID.setText("Enter An Event ID");//sets the error lable to give an error
        }

    }

    private void btnEditEventCloseActionPerformed(java.awt.event.ActionEvent evt) {
        dlgEditEvent.setVisible(false);//hides the dialog
    }

    private void dlgEditEventComponentShown(java.awt.event.ComponentEvent evt) {
        // TODO add your handling code here:
    }

    private void btnEditEventEnterActionPerformed(java.awt.event.ActionEvent evt) {
```

```
        clearErrorLables(); //clears the error lables by calling the clear error lables method
        String name; //holds the event name
        //Presence, Length and Type check
        boolean validName = presenceAndLengthCheck(txfEditEventName, lblEditNameError) &&
typeCheck(txfEditEventName, lblEditNameError, " :,;'");
        //Precence check
        boolean validDate = CheckEditDate(lblEditDateError);
        boolean catering = false;//holds the boolean for if the event is for catering
        boolean av = false;//holds the boolean for if the event is for AV
        boolean photography = false;//holds the boolean for if the event is for photography
        boolean firstAid = false;//holds the boolean for if the event is for first aid
        boolean selected = false;//checks that a portfolio has been selected

        //Precence check
        if(rbnEditEventCatering.isSelected())//checks if the catering radio button is selected
        {
            catering = true;//sets the catering boolean to true

            selected = true;//sets the selected boolean to true
        }
        else if (rbnEditEventAV.isSelected())//checks if the AV radio button is selected
        {
            av = true;//sets the AV boolean to true

            selected = true;//sets the selected boolean to true
        }
        else if (rbnEditEventPhotography.isSelected())//checks if the Photography radio button is selected
        {
            photography = true;//sets the photography boolean to true

            selected = true;//sets the selected boolean to true
        }
        else if (rbnEditEventFirstAid.isSelected())//checks if the first aid radio button is selected
        {
            firstAid = true;//sets the first aid boolean to true

            selected = true;//sets the selected boolean to true
        }
        else//if none of the buttons are selected
        {
            lblEditPortfolioError.setText("Select a portfolio");//sets the lable to an error
        }

        //Type check and range check
        String hours = checkHours(txfEditEventHours, lblEditHoursError);

        double h = Double.parseDouble(hours);//converts hours to a double

        //checks that all the values are valid
```

```
        if(validName == true &&  validDate == true && hours != null && selected == true)
        {
            try {
                name = txfEditEventName.getText();//sets name to the name entered by the user

                LocalDate dateEvent = dpkEditDate.getDate();//gets the date from the date picker
                //creates the update event query
                String sql = ("UPDATE tblEvents SET EventName = '" + name+ "',Hours = " + hours+ ",
EventDate = #" + dateEvent + "#, Catering = " + catering + ", Photography = " + photography +  ", FirstAid
= " + firstAid + ", AV = " + av + " WHERE EventID = " + txfEditEventID.getText() + ";");

                dbman.updateQuery(sql);//executes the query

                dlgEditEvent.setVisible(false);//hides the dialog

                JOptionPane.showMessageDialog(null, "Event successfully Updated");//shows a message to
indicate a successfull entry

            } catch (SQLException ex) {//catches SQL errors
                System.out.println("SQL ERROR");//gives an error
            }
        }
        //reloads the events table for the admin
        AdminEventsTBL((am.getPortfolio("Catering", currentAdmin)),(am.getPortfolio("Photography",
currentAdmin)),(am.getPortfolio("av", currentAdmin)),(am.getPortfolio("First aid", currentAdmin)));

    }

    private void pnlParticipationComponentShown(java.awt.event.ComponentEvent evt) {
        //loads the participation table for the student
        ParticipationTBL((sm.getPortfolio("Catering", currentStu)),(sm.getPortfolio("Photography",
currentStu)),(sm.getPortfolio("av", currentStu)),(sm.getPortfolio("First aid", currentStu)));
    }

    private void pnlParticipation1ComponentShown(java.awt.event.ComponentEvent evt) {
        if(currentAdmin != -1)//checks that there is a valid admin logged in
        {
            //loads the participation table for the admin
            AdminParticipationTBL((am.getPortfolio("Catering",
currentAdmin)),(am.getPortfolio("Photography", currentAdmin)),(am.getPortfolio("av",
currentAdmin)),(am.getPortfolio("First aid", currentAdmin)));
        }
    }

    private void btnSearchNameActionPerformed(java.awt.event.ActionEvent evt) {
        try //Searches for an entry by Event name and returns it to the table if found
        {
            DefaultTableModel dtm = (DefaultTableModel) tblEventsAdmin.getModel(); //Sets the default table
model
```

```
        tblEventsAdmin.removeAll();//Empties the Jtable
        dtm.setRowCount(0); //sets the row count to 0
        model = (DefaultTableModel)tblEventsAdmin.getModel(); //Sets the default table model
        ResultSet rs = dbman.queryDatabase("SELECT * FROM tblEvents WHERE EventName LIKE '*"
+ (txfSearchName.getText()) + "*'"); //Retrives entry(s) from the database
        int count = 0; //sets a counter to 0
        while(rs.next()) //checks if the result set has a next result and moves to it if it does
        {
            String portfolio = ""; //Creates a string to store the portfolio of the event
            if(rs.getBoolean("Catering") == true) //Checks if the event is for catering
            {
                portfolio = "Catering"; //Sets the portfolio string to catering
            }
            else if(rs.getBoolean("Photography") == true) //Checks if the event is for photography
            {
                portfolio = "Photography"; //Sets the portfolio string to photography
            }
            else if(rs.getBoolean("FirstAid") == true) //Checks if the event is for first aid
            {
                portfolio = "First Aid"; ////Sets the portfolio string to First aid
            }
            else if(rs.getBoolean("AV") == true) //Checks if the event is for AV
            {
                portfolio = "AV"; //Sets the portfolio string to AV
            }
            boolean c = am.getPortfolio("Catering", currentAdmin); //checks if the admin oversees catering
            boolean p = am.getPortfolio("Photography", currentAdmin); //checks if the admin oversees
photography
            boolean a = am.getPortfolio("av", currentAdmin); //checks if the admin oversees AV
            boolean f = am.getPortfolio("First aid", currentAdmin);//checks if the admin oversees first aid
            //Checks that the admin has permission to view the event
            if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true &&
portfolio.equalsIgnoreCase("Photography")) || (a == true && portfolio.equalsIgnoreCase("AV")) || (f ==
true && portfolio.equalsIgnoreCase("First Aid")))
            {
                count ++;//adds 1 to count
                model.addRow(new Object[]{rs.getInt("EventID"),rs.getString("EventName"),
                rs.getInt("Hours"),rs.getDate("EventDate"), portfolio}); //Creates a new object to be added to
the table
            }
        }
        if(count == 0) //checks if count is till equal to 0
        {
            JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message to indicate that the event could not be found
        }
        txfSearchName.setText(""); //Clears the text field
        model.addRow(new Object[]{}); //Adds row to the table
    }
```

```
        catch(SQLException e) //Catches any sql Errors
        {
            JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE); //Displays an error message to indicate that the event could not be found
        }
    }

    private void btnSearchIDActionPerformed(java.awt.event.ActionEvent evt) {
        try //Searches for an entry by Event ID and returns it to the table if found
        {
            DefaultTableModel dtm = (DefaultTableModel) tblEventsAdmin.getModel();//Sets the default table
model
            tblEventsAdmin.removeAll();//Empties the Jtable
            dtm.setRowCount(0); //sets the row count to 0
            model = (DefaultTableModel)tblEventsAdmin.getModel(); //Sets the default table model
            ResultSet rs = dbman.queryDatabase("SELECT * FROM tblEvents WHERE EventID ="
+txfSearchID.getText()); //Retrives entry(s) from the database
            int count = 0; //sets a counter to 0
            while(rs.next()) //checks if the result set has a next result and moves to it if it does
            {
                String portfolio = ""; //Creates a string to store the portfolio of the event
                if(rs.getBoolean("Catering") == true) //Checks if the event is for catering
                {
                    portfolio = "Catering"; //Sets the portfolio string to catering
                }
                else if(rs.getBoolean("Photography") == true) //Checks if the event is for photography
                {
                    portfolio = "Photography"; //Sets the portfolio string to photography
                }
                else if(rs.getBoolean("FirstAid") == true) //Checks if the event is for first aid
                {
                    portfolio = "First Aid"; ////Sets the portfolio string to First aid
                }
                else if(rs.getBoolean("AV") == true) //Checks if the event is for AV
                {
                    portfolio = "AV"; //Sets the portfolio string to AV
                }
                boolean c = am.getPortfolio("Catering", currentAdmin); //checks if the admin oversees catering
                boolean p = am.getPortfolio("Photography", currentAdmin); //checks if the admin oversees
photography
                boolean a = am.getPortfolio("av", currentAdmin); //checks if the admin oversees AV
                boolean f = am.getPortfolio("First aid", currentAdmin);//checks if the admin oversees first aid
                //Checks that the admin has permission to view the event
                if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true &&
portfolio.equalsIgnoreCase("Photography")) || (a == true && portfolio.equalsIgnoreCase("AV")) || (f ==
true && portfolio.equalsIgnoreCase("First Aid")))
                {
                    count ++;//adds 1 to count
                    model.addRow(new Object[]{rs.getInt("EventID"),rs.getString("EventName"),
```

```
                rs.getInt("Hours"),rs.getDate("EventDate"), portfolio}); //Creates a new object to be added to
the table
            }
        }
            if(count == 0) //checks if count is till equal to 0
            {
                JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message to indicate that the event could not be found
            }
            txfSearchID.setText(""); //Clears the text field
            model.addRow(new Object[]{}); //Adds row to the table
        }
        catch(SQLException e) //Catches any sql Errors
        {
            JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE); //Displays an error message to indicate that the event could not be found
        }

    }

    private void btnSearchStudentNamePartActionPerformed(java.awt.event.ActionEvent evt) {
        try //Searches for an entry by student name and returns it to the table if found
        {
            DefaultTableModel dtm = (DefaultTableModel) tblParticipationAdmin.getModel(); //Sets the default
table model
            tblParticipationAdmin.removeAll();//Empties the Jtable
            dtm.setRowCount(0); //sets the row count to 0
            model = (DefaultTableModel)tblParticipationAdmin.getModel(); //Sets the default table model
            String fullname = txfSearchStudentNamePart.getText();//creates a string to hold the student name
entered by the admin
            String lName = "";//Creates a string to hold the student's last name
            String fName = "";//creates a string to hold the student's first name
            if (fullname.contains(" "))//checks that the string contains a space
            {
                lName = fullname.substring(fullname.lastIndexOf(" ")+1);//puts the string after the last space
into lname
                fName = fullname.substring(0,fullname.lastIndexOf(" "));//puts the string before the last space
into fname
            }
            ResultSet rs = dbman.queryDatabase("SELECT DISTINCT tblStudents.StudentID,
tblStudents.firstName, tblStudents.Surname, tblEvents.EventID, tblEvents.EventName, tblEvents.hours,
tblEvents.Catering, tblEvents.Photography, tblEvents.AV, tblEvents.FirstAid FROM tblEvents,
tblStudents, tblParticipation Where tblParticipation.StudentID = tblStudents.StudentID AND
tblParticipation.EventID = tblEvents.EventID AND FirstName LIKE '*" + fName + "*' OR surname LIKE '*"
+ lName + "*'"); //Retrives entry(s) from the database
            int count = 0; //sets a counter to 0
            while(rs.next()) //checks if the result set has a next result and moves to it if it does
            {
                String portfolio = "";//Creates a string to store the portfolio of the event
```

```
            if(rs.getBoolean("Catering") == true) //Checks if the event of the participation record is for
catering
            {
               portfolio = "Catering"; //Sets the portfolio string to catering
            }
            else if(rs.getBoolean("Photography") == true) //Checks if the event of the participation record is
for photography
            {
               portfolio = "Photography"; //Sets the portfolio string to photography
            }
            else if(rs.getBoolean("FirstAid") == true) //Checks if the event of the participation record is for
first aid
            {
               portfolio = "First Aid"; //Sets the portfolio string to first aid
            }
            else if(rs.getBoolean("AV") == true) //Checks if the event of the participation record is for AV
            {
               portfolio = "AV"; //Sets the portfolio string to AV
            }
            boolean c = am.getPortfolio("Catering", currentAdmin);//checks if the admin oversees catering
            boolean p = am.getPortfolio("Photography", currentAdmin); //checks if the admin oversees
photography
            boolean a = am.getPortfolio("av", currentAdmin); //checks if the admin oversees AV
            boolean f = am.getPortfolio("First aid", currentAdmin); //checks if the admin oversees first aid

            if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true &&
portfolio.equalsIgnoreCase("Photography")) || (a == true && portfolio.equalsIgnoreCase("AV")) || (f ==
true && portfolio.equalsIgnoreCase("First Aid")))
            {
               String name = rs.getString("firstName") + " " + rs.getString("surname");//gets the student's full
name
               model.addRow(new
Object[]{rs.getInt("StudentID"),name,rs.getInt("EventID"),rs.getString("EventName"),
                rs.getInt("Hours"), portfolio});//Creates a new object to be added to the table
               count++;//adds 1 to count
            }
      }
         if(count == 0) //checks if count is till equal to 0
         {
            JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message to indicate that the event could not be found
         }
         txfSearchStudentNamePart.setText(""); //Clears the text field
         model.addRow(new Object[]{}); //Adds row to the table
      }
      catch(SQLException e) //Catches any sql Errors
      {
         JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message to indicate that the event could not be found
```

```
        }

    }

    private void tbnSearchEventIDPartActionPerformed(java.awt.event.ActionEvent evt) {
        try //Searches for an entry by Event ID and returns it to the table if found
        {
            DefaultTableModel dtm = (DefaultTableModel) tblParticipationAdmin.getModel(); //Sets the default table model
            tblParticipationAdmin.removeAll();//Empties the Jtable
            dtm.setRowCount(0); //sets the row count to 0
            model = (DefaultTableModel)tblParticipationAdmin.getModel(); //Sets the default table model
            ResultSet rs = dbman.queryDatabase("SELECT DISTINCT tblStudents.StudentID, tblStudents.firstName, tblStudents.Surname, tblEvents.EventID, tblEvents.EventName, tblEvents.hours, tblEvents.Catering, tblEvents.Photography, tblEvents.AV, tblEvents.FirstAid FROM tblEvents, tblStudents, tblParticipation Where tblParticipation.StudentID = tblStudents.StudentID AND tblParticipation.EventID = tblEvents.EventID AND EventID = " + txfSearchEventIDPart.getText()); //Retrives entry(s) from the database
            int count = 0; //sets a counter to 0
            while(rs.next()) //checks if the result set has a next result and moves to it if it does
            {
                String portfolio = "";//Creates a string to store the portfolio of the event
                if(rs.getBoolean("Catering") == true) //Checks if the event of the participation record is for catering
                {
                    portfolio = "Catering"; //Sets the portfolio string to catering
                }
                else if(rs.getBoolean("Photography") == true) //Checks if the event of the participation record is for photography
                {
                    portfolio = "Photography"; //Sets the portfolio string to photography
                }
                else if(rs.getBoolean("FirstAid") == true) //Checks if the event of the participation record is for first aid
                {
                    portfolio = "First Aid"; //Sets the portfolio string to first aid
                }
                else if(rs.getBoolean("AV") == true) //Checks if the event of the participation record is for AV
                {
                    portfolio = "AV"; //Sets the portfolio string to AV
                }
                boolean c = am.getPortfolio("Catering", currentAdmin);//checks if the admin oversees catering
                boolean p = am.getPortfolio("Photography", currentAdmin); //checks if the admin oversees photography
                boolean a = am.getPortfolio("av", currentAdmin); //checks if the admin oversees AV
                boolean f = am.getPortfolio("First aid", currentAdmin); //checks if the admin oversees first aid
```

```
            if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true &&
portfolio.equalsIgnoreCase("Photography")) || (a == true && portfolio.equalsIgnoreCase("AV")) || (f ==
true && portfolio.equalsIgnoreCase("First Aid")))
            {
                String name = rs.getString("firstName") + " " + rs.getString("surname");//gets the student's full
name
                model.addRow(new
Object[]{rs.getInt("StudentID"),name,rs.getInt("EventID"),rs.getString("EventName"),
                 rs.getInt("Hours"), portfolio});//Creates a new object to be added to the table
                count++;//adds 1 to count
            }
        }

        txfSearchEventIDPart.setText(""); //Clears the text field
        model.addRow(new Object[]{}); //Adds row to the table
        if(count == 0) //checks if count is till equal to 0
        {
            JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message to indicate that the event could not be found
        }
      }
      catch(SQLException e) //Catches any sql Errors
      {
        JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message to indicate that the event could not be found
      }


    }

    private void btnSearchStuNameActionPerformed(java.awt.event.ActionEvent evt) {
      try
      {
        DefaultTableModel dtm = (DefaultTableModel) tblStudents.getModel(); //Sets the default table
model
        dtm.setRowCount(0);//sets the row count to 0
        tblStudents.removeAll();//removes everything from the table
        model = (DefaultTableModel)tblStudents.getModel(); //Sets the default table model
        String fullname = txfSearchStuName.getText();//creates a string to hold the student name entered
by the admin
        String lName = "";
        String fName = "";
        if (fullname.contains(" "))
        {
          lName = fullname.substring(fullname.lastIndexOf(" ")+1);
          fName = fullname.substring(0,fullname.lastIndexOf(" "));
        }
        fName = fullname;
```

```
        ResultSet rs = dbman.queryDatabase("SELECT * FROM tblStudents WHERE firstName LIKE '*"
+ fName + "*' OR surname LIKE '*" + lName + "*'"); //Retrives entry(s) from the database
        int count = 0; //sets a counter to 0
        while(rs.next()) //Creates a line in the JTable for every database entry
        {
            String stu = rs.getString("StudentID");//Sets the student ID to the ID value of the current record
in the database

            int catering = 0; //Creates a variable to hold the catering hours total

            int photography = 0; //Creates a variable to hold the photography hours total

            int firstAid = 0; //Creates a variable to hold the first aid hours total

            int av = 0; //Creates a variable to hold the AV hours total

        ResultSet ch = dbman.queryDatabase("SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS
CateringHours\n" +
                "FROM tblStudents, tblEvents, tblParticipation\n" +
                "WHERE tblParticipation.StudentID = tblStudents.StudentID AND tblEvents.EventID =
tblParticipation.EventID AND tblEvents.Catering = True\n" +
                "GROUP BY tblStudents.StudentID;"); //Sums the total catering hours for each student
        ResultSet ph = dbman.queryDatabase("SELECT tblStudents.StudentID, SUM(tblEvents.hours)
AS PhotographyHours\n" +
                "FROM tblStudents, tblEvents, tblParticipation\n" +
                "WHERE tblParticipation.StudentID = tblStudents.StudentID AND tblEvents.EventID =
tblParticipation.EventID AND tblEvents.Photography = True\n" +
                "GROUP BY tblStudents.StudentID;"); //Sums the total photography hours for each student
        ResultSet avh = dbman.queryDatabase("SELECT tblStudents.StudentID, SUM(tblEvents.hours)
AS AVHours\n" +
                "FROM tblStudents, tblEvents, tblParticipation\n" +
                "WHERE tblParticipation.StudentID = tblStudents.StudentID AND tblEvents.EventID =
tblParticipation.EventID AND tblEvents.AV = True\n" +
                "GROUP BY tblStudents.StudentID;"); //Sums the total AV hours for each student
        ResultSet fh = dbman.queryDatabase("SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS
FirstAidHours\n" +
                "FROM tblStudents, tblEvents, tblParticipation\n" +
                "WHERE tblParticipation.StudentID = tblStudents.StudentID AND tblEvents.EventID =
tblParticipation.EventID AND tblEvents.FirstAid = True\n" +
                "GROUP BY tblStudents.StudentID;"); //Sums the total first aid hours for each student

            while (ch.next()) //Loops throught the catering hours results
            {
                //checks if the admin oversees catering and that the current student ID from the main result
set matches the catering hours result set matches the catering hours result set
                if(am.getPortfolio("Catering", currentAdmin) == true &&
stu.equals(ch.getString("StudentID")))

                {
```

```
                    catering = ch.getInt("CateringHours");//assigns the current students total catering hours to
the catering variable
                }
            }

            while (ph.next()) //Loops throught the photography hours results
            {
                //checks if the admin oversees catering and that the current student ID from the main result
set matches the catering hours result set matches the photography hours result set
                if(am.getPortfolio("Photography", currentAdmin) == true &&
stu.equals(ph.getString("StudentID")))

                {
                    photography = ph.getInt("PhotographyHours");//assigns the current students total
photography hours to the photography variable
                }
            }

            while (avh.next()) //Loops throught the AV hours results
            {
                //checks if the admin oversees catering and that the current student ID from the main result
set matches the catering hours result set matches the AV hours result set
                if(am.getPortfolio("AV", currentAdmin) == true && stu.equals(avh.getString("StudentID")))

                {
                    av = avh.getInt("AVHours"); //assigns the current students total AV hours to the AV variable
                }
            }

            while (fh.next()) //Loops throught the first aid hours results
            {
                //checks if the admin oversees catering and that the current student ID from the main result
set matches the catering hours result set matches the first aid hours result set
                if(am.getPortfolio("First Aid", currentAdmin) == true &&
stu.equals(fh.getString("StudentID")))
                {
                    firstAid = fh.getInt("FirstAidHours"); //assigns the current students total first aid hours to the
first aid variable
                }
            }
            String grade = rs.getInt("Grade") + "." + rs.getInt("Class"); //Creates a string to contain the
grade and class of the learner in a formatted manner
            String name = rs.getString("FirstName") + " " + rs.getString("Surname"); //Creates a string to
contain the students full name
            String cater = Integer.toString(catering);//Converts the catering hours to a string
            String aV = Integer.toString(av); //converts the AV hours to a string
            String photo = Integer.toString(photography); //Converts the photography hours to a string
            String first =Integer.toString(firstAid); //Converts the first aid hours to a string
```

```
            if (am.getPortfolio("First Aid", currentAdmin) == false) //checks if the admin has permissions for
first aid
            {
               first = "NA"; //if the admin does not have permissions for first aid, sets the first aid hours
collum to 'NA'
            }
            if (am.getPortfolio("AV", currentAdmin) == false)//checks if the admin has permissions for AV
            {
               aV = "NA"; //if the admin does not have permissions for AV, sets the AV hours collum to 'NA'
            }
            if(am.getPortfolio("Catering", currentAdmin) == false) //checks if the admin has permissions for
catering
            {
               cater = "NA"; //if the admin does not have permissions for catering, sets the catering hours
collum to 'NA'
            }
            if (am.getPortfolio("Photography", currentAdmin) == false) //checks if the admin has
permissions for photography
            {
               photo = "NA"; //if the admin does not have permissions for photography, sets the
photography hours collum to 'NA'
            }
            //Creates a new object to be added to the table
             model.addRow(new Object[]{rs.getInt("StudentID"), name, grade, rs.getString("Gender"),cater,
aV, photo,first });

            }
         model.addRow(new Object[]{}); //adds each object to its own row in the table
          if(count == 0) //checks if count is till equal to 0
          {
             JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message to indicate that the event could not be found
          }
      }
      catch(SQLException e)
      {
          JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message to indicate that the event could not be found
      }
          txfSearchStuName.setText(""); //Clears the text field
          model.addRow(new Object[]{}); //Adds row to the table

   }

   private void btnSearchStuGradeActionPerformed(java.awt.event.ActionEvent evt) {
      try //Searches for an entry by Student grade and returns it to the table if found
      {
          DefaultTableModel dtm = (DefaultTableModel) tblStudents.getModel(); //Sets the default table
model
```

```
        tblStudents.removeAll();//Empties the Jtable
        dtm.setRowCount(0); //sets the row count to 0
        model = (DefaultTableModel)tblStudents.getModel(); //Sets the default table model
        ResultSet rs = dbman.queryDatabase("SELECT * FROM tblStudents WHERE Grade =" +
Integer.parseInt(txfSearchStuGrade.getText()) ); //Retrives entry(s) from the database
        int count = 0; //sets a counter to 0
        while(rs.next()) //Creates a line in the JTable for every database entry
        {
            String stu = rs.getString("StudentID");//Sets the student ID to the ID value of the current record
in the database

            int catering = 0; //Creates a variable to hold the catering hours total

            int photography = 0; //Creates a variable to hold the photography hours total

            int firstAid = 0; //Creates a variable to hold the first aid hours total

            int av = 0; //Creates a variable to hold the AV hours total

        ResultSet ch = dbman.queryDatabase("SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS
CateringHours\n" +
                "FROM tblStudents, tblEvents, tblParticipation\n" +
                "WHERE tblParticipation.StudentID = tblStudents.StudentID AND tblEvents.EventID =
tblParticipation.EventID AND tblEvents.Catering = True\n" +
                "GROUP BY tblStudents.StudentID;"); //Sums the total catering hours for each student
        ResultSet ph = dbman.queryDatabase("SELECT tblStudents.StudentID, SUM(tblEvents.hours)
AS PhotographyHours\n" +
                "FROM tblStudents, tblEvents, tblParticipation\n" +
                "WHERE tblParticipation.StudentID = tblStudents.StudentID AND tblEvents.EventID =
tblParticipation.EventID AND tblEvents.Photography = True\n" +
                "GROUP BY tblStudents.StudentID;"); //Sums the total photography hours for each student
        ResultSet avh = dbman.queryDatabase("SELECT tblStudents.StudentID, SUM(tblEvents.hours)
AS AVHours\n" +
                "FROM tblStudents, tblEvents, tblParticipation\n" +
                "WHERE tblParticipation.StudentID = tblStudents.StudentID AND tblEvents.EventID =
tblParticipation.EventID AND tblEvents.AV = True\n" +
                "GROUP BY tblStudents.StudentID;"); //Sums the total AV hours for each student
        ResultSet fh = dbman.queryDatabase("SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS
FirstAidHours\n" +
                "FROM tblStudents, tblEvents, tblParticipation\n" +
                "WHERE tblParticipation.StudentID = tblStudents.StudentID AND tblEvents.EventID =
tblParticipation.EventID AND tblEvents.FirstAid = True\n" +
                "GROUP BY tblStudents.StudentID;"); //Sums the total first aid hours for each student

            while (ch.next()) //Loops throught the catering hours results
            {
                //checks if the admin oversees catering and that the current student ID from the main result
set matches the catering hours result set matches the catering hours result set
```

```
                if(am.getPortfolio("Catering", currentAdmin) == true &&
stu.equals(ch.getString("StudentID")))

                {
                    catering = ch.getInt("CateringHours");//assigns the current students total catering hours to
the catering variable
                }
            }

            while (ph.next()) //Loops throught the photography hours results
            {
                //checks if the admin oversees catering and that the current student ID from the main result
set matches the catering hours result set matches the photography hours result set
                if(am.getPortfolio("Photography", currentAdmin) == true &&
stu.equals(ph.getString("StudentID")))

                {
                    photography = ph.getInt("PhotographyHours");//assigns the current students total
photography hours to the photography variable
                }
            }

            while (avh.next()) //Loops throught the AV hours results
            {
                //checks if the admin oversees catering and that the current student ID from the main result
set matches the catering hours result set matches the AV hours result set
                if(am.getPortfolio("AV", currentAdmin) == true && stu.equals(avh.getString("StudentID")))

                {
                    av = avh.getInt("AVHours"); //assigns the current students total AV hours to the AV variable
                }
            }

            while (fh.next()) //Loops throught the first aid hours results
            {
                //checks if the admin oversees catering and that the current student ID from the main result
set matches the catering hours result set matches the first aid hours result set
                if(am.getPortfolio("First Aid", currentAdmin) == true &&
stu.equals(fh.getString("StudentID")))
                {
                    firstAid = fh.getInt("FirstAidHours"); //assigns the current students total first aid hours to the
first aid variable
                }
            }
            String grade = rs.getInt("Grade") + "." + rs.getInt("Class"); //Creates a string to contain the
grade and class of the learner in a formatted manner
            String name = rs.getString("FirstName") + " " + rs.getString("Surname"); //Creates a string to
contain the students full name
            String cater = Integer.toString(catering);//Converts the catering hours to a string
```

```
            String aV = Integer.toString(av); //converts the AV hours to a string
            String photo = Integer.toString(photography); //Converts the photography hours to a string
            String first =Integer.toString(firstAid); //Converts the first aid hours to a string
            if (am.getPortfolio("First Aid", currentAdmin) == false) //checks if the admin has permissions for
first aid
            {
               first = "NA"; //if the admin does not have permissions for first aid, sets the first aid hours
collum to 'NA'
            }
            if (am.getPortfolio("AV", currentAdmin) == false)//checks if the admin has permissions for AV
            {
               aV = "NA"; //if the admin does not have permissions for AV, sets the AV hours collum to 'NA'
            }
            if(am.getPortfolio("Catering", currentAdmin) == false) //checks if the admin has permissions for
catering
            {
               cater = "NA"; //if the admin does not have permissions for catering, sets the catering hours
collum to 'NA'
            }
            if (am.getPortfolio("Photography", currentAdmin) == false) //checks if the admin has
permissions for photography
            {
               photo = "NA"; //if the admin does not have permissions for photography, sets the
photography hours collum to 'NA'
            }
            //Creates a new object to be added to the table
             model.addRow(new Object[]{rs.getInt("StudentID"), name, grade, rs.getString("Gender"),cater,
aV, photo,first });
             count++;//adds 1 to count
            }
         model.addRow(new Object[]{}); //adds each object to its own row in the table
          if(count == 0) //checks if count is till equal to 0
          {
             JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message to indicate that the event could not be found
          }
       }
       catch(SQLException e)
       {
          JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message to indicate that the event could not be found
       }
          txfSearchStuGrade.setText(""); //Clears the text field
          model.addRow(new Object[]{}); //Adds row to the table
    }

    private void btnNewAdminActionPerformed(java.awt.event.ActionEvent evt) {
       clearErrorLables(); //clears the error lables by calling the clear error lables method
```

```
        dlgNewAdmin.setEnabled(true);//enables the dialog

        dlgNewAdmin.setSize(900,600);//sets the dialog size

        dlgNewAdmin.setLocationRelativeTo(lblLogoHome1);//sets the dialog position

        dlgNewAdmin.setVisible(true);//displays the dialog

        txfNewAdminName.setText("");//clears the text field
        txfNewAdminUsername.setText("");//clears the text field
        txfNewAdminPassword.setText("");//clears the text field
        txfNewAdminConfirmPassword.setText("");//clears the text field

    }

    private void tbnSearchEventNameActionPerformed(java.awt.event.ActionEvent evt) {
        try //Searches for an entry by Event name and returns it to the table if found
        {
            DefaultTableModel dtm = (DefaultTableModel) tblEventStu.getModel(); //Sets the default table
model
            tblEventStu.removeAll();//Empties the Jtable
            dtm.setRowCount(0); //sets the row count to 0
            model = (DefaultTableModel)tblEventStu.getModel(); //Sets the default table model
            ResultSet rs = dbman.queryDatabase("SELECT * FROM tblEvents WHERE EventName LIKE'*" +
txfSearchEventName.getText() +"*'"); //Retrives entry(s) from the database
            int count = 0; //sets a counter to 0
            while(rs.next()) //checks if the result set has a next result and moves to it if it does
            {
                String portfolio = ""; //Creates a string to store the portfolio of the event

                if(rs.getBoolean("Catering") == true) //Checks if the event is for catering
                {
                    portfolio = "Catering";//Sets the portfolio string to catering
                }
                else if(rs.getBoolean("Photography") == true) //Checks if the event is for photography
                {
                    portfolio = "Photography"; //Sets the portfolio string to photography
                }
                else if(rs.getBoolean("FirstAid") == true)//Checks if the event is for first aid
                {
                    portfolio = "First Aid"; //Sets the portfolio string to first aid
                }
                else if(rs.getBoolean("AV") == true)//Checks if the event is for AV
                {
                    portfolio = "AV"; //Sets the portfolio string to AV
                }
                boolean c = sm.getPortfolio("Catering", currentStu); //checks if the student does catering
                boolean p = sm.getPortfolio("Photography", currentStu); //checks if the student does
photography
```

```
            boolean a = sm.getPortfolio("av", currentStu); //checks if the student does AV
            boolean f = sm.getPortfolio("First aid", currentStu); //checks if the student does first aid

            //Checks that the student has permission to view the event
            if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true &&
portfolio.equalsIgnoreCase("Photography")) || (a == true && portfolio.equalsIgnoreCase("AV")) || (f ==
true && portfolio.equalsIgnoreCase("First Aid")))
            {
                model.addRow(new Object[]{rs.getInt("EventID"),rs.getString("EventName"),
                 rs.getInt("Hours"),rs.getDate("EventDate"), portfolio}); //Creates a new object to be added to
the table
                count++; //adds 1 to count
            }
        }
            if(count == 0) //checks if count is till equal to 0
            {
                JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message to indicate that the event could not be found
            }
            txfSearchEventName.setText(""); //Clears the text field
            model.addRow(new Object[]{}); //Adds row to the table
        }
        catch(SQLException e) //Catches any sql Errors
        {
            JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message to indicate that the event could not be found
        }

    }

    private void btnSearchEventIDActionPerformed(java.awt.event.ActionEvent evt) {
        try //Searches for an entry by Event ID  and returns it to the table if found
        {
            DefaultTableModel dtm = (DefaultTableModel) tblEventStu.getModel(); //Sets the default table
model
            tblEventStu.removeAll();//Empties the Jtable
            dtm.setRowCount(0); //sets the row count to 0
            model = (DefaultTableModel)tblEventStu.getModel(); //Sets the default table model
            ResultSet rs = dbman.queryDatabase("SELECT * FROM tblEvents WHERE EventID =" +
tfxSearchEventID.getText() ); //Retrives entry(s) from the database
            int count = 0; //sets a counter to 0
            while(rs.next()) //checks if the result set has a next result and moves to it if it does
            {
                String portfolio = ""; //Creates a string to store the portfolio of the event

                if(rs.getBoolean("Catering") == true) //Checks if the event is for catering
                {
                    portfolio = "Catering";//Sets the portfolio string to catering
                }
```

```
            else if(rs.getBoolean("Photography") == true) //Checks if the event is for photography
            {
               portfolio = "Photography"; //Sets the portfolio string to photography
            }
            else if(rs.getBoolean("FirstAid") == true)//Checks if the event is for first aid
            {
               portfolio = "First Aid"; //Sets the portfolio string to first aid
            }
            else if(rs.getBoolean("AV") == true)//Checks if the event is for AV
            {
               portfolio = "AV"; //Sets the portfolio string to AV
            }
            boolean c = sm.getPortfolio("Catering", currentStu); //checks if the student does catering
            boolean p = sm.getPortfolio("Photography", currentStu); //checks if the student does
photography
            boolean a = sm.getPortfolio("av", currentStu); //checks if the student does AV
            boolean f = sm.getPortfolio("First aid", currentStu); //checks if the student does first aid

            //Checks that the student has permission to view the event
            if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true &&
portfolio.equalsIgnoreCase("Photography")) || (a == true && portfolio.equalsIgnoreCase("AV")) || (f ==
true && portfolio.equalsIgnoreCase("First Aid")))
            {
               model.addRow(new Object[]{rs.getInt("EventID"),rs.getString("EventName"),
                rs.getInt("Hours"),rs.getDate("EventDate"), portfolio}); //Creates a new object to be added to
the table
               count++; //adds 1 to count
            }
      }
         if(count == 0) //checks if count is till equal to 0
         {
            JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message to indicate that the event could not be found
         }
         tfxSearchEventID.setText(""); //Clears the text field
         model.addRow(new Object[]{}); //Adds row to the table
      }
      catch(SQLException e) //Catches any sql Errors
      {
         JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message to indicate that the event could not be found
      }
   }

   private void btnSearchPartNameActionPerformed(java.awt.event.ActionEvent evt) {
      try //Searches for an entry by event name and returns it to the table if found
      {
         DefaultTableModel dtm = (DefaultTableModel) tblParticipation.getModel(); //Sets the default table
model
```

```
            tblParticipation.removeAll();//Empties the Jtable
            dtm.setRowCount(0); //sets the row count to 0
            model = (DefaultTableModel)tblParticipation.getModel(); //Sets the default table model
            ResultSet rs = dbman.queryDatabase("SELECT DISTINCT tblStudents.StudentID,
tblEvents.EventID, tblEvents.EventName, tblEvents.hours, tblEvents.Catering, tblEvents.Photography,
tblEvents.AV, tblEvents.FirstAid FROM tblEvents, tblStudents, tblParticipation Where
tblParticipation.StudentID = tblStudents.StudentID AND tblParticipation.EventID = tblEvents.EventID AND
EventName LIKE '*" + txfSearchPartName.getText() + "*' AND tblStudents.StudentID = " + currentStu);
//Retrives entry(s) from the database
            int count = 0; //sets a counter to 0
            while(rs.next()) //checks if the result set has a next result and moves to it if it does
            {
                String portfolio = ""; //Creates a string to store the portfolio of the event
                if(rs.getBoolean("Catering") == true)//Checks if the event of the participation record is for
catering
                {
                    portfolio = "Catering"; //Sets the portfolio string to catering
                }
                else if(rs.getBoolean("Photography") == true) //Checks if the event of the participation record is
for photography
                {
                    portfolio = "Photography"; //Sets the portfolio string to photography
                }
                else if(rs.getBoolean("FirstAid") == true)//Checks if the event of the participation record is for
first aid
                {
                    portfolio = "First Aid"; //Sets the portfolio string to first aid
                }
                else if(rs.getBoolean("AV") == true) //Checks if the event of the participation record is for AV
                {
                    portfolio = "AV"; //Sets the portfolio string to AV
                }
                boolean c = sm.getPortfolio("Catering", currentStu); //checks if the student does catering
                boolean p = sm.getPortfolio("Photography", currentStu); //checks if the student does
photography
                boolean a = sm.getPortfolio("av", currentStu); //checks if the student does AV
                boolean f = sm.getPortfolio("First aid", currentStu); //checks if the student does first aid
                if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true &&
portfolio.equalsIgnoreCase("Photography")) || (a == true && portfolio.equalsIgnoreCase("AV")) || (f ==
true && portfolio.equalsIgnoreCase("First Aid")))
                {
                    count++;//Adds 1 to count
                    model.addRow(new
Object[]{rs.getInt("StudentID"),rs.getInt("EventID"),rs.getString("EventName"),
                    rs.getInt("Hours"), portfolio});//Creates a new object to be added to the table
                }
        }
            if(count == 0) //checks if count is till equal to 0
            {
```

```
            JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message to indicate that the event could not be found
        }
        txfSearchPartName.setText(""); //Clears the text field
        model.addRow(new Object[]{}); //Adds row to the table
      }
      catch(SQLException e) //Catches any sql Errors
      {
        JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE); //Displays an error message to indicate that the event could not be found
      }
   }

   private void btnSearchPartIDActionPerformed(java.awt.event.ActionEvent evt) {
      try //Searches for an entry by event ID and returns it to the table if found
      {
        DefaultTableModel dtm = (DefaultTableModel) tblParticipation.getModel();//Sets the default table
model
        tblParticipation.removeAll();//Empties the Jtable
        dtm.setRowCount(0); //sets the row count to 0
        model = (DefaultTableModel)tblParticipation.getModel(); //Sets the default table model
        ResultSet rs = dbman.queryDatabase("SELECT DISTINCT tblStudents.StudentID,
tblEvents.EventID, tblEvents.EventName, tblEvents.hours, tblEvents.Catering, tblEvents.Photography,
tblEvents.AV, tblEvents.FirstAid FROM tblEvents, tblStudents, tblParticipation Where
tblParticipation.StudentID = tblStudents.StudentID AND tblParticipation.EventID = tblEvents.EventID AND
tblEvents.EventID = " + Integer.parseInt(txfSearchPartID.getText()) + " AND tblStudents.StudentID = " +
currentStu); //Retrives entry(s) from the database
        int count = 0; //sets a counter to 0
        while(rs.next()) //checks if the result set has a next result and moves to it if it does
        {
          String portfolio = ""; //Creates a string to store the portfolio of the event
          if(rs.getBoolean("Catering") == true)//Checks if the event of the participation record is for
catering
          {
            portfolio = "Catering"; //Sets the portfolio string to catering
          }
          else if(rs.getBoolean("Photography") == true) //Checks if the event of the participation record is
for photography
          {
            portfolio = "Photography"; //Sets the portfolio string to photography
          }
          else if(rs.getBoolean("FirstAid") == true)//Checks if the event of the participation record is for
first aid
          {
            portfolio = "First Aid"; //Sets the portfolio string to first aid
          }
          else if(rs.getBoolean("AV") == true) //Checks if the event of the participation record is for AV
          {
            portfolio = "AV"; //Sets the portfolio string to AV
```

```
            }
            boolean c = sm.getPortfolio("Catering", currentStu); //checks if the student does catering
            boolean p = sm.getPortfolio("Photography", currentStu); //checks if the student does
photography
            boolean a = sm.getPortfolio("av", currentStu); //checks if the student does AV
            boolean f = sm.getPortfolio("First aid", currentStu); //checks if the student does first aid
            if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true &&
portfolio.equalsIgnoreCase("Photography")) || (a == true && portfolio.equalsIgnoreCase("AV")) || (f ==
true && portfolio.equalsIgnoreCase("First Aid")))
            {
                count++;//Adds 1 to count
                model.addRow(new
Object[]{rs.getInt("StudentID"),rs.getInt("EventID"),rs.getString("EventName"),
                rs.getInt("Hours"), portfolio});//Creates a new object to be added to the table
            }
        }
            if(count == 0) //checks if count is till equal to 0
            {
                JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message to indicate that the event could not be found
            }
            txfSearchPartID.setText(""); //Clears the text field
            model.addRow(new Object[]{}); //Adds row to the table
        }
        catch(SQLException e) //Catches any sql Errors
        {
            JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE); //Displays an error message to indicate that the event could not be found
        }
    }

    private void btnDeleteStuPartActionPerformed(java.awt.event.ActionEvent evt) {
        try //Searches for an entry by event ID and returns it to the table if found
        {
            DefaultTableModel dtm = (DefaultTableModel) tblParticipation.getModel(); //Sets the default table
model
            tblParticipation.removeAll();//Empties the Jtable
            dtm.setRowCount(0); //sets the row count to 0
            model = (DefaultTableModel)tblParticipation.getModel(); //Sets the default table model
            ResultSet rs = dbman.queryDatabase("SELECT DISTINCT tblStudents.StudentID,
tblEvents.EventID, tblEvents.EventName, tblEvents.hours, tblEvents.Catering, tblEvents.Photography,
tblEvents.AV, tblEvents.FirstAid FROM tblEvents, tblStudents, tblParticipation Where
tblParticipation.StudentID = tblStudents.StudentID AND tblParticipation.EventID = tblEvents.EventID AND
tblEvents.EventID = " + Integer.parseInt(txfEventIDDelete.getText()) + " AND tblStudents.StudentID = " +
currentStu); //Retrives entry(s) from the database
            while(rs.next())//loops through all the entrys returned
            {
                String portfolio = "";//creates a string to hold the portfolio
                if(rs.getBoolean("Catering") == true)//checks if the participation record is for catering
```

```
            {
                portfolio = "Catering";//sets portfolio to catering
            }
            else if(rs.getBoolean("Photography") == true)//checks if the participation record is for
photography
            {
                portfolio = "Photography";//sets portfolio to photography
            }
            else if(rs.getBoolean("FirstAid") == true)//checks if the participation record is for first aid
            {
                portfolio = "First Aid";//sets portfolio to first aid
            }
            else if(rs.getBoolean("AV") == true)//checks if the participation record is for AV
            {
                portfolio = "AV"; //sets portfolio to AV
            }
            boolean c = sm.getPortfolio("Catering", currentStu);//checks if the current student does catering
            boolean p = sm.getPortfolio("Photography", currentStu);//checks if the current student does
photography
            boolean a = sm.getPortfolio("av", currentStu);//checks if the current student does AV
            boolean f = sm.getPortfolio("First aid", currentStu);//checks if the current student does first aid
            //checks that the student has permision to view the participation record
            if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true &&
portfolio.equalsIgnoreCase("Photography")) || (a == true && portfolio.equalsIgnoreCase("AV")) || (f ==
true && portfolio.equalsIgnoreCase("First Aid")))
            {
                int s = rs.getInt("StudentID");//places the Student ID into a variable
                int e = rs.getInt("EventID");//places the event ID into a variable
                String en = rs.getString("EventName");//places the event name into a variable
                int h = rs.getInt("Hours");//places the number of hours into a variable
                model.addRow(new Object[]{s,e,en,h, portfolio});//adds the row to an object

                int answer = JOptionPane.showConfirmDialog(btnDeleteStuPart,
                "Are you sure you want to delete this record?\n" + s + "-" + e + "-"+ en +"-"+h, "DELETE
PARTICIPATION RECORD",
                JOptionPane.YES_NO_OPTION);//displays a yes/no message to check if the user wants to
delete the record

                if (answer == JOptionPane.NO_OPTION) { //if the user selects no
                    JOptionPane.showMessageDialog(null,"Delete Canceled");//displays a cancelled
message
                } else if(answer == JOptionPane.YES_OPTION) {//if the user selects yes
                    pm.deleteEntry(Integer.parseInt(txfEventIDDelete.getText()),currentStu); //deletes the
record
                    //reloads the participation table for the student
                    ParticipationTBL((sm.getPortfolio("Catering", currentStu)),(sm.getPortfolio("Photography",
currentStu)),(sm.getPortfolio("av", currentStu)),(sm.getPortfolio("First aid", currentStu)));
                }
            }
```

```
        }
            txfEventIDDelete.setText("eg.5"); //Clears the text field
            model.addRow(new Object[]{}); //Adds row to the table
        }
        catch(SQLException e) //Catches any sql Errors
        {
            JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//Displays an error message
        }
        //reloads the participation table for the student
        ParticipationTBL((sm.getPortfolio("Catering", currentStu)),(sm.getPortfolio("Photography",
currentStu)),(sm.getPortfolio("av", currentStu)),(sm.getPortfolio("First aid", currentStu)));
    }

    private void txfEventIDDeleteActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
    }

    private void btnCreateParticipationActionPerformed(java.awt.event.ActionEvent evt) {
        try {
            int Event = Integer.parseInt(txfEventIDCreate.getText());//places the entered event ID into a
variable
            pm.add(Event,currentStu);//adds a participation record with the current student ID and entered
event ID
            JOptionPane.showMessageDialog(null,"Successfull entry");//displays a message to indicate
success
            txfEventIDCreate.setText("e.g 5");//re-sets the text field
        } catch (SQLException ex) {//catches any SQL errors
            Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
        }
        //re-loads the participation table for the student
        ParticipationTBL((sm.getPortfolio("Catering", currentStu)),(sm.getPortfolio("Photography",
currentStu)),(sm.getPortfolio("av", currentStu)),(sm.getPortfolio("First aid", currentStu)));
    }

    private void txfEventIDCreateActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
    }

    private void lblLogoHomeComponentShown(java.awt.event.ComponentEvent evt) {

    }

    private void btnEditStuEnterActionPerformed(java.awt.event.ActionEvent evt) {
        clearErrorLables(); //clears the error lables by calling the clear error lables method
        char gender = ' ';//creates a variable to hold the gender
        if(rbnEditStuGenderM.isSelected())//checks if the radio button is selected
        {
            gender = 'M';//sets the gender character to M
```

```
        }
        else if (rbnEditStuGenderF.isSelected())//checks if the radio button is selected
        {
            gender = 'F';//sets the gender character to F
        }
        else//if neither radio buttons were selected
        {
            lblEditStuGenderError.setText("Please select a gender");//displays an error message
        }

        boolean validPortfolio = checkEditPortfolio();//checks that the portfolio selection is valid
        boolean validGrade = checkInt(txfEditStuGrade, lblEditStuGradeError,8,12);//Type,presence and
range check
        boolean validClass = checkInt(txfEditStuClass, lblEditStuClassError, 1,5);//Type,presence and range
check
        //precence, length and type check
        boolean validName = presenceAndLengthCheck(txfEditStuName, lblEditStuNameError) &&
typeCheck(txfEditStuName, lblEditStuNameError, " -:,;'@#$%^&*()!+~");
        //precence, length and type check
        boolean validSurname = presenceAndLengthCheck(txfEditStuSurname, lblEditStuSurnameError)
&& typeCheck(txfEditStuSurname, lblEditStuSurnameError, " -:,;'@#$%^&*()!+~");
        //confirms that all the data is valid
        if((validName == true) && (validSurname == true) && (gender != ' ') && (validGrade == true) &&
(validClass == true) && (validPortfolio == true))
        {
            String name = txfEditStuName.getText();//sets the name to the value entered by the user
            String surname = txfEditStuSurname.getText();//sets the surname to the value entered by the user
            int grade = Integer.parseInt(txfEditStuGrade.getText());//sets the grade to the value entered by the
user and converts it to an integer
            int classPoint =Integer.parseInt(txfEditStuClass.getText());//sets the class to the value entered by
the user and converts it to an integer

            boolean catering = false;//creates a boolean to hold catering
            boolean photography = false;//creates a boolean to hold photography
            boolean firstAid = false;//creates a boolean to hold first aid
            boolean av = false;//creates a boolean to hold AV

            if(cbxCatering1.isSelected())//checks if the catering checkbox is ticked
            {
                catering = true;//sets the catering boolean to true
            }
            if(cbxPhotography1.isSelected())//checks if the photography checkbox is ticked
            {
                photography = true;//sets the photography boolean to true
            }
            if(cbxFirstAid1.isSelected())//checks if the first aid checkbox is ticked
            {
                firstAid = true;//sets the first aid boolean to true
            }
```

```
        if(cbxAV1.isSelected())//checks if the AV checkbox is ticked
        {
            av = true;//sets the AV boolean to true
        }

        try {
            //updates the student record in the database
            dbman.updateQuery("UPDATE tblStudents SET firstName = '" + name+ "',Surname = '" +
surname+ "', Grade = " + grade+  ",Class = " + classPoint + ", Gender = '" + gender + "', Catering = " +
catering + ", Photography = " + photography + ", AV = " + av + ", FirstAid = " + firstAid + " WHERE
StudentID = " + currentStu + ";");
            dlgEditStudent.setVisible(false);//hides the dialog
            tbdpStudent.setVisible(true);//shows the student tabbed pane
        } catch (SQLException ex) {//catches any SQL errors
            Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
        }
    }
    else//if the data was not valid
    {
        JOptionPane. showMessageDialog(null, "UNABLE TO UPDATE!", "", JOptionPane.
ERROR_MESSAGE);//displays an error message
    }
    try {
        if(currentStu != -1)//checks that the current student has a value
        {
         txaStudent.setText(sm.getCurrent(currentStu));//calls a method to show the student details on
the home page
        }
    } catch (SQLException ex) {//catches SQL errors
        Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
    }
}

private void btnEditStuCancelActionPerformed(java.awt.event.ActionEvent evt) {
    dlgEditStudent.setVisible(false);//hides the dialog
}

private void tbdpStudentComponentResized(java.awt.event.ComponentEvent evt) {
    // TODO add your handling code here:
}

private void pnlHomeComponentShown(java.awt.event.ComponentEvent evt) {
    try {
        if(currentStu != -1)//checks that the current student has a value
        {
         txaStudent.setText(sm.getCurrent(currentStu));//calls a method to show the student details on
the home page
        }
    } catch (SQLException ex) {//catches SQL errors
```

```
            Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
        }
    }

    private void cbxFirstAid1ActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
    }

    private void pnlHomeAdminComponentShown(java.awt.event.ComponentEvent evt) {
      if(currentAdmin != -1)//checks that current admin has a value
        {
            try {
                txaAdminDetails.setText(am.getCurrent(currentAdmin));//calls a method to show the admin
details on the home page
            } catch (SQLException ex) {//catches any SQL errors
                Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
            }
        }

    }

    private void cbxFirstAidAdminActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
    }

    private void btnEditAdminCancelActionPerformed(java.awt.event.ActionEvent evt) {
        dlgEditAdmin.setVisible(false);//hides the dialog
    }

    private void btnEditAdminEnterActionPerformed(java.awt.event.ActionEvent evt) {
        clearErrorLables(); //clears the error lables by calling the clear error lables method
        boolean validPortfolio = checkPortfolioEditAdmin();
        boolean validName = presenceAndLengthCheck(txfEditAdminName, lblEditAdminNameError) &&
typeCheck(txfEditAdminName, lblEditAdminNameError, "-:,;'@#$%^&*()!+~ ");
        boolean validSurname = presenceAndLengthCheck(txfEditAdminSurname,
lblEditAdminSurnameError) && typeCheck(txfEditAdminSurname, lblEditAdminSurnameError, "
:,;'@#$%^&*()!+~");
        if((validName == true) && (validSurname == true) && (validPortfolio == true));
        {
            String name = txfEditAdminName.getText();
            String surname = txfEditAdminSurname.getText();
            boolean catering = false;
            boolean photography = false;
            boolean firstAid = false;
            boolean av = false;

            if(cbxCateringAdmin.isSelected())
            {
                catering = true;
```

```
        }
        if(cbxPhotographyAdmin.isSelected())
        {
            photography = true;
        }
        if(cbxFirstAidAdmin.isSelected())
        {
            firstAid = true;
        }
        if(cbxAVAdmin.isSelected())
        {
            av = true;
        }

        try {
            dbman.updateQuery("UPDATE tblAdmins SET AdminName = '" + name+ "',AdminSurname = '"
+ surname+ "', Catering = " + catering + ", Photography = " + photography + ", AV = " + av + ", [First Aid]
= " + firstAid + " WHERE AdminID = " + currentAdmin + ";");
            dlgEditAdmin.setVisible(false);
            tbdpAdmin.setVisible(true);
        } catch (SQLException ex) {
            System.out.println("Update Error, please try again");
        }
    }

    if(currentAdmin != -1)//checks that current admin has a value
    {
        try {
            txaAdminDetails.setText(am.getCurrent(currentAdmin));//calls a method to show the admin
details on the home page
        } catch (SQLException ex) {//catches any SQL errors
            Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
        }
    }

}

private void btnConfirmPortfolioActionPerformed(java.awt.event.ActionEvent evt) {
    clearErrorLables(); //clears the error lables by calling the clear error lables method
    dlgEditAdmin.setEnabled(true);//enables the dialog
    dlgEditAdmin.setSize(900,600);//sets the dialog size
    dlgEditAdmin.setLocationRelativeTo(lblLogoHome1);//sets the dialog position
    dlgEditAdmin.setVisible(true);//shows the dialog
    txfEditAdminName.setText("");//clears the text field
    txfEditAdminSurname.setText("");//clears the text field

    try
    {
```

```
        ResultSet rs = dbman.queryDatabase("SELECT * FROM tblAdmins WHERE AdminID = " +
currentAdmin);//selects the entry(S) from the table
        while(rs.next()) //Creates a line in the JTable for every database entry
        {
            if(rs.getBoolean("Catering") == true)//checks if the admin oversees catering
            {
                cbxCateringAdmin.setSelected(true);//sets the checkbox to selected
            }
            if(rs.getBoolean("Photography") == true)//checks if the admin oversees photography
            {
                cbxPhotographyAdmin.setSelected(true);//sets the checkbox to selected
            }
            if(rs.getBoolean("First Aid") == true)//checks if the admin oversees first aid
            {
                cbxFirstAidAdmin.setSelected(true);//sets the checkbox to selected
            }
            if(rs.getBoolean("AV") == true)//checks if the admin oversees AV
            {
                cbxAVAdmin.setSelected(true);//sets the checkbox to selected
            }
            txfEditAdminName.setText(rs.getString("AdminName"));//puts the admin name into the text field
            txfEditAdminSurname.setText(rs.getString("AdminSurname"));//puts the admin surname into
the text field
            }
      }
      catch(SQLException e)//catches any SQL errors
      {
          System.out.println("Could Not Edit Admin");//displays an error message
      }
   }

   private void btnEditAdminCancel1ActionPerformed(java.awt.event.ActionEvent evt) {
      dlgNewAdmin.setVisible(false);//hides the dialog
   }

   private void btnEditAdminEnter1ActionPerformed(java.awt.event.ActionEvent evt) {
      clearErrorLables(); //clears the error lables by calling the clear error lables method
      try{ try ( //Creates a new log in
          PrintWriter outFile = new PrintWriter(new FileWriter("UsernamesAndPasswords.txt",true)))
{//creates a file writer
          boolean password = false;//creates a boolean to check that the passwords match
          if (txfNewAdminPassword.getText().equals(txfNewAdminConfirmPassword.getText()))//Checks
that both passwords are the same
          {
              password = true;//sets the password boolean to true
          }
          else//if the passwords dont match
          {
```

```
            lblNewAdminConfirmPasswordError.setText("Passwords Do Not Match");//sets the lable to
display an error
        }
        boolean validPortfolio = checkPortfolioAdmin();//checks that the portfolio entries are valid
        //precence, length and type check
        boolean validUserName = presenceAndLengthCheck(txfNewAdminUsername,
lblNewAdminUsernameError) && typeCheck(txfNewAdminUsername, lblNewAdminUsernameError, ":,;'");
        //precence, length and type check
        boolean validPassword = presenceAndLengthCheck(txfNewAdminPassword,
lblNewAdminPasswordError);
        //precence, length and type check
        boolean validName = presenceAndLengthCheck(txfNewAdminName, lblNewAdminNameError)
&& typeCheck(txfNewAdminName, lblNewAdminNameError, " :,;'@#$%^&*()!+~") &&
SpaceCheck(txfNewAdminName, lblNewAdminNameError);
        //confirms that all the data is valid
        if((validUserName == true) && (validPassword == true) && (validName == true) &&
(validPortfolio == true) && (password == true))
        {
            String name = txfNewAdminName.getText();//sets the name to the value entered by the user
            boolean catering = false;//creates a boolean to check if the admin oversees catering
            boolean photography = false;//creates a boolean to check if the admin oversees photography
            boolean firstAid = false;//creates a boolean to check if the admin oversees first aid
            boolean av = false;//creates a boolean to check if the admin oversees AV
            String lName = name.substring(name.lastIndexOf(" ")+1);//puts the surname into a string
            String fName = name.substring(0,name.lastIndexOf(" "));//puts the first name into a string

            if(cbxNewAdminCatering.isSelected())//checks if the catering checkbox is selected
            {
                catering = true;//sets the boolean to true
            }
            if(cbxNewAdminPhotography.isSelected())//checks if the photography checkbox is selected
            {
                photography = true;//sets the boolean to true
            }
            if(cbxNewAdminFirstAid.isSelected())//checks if the first aid checkbox is selected
            {
                firstAid = true;//sets the boolean to true
            }
            if(cbxNewAdminAv.isSelected())//checks if the AV checkbox is selected
            {
                av = true;//sets the boolean to true
            }

            try {

                am.add(fName,lName , catering, photography, av, firstAid);//adds a new admin
                //creates a line to be added to the text file
                String line = (am.getSize()-1) + "#" + txfNewAdminUsername.getText() + "#" +
txfNewAdminPassword.getText() + "#Admin";
```

```
                outFile.println(line);//adds the line to the textfile
                JOptionPane.showMessageDialog(null,"New Admin Created");//displays a message to
indicate success
                dlgNewAdmin.setVisible(false);//hides the dialog
            } catch (SQLException ex) {//catches any SQL errors
                Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
            }
        }  }
    }
     catch (IOException ex) {//catches any errors
        //displays an error
        JOptionPane. showMessageDialog(null, "Please ensure all fields are filled out", "", JOptionPane.
ERROR_MESSAGE);
    }
  }

  private void txfNewAdminConfirmPasswordActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
  }

  private void txfCreatePartStuIDActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
  }

  private void txfCreatePartEventIDActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
  }

  private void btnDeleteParticipationActionPerformed(java.awt.event.ActionEvent evt) {
    try //Searches for an entry by dive number and returns it to the table if found
    {
        DefaultTableModel dtm = (DefaultTableModel) tblParticipationAdmin.getModel(); //Sets the default
table model
        tblParticipationAdmin.removeAll();//Empties the Jtable
        dtm.setRowCount(0); //sets the row count to 0
        model = (DefaultTableModel)tblParticipationAdmin.getModel(); //Sets the default table model
        ResultSet rs = dbman.queryDatabase("SELECT DISTINCT tblStudents.StudentID,
tblStudents.firstName, tblStudents.surname, tblEvents.EventID, tblEvents.EventName, tblEvents.hours,
tblEvents.Catering, tblEvents.Photography, tblEvents.AV, tblEvents.FirstAid FROM tblEvents,
tblStudents, tblParticipation Where tblParticipation.StudentID = tblStudents.StudentID AND
tblParticipation.EventID = tblEvents.EventID AND tblEvents.EventID = " +
Integer.parseInt(txfDeleteEventID.getText()) + " AND tblStudents.StudentID = " +
txfDeleteStuID.getText()); //Retrives entry(s) from the database
        while(rs.next())//Loops through each result
        {
            String portfolio = "";//creates a string to hold the portfolios
            if(rs.getBoolean("Catering") == true)//checks if the participation record is for catering
            {
                portfolio = "Catering";//sets the string to catering
```

```
            }
            else if(rs.getBoolean("Photography") == true)//checks if the participation record is for
photography
            {
               portfolio = "Photography";//sets the string to photography
            }
            else if(rs.getBoolean("FirstAid") == true)//checks if the participation record is fornfirst aid
            {
               portfolio = "First Aid";//sets the string to first aid
            }
            else if(rs.getBoolean("AV") == true)//checks if the participation record is for AV
            {
               portfolio = "AV";//sets the string to AV
            }
            boolean c = am.getPortfolio("Catering", currentAdmin);//checks if the current admin oversees
catering
            boolean p = am.getPortfolio("Photography", currentAdmin);//checks if the current admin
oversees photography
            boolean a = am.getPortfolio("av", currentAdmin);//checks if the current admin oversees AV
            boolean f = am.getPortfolio("First aid", currentAdmin);//checks if the current admin oversees
first aid
            //checks that all the data entered is valid
            if((c == true && portfolio.equalsIgnoreCase("Catering")) || (p == true &&
portfolio.equalsIgnoreCase("Photography")) || (a == true && portfolio.equalsIgnoreCase("AV")) || (f ==
true && portfolio.equalsIgnoreCase("First Aid")))
            {
               int s = rs.getInt("StudentID");//puts the student ID into a variable
               String name = rs.getString("firstname") + " " + rs.getString("surname");//puts the student full
name into a variable
               int e = rs.getInt("EventID");//puts the event ID into a variable
               String en = rs.getString("EventName");//puts the event name into a variable
               int h = rs.getInt("Hours");//puts the event hours into a variable
               model.addRow(new Object[]{s,name,e,en,h, portfolio});//creates an object for each row
               //displays a message to the user with a yes/no option
               int answer = JOptionPane.showConfirmDialog(btnDeleteParticipation,
               "Are you sure you want to delete this record?\n" + s + "-" + name + "-" + e + "-"+ en +"-"+h,
"DELETE PARTICIPATION RECORD",
               JOptionPane.YES_NO_OPTION);
                  if (answer == JOptionPane.NO_OPTION) {//if the user selects no
                  JOptionPane.showMessageDialog(null,"Delete Canceled");//displays a message to
explain that the delete has been canceled
               } else if(answer == JOptionPane.YES_OPTION) {

pm.deleteEntry(Integer.parseInt(txfDeleteEventID.getText()),Integer.parseInt(txfDeleteStuID.getText()));//
deletes event
               }
            }
      }
         txfDeleteEventID.setText("Enter Event ID"); //Clears the text field
```

```
            txfDeleteStuID.setText("Enter Student ID");//clears the text field
            model.addRow(new Object[]{}); //Adds row to the table
        }
        catch(SQLException e) //Catches any sql Errors
        {
            JOptionPane. showMessageDialog(null, "Entry not found", "", JOptionPane.
ERROR_MESSAGE);//displays an error message
        }
            //re-loads the admin participation table
            AdminParticipationTBL((am.getPortfolio("Catering",
currentAdmin)),(am.getPortfolio("Photography", currentAdmin)),(am.getPortfolio("av",
currentAdmin)),(am.getPortfolio("First aid", currentAdmin)));
    }

    private void btnCreateParticipationAdminActionPerformed(java.awt.event.ActionEvent evt) {
        try {
            int Event = Integer.parseInt(txfCreatePartEventID.getText());//creates a variable to hold the event
ID entered by the user
            pm.add(Event,Integer.parseInt(txfCreatePartStuID.getText()));//creates a variable to hold the
student ID entered by the user
            JOptionPane.showMessageDialog(null,"Successfull entry");//displays a message to indicate
success
        } catch (SQLException ex) {//catches any SQL errors
            Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
        }
        //re-loads the admin participation table
        AdminParticipationTBL((am.getPortfolio("Catering", currentAdmin)),(am.getPortfolio("Photography",
currentAdmin)),(am.getPortfolio("av", currentAdmin)),(am.getPortfolio("First aid", currentAdmin)));
    }

    private void btnHowLogInActionPerformed(java.awt.event.ActionEvent evt) {
        //Code Adapted from https://www.youtube.com/watch?v=Nm-e9aWPH8U
        final URI url;
        try {
            url = new URI("https://youtu.be/7diFPRxfVFs");//creates a link to the youtube video
            play(url);//Attempts to open the browser and play the video
        } catch (URISyntaxException ex) {//catches any errors
            Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
        }
    }

    private void btnHowSignUpActionPerformed(java.awt.event.ActionEvent evt) {
        //Code Adapted from https://www.youtube.com/watch?v=Nm-e9aWPH8U
        final URI url;
        try {
            url = new URI("https://youtu.be/MwpUDZic4O0");//creates a link to the youtube video
            play(url);//Attempts to open the browser and play the video
        } catch (URISyntaxException ex) {//catches any errors
            Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
```

```
    }
}

private void btnHowNewAdminActionPerformed(java.awt.event.ActionEvent evt) {
    //Code Adapted from https://www.youtube.com/watch?v=Nm-e9aWPH8U
    final URI url;
    try {
        url = new URI("https://youtu.be/ADulbxisk6g");//creates a link to the youtube video
        play(url);//Attempts to open the browser and play the video
    } catch (URISyntaxException ex) {//catches any errors
        Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
    }
}

private void btnHowMakeParticipationStudentActionPerformed(java.awt.event.ActionEvent evt) {
    //Code Adapted from https://www.youtube.com/watch?v=Nm-e9aWPH8U
    final URI url;
    try {
        url = new URI("https://youtu.be/wlLID46vXwc");//creates a link to the youtube video
        play(url);//Attempts to open the browser and play the video
    } catch (URISyntaxException ex) {//catches any errors
        Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
    }
}

private void btnHowDeleteParticipationStudentActionPerformed(java.awt.event.ActionEvent evt) {
    //Code Adapted from https://www.youtube.com/watch?v=Nm-e9aWPH8U
    final URI url;
    try {
        url = new URI("https://youtu.be/77i3yiSdTy4");//creates a link to the youtube video
        play(url);//Attempts to open the browser and play the video
    } catch (URISyntaxException ex) {//catches any errors
        Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
    }
}

private void btnHowEditMyAccountStudentActionPerformed(java.awt.event.ActionEvent evt) {
    //Code Adapted from https://www.youtube.com/watch?v=Nm-e9aWPH8U
    final URI url;
    try {
        url = new URI("https://youtu.be/VWespaXbS2I");//creates a link to the youtube video
        play(url);//Attempts to open the browser and play the video
    } catch (URISyntaxException ex) {//catches any errors
        Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
    }
}

private void btnHowSearchEventStuActionPerformed(java.awt.event.ActionEvent evt) {
    //Code Adapted from https://www.youtube.com/watch?v=Nm-e9aWPH8U
```

```
    final URI url;
    try {
        url = new URI("https://youtu.be/ksRwj2zDzQ8");//creates a link to the youtube video
        play(url);//Attempts to open the browser and play the video
    } catch (URISyntaxException ex) {//catches any errors
        Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
    }
}

private void btnHowCreateEventActionPerformed(java.awt.event.ActionEvent evt) {
    //Code Adapted from https://www.youtube.com/watch?v=Nm-e9aWPH8U
    final URI url;
    try {
        url = new URI("https://youtu.be/COV03_8S334");//creates a link to the youtube video
        play(url);//Attempts to open the browser and play the video
    } catch (URISyntaxException ex) {//catches any errors
        Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
    }
}

private void btnHowEditAccountAdminActionPerformed(java.awt.event.ActionEvent evt) {
    //Code Adapted from https://www.youtube.com/watch?v=Nm-e9aWPH8U
    final URI url;
    try {
        url = new URI("https://youtu.be/XGRLSFfkYBs");//creates a link to the youtube video
        play(url);//Attempts to open the browser and play the video
    } catch (URISyntaxException ex) {//catches any errors
        Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
    }
}

private void btnHowEditEventActionPerformed(java.awt.event.ActionEvent evt) {
    //Code Adapted from https://www.youtube.com/watch?v=Nm-e9aWPH8U
    final URI url;
    try {
        url = new URI("https://youtu.be/m_wRiSljjjw");//creates a link to the youtube video
        play(url);//Attempts to open the browser and play the video
    } catch (URISyntaxException ex) {//catches any errors
        Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
    }
}

private void btnHowMakeParticipationActionPerformed(java.awt.event.ActionEvent evt) {
    //Code Adapted from https://www.youtube.com/watch?v=Nm-e9aWPH8U
    final URI url;
    try {
        url = new URI("https://youtu.be/a94-HrQaD8M");//creates a link to the youtube video
        play(url);//Attempts to open the browser and play the video
    } catch (URISyntaxException ex) {//catches any errors
```

```
            Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
        }
    }

    private void btnHowSearchParticipationStuActionPerformed(java.awt.event.ActionEvent evt) {
        //Code Adapted from https://www.youtube.com/watch?v=Nm-e9aWPH8U
        final URI url;
        try {
            url = new URI("https://youtu.be/2_G35DJKo0M");//creates a link to the youtube video
            play(url);//Attempts to open the browser and play the video
        } catch (URISyntaxException ex) {//catches any errors
            Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
        }
    }

    private void btnHowSearchStudentActionPerformed(java.awt.event.ActionEvent evt) {
        //Code Adapted from https://www.youtube.com/watch?v=Nm-e9aWPH8U
        final URI url;
        try {
            url = new URI("https://youtu.be/6JfSBQ9gFT8");//creates a link to the youtube video
            play(url);//Attempts to open the browser and play the video
        } catch (URISyntaxException ex) {//catches any errors
            Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
        }
    }

    private void btnHowDeleteParticipationAdminActionPerformed(java.awt.event.ActionEvent evt) {
        //Code Adapted from https://www.youtube.com/watch?v=Nm-e9aWPH8U
        final URI url;
        try {
            url = new URI("https://youtu.be/iNeotBsmOa8");//creates a link to the youtube video
            play(url);//Attempts to open the browser and play the video
        } catch (URISyntaxException ex) {//catches any errors
            Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
        }
    }

    private void btnHowSearchParticipationAdminActionPerformed(java.awt.event.ActionEvent evt) {
        //Code Adapted from https://www.youtube.com/watch?v=Nm-e9aWPH8U
        final URI url;
        try {
            url = new URI("https://youtu.be/W6J9I_vHoyw");//creates a link to the youtube video
            play(url);//Attempts to open the browser and play the video
        } catch (URISyntaxException ex) {//catches any errors
            Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
        }
    }

    private void btnHowSearchEventAdminActionPerformed(java.awt.event.ActionEvent evt) {
```

```
        //Code Adapted from https://www.youtube.com/watch?v=Nm-e9aWPH8U
        final URI url;
        try {
            url = new URI("https://youtu.be/8ffuBUkUIUA");//creates a link to the youtube video
            play(url);//Attempts to open the browser and play the video
        } catch (URISyntaxException ex) {//catches any errors
            Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);
        }
    }
    private static void play(URI url) //Code Adapted from https://www.youtube.com/watch?v=Nm-
e9aWPH8U
    {
        if(Desktop.isDesktopSupported())//checks if the action can be performed on the pc
        {
            try{
                Desktop.getDesktop().browse(url);//opens the b,video in the browser
            }
            catch(IOException e)//catches any errors
                {
                    System.out.println("Could not play video");//gives an error to indicate that the video could
not be played
                }
        }
    }

    private void clearErrorLables() //clears the lables listed in the method
    {
        lblNewNameError.setText("");//new event name error lable
        lblNewHoursError.setText("");//new event hours lable
        lblNewDateError.setText("");//new event date lable
        lblNewPortfolioError.setText("");//new event portfolio error
        lblNewStudentUsernameError.setText("");//new student username error
        lblNewStudentPasswordError.setText("");//new student password error
        lblNewStudentConfPasswordError.setText("");//new student password confirmation error
        lblNewStudentNameError.setText("");//new student name error
        lblNewStudentGradeError.setText("");//new student grade error
        lblNewStudentClassError.setText("");//new student class error
        lblNewStudentGenderError.setText("");//new student gender error
        lblNewStudentPortfolioError.setText("");//new student portfolio error
        lblEditIDError.setText("");//Edit event ID error
        lblEditNameError.setText("");//edit event name error
        lblEditHoursError.setText("");//edit event hours error
        lblEditDateError.setText("");//edit event date error
        lblEditPortfolioError.setText("");//edit event portfolio error
        lblNewAdminNameError.setText("");//new admin name error
        lblNewAdminUsernameError.setText("");//new admin username error
        lblNewAdminPasswordError.setText("");//new admin password error
        lblNewAdminConfirmPasswordError.setText("");//new admin confirm password error
        lblNewAdminPortfoliosError.setText("");//new admin portfolio error
```

```
        lblEditStuNameError.setText("");//edit student name error
        lblEditStuSurnameError.setText("");//edit student surname error
        lblEditStuGradeError.setText("");//edit student grade error
        lblEditStuClassError.setText("");//edit student class error
        lblEditStuGenderError.setText("");//edit student gender error
        lblEditStuPortfoliosError.setText("");//edit student portfolios error
        lblEditAdminNameError.setText("");//edit admin name error
        lblEditAdminSurnameError.setText("");//edit admin surname error
        lblEditAdminPortfoliosError.setText("");//edit admin portfolios error

    }

    private boolean checkPortfolio()//checks that a portfolio has been selected
    {
        boolean test = false;//creates a boolean to check if any have been selected
        if(cbxCatering.isSelected())//if the checkbox is ticked
        {
            test = true;//sets the boolen to true
        }
        else if (cbxPotography.isSelected())//if the checkbox is ticked
        {
            test = true;//sets the boolen to true
        }
        else if (cbxFirstAid.isSelected())//if the checkbox is ticked
        {
            test = true;//sets the boolen to true
        }
        else if (cbxAV.isSelected())//if the checkbox is ticked
        {
            test = true;//sets the boolen to true
        }
        else//if none of the checkboxes are selected
        {
            lblNewStudentPortfolioError.setText("Select at least one portfolio");//sets the lable to an error
        }
        return test;//returns the boolean
    }

    private boolean checkEditPortfolio()//checks that a portfolio has been selected
    {
        boolean test = false;//creates a boolean to check if any have been selected
        if(cbxCatering1.isSelected())//if the checkbox is ticked
        {
            test = true;//sets the boolen to true
        }
        else if (cbxPhotography1.isSelected())//if the checkbox is ticked
        {
            test = true;//sets the boolen to true
        }
```

```
    else if (cbxFirstAid1.isSelected())//if the checkbox is ticked
    {
        test = true;//sets the boolen to true
    }
    else if (cbxAV1.isSelected())//if the checkbox is ticked
    {
        test = true;//sets the boolen to true
    }
    else//if none of the checkboxes are selected
    {
        lblEditStuPortfoliosError.setText("Select at least one portfolio");//sets the lable to an error
    }
    return test;//returns the boolean
}

private boolean checkPortfolioAdmin()//checks that a portfolio has been selected
{
    boolean test = false;//creates a boolean to check if any have been selected
    if(cbxNewAdminCatering.isSelected())//if the checkbox is ticked
    {
        test = true;//sets the boolen to true
    }
    else if (cbxNewAdminPhotography.isSelected())//if the checkbox is ticked
    {
        test = true;//sets the boolen to true
    }
    else if (cbxNewAdminFirstAid.isSelected())//if the checkbox is ticked
    {
        test = true;//sets the boolen to true
    }
    else if (cbxNewAdminAv.isSelected())//if the checkbox is ticked
    {
        test = true;//sets the boolen to true
    }
    else//if none of the checkboxes are selected
    {
        lblNewAdminPortfoliosError.setText("Select at least one portfolio");//sets the lable to an error
    }
    return test;//returns the boolean
}

private boolean checkPortfolioEditAdmin()//checks that a portfolio has been selected
{
    boolean test = false;//creates a boolean to check if any have been selected
    if(cbxCateringAdmin.isSelected())//if the checkbox is ticked
    {
        test = true;//sets the boolen to true
    }
    else if (cbxPhotographyAdmin.isSelected())//if the checkbox is ticked
```

```
        {
            test = true;//sets the boolen to true
        }
        else if (cbxFirstAidAdmin.isSelected())//if the checkbox is ticked
        {
            test = true;//sets the boolen to true
        }
        else if (cbxAVAdmin.isSelected())//if the checkbox is ticked
        {
            test = true;//sets the boolen to true
        }
        else//if none of the checkboxes are selected
        {
            lblEditAdminPortfoliosError.setText("Select at least one portfolio");//sets the lable to an error
        }
        return test;//returns the boolean
    }

    //Adapted from Exploring IT: Java programming, Grade 12 text book
    //Type check
    private boolean typeCheck(javax.swing.JTextField txf, javax.swing.JLabel lbl, String validStr)
    {
        if(txf.getText().isEmpty())//checks if the text field is empty
        {
            lbl.setText("!Required");//gives an error in the label
            return false;//returns false if invalid
        }
        String txfText = txf.getText();//puts the text field value into a string
        for (int i = 0; i < txfText.length(); i++)//loops through the string untill the end
        {
            if(!Character.isLetterOrDigit(txfText.charAt(i))&& !(validStr.indexOf(txfText.charAt(i))>=0))//checks
that each character is of the correct type
            {
                lbl.setText("Invalid character(s)" + txfText.charAt(i));//gives an invalid character error
                return false;//returns false if invalid
            }
        }
        return true;//returns true if all is valid
    }

    //Presence check
    private boolean CheckDate(javax.swing.JLabel lbl)
    {
        boolean flag = true;//creates a boolean to indicate if the data is valid
        LocalDate dateEvent = dpkDate.getDate();//gets the date
        if(dateEvent == null)//checks if the date is entered
        {
            lblNewDateError.setText("Select a date");//gives an error
            flag = false;//sets the boolean to false
```

```
        }
        return flag;//returns the boolean
    }

    private boolean CheckEditDate(javax.swing.JLabel lbl)
    {
        boolean flag = true;//creates a boolean to indicate if the data is valid
        LocalDate dateEvent = dpkEditDate.getDate();//gets the date
        if(dateEvent == null)//checks if the date is entered
        {
            lblEditDateError.setText("Select a date");//gives an error
            flag = false;//sets the boolean to false
        }
        return flag;//returns the boolean
    }

    //Adapted from Exploring IT: Java programming, Grade 12 text book
    //Presence check and length check
    private boolean presenceAndLengthCheck(javax.swing.JTextField txf, javax.swing.JLabel lbl)
    {
        if ((txf.getText()).length() <= 1)//checks that the length is longer than 1 character
        {
            lbl.setText("Enter at least two characters");//gives an error to indicate that it is not long enough
            return false;//returns false to indicate invalid data
        }
        else
        {
            return true;//returns true to indicate valid data
        }
    }

    private boolean SpaceCheck(javax.swing.JTextField txf, javax.swing.JLabel lbl)
    {
        boolean flag = false;//creates a boolean to check if the data is valid
        for (int i = 1; i < txf.getText().length(); i++)//loops through the whole string
        {
            if (txf.getText().charAt(i) == ' ')//checks if each character is a space or not
            {
                flag = true;//sets the boolean to true, to indicate valid data
            }
        }
        if(flag == false)//if no space was found
        {
            lbl.setText("Name must have a space");//gives an error to tell the user they must enter a name
with a space
        }
        return flag;//returns the boolean
    }
```

```
    private boolean checkInt(javax.swing.JTextField txf, javax.swing.JLabel lbl, int range, int end)
    {
        boolean check;//creates a boolean to check if the data is valid
        int num = Integer.parseInt(txf.getText());//converts the entered information to an integer
        if(txf.getText().isEmpty() || Integer.parseInt(txf.getText()) == 0)//checks if the text is blank
        {
            lbl.setText("Enter a number");//sets the label to an error
            check = false;//sets the boolean to false
            return false;//returns false to indicate that the data was not valid
        }
        else//if the text is not blank
        {
            int i = 0;//creates a variable to count how many characters have been checked
            boolean test = true;//creates a variable to check if each character is valid
                while (test == true && i < txf.getText().length())//loops untill the end of the string is reached or a
character is not valid
                {
                    if (Character.isDigit(txf.getText().charAt(i)))//checks if the character is a digit
                    {
                        i++;//adds 1 to i
                    }
                    else//if the character is not a digit
                    {
                        test = false;//sets test to false, to end the loop
                        check = false;//sets boolean to false
                        lbl.setText("This field must only contain digits");//sets the lable to an error
                    }
                }
        }
        if(num >= range && num <= end)//checks that the value entered by the user is within the set range
        {
            return true;//returns true to indicate that the data is valid
        }
        else//if it is not within the range
        {
            check = false;//sets boolean to false
            lbl.setText("Number must be between " + range + " and " + end);//sets the lable to an error
        }
        return check;//returns true if data is valid or false if the data is not valid
    }

    //Adapted from Exploring IT: Java programming, Grade 12 text book
    //Type check and range check
    private String checkHours(javax.swing.JTextField txf, javax.swing.JLabel lbl)
    {
        String temp = "";//creates a string to hold the number of hours
        String txfText = txf.getText();//puts the value entered by the user into a string
        if(txf.getText().equals(""))//checks if the string from the user is empty
        {
```

```
            lbl.setText("Enter a number");//sets the label to an error
        } else//if the string is not empty
        {
            boolean test = true;//creates a boolean to check if each character is valid
            int i = 0;//creates a variable to count how many characters have been checked
            while (test == true && i < txfText.length())//loops untill the end of the string is reached or a
character is not valid
            {
                if (Character.isDigit(txfText.charAt(i)) || (txfText.charAt(i)) == '.')//checks if the character is a
digit or a full stop
                {
                    temp += txfText.charAt(i);//adds the character to the string
                    i++;//adds 1 to i
                } else //if the character is not a digit
                {
                    test = false;//sets test to false, to end the loop
                    lbl.setText("Hours number must only contain digits or a full stop");//sets the lable to an error
                    return null;//returns null to indicate that the data is not valid
                }
            }
            String hours = temp;//makes a new string to contain temp
            double num;//creates a double to hold the number of hours
            int lastDigitPos = hours.length() - 1;//creates an integer to hold the length of the string
            int checkDigit = Character.getNumericValue(hours.charAt(lastDigitPos));//checks the last
character of the string
            num = Double.parseDouble(hours);//converts the string to a double
            if(num < 0)//checks that the number is bigger than 0
            {
                lblNewHoursError.setText("Hours must be more than zero");//sets the lable to an error
                return null;//returns null to indicate that the data is not valid
            }
            else if (num > 12)//checks that the number is less than 12
            {
                lblNewHoursError.setText("Hours must be less than 12");//sets the lable to an error
                return null;//returns null to indicate that the data is not valid
            }
            else
            {
                return hours;//returns the number of hours if all data is valid
            }
        }
        return null;//returns null to indicate that the data is not valid
    }

    /**

     * @param args the command line arguments

     */
```

```
    public static void main(String args[]) {
        /* Set the Nimbus look and feel */
        //<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
        /* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
         * For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
         */
        try {
            for (javax.swing.UIManager.LookAndFeelInfo info :
javax.swing.UIManager.getInstalledLookAndFeels()) {
                if ("Nimbus".equals(info.getName())) {
                    javax.swing.UIManager.setLookAndFeel(info.getClassName());
                    break;
                }
            }
        } catch (ClassNotFoundException | InstantiationException | IllegalAccessException |
javax.swing.UnsupportedLookAndFeelException ex) {
            java.util.logging.Logger.getLogger(frmMain.class.getName()).log(java.util.logging.Level.SEVERE,
null, ex);
        }
        //</editor-fold>

        //</editor-fold>

        /* Create and display the form */
        java.awt.EventQueue.invokeLater(() -> {
            try {
```

```
            new frmMain().setVisible(true);
        } catch (SQLException ex) {
            Logger.getLogger(frmMain.class.getName()).log(Level.SEVERE, null, ex);

        }

    });
}


// Variables declaration - do not modify
private javax.swing.ButtonGroup bngEditEvent;
private javax.swing.ButtonGroup bngEditStuGender;
private javax.swing.ButtonGroup bngNewAdmin;
private javax.swing.ButtonGroup bngNewEvent;
private javax.swing.ButtonGroup bngNewStudentGender;
private javax.swing.ButtonGroup btgChangeAdmin;
private javax.swing.JButton btnChangeStudent;
private javax.swing.JButton btnConfirmPortfolio;
private javax.swing.JButton btnCreateParticipation;
private javax.swing.JButton btnCreateParticipationAdmin;
private javax.swing.JButton btnDeleteParticipation;
private javax.swing.JButton btnDeleteStuPart;
private javax.swing.JButton btnEditAdminCancel;
private javax.swing.JButton btnEditAdminCancel1;
private javax.swing.JButton btnEditAdminEnter;
private javax.swing.JButton btnEditAdminEnter1;
private javax.swing.JButton btnEditEvent;
private javax.swing.JButton btnEditEventClose;
private javax.swing.JButton btnEditEventEnter;
private javax.swing.JButton btnEditEventSearch;
private javax.swing.JButton btnEditStuCancel;
private javax.swing.JButton btnEditStuEnter;
private javax.swing.JButton btnExitHelp;
private javax.swing.JButton btnHelp10;
private javax.swing.JButton btnHelp11;
private javax.swing.JButton btnHelp12;
private javax.swing.JButton btnHelp13;
private javax.swing.JButton btnHelp14;
private javax.swing.JButton btnHelp2;
private javax.swing.JButton btnHelp3;
private javax.swing.JButton btnHelp4;
private javax.swing.JButton btnHelp5;
private javax.swing.JButton btnHowCreateEvent;
private javax.swing.JButton btnHowDeleteParticipationAdmin;
private javax.swing.JButton btnHowDeleteParticipationStudent;
private javax.swing.JButton btnHowEditAccountAdmin;
private javax.swing.JButton btnHowEditEvent;
```

```
private javax.swing.JButton btnHowEditMyAccountStudent;
private javax.swing.JButton btnHowLogIn;
private javax.swing.JButton btnHowMakeParticipation;
private javax.swing.JButton btnHowMakeParticipationStudent;
private javax.swing.JButton btnHowNewAdmin;
private javax.swing.JButton btnHowSearchEventAdmin;
private javax.swing.JButton btnHowSearchEventStu;
private javax.swing.JButton btnHowSearchParticipationAdmin;
private javax.swing.JButton btnHowSearchParticipationStu;
private javax.swing.JButton btnHowSearchStudent;
private javax.swing.JButton btnHowSignUp;
private javax.swing.JButton btnLogIn;
private javax.swing.JButton btnNewAdmin;
private javax.swing.JButton btnNewEvent;
private javax.swing.JButton btnNewEventClose;
private javax.swing.JButton btnNewEventEnter;
private javax.swing.JButton btnSearchDate;
private javax.swing.JButton btnSearchEventDate;
private javax.swing.JButton btnSearchEventID;
private javax.swing.JButton btnSearchEventNamePart;
private javax.swing.JButton btnSearchID;
private javax.swing.JButton btnSearchName;
private javax.swing.JButton btnSearchPartID;
private javax.swing.JButton btnSearchPartName;
private javax.swing.JButton btnSearchPartPortfolio;
private javax.swing.JButton btnSearchPortfolioPart;
private javax.swing.JButton btnSearchStuGrade;
private javax.swing.JButton btnSearchStuID;
private javax.swing.JButton btnSearchStuName;
private javax.swing.JButton btnSearchStudentNamePart;
private javax.swing.JButton btnSignUp;
private javax.swing.JButton btnSignUpStu;
private javax.swing.JCheckBox cbxAV;
private javax.swing.JCheckBox cbxAV1;
private javax.swing.JCheckBox cbxAVAdmin;
private javax.swing.JCheckBox cbxCatering;
private javax.swing.JCheckBox cbxCatering1;
private javax.swing.JCheckBox cbxCateringAdmin;
private javax.swing.JCheckBox cbxFirstAid;
private javax.swing.JCheckBox cbxFirstAid1;
private javax.swing.JCheckBox cbxFirstAidAdmin;
private javax.swing.JCheckBox cbxNewAdminAv;
private javax.swing.JCheckBox cbxNewAdminCatering;
private javax.swing.JCheckBox cbxNewAdminFirstAid;
private javax.swing.JCheckBox cbxNewAdminPhotography;
private javax.swing.JCheckBox cbxPhotography1;
private javax.swing.JCheckBox cbxPhotographyAdmin;
private javax.swing.JCheckBox cbxPotography;
private javax.swing.JDialog dlgEditAdmin;
```

```
private javax.swing.JDialog dlgEditEvent;
private javax.swing.JDialog dlgEditStudent;
private javax.swing.JDialog dlgEvents;
private javax.swing.JDialog dlgNewAdmin;
private com.github.lgooddatepicker.components.DatePicker dpkDate;
private com.github.lgooddatepicker.components.DatePicker dpkEditDate;
private com.github.lgooddatepicker.components.DatePicker dpkSearch;
private com.github.lgooddatepicker.components.DatePicker dpkSearchDate;
private javax.swing.JScrollPane jScrollPane1;
private javax.swing.JScrollPane jScrollPane10;
private javax.swing.JScrollPane jScrollPane11;
private javax.swing.JScrollPane jScrollPane2;
private javax.swing.JScrollPane jScrollPane3;
private javax.swing.JScrollPane jScrollPane4;
private javax.swing.JScrollPane jScrollPane5;
private javax.swing.JScrollPane jScrollPane6;
private javax.swing.JScrollPane jScrollPane7;
private javax.swing.JScrollPane jScrollPane8;
private javax.swing.JScrollPane jScrollPane9;
private javax.swing.JLabel lblCPassword4;
private javax.swing.JLabel lblCreateParticipation;
private javax.swing.JLabel lblDeleteParticipation;
private javax.swing.JLabel lblEditAdmin;
private javax.swing.JLabel lblEditAdminName;
private javax.swing.JLabel lblEditAdminNameError;
private javax.swing.JLabel lblEditAdminPortfolios;
private javax.swing.JLabel lblEditAdminPortfoliosError;
private javax.swing.JLabel lblEditAdminSurname;
private javax.swing.JLabel lblEditAdminSurnameError;
private javax.swing.JLabel lblEditDateError;
private javax.swing.JLabel lblEditEvent;
private javax.swing.JLabel lblEditEventDate;
private javax.swing.JLabel lblEditEventHours;
private javax.swing.JLabel lblEditEventID;
private javax.swing.JLabel lblEditEventName;
private javax.swing.JLabel lblEditEventPortfolio;
private javax.swing.JLabel lblEditHoursError;
private javax.swing.JLabel lblEditIDError;
private javax.swing.JLabel lblEditNameError;
private javax.swing.JLabel lblEditPortfolioError;
private javax.swing.JLabel lblEditStuClass;
private javax.swing.JLabel lblEditStuClassError;
private javax.swing.JLabel lblEditStuGender;
private javax.swing.JLabel lblEditStuGenderError;
private javax.swing.JLabel lblEditStuGrade;
private javax.swing.JLabel lblEditStuGradeError;
private javax.swing.JLabel lblEditStuName;
private javax.swing.JLabel lblEditStuNameError;
private javax.swing.JLabel lblEditStuPortfolios;
```

```
private javax.swing.JLabel lblEditStuPortfoliosError;
private javax.swing.JLabel lblEditStuSurname;
private javax.swing.JLabel lblEditStuSurnameError;
private javax.swing.JLabel lblEditStudent;
private javax.swing.JLabel lblHelpContact;
private javax.swing.JLabel lblLogo;
private javax.swing.JLabel lblLogo1;
private javax.swing.JLabel lblLogoHome;
private javax.swing.JLabel lblLogoHome1;
private javax.swing.JLabel lblNewAdminAccount;
private javax.swing.JLabel lblNewAdminConfirmPassword;
private javax.swing.JLabel lblNewAdminConfirmPasswordError;
private javax.swing.JLabel lblNewAdminName;
private javax.swing.JLabel lblNewAdminNameError;
private javax.swing.JLabel lblNewAdminPassword;
private javax.swing.JLabel lblNewAdminPasswordError;
private javax.swing.JLabel lblNewAdminPortfolios;
private javax.swing.JLabel lblNewAdminPortfoliosError;
private javax.swing.JLabel lblNewAdminUsername;
private javax.swing.JLabel lblNewAdminUsernameError;
private javax.swing.JLabel lblNewDateError;
private javax.swing.JLabel lblNewEvent;
private javax.swing.JLabel lblNewEventDate;
private javax.swing.JLabel lblNewEventHours;
private javax.swing.JLabel lblNewEventName;
private javax.swing.JLabel lblNewEventPortfolio;
private javax.swing.JLabel lblNewHoursError;
private javax.swing.JLabel lblNewNameError;
private javax.swing.JLabel lblNewPortfolioError;
private javax.swing.JLabel lblNewStudentClass;
private javax.swing.JLabel lblNewStudentClassError;
private javax.swing.JLabel lblNewStudentConfPasswordError;
private javax.swing.JLabel lblNewStudentGender;
private javax.swing.JLabel lblNewStudentGenderError;
private javax.swing.JLabel lblNewStudentGrade;
private javax.swing.JLabel lblNewStudentGradeError;
private javax.swing.JLabel lblNewStudentName;
private javax.swing.JLabel lblNewStudentNameError;
private javax.swing.JLabel lblNewStudentPasswordError;
private javax.swing.JLabel lblNewStudentPortfolioError;
private javax.swing.JLabel lblNewStudentPortfolios;
private javax.swing.JLabel lblNewStudentUsernameError;
private javax.swing.JLabel lblPassword1;
private javax.swing.JLabel lblPassword2;
private javax.swing.JLabel lblUsername;
private javax.swing.JLabel lblUsername1;
private javax.swing.JPanel pnlEvents;
private javax.swing.JPanel pnlEvents1;
private javax.swing.JPanel pnlHelp;
```

```
private javax.swing.JPanel pnlHome;
private javax.swing.JPanel pnlHomeAdmin;
private javax.swing.JPanel pnlLogIn;
private javax.swing.JPanel pnlParticipation;
private javax.swing.JPanel pnlParticipation1;
private javax.swing.JPanel pnlSignUp;
private javax.swing.JPanel pnlStudent;
private javax.swing.JRadioButton rbnEditEventAV;
private javax.swing.JRadioButton rbnEditEventCatering;
private javax.swing.JRadioButton rbnEditEventFirstAid;
private javax.swing.JRadioButton rbnEditEventPhotography;
private javax.swing.JRadioButton rbnEditStuGenderF;
private javax.swing.JRadioButton rbnEditStuGenderM;
private javax.swing.JRadioButton rbnNewEventAV;
private javax.swing.JRadioButton rbnNewEventCatering;
private javax.swing.JRadioButton rbnNewEventFirstAid;
private javax.swing.JRadioButton rbnNewEventPhotography;
private javax.swing.JRadioButton rbnNewStudentF;
private javax.swing.JRadioButton rbnNewStudentM;
private javax.swing.JTabbedPane tbdpAdmin;
private javax.swing.JTabbedPane tbdpStudent;
private javax.swing.JTable tblEventStu;
private javax.swing.JTable tblEventsAdmin;
private javax.swing.JTable tblParticipation;
private javax.swing.JTable tblParticipationAdmin;
private javax.swing.JTable tblStudents;
private javax.swing.JButton tbnSearchEventIDPart;
private javax.swing.JButton tbnSearchEventName;
private javax.swing.JTextField tfxSearchEventID;
private javax.swing.JTextArea txaAdmin;
private javax.swing.JTextArea txaAdmin1;
private javax.swing.JTextArea txaAdmin2;
private javax.swing.JTextArea txaAdmin3;
private javax.swing.JTextArea txaAdminDetails;
private javax.swing.JTextArea txaStudent;
private javax.swing.JTextField txfConfirmPassword1;
private javax.swing.JTextField txfCreatePartEventID;
private javax.swing.JTextField txfCreatePartStuID;
private javax.swing.JTextField txfDeleteEventID;
private javax.swing.JTextField txfDeleteStuID;
private javax.swing.JTextField txfEditAdminName;
private javax.swing.JTextField txfEditAdminSurname;
private javax.swing.JTextField txfEditEventHours;
private javax.swing.JTextField txfEditEventID;
private javax.swing.JTextField txfEditEventName;
private javax.swing.JTextField txfEditStuClass;
private javax.swing.JTextField txfEditStuGrade;
private javax.swing.JTextField txfEditStuName;
private javax.swing.JTextField txfEditStuSurname;
```

```
    private javax.swing.JTextField txfEventIDCreate;
    private javax.swing.JTextField txfEventIDDelete;
    private javax.swing.JTextField txfNewAdminConfirmPassword;
    private javax.swing.JTextField txfNewAdminName;
    private javax.swing.JTextField txfNewAdminPassword;
    private javax.swing.JTextField txfNewAdminUsername;
    private javax.swing.JTextField txfNewEventHours;
    private javax.swing.JTextField txfNewEventName;
    private javax.swing.JTextField txfNewStudentClass;
    private javax.swing.JTextField txfNewStudentGrade;
    private javax.swing.JTextField txfNewStudentName;
    private javax.swing.JTextField txfPassword;
    private javax.swing.JTextField txfPassword2;
    private javax.swing.JTextField txfSearchEventIDPart;
    private javax.swing.JTextField txfSearchEventName;
    private javax.swing.JTextField txfSearchEventNamePart;
    private javax.swing.JTextField txfSearchID;
    private javax.swing.JTextField txfSearchName;
    private javax.swing.JTextField txfSearchPartID;
    private javax.swing.JTextField txfSearchPartName;
    private javax.swing.JTextField txfSearchPartPortfolio;
    private javax.swing.JTextField txfSearchPortfolioPart;
    private javax.swing.JTextField txfSearchStuGrade;
    private javax.swing.JTextField txfSearchStuID;
    private javax.swing.JTextField txfSearchStuName;
    private javax.swing.JTextField txfSearchStudentNamePart;
    private javax.swing.JTextField txfUsername;
    private javax.swing.JTextField txfUsername1;
    // End of variables declaration
}
```

AdminManager Class:

```
/*
 * To change this license header, choose License Headers in Project Properties.
 * To change this template file, choose Tools | Templates
 * and open the template in the editor.
 */
package Mannager;

import Objects.Admin;
import Objects.DBService;
import Objects.Student;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.logging.Level;
import java.util.logging.Logger;
import javax.swing.table.DefaultTableModel;
```

```java
/**
 * Manages an array of admin objects from the database
 * @author ruth2
 */
public class AdminManager {

    private int size = 0; //Stores the size of the array
    DBService d = new DBService(); //Creates a connection to the database
    private final Admin[] AdminArr = new Admin[500]; //Creates an array to store up to 500 Admin objects
    DefaultTableModel model = new DefaultTableModel(); //sets the table moddle to the default settings



    /**
     * Adds an admin to the array and to the database
     * @param fn String first name of the admin
     * @param sn String surname of the admin
     * @param ca Boolean to check if the admin oversees catering
     * @param p Boolean to check if the admin oversees photography
     * @param a Boolean to check if the admin oversees AV
     * @param f Boolean to check if the admin oversees First Aid
     * @throws SQLException
     */
    public void add(String fn, String sn, boolean ca, boolean p, boolean a, boolean f) throws
SQLException //Recives all the information for a new admin except the auto generated Admin ID
    {
        int num = AdminArr[size-1].getID() + 1; //Finds the last ID in the array and sets the value of num one
higher
        Admin temp = new Admin(num,fn,sn,ca,p,a,f); //creates an admin admin object with all the new
details
        AdminArr[size] = new Admin(num,fn,sn,ca,p,a,f);//adds new Admin to the Array
        insertAdmin(AdminArr[size]); //calls insert admin method to add the admin to the databse
        size++; //Incriments size by 1
    }

    /**
     * Parameterized constructor to assign values to an admin
     * @throws SQLException
     */
    public AdminManager() throws SQLException //Constructor Method
    {
        String sql = "SELECT * FROM tblAdmins;"; //Select all from database and initialize the array with
returned values from the database
        ResultSet s = d.queryDatabase(sql); //Puts the results of the query into a result set
        while(s.next()) //checks through each element of the result set
        {
          int n = s.getInt("AdminID"); //Assigns the current admin ID to n
          String name = s.getString("AdminName"); //Assigns the current admin first name to name
```

```
            String surName = s.getString("AdminSurname"); //Assigns the current admin surname to surName
            boolean catering = s.getBoolean("Catering"); //Assigns the current Admin Boolean indicating if the admin does or does not oversee catering
            boolean photography = s.getBoolean("Photography");//Assigns the current Admin Boolean indicating if the admin does or does not oversee photography
            boolean AV = s.getBoolean("AV"); //Assigns the current Admin Boolean indicating if the admin does or does not oversee AV
            boolean firstAid = s.getBoolean("First Aid");//Assigns the current Admin Boolean indicating if the admin does or does not oversee first aid
            AdminArr[size] = new Admin(n,name,surName, catering, photography, AV, firstAid); //Assigns all the details of the current admin to the object array in the next position
            size++; //Incriments size by 1
        }
    }

    /**
     * Receives a position from which elements will be moved to the left within the array, can be used to remove an element from the array
     * @param pos Integer value of the Admin position in the array
     */
    public void shiftLeft(int pos)
    {
        for (int i = pos; i < size -1; i++) //Loops from position to the end of the array
        {
            AdminArr[i] = AdminArr[i +1]; //moves each element one forward
        }
        size--; //Decreases size by 1 to accomidate for the removed element
    }

    /**
     * Receives an admin object and creates a query with all relevant details to enter it into the database
     * @param f Admin object containing all the relevant details
     * @throws SQLException
     */
    public void insertAdmin(Admin f) throws SQLException
    {
        String sql = "INSERT INTO tblAdmins([AdminName], [AdminSurname], [Catering], [Photography], [AV], [First Aid]) VALUES ('"
        +  f.getAdminName() + "','" + f.getAdminSurname() + "','" + f.isCatering() + "','"
        + f.isPhotography() + "','" + f.isAv() + "','" + f.isFirstAid()+"');"; //SQL query to insert an admin into the database

        try
        {
            d.updateQuery(sql); //Executes the query
        }
        catch(SQLException e)
        {
            System.out.println("Insert error: Please try again");
```

```
    }
  }

  /**
   * Returns the size of the array
   * @return the size of the array
   */
  public int getSize()
  {
    return size; //Returns the size of the array
  }

  /**
   * checks if the current Admin oversees the portfolio specified by the string
   * @param c String name of the portfolio to be checked
   * @param i Integer value of the Admin ID
   * @return false if no match was found, returns true if the Admin does oversee the specified portfolio
   */
  public boolean getPortfolio(String c, int i)
  {
    boolean test = false;
    for(int j =0; j < size; j++)//loops through all Admins in the array
    {
      if((AdminArr[j].getID()) == i)//checks each Admin ID aginst i
      {
        i = i-1;
      if(c.equalsIgnoreCase("Catering") && AdminArr[i].isCatering())//checks if the string entered is
catering and if the current Admin oversees catering
      {
        test = true;
      }
      else if (c.equalsIgnoreCase("Photography") && AdminArr[i].isPhotography())//checks if the string
entered is photography and if the current Admin oversees photography
      {
        test = true;
      }
      else if (c.equalsIgnoreCase("av") && AdminArr[i].isAv())//checks if the string entered is AV and if
the current Admin oversees AV
      {
        test = true;
      }
      else if (c.equalsIgnoreCase("First Aid") && AdminArr[i].isFirstAid())//checks if the string entered is
First Aid and if the current Admin oversees First Aid
      {
        test = true;
      }
      }
    }
```

```
        return test; //Returns false if no match was found, returns true if the Admin does oversee the specified portfolio
    }

    /**
     * returns a string containing all the details of the Admin with Admin ID equal to i
     * @param i Integer value of Admin ID
     * @return the string containing all of the Admins formatted details
     */
    public String getCurrent(int i) throws SQLException
    {
        //Components
        String temp = "";
        size = 0;
        String sql = "SELECT * FROM tblAdmins;"; //Select all from database and initialize the array with returned values from the database
        ResultSet s = d.queryDatabase(sql); //Puts the results of the query into a result set
        while(s.next()) //checks through each element of the result set
        {
            int n = s.getInt("AdminID"); //Assigns the current admin ID to n
            String name = s.getString("AdminName"); //Assigns the current admin first name to name
            String surName = s.getString("AdminSurname"); //Assigns the current admin surname to surName
            boolean catering = s.getBoolean("Catering"); //Assigns the current Admin Boolean indicating if the admin does or does not oversee catering
            boolean photography = s.getBoolean("Photography");//Assigns the current Admin Boolean indicating if the admin does or does not oversee photography
            boolean AV = s.getBoolean("AV"); //Assigns the current Admin Boolean indicating if the admin does or does not oversee AV
            boolean firstAid = s.getBoolean("First Aid");//Assigns the current Admin Boolean indicating if the admin does or does not oversee first aid
            AdminArr[size] = new Admin(n,name,surName, catering, photography, AV, firstAid); //Assigns all the details of the current admin to the object array in the next position
            size++; //Incriments size by 1
        }
        for(int j =0; j < size; j++) //Loops through the array
        {
            if((AdminArr[j].getID()) == i) //checks if the current element in the array has the same ID value as the imputted value
            {
                i = j;
                temp = temp + "Name: " + AdminArr[i].getAdminName() + " " + AdminArr[i].getAdminSurname() + "\t\tAdmin ID: " + AdminArr[i].getID() + "\nPortfolios: ";
                if(AdminArr[i].isCatering()) //If the admin oversees catering, 'catering' is added to the string
                {
                    temp = temp + "Catering\t";//adds 'catering' to the string
                }
                if(AdminArr[i].isAv()) //If the admin oversees AV, 'AV' is added to the string
                {
                    temp = temp + "AV\t";//adds 'AV' to the string
```

```
            }
            if(AdminArr[i].isPhotography()) //If the admin oversees photography, 'photography' is added to
the string
            {
                temp = temp + "Photography\t";//adds 'photography' to the string
            }
            if(AdminArr[i].isFirstAid()) //If the admin oversees first aid, 'first aid' is added to the string
            {
                temp = temp + "First Aid\t";//adds 'first aid' to the string
            }
        }
    }
    return temp; //Returns the string containing all of the Admin's formatted details
  }

  public String toString() //To String method returns all Admins in the array
  {
    String temp = "";
    for(int i = 0; i < size; i++)
    {
      temp = temp + AdminArr[i].toString() + "\n";
    }
    return temp;
  }
}
```

EventManager Class:

```
/*
 * To change this license header, choose License Headers in Project Properties.
 * To change this template file, choose Tools | Templates
 * and open the template in the editor.
 */
package Mannager;

import Objects.DBService;
import Objects.Event;
import java.sql.Date;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.time.LocalDate;
import javax.lang.model.SourceVersion;
import javax.swing.table.DefaultTableModel;

/**
 * Manages an array of Event Objects from the database
 * @author ruth2
 */
```

```
public class EventManager {//Class name
    //Components
    private int size = 0; //Stores the size of the array
    DBService d = new DBService(); //Creates a connection to the database
    Event[] EventArr = new Event[1000]; //Creates an array to store up to 1000 Event objects
    DefaultTableModel model = new DefaultTableModel(); //sets the table moddle to the default settings

    /**
     * Receives all information for a new event except the auto generated Event number
     * @param n String name of the event
     * @param h Integer hours of the event
     * @param d LocalDate date of the event
     * @param c the Boolean to check if the event is for catering
     * @param p the Boolean to check if the event is for photography
     * @param f the Boolean to check if the event is for First Aid
     * @param a the Boolean to check if the event is for AV
     * @throws SQLException
     */
    public void add(String n, double h,LocalDate d, boolean c, boolean p, boolean f, boolean a) throws
SQLException
    {
        int x = size -1; //Sets x which will be used to allocate the place in the array where the new record will
be stored
        int num = EventArr[x].getEventNum() + 1; //Finds the last ID in the array and sets the value of num
one higher
        EventArr[size] = new Event(num,n,h,d,c,p,f,a);//Adds a new event to the array
        insertEvent(EventArr[size]);//calls the insert event method
        size++; //Adds 1 to size
    }

    /**
     * Constructor Method to assign Events to an array from the database
     * @throws SQLException
     */
    public EventManager() throws SQLException
    {
        String sql = "SELECT * FROM tblEvents;";   //Select all from database and initialize the array with
returned values from the database
        ResultSet s = d.queryDatabase(sql); //Puts the results of the query into a result set
        while(s.next()) //checks through each element of the result set
        {
          int n = s.getInt("EventID"); //Assigns the current Event ID to n
          String name = s.getString("EventName"); //Assigns the current event name to name
          double h = s.getDouble("Hours"); //Assigns the current Event hours to hours
          Date date = s.getDate("EventDate"); //Assigns the current Event date to date
          LocalDate localDate = date.toLocalDate(); //Converts date to local date
          boolean catering = s.getBoolean("Catering"); //Assigns the current event Boolean indicating if the
event is for catering
```

```
            boolean photography = s.getBoolean("Photography"); //Assigns the current event Boolean indicating if the event is for photography
            boolean firstAid = s.getBoolean("FirstAid"); //Assigns the current event Boolean indicating if the event is for first aid
            boolean AV = s.getBoolean("AV"); //Assigns the current event Boolean indicating if the event is for AV
            EventArr[size] = new Event(n,name,h,localDate,catering,photography,firstAid,AV); //Assigns all the details of the current event to the object array in the next position
            size++;//Adds 1 to size
        }
    }

    /**
     * Receives a position from which elements will be moved to the left within the array, can be used to remove an element from the array
     * @param pos position of the event in the array
     */
    public void shiftLeft(int pos)
    {
        for (int i = pos; i < size -1; i++) //Loops from position to the end of the array
        {
            EventArr[i] = EventArr[i +1]; //moves each element one forward
        }
        size--; //Decreases size by 1 to accomidate for the removed element
    }

    /**
     * Receives an event object and creates a query with all relevant details to enter it into the database
     * @param f Event Object containing all the event details
     * @throws SQLException
     */
    public void insertEvent(Event f) throws SQLException
    {
        String sql = "INSERT INTO tblEvents([EventName], [Hours], [EventDate], [Catering], [Photography], [FirstAid], [AV]) VALUES ('"
            + f.getEventName() + "','" + f.getNumHours() + "','" + f.getDate() + "','" + f.isCatering() + "','"
            + f.isPhotography() + "','" + f.isFirstAid() + "','" + f.isAudioVisual()+"');"; //SQL query to insert an event into the database

        try
        {
            d.updateQuery(sql); //Executes the query
        }
        catch(SQLException e)
        {
            System.out.println("Insert error: could not insert event");
        }
    }
```

```
    /**
     * Returns the size of the array
     * @return Integer value of the array size
     */
    public int getSize()
    {
        return size; //Returns the size of the array
    }

    public String toString() //To String method returns all events in the array
    {
        String temp = "";
        for(int i = 0; i < size; i++)
        {
            temp = temp + EventArr[i].toString() + "\n";
        }
        return temp;
    }
}
```

ParticipationManager Class:

```
/*
 * To change this license header, choose License Headers in Project Properties.
 * To change this template file, choose Tools | Templates
 * and open the template in the editor.
 */
package Mannager;

import Objects.DBService;
import Objects.Event;
import Objects.Participation;
import java.sql.Date;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.time.LocalDate;
import javax.swing.table.DefaultTableModel;

/**
 * Manages an array of Participation objects
 * @author ruth2
 */
public class ParticipationManager {//Class Name
    //Components
    private int size = 0; //Stores the size of the array
    DBService d = new DBService(); //Creates a connection to the database
```

```
    private final Participation[] PartArr = new Participation[2000]; //Creates an array to store up to 2000 participation objects
    DefaultTableModel model = new DefaultTableModel(); //sets the table moddle to the default settings


    /**
     * adds a new participation object to the array and database
     * @param studentID Integer ID of the student
     * @param eventID Integer ID of the event
     * @throws SQLException
     */
    public void add(int studentID, int eventID) throws SQLException//Recives a Student ID and Event ID for a new participation record
    {
        PartArr[size] = new Participation(studentID, eventID);//Adds a new Participation to the array
        size++; //Adds 1 to size
        insertParticipation(eventID,studentID);//calls the insert participation mathod
    }

    /**
     * Creates an array of participation objects with all their relevant details from the database
     * @throws SQLException
     */
    public ParticipationManager() throws SQLException //Constructor Method
    {
        String sql = "SELECT * FROM tblParticipation;";   //Select all from database and initialize the array with returned values from the database
        ResultSet s = d.queryDatabase(sql); //Puts the results of the query into a result set
        while(s.next()) //checks through each element of the result set
        {
            int sn = s.getInt("StudentID"); //Assigns the current student ID to sn
            int n = s.getInt("EventID"); //Assigns the current Event ID to n
            PartArr[size] = new Participation(sn,n); //Assigns all the details of the current participation to the object array in the next position
            size++; //Adds 1 to size
        }
    }

    /**
     * Receives a position from which elements will be moved to the left within the array, can be used to remove an element from the array
     * @param pos the Integer position of the object in the array
     */
    public void shiftLeft(int pos)
    {
        for (int i = pos; i < size -1; i++) //Loops from position to the end of the array
        {
            PartArr[i] = PartArr[i +1]; //moves each element one forward
        }
```

```
        size--; //Decreases size by 1 to accomidate for the removed element
    }

    /**
     * Receives an object and creates a query with all relevant details to enter it into the database
     * @param s Integer value of the student ID
     * @param e Integer value of the event ID
     * @throws SQLException
     */
    public void insertParticipation(int s,int e) throws SQLException
    {
        String sql = "INSERT INTO tblParticipation([EventID],[StudentID]) VALUES ('"
            + e + "','" + s+ "');"; //SQL query to insert a participation record into the database
        try
        {
            d.updateQuery(sql); //Executes the query
        }
        catch(SQLException p)
        {
            System.out.println("Insert error: Could not create participation record");
        }
    }

    /**
     * Receives a row number and deletes the corresponding entry in the database and the object array
     * @param event Integer value of the Event ID
     * @param student Integer value of the Student ID
     * @return a string to indicate success or failure
     */
    public String deleteEntry(int event, int student)
    {
        String sql = "DELETE * FROM tblParticipation WHERE EventID =" + event + " AND StudentID =" +
student; //Deletes all columbs where the row number is equal to what the user has entered
        try
        {
            d.updateQuery(sql); //Executes the query
            for (int i =0; i < size-1; i++) //Loops through the array
            {
                if(PartArr[i].getEventID() == event && PartArr[i].getStudentID() == student) //Checks if the
current array element in the loop matches the information given
                {
                    shiftLeft(event-1); //Calls the shift left method to remove the event from the array
                    return "You have successfully deleted the record from the database";
                }
            }
        }
        catch(SQLException e)
        {
            return "Error: Could not delete record";
```

```
        }
        return""; //Returns a string to indicate success or failure
    }

    /**
     * Returns the size of the array
     * @return Integer value of how many objects are in the array
     */
    public int getSize()//Returns the size of the array
    {
        return size; //Returns the size of the array
    }

    public String toString() //To String method returns all Participation records in the array
    {
        String temp = "";
        for(int i = 0; i < size; i++)
        {
            temp = temp + PartArr[i].toString() + "\n";
        }
        return temp;
    }
}
```

StudentManager Class:

```
/*
 * To change this license header, choose License Headers in Project Properties.
 * To change this template file, choose Tools | Templates
 * and open the template in the editor.
 */
package Mannager;

import Objects.DBService;
import Objects.Student;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.time.LocalDate;
import javax.swing.table.DefaultTableModel;

/**
 * Manage an array of student objects
 * @author ruth2
 */
public class StudentManager {//Class Name
    //Components
    private int size = 0; //Stores the size of the array
    DBService d = new DBService(); //Creates a connection to the database
```

```
    private Student[] StudentArr = new Student[1000];//Creates an array to store up to 1000 Student objects
    DefaultTableModel model = new DefaultTableModel(); //sets the table moddle to the default settings



    /**
     * adds a new Student object to the database and the array
     * @param fn String value of the student name
     * @param sn String value of the student surname
     * @param g Integer value of the student grade
     * @param c Integer value of the student class
     * @param gen character value of the student gender (M/F)
     * @param ca the Boolean indicating if the student does or does not do catering
     * @param p the Boolean indicating if the student does or does not do photography
     * @param a the Boolean indicating if the student does or does not do AV
     * @param f the Boolean indicating if the student does or does not do First Aid
     * @throws SQLException
     */
    public void add(String fn, String sn, int g, int c, char gen, boolean ca, boolean p, boolean a, boolean f) throws SQLException//recives all the details of a student object except the auto generated StudentID
    {
        int x = size -1; //Sets x which will be used to allocate the place in the array where the new record will be stored
        int num = (StudentArr[x].getStuID()) + 1; //Finds the last ID in the array and sets the value of num one higher
        size++; //Increases size by 1
        StudentArr[size] = new Student(num,fn,sn,g,c,gen,ca,p,a,f);//adds new Student to the Array
        insertStudent(StudentArr[size]);//calls insert student method to add the student to the databse
    }

    /**
     * Assigns values to each student object and adds them to the array
     * @throws SQLException
     */
    public StudentManager() throws SQLException
    {
        String sql = "SELECT * FROM tblStudents;";   //Select all from database and initialize the array with returned values from the database
        ResultSet s = d.queryDatabase(sql); //Puts the results of the query into a result set
        while(s.next()) //checks through each element of the result set
        {
          int n = s.getInt("StudentID"); //Assigns the current student ID to n
          String name = s.getString("FirstName");//Assigns the current student first name to name
          String surName = s.getString("Surname");//Assigns the current student surname to surName
          int grade = s.getInt("Grade");//Assigns the current student grade to grade
          int classPoint = s.getInt("Class");//Assigns the current student class to classPoint
          char g = (s.getString("Gender")).charAt(0);//Assigns the current student gender to g
```

```
        boolean catering = s.getBoolean("Catering");//Assigns the current student Boolean indicating if the student does or does not do catering
        boolean photography = s.getBoolean("Photography");//Assigns the current student Boolean indicating if the student does or does not do photography
        boolean AV = s.getBoolean("AV");//Assigns the current student Boolean indicating if the student does or does not do AV
        boolean firstAid = s.getBoolean("FirstAid");//Assigns the current student Boolean indicating if the student does or does not do first aid
        StudentArr[size] = new Student(n,name,surName,grade,classPoint, g, catering, photography, AV, firstAid); //Assigns all the details of the current student to the object array in the next position
        size++; //incriments size by 1
      }
    }

    /**
     * Receives a position from which elements will be moved to the left within the array, can be used to remove an element from the array
     * @param pos integer value of the students position in the array
     */
    public void shiftLeft(int pos)
    {
      pos--; //moves to the eliment before the position received
      for (int i = pos; i < size -1; i++) //Loops from position to the end of the array
      {
        StudentArr[i] = StudentArr[i +1];//moves each element one forward
      }
      size--; //Decreases size by 1 to accomidate for the removed element
    }

    /**
     * Returns the ID of the last student in the array
     * @return the integer value of the ID belonging to the last student in the array
     */
    public int getLast()
    {
      return StudentArr[size].getStuID();//returns the ID of the element in the array
    }

    /**
     * Receives a student object and creates a query with all relevant details to enter it into the database
     * @param f Student object containing all the students information
     * @throws SQLException
     */
    public void insertStudent(Student f) throws SQLException
    {
      String sql = "INSERT INTO tblStudents([firstName], [Surname], [Grade], [Class], [Gender], [Catering], [Photography], [AV], [FirstAid]) VALUES ('"
          + f.getStuName() + "','" + f.getStuSurname() + "','" + f.getGrade() + "','" + f.getClassPoint() + "','"  + f.getGender() + "','" + f.isCatering() + "','"
```

```
            + f.isPhotography() + "','" + f.isAv() + "','" + f.isFirstAid()+"');"; //SQL query to insert a student into
the database
        try
        {
            d.updateQuery(sql); //Executes the query
        }
        catch(SQLException e)
        {
            System.out.println("Insert error: Could not insert Student");
        }
    }

    /**
     * Returns the size of the array
     * @return the size of the array/ number of students in the array
     */
    public int getSize()
    {
        return size;//Returns the size of the array
    }

    /**
     * Returns a string containing all the details of the student with student ID equal to i
     * @param i the integer value of the Student ID
     * @return all the formatted details of the student as a String
     * @throws SQLException
     */
    public String getCurrent(int i) throws SQLException
    {
        //Components
        String temp = "";
        int c =0; //holds the number of catering hours
        int p = 0;//holds the number of photography hours
        int a = 0;//holds the number of AV hours
        int f = 0;//holds the number of First Aid hours
        String sql = "SELECT * FROM tblStudents;";   //Select all from database and initialize the array with
returned values from the database
        ResultSet s = d.queryDatabase(sql);
        size = 0; //Sets size back to 0
        while(s.next()) //checks through each element of the result set
        {
            int n = s.getInt("StudentID"); //Assigns the current student ID to n
            String name = s.getString("FirstName");//Assigns the current student first name to name
            String surName = s.getString("Surname");//Assigns the current student surname to surName
            int grade = s.getInt("Grade");//Assigns the current student grade to grade
            int classPoint = s.getInt("Class");//Assigns the current student class to classPoint
            char g = (s.getString("Gender")).charAt(0);//Assigns the current student gender to g
            boolean catering = s.getBoolean("Catering");//Assigns the current student Boolean indicating if the
student does or does not do catering
```

```
        boolean photography = s.getBoolean("Photography");//Assigns the current student Boolean indicating if the student does or does not do photography
        boolean AV = s.getBoolean("AV");//Assigns the current student Boolean indicating if the student does or does not do AV
        boolean firstAid = s.getBoolean("FirstAid");//Assigns the current student Boolean indicating if the student does or does not do first aid
        StudentArr[size] = new Student(n,name,surName,grade,classPoint, g, catering, photography, AV, firstAid); //Assigns all the details of the current student to the object array in the next position
        size++; //incriments size by 1
      }
      for(int j =0; j < size; j++) //Loops through the array
      {
        if((StudentArr[j].getStuID()) == i)//checks if the current element in the array has the same ID value as the imputted value
        {
          ResultSet ch = d.queryDatabase("SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS CateringHours\n" +
          "FROM tblStudents, tblEvents, tblParticipation\n" +
          "WHERE tblParticipation.StudentID = " + i + " And " + i + " = tblStudents.StudentID AND tblEvents.EventID = tblParticipation.EventID AND tblEvents.Catering = True\n" +
          "GROUP BY tblStudents.StudentID;"); //Selects the total number of catering hours
          ResultSet ph = d.queryDatabase("SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS PhotographyHours\n" +
          "FROM tblStudents, tblEvents, tblParticipation\n" +
          "WHERE tblParticipation.StudentID = " + i + " And " + i + " = tblStudents.StudentID AND tblEvents.EventID = tblParticipation.EventID AND tblEvents.Photography = True\n" +
          "GROUP BY tblStudents.StudentID;");//Selects the total number of Photography hours
          ResultSet avh = d.queryDatabase("SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS AVHours\n" +
          "FROM tblStudents, tblEvents, tblParticipation\n" +
          "WHERE tblParticipation.StudentID = " + i + " And " + i + " = tblStudents.StudentID AND tblEvents.EventID = tblParticipation.EventID AND tblEvents.AV = True\n" +
          "GROUP BY tblStudents.StudentID;");//Selects the total number of AV hours
          ResultSet fh = d.queryDatabase("SELECT tblStudents.StudentID, SUM(tblEvents.hours) AS FirstAidHours\n" +
          "FROM tblStudents, tblEvents, tblParticipation\n" +
          "WHERE tblParticipation.StudentID = " + i + " And " + i + " = tblStudents.StudentID AND tblEvents.EventID = tblParticipation.EventID AND tblEvents.FirstAid = True\n" +
          "GROUP BY tblStudents.StudentID;");//Selects the total number of First Aid hours
          i = j;
          temp = temp + "Name: " + StudentArr[i].getStuName() + " " + StudentArr[i].getStuSurname() + "\t\tStudent ID: " + StudentArr[i].getStuID() + "\nGrade: ";
          temp = temp + StudentArr[i].getGrade() + "." + StudentArr[i].getClassPoint() + "\t\tGender: " + StudentArr[i].getGender() + "\nPortfolios: "; // formats the Student details
          if(StudentArr[i].isCatering()) //If the student does catering, the total number of catering hours is added to the string
          {
            temp = temp + "Catering "; //Adds 'catering' to the string
            while(ch.next())
```

```
                {
                    c = ch.getInt("CateringHours");//Assigns the total number of catering hours to c
                    temp = temp + "-" + c; //Adds the number of catering hours to the string
                }
                temp = temp + "\t";
            }
            if(StudentArr[i].isAv()) //If the student does AV, the total number of AV hours is added to the
string
            {
                temp = temp + "AV "; //Adds 'AV' to the string
                while(avh.next())
                {
                    a = avh.getInt("AVHours");//Assigns the total number of AV hours hours to a
                    temp = temp + "-" + a; //Adds the number of av hours to the string
                }
                temp = temp + "\t";
            }
            if(StudentArr[i].isPhotography()) //If the student does photography, the total number of
photography hours is added to the string
            {
                temp = temp + "Photography "; //Adds 'photography' to the string
                while(ph.next())
                {
                    p = ph.getInt("PhotographyHours"); //Assigns the total number of photography hours to p
                    temp = temp + "-" + p; //Adds the number of photography hours to the string
                }
                temp = temp + "\t";
            }
            if(StudentArr[i].isFirstAid()) //If the student does FirstAid, the total number of FirstAid hours is
added to the string
            {
                temp = temp + "First Aid "; //Adds 'first aid' to the string
                while(fh.next())
                {
                    f= fh.getInt("FirstAidHours");//Assigns the total number of first aid hours to f
                    temp = temp + "-" + f; //Adds the number of first aid hours to the string
                }
                temp = temp + "\t";
            }
        }
    }
    return temp; //Returns the string containing all of the student's formatted details
}

/**
 * checks if the current Student(position in database at i) participates in the portfolio specified by the
string
 * @param c the String name of the portfolio to be checked
 * @param i the Integer value of the student ID
```

```
     * @return false if no match was found, returns true if the Admin does oversee the specified portfolio
     */
    public boolean getPortfolio(String c, int i)
    {
        boolean test = false;
        for(int j =0; j < size; j++)//loops through all studetents in the array
        {
            if((StudentArr[j].getStuID()) == i)//checks each Student ID aginst i
            {
                if(c.equalsIgnoreCase("Catering") && StudentArr[j].isCatering())//checks if the string entered is
catering and if the current student does catering
                {
                    test = true;
                }
                else if (c.equalsIgnoreCase("Photography") && StudentArr[j].isPhotography())//checks if the
string entered is photography and if the current student does photography
                {
                    test = true;
                }
                else if (c.equalsIgnoreCase("av") && StudentArr[j].isAv())//checks if the string entered is AV
and if the current student does AV
                {
                    test = true;
                }
                else if (c.equalsIgnoreCase("First Aid") && StudentArr[j].isFirstAid())//checks if the string
entered is First Aid and if the current student does First Aid
                {
                    test = true;
                }
            }
        }
        return test; //Returns false if no match was found, returns true if the student does participate in the
specified portfolio
    }

    public String toString() //To String method returns all Students in the array
    {
        String temp = "";
        for(int i = 0; i < size; i++)
        {
            temp = temp + StudentArr[i].toString() + "\n";
        }
        return temp;
    }

}
```

Admin Object Class:

```
/*
 * To change this license header, choose License Headers in Project Properties.
 * To change this template file, choose Tools | Templates
 * and open the template in the editor.
 */
package Objects;

/**
 * Store details of an Admin
 * @author ruth2
 */
public class Admin {//Class Name
    //Components
    private int ID; //Admin ID
    private String adminName; //Admin first name
    private String adminSurname; //Admin surname
    private boolean photography; // Boolean to check if the admin oversees Photography
    private boolean catering; //Boolean to check if the admin oversees Catering
    private boolean firstAid; //Boolean to check if the admin oversees FirstAid
    private boolean av; //Boolean to check if the admin oversees AV

    //Constructor Method

    /**
     * Parameterized constructor to assign default values to an admin
     * @param ID integer ID of the Admin
     * @param adminName String first name of the Admin
     * @param adminSurname String surname of the Admin
     * @param catering Boolean to check if the admin oversees catering
     * @param photography Boolean to check if the admin oversees photography
     * @param av Boolean to check if the admin oversees AV
     * @param firstAid Boolean to check if the admin oversees FirstAid
     */
    public Admin(int ID, String adminName, String adminSurname, boolean catering, boolean photography,
boolean av, boolean firstAid)
    {
        this.ID = ID; //Assigns imput value to the variable
        this.adminName = adminName; //Assigns imput value to the variable
        this.adminSurname = adminSurname; //Assigns imput value to the variable
        this.photography = photography; //Assigns imput value to the variable
        this.catering = catering; //Assigns imput value to the variable
        this.firstAid = firstAid; //Assigns imput value to the variable
        this.av = av; //Assigns imput value to the variable
    }

    //Accessor and Mutator Methods
```

```java
/**
 * getter method for Admin ID
 * @return the Admin ID as an integer
 */
public int getID()
{
    return ID;//Returns Admin ID
}

/**
 * getter method for Admin first name
 * @return the Admins first name as a String
 */
public String getAdminName()
{
    return adminName; //Returns the admin first name
}

/**
 * getter method for admin surname
 * @return the admins surname as a String
 */
public String getAdminSurname()
{
    return adminSurname; //Returns the admin surname
}

/**
 * getter method for Boolean photography
 * @return a Boolean to check if the admin oversees photography
 */
public boolean isPhotography()
{
    return photography; //Returns the Boolean to check if the admin oversees photography
}

/**
 * getter method for Boolean Catering
 * @return a Boolean to check if the admin oversees catering
 */
public boolean isCatering()
{
    return catering;//Returns the Boolean to check if the admin oversees Catering
}

/**
 * getter method for Boolean FirstAid
 * @return a Boolean to check if the admin oversees first aid
 */
```

```
    public boolean isFirstAid()
    {
        return firstAid; //Returns the Boolean to check if the admin oversees FirstAid
    }

    /**
     * getter method for Boolean AV
     * @return a Boolean to check if the admin oversees AV
     */
    public boolean isAv()
    {
        return av; //Returns the Boolean to check if the admin oversees AV
    }

    @Override
    //toString Method
    public String toString() //Returns all the details of an admin as a formatted string
    {
        return "Admin{" + "ID=" + ID + ", adminName=" + adminName + ", adminSurname=" +
adminSurname + ", photography=" + photography + ", catering=" + catering + ", firstAid=" + firstAid + ",
av=" + av + '}';
    }
}
```

Event Object Class:

```
/*
 * To change this license header, choose License Headers in Project Properties.
 * To change this template file, choose Tools | Templates
 * and open the template in the editor.
 */
package Objects;

import java.time.LocalDate;

/**
 * Stores details of an event
 * @author ruth2
 */
public class Event {//Class Name
    //Components
    private int eventNum; //Event number
    private String eventName; //Event Name
    private double numHours; //Event number of hours
    private LocalDate date; //Event date
    private boolean catering; //Boolean to check if the event is for catering
    private boolean photography;//Boolean to check if the event is for photography
    private boolean firstAid; //Boolean to check if the event is for FirstAid
```

```
private boolean audioVisual; //Boolean to check if the event is for AV

//Constructor Method

/**
 * parameterized constructor to assign default values to an event
 * @param num the integer event number
 * @param n the String name of the event
 * @param h the integer number of hours of the event
 * @param d the date of the event
 * @param c the Boolean to check if the event is for catering
 * @param p the Boolean to check if the event is for photography
 * @param f the Boolean to check if the event is for first aid
 * @param a the Boolean to check if the event is for AV
 */
public Event(int num, String n, double h,LocalDate d, boolean c, boolean p, boolean f, boolean a)
{
    eventNum = num; //Assigns imput value to the variable
    eventName = n; //Assigns imput value to the variable
    numHours = h; //Assigns imput value to the variable
    date = d; //Assigns imput value to the variable
    catering = c; //Assigns imput value to the variable
    photography = p; //Assigns imput value to the variable
    firstAid = f; //Assigns imput value to the variable
    audioVisual = a; //Assigns imput value to the variable
}

//Accessor and Mutator Methods

/**
 * getter method for the event number
 * @return the the number of the event as an integer
 */
public int getEventNum()
{
    return eventNum;//Returns the event number
}

/**
 * getter method for the event name
 * @return the name of the event as a String
 */
public String getEventName()
{
    return eventName;//Returns the event name
}

/**
 * getter method for hours (number of hours for which the event lasts)
```

```
     * @return the number of hours as an integer
     */
    public double getNumHours()
    {
        return numHours;//Returns the event hours
    }

    /**
     * getter method for the event date
     * @return the date of the event as a LocalDate
     */
    public LocalDate getDate()
    {
        return date; //Returns the event date
    }

    /**
     * getter method for the Boolean catering
     * @return a Boolean to check if the event is for catering
     */
    public boolean isCatering()
    {
        return catering; //Returns the Boolean to check if the event is for catering
    }

    /**
     * getter method for the Boolean photography
     * @return a Boolean to check if the event is for photography
     */
    public boolean isPhotography()
    {
        return photography;//Returns the Boolean to check if the event is for photography
    }

    /**
     * getter method for the Boolean FirstAid
     * @return a Boolean to check if the event is for first aid
     */
    public boolean isFirstAid()
    {
        return firstAid; //Return the Boolean to check if the event is for first aid
    }

    /**
     * getter method for the Boolean Audio Visual
     * @return a Boolean to check if the event is for AV
     */
    public boolean isAudioVisual()
    {
```

```
        return audioVisual; //Returns the Boolean to check if the event is for AV
    }

    @Override
    //toString Method
    public String toString()//Returns all the details of the event as a formatted string
    {
        return "Event{" + "eventNum=" + eventNum + ", eventName=" + eventName + ", numHours=" +
numHours + ", date=" + date + ", catering=" + catering + ", photography=" + photography + ", firstAid=" +
firstAid + ", audioVisual=" + audioVisual + '}';
    }
}
```

Participation Object Class:

```
/*
 * To change this license header, choose License Headers in Project Properties.
 * To change this template file, choose Tools | Templates
 * and open the template in the editor.
 */
package Objects;

/**
 * Stores the details of a participation record
 * @author ruth2
 */
public class Participation {//Class Name
    //Components
    private int eventID; //Event ID
    private int studentID; //Student ID

    //Constructor Method

    /**
     * parameterized constructor to assign default values to a participation record
     * @param studentID the integer student ID
     * @param eventID the integer event ID
     */
    public Participation(int studentID,int eventID)
    {
        this.eventID = eventID; //assigns imput value to private variable
        this.studentID = studentID; //assigns imput value to private variable
    }

    //Accessor and Mutator Methods

    /**
     * getter method for the event ID of the participation record
```

```
     * @return the Event ID as an integer
     */
    public int getEventID()
    {
        return eventID; //Returns EventID
    }

    /**
     * getter method for the Student ID of the participation record
     * @return the Student ID as an integer
     */
    public int getStudentID()
    {
        return studentID; //Returns Student ID
    }

    @Override
    //toString Method
    public String toString()
    {
        return "Participation{" + "eventID=" + eventID + ", studentID=" + studentID + '}'; //Returns all the
values as a formatted string
    }
}
```

Student Object Class:

```
/*
 * To change this license header, choose License Headers in Project Properties.
 * To change this template file, choose Tools | Templates
 * and open the template in the editor.
 */
package Objects;

/**
 * Store Details of a Student
 * @author ruth2
 */
public class Student {//Class Name
    //Components
    private int stuID; //Student ID
    private String stuName; //Studnet First Name
    private String stuSurname; //Student surname
    private int grade; //Student grade (8-12)
    private int classPoint; //Student class (1-5)
    private char gender; //Student gender (M/F)
    private boolean catering; // the Boolean indicating if the student does or does not do catering
    private boolean photography; // the Boolean indicating if the student does or does not do photography
```

```
    private boolean av; // the Boolean indicating if the student does or does not do AV
    private boolean firstAid; //the Boolean indicating if the student does or does not do First Aid

    /**
     * Parameterized Constructor Method to assign values to a student
     * @param stuID the integer ID of the student
     * @param stuName the String first name of the student
     * @param stuSurname the String surname of the student
     * @param grade the integer grade of the student (8 to 12)
     * @param classPoint the integer class of the student (1 to 5)
     * @param gender the character gender of the student (M or F)
     * @param catering the Boolean indicating if the student does or does not do catering
     * @param photography the Boolean indicating if the student does or does not do photography
     * @param av the Boolean indicating if the student does or does not do AV
     * @param firstAid the Boolean indicating if the student does or does not do First Aid
     */
    public Student(int stuID, String stuName, String stuSurname, int grade, int classPoint, char gender,
boolean catering, boolean photography, boolean av, boolean firstAid)
    {
        this.stuID = stuID; //Assigns imput value to the variable
        this.stuName = stuName; //Assigns imput value to the variable
        this.stuSurname = stuSurname; //Assigns imput value to the variable
        this.grade = grade; //Assigns imput value to the variable
        this.classPoint = classPoint; //Assigns imput value to the variable
        this.gender = gender; //Assigns imput value to the variable
        this.catering = catering; //Assigns imput value to the variable
        this.photography = photography; //Assigns imput value to the variable
        this.av = av; //Assigns imput value to the variable
        this.firstAid = firstAid; //Assigns imput value to the variable
    }

    //Accessor and Mutator Methods
    /**
     * Getter method for the student ID
     * @return the Student ID as an integer
     */
    public int getStuID()
    {
        return stuID; //Returns the Student ID
    }

    /**
     * Getter method for the student first name
     * @return the student's first name as a string
     */
    public String getStuName()
    {
        return stuName; //Returns the Student First Name
    }
```

```java
/**
 * Getter method for the student surname
 * @return String value of the student surname
 */
public String getStuSurname()
{
    return stuSurname; //Resturns the Student surname
}

/**
 * Getter method for the student grade
 * @return Integer value of the students grade
 */
public int getGrade()
{
    return grade; //Returns the student grade
}

/**
 * Getter method for the students class
 * @return Integer value of the students class
 */
public int getClassPoint()
{
    return classPoint; //Returns the students class
}

/**
 * Setter method for the students Class
 * @param classPoint receives the class of the learner
 */
public void setClassPoint(int classPoint)
{
    this.classPoint = classPoint; //Assigns the imput value to the variable
}

/**
 * Getter method for the students gender
 * @return the character value of the students gender (M/F)
 */
public char getGender()
{
    return gender; //Returns the students gender
}

/**
 * Setter method for the students gender
 * @param gender the character value of the gender
```

```
     */
    public void setGender(char gender)
    {
        this.gender = gender; //Assigns the imput value to the variable
    }

    /**
     * Getter method for Boolean indicating if the student does or does not do catering
     * @return the Boolean indicating if the student does or does not do catering
     */
    public boolean isCatering()
    {
        return catering;
    }

    /**
     * Setter method for the Boolean indicating if the student does or does not do catering
     * @param catering receives the Boolean indicating if the student does or does not do catering
     */
    public void setCatering(boolean catering)
    {
        this.catering = catering; //Assigns the imput value to the variable
    }

    /**
     * Getter method for the Boolean indicating if the student does or does not do photography
     * @return the Boolean indicating if the student does or does not do photography
     */
    public boolean isPhotography()
    {
        return photography; //Returns for the Boolean indicating if the student does or does not do
photography
    }

    /**
     * Setter method for the Boolean indicating if the student does or does not do photography
     * @param photography receives the Boolean indicating if the student does or does not do
photography
     */
    public void setPhotography(boolean photography)
    {
        this.photography = photography;//Assigns  the imput value to the variable
    }

    /**
     * Getter method for the Boolean indicating if the student does or does not do AV
     * @return the Boolean indicating if the student does or does not do AV
     */
    public boolean isAv()
```

```
    {
        return av; //Return the Boolean indicating if the student does or does not do AV
    }

    /**
     * Setter method for the Boolean indicating if the student does or does not do AV
     * @param av the Boolean indicating if the student does or does not do AV
     */
    public void setAv(boolean av)
    {
        this.av = av; //Assigns the imput value to the variable
    }

    /**
     * Getter method for the Boolean indicating if the student does or does not do FirstAid
     * @return the Boolean indicating if the student does or does not do FirstAid
     */
    public boolean isFirstAid()
    {
        return firstAid;//Returns the Boolean indicating if the student does or does not do FirstAid
    }

    /**
     * Setter method the Boolean indicating if the student does or does not do FirstAid
     * @param firstAid receives the Boolean indicating if the student does or does not do FirstAid
     */
    public void setFirstAid(boolean firstAid)
    {
        this.firstAid = firstAid; //Assigns the imput value to the variable
    }

    @Override
    //toString Method
    public String toString() //Returns the values of the student as a formatted String
    {
        return "Student{" + "stuID=" + stuID + ", stuName=" + stuName + ", stuSurname=" + stuSurname +
", grade=" + grade + ", classPoint=" + classPoint + ", gender=" + gender + ", catering=" + catering + ",
photography=" + photography + ", av=" + av + ", firstAid=" + firstAid + '}';
    }
}
```

DBService Object Class:

```
/*
 * To change this license header, choose License Headers in Project Properties.
 * To change this template file, choose Tools | Templates
 * and open the template in the editor.
 */
```

```java
package Objects;
//Code adapted from Petri Mostert, previous teacher
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

/**
 * Establishes a connection to the database
 * @author ruth2
 */
public class DBService {
    private String url = "jdbc:ucanaccess://2022PAT.accdb";
    private Connection conn = null;

    /**
     * Establishes a connection to the database or throws an error
     */
    public DBService()
    {
        try
        {
            conn = DriverManager.getConnection(url); // Tries to establish a connection using the connection
string
        }
        catch(SQLException e) // Throws SQLException if connection cannot be established.
        {

        }
    }

    /**
     * Sends and receives SELECT queries between DB and Java
     * @param sql the SQL query as a String
     * @return the result of the SQL query
     * @throws SQLException
     */
    public ResultSet queryDatabase(String sql) throws SQLException    {
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet set = stmt.executeQuery(); // Executes the query received from paramater
        return set;
    }

    /**
     *Updates the DB using queries in Java
     * @param sql the SQl query as a String
     * @throws SQLException
```

```
     */
    public void updateQuery(String sql) throws SQLException
    {
        PreparedStatement stmt = conn.prepareStatement(sql);
        stmt.execute(); // Executes the query received from paramater
    }
}
```